

---

# ArkTS Specification

*Release 1.1.0*

2025.03.04



---

# Contents

---

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                      | <b>1</b>  |
| 1.1      | Common Description . . . . .             | 1         |
| 1.2      | Lexical and Syntactic Notation . . . . . | 3         |
| 1.3      | Terms and Definitions . . . . .          | 4         |
| <b>2</b> | <b>Lexical Elements</b>                  | <b>7</b>  |
| 2.1      | Use of Unicode Characters . . . . .      | 7         |
| 2.2      | Lexical Input Elements . . . . .         | 7         |
| 2.3      | White Spaces . . . . .                   | 8         |
| 2.4      | Line Separators . . . . .                | 8         |
| 2.5      | Tokens . . . . .                         | 8         |
| 2.6      | Identifiers . . . . .                    | 9         |
| 2.7      | Keywords . . . . .                       | 10        |
| 2.8      | Operators and Punctuators . . . . .      | 11        |
| 2.9      | Literals . . . . .                       | 12        |
| 2.9.1    | Numeric Literals . . . . .               | 12        |
| 2.9.2    | Integer Literals . . . . .               | 12        |
| 2.9.3    | Floating-Point Literals . . . . .        | 14        |
| 2.9.4    | Bigint Literals . . . . .                | 15        |
| 2.9.5    | Boolean Literals . . . . .               | 15        |
| 2.9.6    | String Literals . . . . .                | 16        |
| 2.9.7    | Multiline String Literal . . . . .       | 17        |
| 2.9.8    | Null Literal . . . . .                   | 18        |
| 2.9.9    | Undefined Literal . . . . .              | 18        |
| 2.10     | Comments . . . . .                       | 18        |
| 2.11     | Semicolons . . . . .                     | 19        |
| <b>3</b> | <b>Types</b>                             | <b>21</b> |
| 3.1      | Predefined Types . . . . .               | 21        |
| 3.1.1    | Primitive Types . . . . .                | 22        |
| 3.1.2    | Boxed Types . . . . .                    | 22        |
| 3.1.3    | Numeric Types . . . . .                  | 22        |
| 3.2      | User-Defined Types . . . . .             | 23        |
| 3.3      | Types by Category . . . . .              | 23        |
| 3.4      | Using Types . . . . .                    | 23        |
| 3.5      | Named Types . . . . .                    | 25        |
| 3.6      | Type References . . . . .                | 25        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 3.7      | Value Types . . . . .                           | 26        |
| 3.7.1    | Integer Types and Operations . . . . .          | 27        |
| 3.7.2    | Floating-Point Types and Operations . . . . .   | 28        |
| 3.7.3    | Numeric Types Hierarchy . . . . .               | 29        |
| 3.7.4    | Boolean Types and Operations . . . . .          | 30        |
| 3.8      | Reference Types . . . . .                       | 30        |
| 3.9      | Type Object . . . . .                           | 31        |
| 3.10     | Type never . . . . .                            | 31        |
| 3.11     | Type void . . . . .                             | 32        |
| 3.12     | Type undefined . . . . .                        | 32        |
| 3.13     | Type null . . . . .                             | 33        |
| 3.14     | Type string . . . . .                           | 33        |
| 3.15     | Type bigint . . . . .                           | 33        |
| 3.16     | Literal Types . . . . .                         | 34        |
| 3.16.1   | Supertypes of Literal Types . . . . .           | 34        |
| 3.16.2   | Operations on Literal Types . . . . .           | 35        |
| 3.17     | Array Types . . . . .                           | 35        |
| 3.17.1   | Resizable Array Types . . . . .                 | 35        |
| 3.18     | Tuple Types . . . . .                           | 37        |
| 3.19     | Function Types . . . . .                        | 37        |
| 3.20     | Union Types . . . . .                           | 39        |
| 3.20.1   | Union Types Normalization . . . . .             | 41        |
| 3.20.2   | Access to Common Union Members . . . . .        | 42        |
| 3.21     | Nullish Types . . . . .                         | 43        |
| 3.22     | Default Values for Types . . . . .              | 44        |
| <b>4</b> | <b>Names, Declarations and Scopes</b> . . . . . | <b>47</b> |
| 4.1      | Names . . . . .                                 | 47        |
| 4.2      | Declarations . . . . .                          | 48        |
| 4.3      | Distinguishable Declarations . . . . .          | 48        |
| 4.4      | Scopes . . . . .                                | 49        |
| 4.5      | Accessible . . . . .                            | 50        |
| 4.6      | Type Declarations . . . . .                     | 51        |
| 4.6.1    | Type Alias Declaration . . . . .                | 51        |
| 4.7      | Variable and Constant Declarations . . . . .    | 52        |
| 4.7.1    | Variable Declarations . . . . .                 | 52        |
| 4.7.2    | Constant Declarations . . . . .                 | 54        |
| 4.7.3    | Assignability with Initializer . . . . .        | 55        |
| 4.7.4    | Type Inference from Initializer . . . . .       | 55        |
| 4.8      | Function Declarations . . . . .                 | 56        |
| 4.8.1    | Signatures . . . . .                            | 56        |
| 4.8.2    | Parameter List . . . . .                        | 56        |
| 4.8.3    | Readonly Parameters . . . . .                   | 57        |
| 4.8.4    | Optional Parameters . . . . .                   | 57        |
| 4.8.5    | Rest Parameter . . . . .                        | 58        |
| 4.8.6    | Shadowing by Parameter . . . . .                | 60        |
| 4.8.7    | Return Type . . . . .                           | 60        |
| 4.8.8    | Return Type Inference . . . . .                 | 61        |
| <b>5</b> | <b>Generics</b> . . . . .                       | <b>63</b> |
| 5.1      | Type Parameters . . . . .                       | 63        |
| 5.1.1    | Type Parameter Constraint . . . . .             | 64        |
| 5.1.2    | Type Parameter Default . . . . .                | 65        |
| 5.1.3    | Type Parameter Variance . . . . .               | 66        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 5.2      | Generic Instantiations . . . . .                                | 67        |
| 5.2.1    | Type Arguments . . . . .                                        | 68        |
| 5.2.2    | Explicit Generic Instantiations . . . . .                       | 68        |
| 5.2.3    | Implicit Generic Instantiations . . . . .                       | 69        |
| 5.3      | Utility Types . . . . .                                         | 70        |
| 5.3.1    | Partial Utility Type . . . . .                                  | 70        |
| 5.3.2    | Required Utility Type . . . . .                                 | 71        |
| 5.3.3    | Readonly Utility Type . . . . .                                 | 71        |
| 5.3.4    | Record Utility Type . . . . .                                   | 72        |
| 5.3.5    | Utility Type Private Fields . . . . .                           | 73        |
| <b>6</b> | <b>Contexts and Conversions</b>                                 | <b>75</b> |
| 6.1      | Assignment-like Contexts . . . . .                              | 76        |
| 6.2      | String Operator Contexts . . . . .                              | 77        |
| 6.3      | Numeric Operator Contexts . . . . .                             | 78        |
| 6.4      | Casting Contexts and Conversions . . . . .                      | 79        |
| 6.4.1    | Numeric Casting Conversions . . . . .                           | 79        |
| 6.4.2    | Class or Interface Casting Conversions . . . . .                | 80        |
| 6.4.3    | Casting Conversions from <code>Object</code> . . . . .          | 80        |
| 6.4.4    | Casting Conversions from Type Parameter . . . . .               | 81        |
| 6.4.5    | Casting Conversions to Type Parameter . . . . .                 | 81        |
| 6.4.6    | Casting Conversions from Union . . . . .                        | 82        |
| 6.4.7    | Casting Conversions to Enumeration . . . . .                    | 82        |
| 6.5      | Implicit Conversions . . . . .                                  | 84        |
| 6.5.1    | Primitive Types Conversions . . . . .                           | 84        |
| 6.5.2    | Widening Primitive Conversions . . . . .                        | 84        |
| 6.5.3    | Constant Narrowing Integer Conversions . . . . .                | 85        |
| 6.5.4    | Boxing Conversions . . . . .                                    | 85        |
| 6.5.5    | Unboxing Conversions . . . . .                                  | 85        |
| 6.5.6    | Widening Union Conversions . . . . .                            | 86        |
| 6.5.7    | Widening Reference Conversions . . . . .                        | 87        |
| 6.5.8    | Character to String Conversions . . . . .                       | 87        |
| 6.5.9    | Constant String to Character Conversions . . . . .              | 87        |
| 6.5.10   | Function Types Conversions . . . . .                            | 88        |
| 6.5.11   | Tuple Types Conversions . . . . .                               | 89        |
| 6.5.12   | Enumeration to Constants Type Conversions . . . . .             | 89        |
| 6.5.13   | Constant to Enumeration Conversions . . . . .                   | 89        |
| 6.5.14   | Literal Type to its Supertype Conversions . . . . .             | 90        |
| <b>7</b> | <b>Expressions</b>                                              | <b>91</b> |
| 7.1      | Evaluation of Expressions . . . . .                             | 93        |
| 7.1.1    | Normal and Abrupt Completion of Expression Evaluation . . . . . | 93        |
| 7.1.2    | Order of Expression Evaluation . . . . .                        | 94        |
| 7.1.3    | Operator Precedence . . . . .                                   | 94        |
| 7.1.4    | Evaluation of Arguments . . . . .                               | 95        |
| 7.1.5    | Evaluation of Other Expressions . . . . .                       | 96        |
| 7.2      | Literal . . . . .                                               | 96        |
| 7.3      | Named Reference . . . . .                                       | 96        |
| 7.4      | Array Literal . . . . .                                         | 97        |
| 7.4.1    | Array Literal Type Inference from Context . . . . .             | 98        |
| 7.4.2    | Array Type Inference from Types of Elements . . . . .           | 100       |
| 7.5      | Object Literal . . . . .                                        | 100       |
| 7.5.1    | Object Literal of Class Type . . . . .                          | 101       |
| 7.5.2    | Object Literal of Interface Type . . . . .                      | 102       |

|        |                                              |     |
|--------|----------------------------------------------|-----|
| 7.5.3  | Object Literal of <code>Record</code> Type   | 103 |
| 7.5.4  | Object Literal Evaluation                    | 104 |
| 7.6    | Spread Expression                            | 105 |
| 7.7    | Parenthesized Expression                     | 106 |
| 7.8    | <code>this</code> Expression                 | 106 |
| 7.9    | Field Access Expression                      | 107 |
| 7.9.1  | Accessing Current Object Fields              | 107 |
| 7.9.2  | Accessing <code>SuperClass</code> Properties | 108 |
| 7.10   | Method Call Expression                       | 108 |
| 7.10.1 | Step 1: Selection of Type to Use             | 108 |
| 7.10.2 | Step 2: Selection of Method                  | 109 |
| 7.10.3 | Step 3: Checking Method Modifiers            | 109 |
| 7.10.4 | Type of Method Call Expression               | 109 |
| 7.11   | Function Call Expression                     | 110 |
| 7.12   | Indexing Expressions                         | 111 |
| 7.12.1 | Array Indexing Expression                    | 111 |
| 7.12.2 | Record Indexing Expression                   | 113 |
| 7.13   | Chaining Operator                            | 114 |
| 7.14   | New Expressions                              | 115 |
| 7.15   | Cast Expressions                             | 116 |
| 7.16   | <code>InstanceOf</code> Expression           | 116 |
| 7.17   | <code>typeof</code> Expression               | 117 |
| 7.18   | Ensure-Not-Nullish Expression                | 119 |
| 7.19   | Nullish-Coalescing Expression                | 119 |
| 7.20   | Unary Expressions                            | 120 |
| 7.20.1 | Postfix Increment                            | 120 |
| 7.20.2 | Postfix Decrement                            | 121 |
| 7.20.3 | Prefix Increment                             | 121 |
| 7.20.4 | Prefix Decrement                             | 122 |
| 7.20.5 | Unary Plus                                   | 122 |
| 7.20.6 | Unary Minus                                  | 122 |
| 7.20.7 | Bitwise Complement                           | 123 |
| 7.20.8 | Logical Complement                           | 123 |
| 7.21   | Multiplicative Expressions                   | 124 |
| 7.21.1 | Multiplication                               | 124 |
| 7.21.2 | Division                                     | 125 |
| 7.21.3 | Remainder                                    | 126 |
| 7.22   | Additive Expressions                         | 127 |
| 7.22.1 | String Concatenation                         | 127 |
| 7.22.2 | Additive Operators for Numeric Types         | 128 |
| 7.23   | Shift Expressions                            | 129 |
| 7.24   | Relational Expressions                       | 130 |
| 7.24.1 | Numerical Relational Operators               | 130 |
| 7.24.2 | String Relational Operators                  | 131 |
| 7.24.3 | BigInt Relational Operators                  | 131 |
| 7.24.4 | Boolean Relational Operators                 | 131 |
| 7.24.5 | Enumeration Relational Operators             | 132 |
| 7.25   | Equality Expressions                         | 132 |
| 7.25.1 | Numerical Equality Operators                 | 133 |
| 7.25.2 | String Equality Operators                    | 134 |
| 7.25.3 | BigInt Equality Operators                    | 134 |
| 7.25.4 | Boolean Equality Operators                   | 134 |
| 7.25.5 | Enumeration Equality Operators               | 135 |
| 7.25.6 | Reference Equality Based on Actual Type      | 135 |

|          |                                                                              |            |
|----------|------------------------------------------------------------------------------|------------|
| 7.25.7   | Reference Equality . . . . .                                                 | 137        |
| 7.25.8   | Extended Equality with <code>null</code> or <code>undefined</code> . . . . . | 137        |
| 7.26     | Bitwise and Logical Expressions . . . . .                                    | 138        |
| 7.26.1   | Integer Bitwise Operators . . . . .                                          | 139        |
| 7.26.2   | Boolean Logical Operators . . . . .                                          | 139        |
| 7.27     | Conditional-And Expression . . . . .                                         | 139        |
| 7.28     | Conditional-Or Expression . . . . .                                          | 140        |
| 7.29     | Assignment . . . . .                                                         | 141        |
| 7.29.1   | Simple Assignment Operator . . . . .                                         | 141        |
| 7.29.2   | Compound Assignment Operators . . . . .                                      | 143        |
| 7.29.3   | Left-Hand-Side Expressions . . . . .                                         | 144        |
| 7.30     | Conditional Expressions . . . . .                                            | 145        |
| 7.31     | String Interpolation Expressions . . . . .                                   | 145        |
| 7.32     | Lambda Expressions . . . . .                                                 | 146        |
| 7.32.1   | Lambda Signature . . . . .                                                   | 147        |
| 7.32.2   | Lambda Body . . . . .                                                        | 148        |
| 7.32.3   | Lambda Expression Type . . . . .                                             | 148        |
| 7.32.4   | Runtime Evaluation of Lambda Expressions . . . . .                           | 149        |
| 7.33     | Constant Expressions . . . . .                                               | 150        |
| <b>8</b> | <b>Statements</b> . . . . .                                                  | <b>151</b> |
| 8.1      | Normal and Abrupt Statement Execution . . . . .                              | 151        |
| 8.2      | Expression Statements . . . . .                                              | 152        |
| 8.3      | Block . . . . .                                                              | 152        |
| 8.4      | Local Declarations . . . . .                                                 | 152        |
| 8.5      | <code>if</code> Statements . . . . .                                         | 153        |
| 8.6      | Loop Statements . . . . .                                                    | 153        |
| 8.7      | <code>while</code> Statements and <code>do</code> Statements . . . . .       | 154        |
| 8.8      | <code>for</code> Statements . . . . .                                        | 154        |
| 8.9      | <code>for-of</code> Statements . . . . .                                     | 155        |
| 8.10     | <code>break</code> Statements . . . . .                                      | 156        |
| 8.11     | <code>continue</code> Statements . . . . .                                   | 156        |
| 8.12     | <code>return</code> Statements . . . . .                                     | 157        |
| 8.13     | <code>switch</code> Statements . . . . .                                     | 157        |
| 8.14     | <code>throw</code> Statements . . . . .                                      | 158        |
| 8.15     | <code>try</code> Statements . . . . .                                        | 159        |
| 8.15.1   | <code>catch</code> Clause . . . . .                                          | 159        |
| 8.15.2   | <code>finally</code> Clause . . . . .                                        | 160        |
| 8.15.3   | <code>try</code> Statement Execution . . . . .                               | 160        |
| <b>9</b> | <b>Classes</b> . . . . .                                                     | <b>163</b> |
| 9.1      | Class Declarations . . . . .                                                 | 164        |
| 9.1.1    | Abstract Classes . . . . .                                                   | 164        |
| 9.1.2    | Final Classes . . . . .                                                      | 165        |
| 9.1.3    | Class Extension Clause . . . . .                                             | 165        |
| 9.1.4    | Class Implementation Clause . . . . .                                        | 166        |
| 9.1.5    | Implementing Interface Properties . . . . .                                  | 168        |
| 9.2      | Class Body . . . . .                                                         | 169        |
| 9.3      | Class Members . . . . .                                                      | 170        |
| 9.4      | Access Modifiers . . . . .                                                   | 170        |
| 9.4.1    | Private Access Modifier . . . . .                                            | 171        |
| 9.4.2    | Internal Access Modifier . . . . .                                           | 171        |
| 9.4.3    | Protected Access Modifier . . . . .                                          | 171        |
| 9.4.4    | Public Access Modifier . . . . .                                             | 172        |

|           |                                                |            |
|-----------|------------------------------------------------|------------|
| 9.5       | Field Declarations . . . . .                   | 172        |
| 9.5.1     | Static Fields . . . . .                        | 173        |
| 9.5.2     | Readonly (Constant) Fields . . . . .           | 173        |
| 9.5.3     | Field Initialization . . . . .                 | 173        |
| 9.5.4     | Overriding Fields . . . . .                    | 175        |
| 9.6       | Method Declarations . . . . .                  | 175        |
| 9.6.1     | Static Methods . . . . .                       | 176        |
| 9.6.2     | Instance Methods . . . . .                     | 176        |
| 9.6.3     | Abstract Methods . . . . .                     | 176        |
| 9.6.4     | Final Methods . . . . .                        | 177        |
| 9.6.5     | Async Methods . . . . .                        | 177        |
| 9.6.6     | Overriding Methods . . . . .                   | 177        |
| 9.6.7     | Native Methods . . . . .                       | 178        |
| 9.6.8     | Method Body . . . . .                          | 178        |
| 9.6.9     | Methods Returning <code>this</code> . . . . .  | 178        |
| 9.7       | Accessor Declarations . . . . .                | 179        |
| 9.8       | Class Initializer . . . . .                    | 181        |
| 9.9       | Constructor Declaration . . . . .              | 181        |
| 9.9.1     | Formal Parameters . . . . .                    | 182        |
| 9.9.2     | Constructor Body . . . . .                     | 182        |
| 9.9.3     | Explicit Constructor Call . . . . .            | 184        |
| 9.9.4     | Default Constructor . . . . .                  | 185        |
| 9.10      | Inheritance . . . . .                          | 186        |
| 9.11      | Local Classes and Interfaces . . . . .         | 186        |
| <b>10</b> | <b>Interfaces</b>                              | <b>189</b> |
| 10.1      | Interface Declarations . . . . .               | 189        |
| 10.2      | Superinterfaces and Subinterfaces . . . . .    | 190        |
| 10.3      | Interface Body . . . . .                       | 191        |
| 10.4      | Interface Members . . . . .                    | 192        |
| 10.5      | Interface Properties . . . . .                 | 192        |
| 10.6      | Interface Method Declarations . . . . .        | 193        |
| 10.6.1    | Interface Method Overloading . . . . .         | 193        |
| 10.7      | Interface Inheritance . . . . .                | 193        |
| <b>11</b> | <b>Enumerations</b>                            | <b>195</b> |
| 11.1      | Enumeration Integer Values . . . . .           | 196        |
| 11.2      | Enumeration String Values . . . . .            | 196        |
| 11.3      | Enumeration Operations . . . . .               | 197        |
| <b>12</b> | <b>Error Handling</b>                          | <b>199</b> |
| 12.1      | Errors . . . . .                               | 199        |
| <b>13</b> | <b>Compilation Units</b>                       | <b>201</b> |
| 13.1      | Separate Modules . . . . .                     | 202        |
| 13.2      | Separate Module Initializer . . . . .          | 202        |
| 13.3      | Import Directives . . . . .                    | 202        |
| 13.3.1    | Bind All with Qualified Access . . . . .       | 203        |
| 13.3.2    | Simple Name Binding . . . . .                  | 204        |
| 13.3.3    | Several Bindings for One Import Path . . . . . | 205        |
| 13.3.4    | Default Import Binding . . . . .               | 206        |
| 13.3.5    | Type Binding . . . . .                         | 207        |
| 13.3.6    | Import Path . . . . .                          | 207        |
| 13.4      | Standard Library Usage . . . . .               | 208        |
| 13.5      | Declaration Modules . . . . .                  | 208        |



|           |                                                            |            |
|-----------|------------------------------------------------------------|------------|
| 13.6      | Compilation Unit Initialization . . . . .                  | 209        |
| 13.7      | Top-Level Declarations . . . . .                           | 210        |
| 13.7.1    | Exported Declarations . . . . .                            | 210        |
| 13.8      | Namespace Declarations . . . . .                           | 211        |
| 13.9      | Export Directives . . . . .                                | 214        |
| 13.9.1    | Selective Export Directive . . . . .                       | 214        |
| 13.9.2    | Single Export Directive . . . . .                          | 215        |
| 13.9.3    | Export Type Directive . . . . .                            | 215        |
| 13.9.4    | Re-Export Directive . . . . .                              | 216        |
| 13.10     | Top-Level Statements . . . . .                             | 216        |
| 13.11     | Program Entry Point . . . . .                              | 218        |
| 13.12     | Program Exit . . . . .                                     | 219        |
| <b>14</b> | <b>Ambient Declarations</b>                                | <b>221</b> |
| 14.1      | Ambient Constant Declarations . . . . .                    | 222        |
| 14.2      | Ambient Function Declarations . . . . .                    | 222        |
| 14.3      | Ambient Class Declarations . . . . .                       | 222        |
| 14.3.1    | Ambient Indexer . . . . .                                  | 223        |
| 14.3.2    | Ambient Call Signature . . . . .                           | 224        |
| 14.3.3    | Ambient Iterable . . . . .                                 | 224        |
| 14.4      | Ambient Interface Declarations . . . . .                   | 225        |
| 14.5      | Ambient Namespace Declarations . . . . .                   | 225        |
| 14.5.1    | Implementing Ambient Namespace Declaration . . . . .       | 226        |
| <b>15</b> | <b>Semantic Rules</b>                                      | <b>227</b> |
| 15.1      | Subtyping . . . . .                                        | 227        |
| 15.1.1    | Supertyping . . . . .                                      | 228        |
| 15.2      | Type Identity . . . . .                                    | 228        |
| 15.3      | Assignability . . . . .                                    | 228        |
| 15.4      | Invariance, Covariance and Contravariance . . . . .        | 228        |
| 15.5      | Compatibility of Call Arguments . . . . .                  | 229        |
| 15.6      | Type Inference . . . . .                                   | 230        |
| 15.6.1    | Smart Types . . . . .                                      | 230        |
| 15.7      | Overloading and Overriding . . . . .                       | 231        |
| 15.7.1    | Overload-Equivalent Signatures . . . . .                   | 232        |
| 15.7.2    | Override-Compatible Signatures . . . . .                   | 233        |
| 15.7.3    | Overloading for Functions . . . . .                        | 237        |
| 15.7.4    | Overloading and Overriding in Classes . . . . .            | 237        |
| 15.7.5    | Overloading and Overriding in Interfaces . . . . .         | 239        |
| 15.8      | Overload Resolution . . . . .                              | 240        |
| 15.8.1    | Selection of Applicable Candidates . . . . .               | 240        |
| 15.8.2    | Selection of Best Candidate . . . . .                      | 241        |
| 15.9      | Initializer Block . . . . .                                | 243        |
| 15.10     | Compatibility Features . . . . .                           | 244        |
| 15.10.1   | Extended Conditional Expressions . . . . .                 | 244        |
| <b>16</b> | <b>Support for GUI Programming</b>                         | <b>247</b> |
| <b>17</b> | <b>Experimental Features</b>                               | <b>249</b> |
| 17.1      | Character Type and Literals . . . . .                      | 250        |
| 17.1.1    | Character Literals . . . . .                               | 250        |
| 17.1.2    | Character Type and Operations . . . . .                    | 251        |
| 17.2      | Fixed Array Types . . . . .                                | 251        |
| 17.3      | Array Creation Expressions . . . . .                       | 252        |
| 17.3.1    | Runtime Evaluation of Array Creation Expressions . . . . . | 254        |

|           |                                                         |            |
|-----------|---------------------------------------------------------|------------|
| 17.4      | Indexable Types                                         | 254        |
| 17.5      | Iterable Types                                          | 256        |
| 17.6      | Callable Types                                          | 257        |
| 17.6.1    | Callable Types with <code>\$_invoke</code> Method       | 257        |
| 17.6.2    | Callable Types with <code>\$_instantiate</code> Method  | 258        |
| 17.7      | Statements                                              | 258        |
| 17.7.1    | For-of Type Annotation                                  | 259        |
| 17.8      | Function, Method and Constructor Overloading            | 259        |
| 17.8.1    | Function Overloading                                    | 259        |
| 17.8.2    | Class Method Overloading                                | 259        |
| 17.8.3    | Constructor Overloading                                 | 260        |
| 17.8.4    | Declaration Distinguishable by Signatures               | 260        |
| 17.9      | Native Functions and Methods                            | 261        |
| 17.9.1    | Native Functions                                        | 261        |
| 17.9.2    | Native Methods                                          | 261        |
| 17.9.3    | Native Constructors                                     | 261        |
| 17.10     | Final Classes and Methods                               | 262        |
| 17.10.1   | Final Classes                                           | 262        |
| 17.10.2   | Final Methods                                           | 262        |
| 17.11     | Default Interface Method Declarations                   | 262        |
| 17.12     | Adding Functionality to Existing Types                  | 263        |
| 17.12.1   | Functions with Receiver                                 | 263        |
| 17.12.2   | Receiver Type                                           | 265        |
| 17.12.3   | Accessors with Receiver                                 | 266        |
| 17.12.4   | Function Types with Receiver                            | 266        |
| 17.12.5   | Lambda Expressions with Receiver                        | 267        |
| 17.12.6   | Implicit <code>this</code> in Lambda with Receiver Body | 268        |
| 17.13     | Trailing Lambdas                                        | 269        |
| 17.14     | Enumeration Types Conversions                           | 271        |
| 17.15     | Enumeration Methods                                     | 271        |
| 17.15.1   | Errors and Initialization Expression                    | 271        |
| 17.16     | Coroutines                                              | 272        |
| 17.16.1   | Create and Launch a Coroutine                           | 272        |
| 17.16.2   | Awaiting a Promise                                      | 273        |
| 17.16.3   | <code>Promise&lt;T&gt;</code> Class                     | 274        |
| 17.17     | Async Functions and Methods                             | 274        |
| 17.17.1   | Async Functions                                         | 274        |
| 17.17.2   | Experimental Async Methods                              | 274        |
| 17.18     | DynamicObject Type                                      | 275        |
| 17.18.1   | DynamicObject Field Access                              | 275        |
| 17.18.2   | DynamicObject Method Call                               | 276        |
| 17.18.3   | DynamicObject Indexing Access                           | 276        |
| 17.18.4   | DynamicObject New Expression                            | 276        |
| 17.18.5   | DynamicObject Cast Expression                           | 277        |
| 17.19     | Packages                                                | 277        |
| 17.19.1   | Internal Access Modifier                                | 278        |
| 17.19.2   | Package Initializer                                     | 279        |
| 17.19.3   | Import and Overloading of Function Names                | 279        |
| 17.20     | Generics Experimental                                   | 280        |
| 17.20.1   | Nonnullish Type Parameter                               | 280        |
| <b>18</b> | <b>Annotations</b>                                      | <b>283</b> |
| 18.1      | Declaring Annotations                                   | 284        |
| 18.1.1    | Types of Annotation Fields                              | 284        |

|           |                                                                              |            |
|-----------|------------------------------------------------------------------------------|------------|
| 18.2      | Using Annotations . . . . .                                                  | 285        |
| 18.2.1    | Using Single Field Annotations . . . . .                                     | 286        |
| 18.3      | Exporting and Importing Annotations . . . . .                                | 287        |
| 18.4      | Ambient Annotations . . . . .                                                | 287        |
| 18.5      | Standard Annotations . . . . .                                               | 288        |
| 18.5.1    | Retention Annotation . . . . .                                               | 289        |
| <b>19</b> | <b>Standard Library</b>                                                      | <b>291</b> |
| <b>20</b> | <b>Implementation Details</b>                                                | <b>293</b> |
| 20.1      | Import Path Lookup . . . . .                                                 | 293        |
| 20.2      | Compilation Units in Host System . . . . .                                   | 293        |
| 20.3      | How to Get Type Via Reflection . . . . .                                     | 294        |
| 20.4      | Methods for <code>T[]</code> Types . . . . .                                 | 294        |
| 20.5      | Generic and Function Types Peculiarities . . . . .                           | 294        |
| 20.6      | Keyword <code>struct</code> and <code>ArkUI</code> . . . . .                 | 295        |
| <b>21</b> | <b>ArkTS-TypeScript compatibility</b>                                        | <b>297</b> |
| 21.1      | Reserved Names of TypeScript Types . . . . .                                 | 297        |
| 21.2      | Undefined is Not a Universal Value . . . . .                                 | 298        |
| 21.3      | Numeric Semantics . . . . .                                                  | 298        |
| 21.4      | Covariant Overriding . . . . .                                               | 299        |
| 21.5      | Function Types Compatibility . . . . .                                       | 299        |
| 21.6      | Compatibility for Utility Types . . . . .                                    | 299        |
| 21.7      | TypeScript Overload Signatures . . . . .                                     | 300        |
| 21.8      | Class Fields While Inheriting . . . . .                                      | 300        |
| 21.9      | Overriding for Primitive Types . . . . .                                     | 301        |
| 21.10     | Type <code>void</code> Compatibility . . . . .                               | 301        |
| 21.11     | Invariant Array Assignment . . . . .                                         | 301        |
| 21.12     | Tuples and Arrays . . . . .                                                  | 302        |
| 21.13     | Extending Class Object . . . . .                                             | 302        |
| 21.14     | Syntax of <code>extends</code> and <code>implements</code> Clauses . . . . . | 303        |
| 21.15     | Uniqueness of Functional Objects . . . . .                                   | 303        |
| 21.16     | Functional Objects for Methods . . . . .                                     | 303        |
| 21.17     | Differences in Namespaces . . . . .                                          | 303        |
| 21.18     | Differences in <code>Math.pow</code> . . . . .                               | 304        |
| <b>22</b> | <b>Grammar Summary</b>                                                       | <b>305</b> |
| <b>23</b> | <b>Contributors</b>                                                          | <b>327</b> |
|           | <b>Index</b>                                                                 | <b>329</b> |



This document presents complete information on the new common-purpose, multiparadigm programming language called ArkTS.

### 1.1 Common Description

The ArkTS language combines and supports features that are in use in many well-known programming languages, where these tools have already proven helpful and powerful.

ArkTS supports imperative, object-oriented, functional, and generic programming paradigms, and combines them safely and consistently.

At the same time, ArkTS does not support features that allow software developers to write dangerous, unsafe, or inefficient code. In particular, the language uses the strong static typing principle. It allows no dynamic type changes, as object types are determined by their declarations. Their semantic correctness is checked at compile time.

The following major aspects characterize the ArkTS language as a whole:

- Object orientation

The ArkTS language supports conventional class-based, *object-oriented approach* to programming (OOP). The major notions of this approach are as follows:

- Classes with single inheritance,
- Interfaces as abstractions to be implemented by classes, and
- Methods (class instance or interface methods) with a dynamically dispatched overriding mechanism.

Common in many (if not all) modern programming languages, object orientation enables powerful, flexible, safe, clear, and adequate software design.

- Modularity

The ArkTS language supports the *component programming approach*. It presumes that software is designed and implemented as a composition of *compilation units*. A compilation unit is typically represented as a *module* or as a *package*.

A module in ArkTS is a standalone, independently compiled unit that combines various programming resources (types, classes, functions, and so on). A module can communicate with other modules by exporting all or some of its resources to, or importing from other modules.

This feature provides a high level of software development process and software maintainability, supports flexible module reuse and efficient version control.

- Genericity

Some program entities in ArkTS can be *type-parameterized*. This means that an entity can represent a very high-level (abstract) concept. Providing more concrete type information makes the entity instantiated for a particular use case.

A classical illustration is the notion of a list that represents the ‘idea’ of an abstract data structure. This abstract notion can be turned into a concrete list by providing additional information (i.e., the type of list elements).

Supported by many programming languages, a similar feature (*generics* or *templates*) serves as the basis of the generic programming paradigm. It enables making programs and program structures more generic and reusable.

- Multitargeting

ArkTS provides an efficient application development solution for a wide range of devices. The language ecosystem is a developer-friendly and uniform programming environment for a range of popular platforms (called *cross-platform development*). It can generate optimized applications capable of operating under the limitations of lightweight devices, or realizing the full potential of any specific target hardware.

ArkTS is designed as a part of the modern language manifold. To provide an efficient and safely executable code, the language takes flexibility and power from TypeScript and its predecessor JavaScript, and the static typing principle from Java and Kotlin. The overall design keeps the ArkTS syntax style similar to that of those languages, and some of its important constructs are almost identical to theirs on purpose.

In other words, there is a significant *common subset* of features of ArkTS on the one hand, and of TypeScript, JavaScript, Java, and Kotlin on the other. Consequently, the ArkTS style and constructs are no puzzle for the TypeScript and Java users who can sense the meaning of most constructs of the new language even if not understand them completely.

This stylistic and semantic similarity permits smoothly migrating the applications originally written in TypeScript, Java, or Kotlin to ArkTS.

Like its predecessors, ArkTS is a relatively high-level language. It means that the language provides no access to low-level machine representations. As a high-level language, ArkTS supports automatic storage management. It means that dynamically created objects are deallocated automatically soon after they are no longer available, and deallocating them explicitly is not required.

ArkTS is not merely a language, but rather a comprehensive software development ecosystem that facilitates the creation of software solutions in various application domains.

The ArkTS ecosystem includes the language along with its compiler, accompanying documents, guidelines, tutorials, the standard library (see [Standard Library](#)), and a set of additional tools that perform automatic or semi-automatic transition from other languages (currently, TypeScript and Java) to ArkTS.

## 1.2 Lexical and Syntactic Notation

This section introduces the notation known as *context-free grammar*. It is used in this specification to define the lexical and syntactic structure of a program.

The ArkTS lexical notation defines a set of productions (rules) that specify the structure of the elementary language parts called *tokens*. All tokens are defined in [Lexical Elements](#). The set of tokens (identifiers, keywords, numbers/numeric literals, operator signs, delimiters), special characters (white spaces and line separators), and comments comprises the language's *alphabet*.

The tokens defined by the lexical grammar are terminal symbols of syntactic notation. Syntactic notation defines a set of productions starting from the goal symbol *compilationUnit* (see [Compilation Units](#)). It is a sentence that consists of a single distinguished nonterminal, and describes how sequences of tokens can form syntactically correct programs.

Lexical and syntactic grammars are defined as a range of productions. Each production:

- Is comprised of an abstract symbol (*nonterminal*) as its left-hand side, and a sequence of one or more *nonterminal* and *terminal* symbols as its *right-hand side*.
- Includes the ':' character as a separator between the left- and right-hand sides, and the ';' character as the end marker.

Grammars draw terminal symbols from a fixed-width form. Starting from the goal symbol, grammars specify the language itself, i.e., the set of possible sequences of terminal symbols that can result from repeatedly replacing any nonterminal in the left-hand-side sequence for a right-hand side of the production.

Grammars can use the following additional symbols—sometimes called *metasymbols*—in the right-hand side of a grammar production along with terminal and nonterminal symbols:

- Vertical line '|' to specify alternatives.
- Question mark '?' to specify an optional (zero- or one-time) occurrence of the preceding terminal or nonterminal.
- Asterisk '\*' to mark a *terminal* or *nonterminal* that can occur zero or more times.
- Parentheses '(' and ')' to enclose any sequence of terminals and/or nonterminals marked with the '?' or '\*' metasymbols.

Such additional symbols specify the structuring rules for terminal and nonterminal sequences. However, they are not part of the terminal symbol sequences that comprise the resultant program text.

The production below is an example that specifies a list of expressions:

```
expressionList:
    expression (',' expression) * ',' '?'
    ;
```

This production introduces the following structure defined by the nonterminal *expressionList*. The expression list must consist of a sequence of *expressions* separated by the terminal ',' symbol. The sequence must have at least one *expression*. The list is optionally terminated by the terminal ',' symbol.

All grammar rules are presented in the Grammar section (see [Grammar Summary](#)) of this specification.

## 1.3 Terms and Definitions

This section contains the alphabetical list of important terms found in the Specification, and their ArkTS-specific definitions. Such definitions are not generic and can differ significantly from the definitions of same terms as used in other languages, application areas, or industries.

**abstract declaration** – ordinary interface method declaration that specifies the method’s name and signature.

**casting conversion** – conversion of an operand of a cast expression to an explicitly specified type.

**class level scope** – a name declared inside a class. It is accessible inside and sometimes—by means of an access modifier, or via a derived class—outside that class.

**comment** – a piece of text, insignificant for the syntactic grammar, that is added to the stream in order to document and compliment the source code.

**constant** – see *constant declaration*.

**constant declaration** – a declaration that introduces a new variable to which an immutable initial value can be assigned only once at the time of instantiation.

**context-free grammar** – grammar in which the left-hand side of each production rule consists of only a single non-terminal symbol.

**expression** – a formula for calculating values. An expression has the syntactic form that is a composition of operators and parentheses, where parentheses are used to change the order of calculation. By default, the order of calculation is determined by operator preferences.

**fit into (v.)** – belong, or be implicitly convertible (see *Widening Primitive Conversions*) to an entity.

**function declaration** – a declaration that specifies names, signatures, and bodies when introducing a named function.

**function parameter scope** – the scope of a type parameter name in a function declaration. It is identical to that entire declaration.

**function scope** – same as *method scope*.

**function types conversion** – conversion of one function type to another.

**generic** – see *generic type*.

**generic type** – a named type (class or interface) that has type parameters.

**goal symbol** – a sentence that consists of a single distinguished nonterminal (*compilationUnit*). The *goal symbol* describes how sequences of tokens can form syntactically correct programs.

**grammar** – a set of rules that describe what possible sequences of terminal and nonterminal symbols a programming language interprets as correct.

Grammar is a range of productions. Each production comprises an abstract symbol (nonterminal) as its left-hand side, and a sequence of nonterminal and terminal symbols as its right-hand side. Each production has the character ‘:’ as a separator between the left- and right-hand sides, and the character ‘;’ as the end marker.

**interface level scope** – a name declared inside an interface. It is accessible inside and outside that interface.

**keyword** – one of the *reserved words* that have their meanings permanently predefined in the language.

**linearization** – de-nesting of all nested types in a union type to present them in the form of a flat line that has no more union types included.

**literal** – a representation of a certain value type.

**match (v.)** – correspond to an entity.



**metasymbol** – additional symbols ‘|’, ‘?’, ‘\*’, ‘(’, and ‘)’ that can be used along with terminal and nonterminal symbols in the right-hand side of a grammar production.

**method** – ordered 3-tuple consisting of type parameters, argument types, return type.

**method scope** – the scope of a name declared immediately inside the body of a method (function) declaration. Method scope is identical to the body of that method (function) declaration from the place of declaration, and up to the end of the body.

**module level scope** – a name that is applicable to separate modules only. It is accessible throughout the entire module and in other packages if exported.

**narrowing conversion** – a conversion that can cause a loss information about the overall magnitude of a numeric value, and potentially a loss of precision and range.

**non-generic** – see *non-generic type*.

**non-generic type** – a named type (class or interface) that has no type parameters.

**nonterminal** – see *nonterminal symbol*.

**nonterminal symbol** – a syntactically variable token that results from the successive application of the production rules.

**nullable type** – a variable declared to have the value `null`, or `type T | null` that can hold values of type `T` and its derived types.

**nullish value** – a reference which is null or undefined.

**operand** – an argument of an operation. Syntactically, operands have the form of simple or qualified identifiers that refer to variables or members of structured objects. In turn, operands can be operators whose preferences (‘priorities’) are higher than the preference of the given operator.

**operation** – an informal notion that means an action or a process of operator evaluation.

**operation sign** – a language token that signifies an operator and conventionally denotes a usual mathematical operator, for example, ‘+’ for addition operator, ‘/’ for division, etc. However, some languages allow using identifiers to denote operators, and/or arbitrarily combining characters that are not tokens in the alphabet of that language, i.e., operator signs.

**operator (in programming languages)** – the term can have several meanings.

(1) a token that denotes the action to be performed on a value (addition, subtraction, comparison, etc.).

(2) a syntactic construct that denotes an elementary calculation within an expression. Normally, an operator consists of an operator sign and one or more operands.

In unary operators that have a single operand, the operator sign can be placed either in front of (*prefix* unary operator) or after an operand (*postfix* unary operator).

If both operands are available, then the operator sign can be placed between the two (*infix* binary operator). A conditional operator with three operands is called *ternary*.

Some operators have special notations. For example, the indexing operator, while formally being a binary operator, has a conventional form like `a[i]`.

Some languages treat operators as *syntactic sugar*—a conventional version of a more common construct, i.e., *function call*. Therefore, an operator like `a+b` is conceptually treated as the call `+(a, b)`, where the operator sign plays the role of the function name, and the operands are function call arguments.

**overloading** – situation where different functions or methods inherited by or declared in the same class or interface have the same name but different signatures.

**own (adj.)** – of a member textually declared in a class, interface, type, etc., as opposed to members inherited from base class (superclass), base interfaces (superinterface), base type (supertype), etc.

**package level scope** – a name that is declared on the package level, and accessible throughout the entire package, and in other packages if exported.

**primitive type** – numeric value types, character, and boolean value types whose names are reserved, and cannot be used for user-defined type names.

**production** – a sequence of terminal and nonterminal symbols that a programming language interprets as correct.

**punctuator** – a token that serves for separating, completing, or otherwise organizing program elements and parts: commas, semicolons, parentheses, square brackets, etc.

**qualified name** – a name that consists of a sequence of identifiers separated with the token ‘.’.

**scope of a name** – a region of program code within which an entity—as declared by that name—can be accessed or referred to by its simple name without any qualification.

**shadowing** – situation where a derived class introduces a field with the same name as that of its base class.

**simple name** – a name that consists of a single identifier.

**static member** – a class member that is not related to a particular class instance. A static member can be used across an entire program by using a qualified name notation (qualification is the name of a class).

**subcomponent (derived component, child component)** – a component produced by, inherited from, and dependent from another component.

**supercomponent (base component, parent component)** – a component from which another component is derived.

**terminal** – see *terminal symbol*.

**terminal symbol** – a syntactically invariable token (i.e., a syntactic notation defined directly by an invariable form of the lexical grammar that defines a set of productions starting from the *goal symbol*).

**token** – an elementary part of a programming language: identifier, keyword, operator and punctuator, or literal. Tokens are lexical input elements that form the vocabulary of a language, and can act as terminal symbols of the language’s syntactic grammar.

**tokenization** – finding the longest sequence of characters that forms a valid token (i.e., establishing a token) in the process of codebase reading by the machine.

**truthiness** – concept that extends the Boolean logic to operands and results of non-Boolean types, and allows handling the value of a valid expression of a non-void type as `Truthy` or `Falsy`, depending on the kind of the value type.

**type parameter scope** – the name of a type parameter declared in a class or an interface. The type parameter scope is identical to the entire declaration (except static member declarations).

**type reference** – references that refer to named types by specifying their type names, and type arguments, where applicable, to be substituted for type parameters of the named type.

**variable** – see *variable declaration*.

**variable declaration** – a declaration that introduces a new named variable a modifiable initial value can be assigned to.

**white space** – one of lexical input elements that separate tokens from one another in order to improve the source code readability and avoid ambiguities.

**widening conversion** – a conversion that causes no loss of information about the overall magnitude of a numeric value.

This chapter discusses the lexical structure of the ArkTS programming language, and the analytical conventions.

### 2.1 Use of Unicode Characters

The ArkTS programming language uses characters of the Unicode Character set<sup>1</sup> as its terminal symbols. It uses the Unicode UTF-16 encoding to represent text in sequences of 16-bit code units.

The term *Unicode code point* is used in this specification only where such representation is relevant to refer the reader to Unicode Character set and UTF-16 encoding. Where such representation is irrelevant to the discussion, the generic term *character* is used.

### 2.2 Lexical Input Elements

The language has the following types of *lexical input elements*:

- *White Spaces*,
- *Line Separators*,
- *Tokens*, and
- *Comments*.

---

<sup>1</sup> Unicode Standard Version 15.0.0, <https://www.unicode.org/versions/Unicode15.0.0/>

## 2.3 White Spaces

*White spaces* are lexical input elements that separate tokens from one another. White spaces include the following:

- Space (U+0020),
- Horizontal tabulation (U+0009),
- Vertical tabulation (U+000B),
- Form feed (U+000C),
- No-break space (U+00A0), and
- Zero-width no-break space (U+FEFF).

White spaces improve source code readability and help avoiding ambiguities. White spaces are ignored by the syntactic grammar (see [Grammar Summary](#)). White spaces never occur within a single token, but can occur within a comment.

## 2.4 Line Separators

*Line separators* are lexical input elements that separate tokens from one another and divide sequences of Unicode input characters into lines. Line separators include the following:

- Newline character (U+000A or ASCII <LF>),
- Carriage return character (U+000D or ASCII <CR>),
- Line separator character (U+2028 or ASCII <LS>), and
- Paragraph separator character (U+2029 or ASCII <PS>).

Line separators improve source code readability. Any sequence of line separators is considered a single separator.

## 2.5 Tokens

Tokens form the vocabulary of the language. There are four classes of tokens:

- *Identifiers*,
- *Keywords*,
- *Operators and Punctuators*, and
- *Literals*.

*Token* is the only lexical input element that can act as a terminal symbol of the syntactic grammar (see *Grammar Summary*). In the process of tokenization, the next token is always the longest sequence of characters that form a valid token. Tokens are separated by white spaces (see *White Spaces*), operators, or punctuators (see *Operators and Punctuators*). White spaces are ignored by the syntactic grammar (see *Grammar Summary*).

Line separators are often treated as white spaces, except where line separators have special meanings. See *Semicolons* for more details.

## 2.6 Identifiers

*Identifier* is a sequence of one or more valid Unicode characters. The Unicode grammar of identifiers is based on character properties specified by the Unicode Standard.

The first character in an identifier must be ‘\$’, ‘\_’, or any Unicode code point with the Unicode property ‘ID\_Start’<sup>2</sup>. Other characters must be Unicode code points with the Unicode property, or one of the following characters:

- ‘\$’ (\U+0024),
- ‘Zero-Width Non-Joiner’ (<ZWJ>, \U+200C), or
- ‘Zero-Width Joiner’ (<ZWJ>, \U+200D).

```
Identifier:
  IdentifierStart IdentifierPart*
;

IdentifierStart:
  UnicodeIDStart
  | '$'
  | '_'
  | '\\ ' EscapeSequence
;

IdentifierPart:
  UnicodeIDContinue
  | '$'
  | ZWJ
  | ZWJ
  | '\\ ' EscapeSequence
;

ZWJ:
  '\u200C'
;

ZWJ:
  '\u200D'
;

UnicodeIDStart
: Letter
| ['$']
| '\\ ' UnicodeEscapeSequence;
```

(continues on next page)

<sup>2</sup> <https://unicode.org/reports/tr31/>

(continued from previous page)

```

UnicodeIDContinue
: UnicodeIDStart
| UnicodeDigit
| '\u200C'
| '\u200D';

UnicodeEscapeSequence:
'u' HexDigit HexDigit HexDigit HexDigit
| 'u' '{' HexDigit HexDigit+ '}'
;

Letter
: UNICODE_CLASS_LU
| UNICODE_CLASS_LL
| UNICODE_CLASS_LT
| UNICODE_CLASS_LM
| UNICODE_CLASS_LO
;

UnicodeDigit
: UNICODE_CLASS_ND
;

```

See *Grammar Summary* for the Unicode character categories `UNICODE_CLASS_LU`, `UNICODE_CLASS_LL`, `UNICODE_CLASS_LT`, `UNICODE_CLASS_LM`, `UNICODE_CLASS_LO`, and `UNICODE_CLASS_ND`.

## 2.7 Keywords

*Keywords* are reserved words with permanently predefined meanings in ArkTS. Keywords are case-sensitive, see the exact spelling in the tables below. Kinds of keywords are discussed below.

1. The following keywords are reserved in any context (*hard keywords*), and cannot be used as identifiers:

|             |            |           |           |
|-------------|------------|-----------|-----------|
|             |            |           |           |
| abstract    | else       | internal  | static    |
| as          | enum       | launch    | switch    |
| async       | export     | let       | super     |
| await       | extends    | native    | this      |
| break       | false      | new       | throw     |
| case        | final      | null      | true      |
| class       | for        | override  | try       |
| const       | function   | package   | undefined |
| constructor | if         | private   | while     |
| continue    | implements | protected |           |
| default     | import     | public    |           |
| do          | interface  | return    |           |

2. Names of primitive built-in types as well as their boxed versions also are *hard keywords*, and cannot be used as identifiers:

|         |        |        |         |        |        |
|---------|--------|--------|---------|--------|--------|
|         |        |        |         |        |        |
| boolean | double | number | Boolean | Double | Number |
| byte    | float  | object | Byte    | Float  | Object |
| bigint  | int    | short  | Bigint  | Int    | Short  |
| char    | long   | string | Char    | Long   | String |
| void    |        |        |         |        |        |

3. The following words have special meaning in certain contexts (*soft keywords*) but are valid identifiers elsewhere:

|         |            |          |
|---------|------------|----------|
|         |            |          |
| catch   | in         | readonly |
| declare | instanceof | set      |
| finally | namespace  | type     |
| from    | of         | typeof   |
| get     | out        |          |

4. The following identifiers are also treated as *soft keywords* reserved for the future use (or used in TypeScript):

|       |    |        |     |       |
|-------|----|--------|-----|-------|
|       |    |        |     |       |
| keyof | is | struct | var | yield |

5. The following words cannot be used as user-defined type names but are not otherwise restricted: see [Reserved Names of TypeScript Types](#).

## 2.8 Operators and Punctuators

*Operators* are tokens that denote various actions to be performed on values: addition, subtraction, comparison, and other. The keywords `instanceof` and `typeof` also act as operators.

*Punctuators* are tokens that separate, complete, or otherwise organize program elements and parts: commas, semicolons, parentheses, square brackets, etc.

The following character sequences represent operators and punctuators:

|   |    |    |      |    |     |     |
|---|----|----|------|----|-----|-----|
|   |    |    |      |    |     |     |
| + | &  | += | =    | &= | <   | ?.  |
| - |    | -= | ^=   | && | >   | !   |
| * | ^  | *= | <<=  |    | === | <=  |
| / | >> | /= | >>=  | ++ | ==  | >=  |
| % | << | %= | >>>= | -- | =   | ... |
| ( | )  | [  | ]    | {  | }   | ??  |
| , | ;  | .  | :    | != | !== |     |

## 2.9 Literals

*Literals* are values of certain types (see *Predefined Types* and *Literal Types*).

```
Literal:
  IntegerLiteral
  | FloatLiteral
  | BigIntLiteral
  | BooleanLiteral
  | StringLiteral
  | MultilineStringLiteral
  | NullLiteral
  | UndefinedLiteral
  | CharLiteral
;
```

See *Character Literals* for the experimental `char` literal.

Each literal is described in detail below.

### 2.9.1 Numeric Literals

Integer and floating-point literals are numeric literals.

### 2.9.2 Integer Literals

Integer literals represent numbers that have neither a decimal point nor an exponential part. Integer literals can be written with radices 16 (hexadecimal), 10 (decimal), 8 (octal), and 2 (binary) as follows:

```
IntegerLiteral:
  DecimalIntegerLiteral
  | HexIntegerLiteral
  | OctalIntegerLiteral
  | BinaryIntegerLiteral
;

DecimalIntegerLiteral:
  '0'
  | DecimalDigitNotNull ('_'? DecimalDigit)*
;

DecimalDigit:
  [0-9]
;

DecimalDigitNotNull:
  [1-9]
```

(continues on next page)



(continued from previous page)

```

;

HexIntegerLiteral:
'0' [xX] ( HexDigit
| HexDigit (HexDigit | '_' ) * HexDigit
)
;

HexDigit:
[0-9a-fA-F]
;

OctalIntegerLiteral:
'0' [oO] ( OctalDigit
| OctalDigit (OctalDigit | '_' ) * OctalDigit )
;

OctalDigit:
[0-7]
;

BinaryIntegerLiteral:
'0' [bB] ( BinaryDigit
| BinaryDigit (BinaryDigit | '_' ) * BinaryDigit )
;

BinaryDigit:
[0-1]
;

```

Integral literals with different radices are represented by the examples below:

```

1 153 // decimal literal
2 1_153 // decimal literal
3 0xBAD3 // hex literal
4 0xBAD_3 // hex literal
5 0o777 // octal literal
6 0b101 // binary literal

```

The underscore character ‘\_’ after the radix prefix or between successive digits can be used to denote an integer literal and improve readability. Underscore characters in such positions do not change the values of literals. However, the underscore character must be neither the very first nor the very last symbol of an integer literal.

Integer literals are of integer types that match literals as follows:

- For *decimal* integer literals
  - `int` if the literal value can be represented by a non-negative 32-bit number, i.e., the value is in the range `0..max(int)`; or
  - `long` otherwise.
- For *hex*, *octal*, and *binary* integer literals
  - `int` if bit representation of the value fits in 32-bits, i.e., the value is in the range `0..max(unsigned 32-bit integer)`; or
  - `long` otherwise.

A compile-time error occurs if an integer literal value is too large for the values of type `long`. The concept is represented by the examples below:

```

1 // literals of type int:
2 1
3 0x7F
4 0x7FFFFFFF // max(int)
5 0x80000000 // min(int)
6
7 // literals of type long:
8 0x7FFF_FFFF_1
9 9223372036854775807 // max(long)
10
11 // compile-time error as value is too large:
12 9223372036854775808 // max(long) + 1
13 0xFFFF_FFFF_FFFF_FFFF_0

```

An integer literal in variable and constant declarations can be implicitly converted to another numeric type (see *Numeric Types*) or type `char` (see *Primitive Types Conversions*). An casting conversion must be used elsewhere (see *Cast Expressions*).

## 2.9.3 Floating-Point Literals

*Floating-point literals* represent decimal numbers and consist of a whole-number part, a decimal point, a fraction part, an exponent, and a `float` type suffix as follows:

```

FloatLiteral:
    DecimalIntegerLiteral '.' FractionalPart? ExponentPart? FloatTypeSuffix?
    | '.' FractionalPart ExponentPart? FloatTypeSuffix?
    | DecimalIntegerLiteral ExponentPart FloatTypeSuffix?
    ;

ExponentPart:
    [eE] [+]? DecimalIntegerLiteral
    ;

FractionalPart:
    DecimalDigit
    | DecimalDigit (DecimalDigit | '_' ) * DecimalDigit
    ;

FloatTypeSuffix:
    'f'
    ;

```

The concept is represented by the examples below:

```

1 3.14
2 3.14f
3 3.141_592
4 .5
5 1e10
6 1e10f

```

The underscore character ‘\_’ after the radix prefix or between successive digits can be used to denote a floating-point literal and improve readability. Underscore characters in such positions do not change the values of literals. However, the underscore character must be neither the very first nor the very last symbol of an integer literal.

Floating-point literals are of floating-point types that match literals as follows:

- `float` if *float type suffix* is present; or
- `double` otherwise (*type number* is an alias to `double`).

A floating-point literal in variable and constant declarations can be implicitly converted to type `float` (see *Assignability with Initializer*).

A compile-time error occurs if a non-zero floating-point literal is too large for its type.

## 2.9.4 Bigint Literals

*Bigint literals* represent integer numbers with unlimited number of digits. *Bigint literals* use decimal radix only.

*Bigint literals* are always of the `bigint` type (see *Type bigint*).

A `bigint` literal is a sequence of digits followed by the symbol ‘n’:

```
BigIntLiteral:
  '0n'
  | [1-9] ('_'? [0-9]) * 'n'
  ;
```

The concept is presented by the examples below:

```
153n // bigint literal
1_153n // bigint literal
-153n // negative bigint literal
```

The underscore character ‘\_’ used between successive digits can be used to denote a `bigint` literal and improve readability. Underscore characters in such positions do not change the values of literals. However, the underscore character must be neither the very first nor the very last symbol of a `bigint` literal.

Strings that represent numbers or any integer value can be converted to `bigint` by using built-in functions:

```
BigInt(other: string): bigint
BigInt(other: long): bigint
```

Two methods allow taking *bitsCount* lower bits of a `bigint` number and return them as a result. Signed and unsigned versions are both possible as seen below:

```
asIntN(bitsCount: long, bigintToCut: bigint): bigint
asUIntN(bitsCount: long, bigintToCut: bigint): bigint
```

## 2.9.5 Boolean Literals

The two *boolean literal* values are represented by the keywords `true` and `false`.

```
BooleanLiteral:
    'true' | 'false'
    ;
```

*Boolean literals* are of the `boolean` type.

## 2.9.6 String Literals

*String literals* consist of zero or more characters enclosed between single or double quotes. A special form of string literals is *multiline string literal* (see [Multiline String Literal](#)).

*String literals* are of the literal type that corresponds to the literal. If an operator is applied to the literal, then the literal type is replaced for `string` (see [Type string](#)).

```
StringLiteral:
    '"' DoubleQuoteCharacter* '"'
    | '\'' SingleQuoteCharacter* '\''
    ;

DoubleQuoteCharacter:
    ~["\\r\n]
    | '\\' EscapeSequence
    ;

SingleQuoteCharacter:
    ~['\\r\n]
    | '\\' EscapeSequence
    ;

EscapeSequence:
    ['"bfnrvt0\\]
    | 'x' HexDigit HexDigit
    | 'u' HexDigit HexDigit HexDigit HexDigit
    | 'u' '{' HexDigit+ '}'
    | ~[1-9xu\r\n]
    ;
```

Characters in *string literals* normally represent themselves. However, certain non-graphic characters can be represented by explicit specifications or Unicode codes. Such constructs are called *escape sequences*.

Escape sequences can represent graphic characters within a *string literal*, e.g., single quotes “'”, double quotes “””, backslashes “\”, and some others. An escape sequence always starts with the backslash character “\”, followed by one of the following characters:

- " (double quote, U+0022),
- ' (neutral single quote, U+0027),
- b (backspace, U+0008),
- f (form feed, U+000c),
- n (linefeed, U+000a),
- r (carriage return, U+000d),

- `␣` (horizontal tab, U+0009),
- `␣` (vertical tab, U+000b),
- `\` (backslash, U+005c),
- `x` and two hexadecimal digits (like `7F`),
- `u` and four hexadecimal digits (forming a fixed Unicode escape sequence like `\u005c`),
- `u{` and at least one hexadecimal digit followed by `}` (forming a bounded Unicode escape sequence like `\u{5c}`), and
- any single character except digits from `'1'` to `'9'`, and characters `'x'`, `'u'`, `'CR'` and `'LF'`.

The examples are provided below:

```

1  let s1 = 'Hello, world!'
2  let s2 = "Hello, world!"
3  let s3 = "\\\"
4  let s4 = ""
5  let s5 = "don't worry, be happy"
6  let s6 = 'don\'t worry, be happy'
7  let s7 = 'don\u0027t worry, be happy'

```

## 2.9.7 Multiline String Literal

*Multiline strings* can contain arbitrary text delimited by backtick characters ````. Multiline strings can contain any character, except the escape character `'\'`. Multiline strings can contain newline characters:

```

MultilineStringLiteral:
    '`' (BacktickCharacter) * '`'
    ;

BacktickCharacter:
    ~['\\r\n]
    | '\\' EscapeSequence
    | LineContinuation
    ;

LineContinuation:
    '\\' [\r\n\u2028\u2029]+
    ;

```

The grammar of *embeddedExpression* is described in *String Interpolation Expressions*.

An example of a multiline string is provided below:

```

1  let sentence = `This is an example of
2                  a multiline string,
3                  which should be enclosed in
4                  backticks`

```

*MultilineString* literals are of the literal type that corresponds to the literal. If an operator is applied to the literal, then the literal type is replaced for *string* (see *Type string*).

## 2.9.8 Null Literal

*Null literal* is the only literal of type `null` (see *Type null*) to denote a reference without pointing at any entity. The null literal is represented by the keyword `null`:

```
NullLiteral:  
    'null'  
    ;
```

The value is typically used for types like `T | null` (see *Nullish Types*).

## 2.9.9 Undefined Literal

*Undefined literal* is the only literal of type `undefined` (see *Type undefined*) to denote a reference with a value that is not defined. The undefined literal is represented by the keyword `undefined`:

```
UndefinedLiteral:  
    'undefined'  
    ;
```

## 2.10 Comments

*Comment* is a piece of text added in the stream to document and compliment the source code. Comments are insignificant for the syntactic grammar (see *Grammar Summary*).

*Line comments* begin with the sequence of characters `'//'` (as seen in the example below) and end with the line separator character. Any character or sequence of characters between them is allowed but ignored.

```
1 // This is a line comment
```

*Multiline comments* begin with the sequence of characters `'/*'` (as seen in the example below) and end with the first subsequent sequence of characters `'*/'`. Any character or sequence of characters between them is allowed but ignored.

```
1 /*  
2    This is a multiline comment  
3 */
```

Comments cannot be nested.

## 2.11 Semicolons

Declarations and statements are usually terminated by a line separator (see [Line Separators](#)). In some cases, a semicolon must be used to separate syntax productions written in one line, or to avoid ambiguity.

```
1  function foo(x: number): number {  
2      x++;  
3      x *= x;  
4      return x  
5  }  
6  
7  let i = 1  
8  i-i++ // one expression  
9  i;-i++ // two expressions
```

---





This chapter introduces the notion of type that is one of the fundamental concepts of ArkTS and other programming languages. Type classification as accepted in ArkTS is discussed below—along with all aspects of using types in programs written in the language.

The type of an entity is conventionally defined as the set of *values* the entity (variable) can take, and the set of *operators* applicable to the entity of a given type.

ArkTS is a statically typed language. It means that the type of every declared entity and every expression is known at compile time. The type of an entity is either set explicitly by a developer, or inferred implicitly (see [Type Inference](#)) by the compiler.

There are two categories of types:

1. *Value Types*, and
2. *Reference Types*.

The types integral to ArkTS are called *predefined types* (see [Predefined Types](#)).

The types introduced, declared, and defined by a developer are called *user-defined types*. All *user-defined types* must always have complete type definitions presented as source code in ArkTS.

## 3.1 Predefined Types

Predefined types include the following:

- Basic numeric value type: `number`
- High-performance value types:
  - Numeric types: `byte`, `short`, `int`, `long`, `float`, and `double`;
  - Character type: `char`;

- Boolean type: `boolean`;
- Reference types:
  - *Type Object*;
  - *Type never*;
  - *Type void*;
  - *Type undefined*;
  - *Type null*;
  - *Type string*;
  - *Type bigint*;
  - *Array Types* (`Array<T>` or `T[]`);
  - *Fixed Array Types*;
  - *Boxed Types*.

### 3.1.1 Primitive Types

The predefined value types are called *primitive types*. Primitive types have no methods. All primitive type operations are discussed in *Value Types*.

Primitive type names are reserved, i.e., they cannot be used for user-defined type names.

Type `double` is an alias to `number`. Type `Double` is an alias to `Number`.

### 3.1.2 Boxed Types

*Boxed type* is a predefined class type that corresponds to and wraps the value of each predefined value type: `Number`, `Byte`, `Short`, `Int`, `Long`, `Float`, `Double`, `Char`, and `Boolean`.

### 3.1.3 Numeric Types

`Integer` (see *Integer Types and Operations*) and floating-point (see *Floating-Point Types and Operations*) types are *numeric types*.

## 3.2 User-Defined Types

User-defined types include the following:

- Class types (see *Classes*);
- Interface types (see *Interfaces*);
- Enumeration types (see *Enumerations*);
- *Function Types*;
- *Tuple Types*;
- *Union Types*;
- *Type Parameters*; and
- *Literal Types*.

## 3.3 Types by Category

All ArkTS types are summarized in the following table:

| Predefined Types   |                                                                                                      | User-Defined Types |                                                                                      |
|--------------------|------------------------------------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------|
| <i>Value Types</i> | <i>Reference Types</i>                                                                               | <i>Value Types</i> | <i>Reference Types</i>                                                               |
| (Primitive Types)  |                                                                                                      |                    |                                                                                      |
| number, byte,      | Number, Byte,                                                                                        | enumeration types  | class types,                                                                         |
| short, int,        | Short, Int,                                                                                          |                    | interface types,                                                                     |
| long, float,       | Long, Float,                                                                                         |                    | array types,                                                                         |
| double, char,      | Double, Char,                                                                                        |                    | fixed array types,                                                                   |
| boolean            | Boolean,<br>String, string,<br>BigInt, bigint,<br>Object, object,<br>never, void,<br>undefined, null |                    | tuple types,<br>union types,<br>literal types,<br>function types,<br>type parameters |

**Note.** Type *string* (see *Type string*), type *bigint* (see *Type bigint*), literal types (see *Literal Types*), and all boxed types (*Boxed Types*) have value type semantics in equality expressions (*Equality Expressions*).

## 3.4 Using Types

A type can be referred to in source code by the following:

- Type reference for:
  - *Named Types*, or

- Type aliases (see *Type Alias Declaration*);
- In-place type definition for:
  - *Array Types*,
  - *Tuple Types*,
  - *Function Types*,
  - *Function Types with Receiver*,
  - *Union Types*, or
  - Type in parentheses.

```
type:
  annotationUsage?
  ( typeReference
  | 'readonly'? arrayType
  | 'readonly'? tupleType
  | functionType
  | functionTypeWithReceiver
  | unionType
  | StringLiteral
  )
  | '(' type ') '
;
```

The usage of annotations is discussed in *Using Annotations*.

The usage of types is presented by the example below:

```
1 let n: number // using identifier as a primitive value type name
2 let o: Object // using identifier as a predefined class type name
3 let a: number[] // using array type
4 let t: [number, number] // using tuple type
5 let f: () => number // using function type
6 let u: number|string // using union type
7 let l: "xyz" // using string literal type
```

Parentheses in types (where a type is a combination of array, function, or union types) are used to specify the required type structure. Without parentheses, the symbol ‘|’ that constructs a union type has the lowest precedence as presented in the following example:

```
1 // a nullable array with elements of type string:
2 let a: string[] | null
3 let s: string[] = []
4 a = s // ok
5 a = null // ok, a is nullable
6
7 // an array with elements whose types are string or null:
8 let b: (string | null)[]
9 b = null // error, b is an array and is not nullable
10 b = ["aa", null] // ok
11
12 // a function type that returns string or null
13 let c: () => string | null
14 c = null // error, c is not nullable
15 c = (): string | null => { return null } // ok
16
```

(continues on next page)

(continued from previous page)

```

17 // (a function type that returns string) or null
18 let d: (() => string) | null
19 d = null // ok, d is nullable
20 d = (): string => { return "hi" } // ok

```

If annotation is used in front of type in parentheses, then parentheses become a mandatory part of the annotation to prevent ambiguity.

```

1 let var_name1: @my_annotation() (A|B) // OK
2 let var_name2: @my_annotation (A|B) // Compile-time error

```

## 3.5 Named Types

Classes, interfaces, enumerations, aliases, type parameters, and predefined types (see *Predefined Types*), except built-in arrays, are named types. Other types (i.e., array, function, and union types) are anonymous unless aliased. Respective named types are introduced by the following:

- Class declarations (see *Classes*),
- Interface declarations (see *Interfaces*),
- Enumeration declarations (see *Enumerations*),
- Type alias declarations (see *Type Alias Declaration*), and
- Type parameter declarations (see *Type Parameters*).

Classes, interfaces and type aliases with type parameters are *generic types* (see *Generics*). Named types without type parameters are *non-generic types*.

*Type references* (see *Type References*) refer to named types by specifying their type names and (where applicable) type arguments to be substituted for the type parameters of a named type.

## 3.6 Type References

A type reference refers to a type by one of the following:

- *Simple* or *qualified* type name (see *Names*),
- Type alias (see *Type Alias Declaration*), or
- Type parameter (see *Type Parameters*) name with the ‘!’ sign (see *Nonnullish Type Parameter*).

A type name denoted by `identifier` is a valid type reference if it is a valid instantiation of a generic when referring to a generic class or an interface type. A type reference is valid if its type arguments (see *Type Arguments*) are provided explicitly or implicitly based on defaults.

```

typeReference:
    typeReferencePart ('.' typeReferencePart)*
    | identifier '!'
    ;

typeReferencePart:
    identifier typeArguments?
    ;

```

```

1  let map: Map<string, number> // Map<string, number> is the type reference
2
3  class A<T> {
4      field1: A<T> // A<T> is a type reference - class type reference
5      field2: A<number> // A<number> is a type reference - class type reference
6      foo (p: T) {} // T is a type reference - type parameter
7      constructor () { /* some body to init fields */ }
8  }
9
10 type MyType<T> = []A<T>
11 let x: MyType<number> = [new A<number>, new A<number>]
12 // MyType<number> is a type reference - alias reference
13 // A<number> is a type reference - class type reference

```

If a type reference refers to the type by a type alias (see *Type Alias Declaration*), then the type alias is replaced (potentially recursively) for a non-aliased type in all cases when dealing with types in this document.

```

1  type T1 = Object
2  type T2 = number
3  function foo(t1: T1, t2: T2) {
4      t1 = t2 // Type compatibility test will use Object and number
5      t2 = t2 + t2 // Operator validity test will use type number not T2
6  }

```

## 3.7 Value Types

Predefined integer types (see *Integer Types and Operations*), floating-point types (see *Floating-Point Types and Operations*), the boolean type (see *Boolean Types and Operations*), character types (see *Character Type and Operations*), and user-defined enumeration types (see *Enumerations*) are *value types*. The values of such types do *not* share state with other values.

### 3.7.1 Integer Types and Operations

| Type   | Corresponding Set of Values                              | Corresponding Class Type |
|--------|----------------------------------------------------------|--------------------------|
| byte   | All signed 8-bit integers ( $-2^7$ to $2^7 - 1$ )        | Byte                     |
| short  | All signed 16-bit integers ( $-2^{15}$ to $2^{15} - 1$ ) | Short                    |
| int    | All signed 32-bit integers ( $-2^{31}$ to $2^{31} - 1$ ) | Int                      |
| long   | All signed 64-bit integers ( $-2^{63}$ to $2^{63} - 1$ ) | Long                     |
| bigint | All integers with no limits                              | BigInt                   |

ArkTS provides a number of operators to act on integer values as discussed below.

- Comparison operators that produce a value of type `boolean`:
  - Numerical relational operators '`<`', '`<=`', '`>`', and '`>=`' (see *Numerical Relational Operators*);
  - Numerical equality operators '`==`' and '`!=`' (see *Numerical Equality Operators*);
- Numerical operators that produce values of types `int`, `long`, or `bigint`:
  - Unary plus '`+`' and minus '`-`' operators (see *Unary Plus* and *Unary Minus*);
  - Multiplicative operators '`*`', '`/`', and '`%`' (see *Multiplicative Expressions*);
  - Additive operators '`+`' and '`-`' (see *Additive Expressions*);
  - Increment operator '`++`' used as prefix (see *Prefix Increment*) or postfix (see *Postfix Increment*);
  - Decrement operator '`--`' used as prefix (see *Prefix Decrement*) or postfix (see *Postfix Decrement*);
  - Signed and unsigned shift operators '`<<`', '`>>`', and '`>>>`' (see *Shift Expressions*);
  - Bitwise complement operator '`~`' (see *Bitwise Complement*);
  - Integer bitwise operators '`&`', '`^`', and '`|`' (see *Integer Bitwise Operators*);
- Conditional operator '`? :`' (see *Conditional Expressions*);
- Cast operator (see *Cast Expressions*) that converts an integer value to a value of any specified numeric type (see *Numeric Types*);
- String concatenation operator '`+`' (see *String Concatenation*) that, if one operand is `string` and the other is of an integer type, converts the integer operand to `string` with the decimal form, and then creates a concatenation of the two strings as a new `string`.

The classes `Byte`, `Short`, `Int`, and `Long` predefine constructors, methods, and constants that are parts of the ArkTS standard library (see *Standard Library*).

If one operand is not of type `long`, then the numeric promotion (see *Primitive Types Conversions*) must be used to widen it first to type `long`.

If neither operand is of type `long`, then:

- The operation implementation uses 32-bit precision.
- The result of the numerical operator is of type `int`.

If one operand (or neither operand) is of type `int`, then the numeric promotion must be used to widen it first to type `int`.

Any integer type value can be converted to or from any numeric type (see *Numeric Types*).

Conversions between integer types and type `boolean` are not allowed.

The integer operators cannot indicate an overflow or an underflow.

An integer operator can throw errors (see [Error Handling](#)) as follows:

- An integer division operator `'/'` (see [Division](#)), and an integer remainder operator `'%'` (see [Remainder](#)) throw `ArithmeticError` if their right-hand-side operand is zero.
- An increment operator `'++'` and a decrement operator `'--'` (see [Additive Expressions](#)) throw `OutOfMemoryError` if boxing conversion (see [Boxing Conversions](#)) is required but the available memory is not sufficient to perform it.

### 3.7.2 Floating-Point Types and Operations

| Type                        | Corresponding Set of Values                                        | Corresponding Class Type   |
|-----------------------------|--------------------------------------------------------------------|----------------------------|
| <code>float</code>          | The set of all IEEE 754 <sup>3</sup> 32-bit floating-point numbers | <code>Float</code>         |
| <code>number, double</code> | The set of all IEEE 754 64-bit floating-point numbers              | <code>Number Double</code> |

ArkTS provides a number of operators to act on floating-point type values as discussed below.

- Comparison operators that produce a value of type `boolean`:
  - Numerical relational operators `'<'`, `'<='`, `'>'`, and `'>='` (see [Numerical Relational Operators](#));
  - Numerical equality operators `'=='` and `'!=='` (see [Numerical Equality Operators](#));
- Numerical operators that produce values of type `float` or `double`:
  - Unary plus `'+'` and minus `'-'` operators (see [Unary Plus](#) and [Unary Minus](#));
  - Multiplicative operators `'*'`, `'/'`, and `'%'` (see [Multiplicative Expressions](#));
  - Additive operators `'+'` and `'-'` (see [Additive Expressions](#));
  - Increment operator `'++'` used as prefix (see [Prefix Increment](#)) or postfix (see [Postfix Increment](#));
  - Decrement operator `'--'` used as prefix (see [Prefix Decrement](#)) or postfix (see [Postfix Decrement](#));
- Numerical operators that produce values of type `int` or `long`:
  - Signed and unsigned shift operators `'<<'`, `'>>'`, and `'>>>'` (see [Shift Expressions](#));
  - Bitwise complement operator `'~'` (see [Bitwise Complement](#));
  - Integer bitwise operators `'&'`, `'^'`, and `'|'` (see [Integer Bitwise Operators](#));
- Conditional operator `'?:'` (see [Conditional Expressions](#));
- Cast operator (see [Cast Expressions](#)) that converts a floating-point value to a value of any specified numeric type (see [Numeric Types](#));
- The string concatenation operator `'+'` (see [String Concatenation](#)) that, if one operand is of type `string` and the other is of a floating-point type, converts the floating-point type operand to type `string` with a value represented in the decimal form (without loss of information), and then creates a concatenation of the two strings as a new `string`.

The classes `Float` and `Double` predefine constructors, methods, and constants that are parts of the ArkTS standard library (see [Standard Library](#)).

<sup>3</sup> Any mention of IEEE 754 in this Specification refers to the latest revision of “754-2019–IEEE Standard for Floating-Point Arithmetic”.



An operation is called a *floating-point operation* if at least one of the operands in a binary operator is of a floating-point type (even if the other operand is integer).

If at least one operand of the numerical operator is of type `double`, then the operation implementation uses the 64-bit floating-point arithmetic. The result of the numerical operator is a value of type `double`.

If the other operand is not of type `double`, then the numeric promotion (see [Primitive Types Conversions](#)) must be used to widen it first to type `double`.

If neither operand is of type `double`, then the operation implementation is to use the 32-bit floating-point arithmetic. The result of the numerical operator is a value of type `float`.

If the other operand is not of type `float`, then the numeric promotion must be used to widen it first to type `float`.

Any floating-point type value can be cast to or from any numeric type (see [Numeric Types](#)).

Conversions between floating-point types and type `boolean` are not allowed.

Operators on floating-point numbers, except the remainder operator (see [Remainder](#)), behave in compliance with the IEEE 754 Standard. For example, ArkTS requires the support of IEEE 754 *denormalized* floating-point numbers and *gradual underflow* which facilitate proving the desirable properties of a particular numerical algorithm. Floating-point operations do not *flush to zero* if the calculated result is a denormalized number.

ArkTS requires the floating-point arithmetic to behave as if the floating-point result of every floating-point operator is rounded to the result precision. An *inexact* result is rounded to a representable value nearest to the infinitely precise result. ArkTS uses the *round to nearest* principle (the default rounding mode in IEEE 754), and prefers the representable value with the least significant bit zero out of any two equally near representable values.

ArkTS uses *round toward zero* to convert a floating-point value to an integer value (see [Primitive Types Conversions](#)). In this case it acts as if the number is truncated, and the mantissa bits are discarded. The result of *rounding toward zero* is the value of that format that is closest to and no greater in magnitude than the infinitely precise result.

A floating-point operation with overflow produces a signed infinity.

A floating-point operation with underflow produces a denormalized value or a signed zero.

A floating-point operation with no mathematically definite result produces NaN.

All numeric operations with a NaN operand result in NaN.

A floating-point operator (the increment ‘++’ operator and decrement ‘--’ operator, see [Additive Expressions](#)) can throw `OutOfMemoryError` (see [Error Handling](#)) if boxing conversion (see [Boxing Conversions](#)) is required but the available memory is not sufficient to perform it.

### 3.7.3 Numeric Types Hierarchy

Integer and floating-point types are numeric types.

Larger type values include all values of smaller types:

- `double > float > long > int > short > byte`

Consequently, a value of a smaller type can be assigned to a variable of a larger type.

Type `bigint` does not belong to this hierarchy. There is no implicit conversion from a numeric type (see [Numeric Types](#)) to `bigint`. Standard library (see [Standard Library](#)) class `BigInt` methods must be used to create `bigint` values from numeric types.

### 3.7.4 Boolean Types and Operations

Type `boolean` represents logical values `true` and `false` that correspond to the class type `Boolean`.

The boolean operators are as follows:

- Relational operators `'=='` and `'!='` (see *Relational Expressions*);
- Logical complement operator `'!'` (see *Logical Complement*);
- Logical operators `'&'`, `'^'`, and `'|'` (see *Integer Bitwise Operators*);
- Conditional-and operator `'&&'` (see *Conditional-And Expression*) and conditional-or operator `'||'` (see *Conditional-Or Expression*);
- Conditional operator `'?:'` (see *Conditional Expressions*);
- String concatenation operator `'+'` (see *String Concatenation*) that converts an operand of type `boolean` to type `string` (`true` or `false`), and then creates a concatenation of the two strings as a new `string`.

The conversion of an integer or floating-point expression  $x$  to a boolean value must follow the *C* language convention: any nonzero value is converted to `true`, and the value of zero is converted to `false`. In other words, the result of expression  $x$  conversion to type `boolean` is always the same as the result of comparison  $x \neq 0$ .

## 3.8 Reference Types

*Reference types* can be of the following kinds:

- *Class types* (see *Classes*);
- *Interface types* (see *Interfaces*);
- *Array Types*;
- *Fixed Array Types*;
- *Tuple Types*;
- *Function Types*;
- *Union Types*;
- *Literal Types*;
- *Type string*;
- *Type bigint*;
- *Type never*;
- *Type null*;
- *Type undefined*;
- *Type void*; and
- *Type Parameters*.

## 3.9 Type Object

Type `Object` is the superclass of all other classes, interfaces, arrays, tuples, function types, string and bigint types, unions except nullish ones, type parameters, and enum types. All these types inherit the methods of class `Object` (see [Inheritance](#)). All methods of class `Object` are described in full in [Standard Library](#).

The method `toString` as used in the examples in this document returns a string representation of the object.

Using `Object` is recommended in all cases, although the name `object` refers to type `Object`.

The term *object* is used in the Specification to refer to an instance of any type that is a subtype of `Object` or of type `Object` itself.

Pointers to these objects are called *references*. Multiple references to an object are possible.

Most objects have state. The state is stored in the field if an object is a class instance, or in a variable that is an element of an array object.

If two variables contain references to the same object, and the state of that object is modified in the reference of either variable, then the state so modified can be seen in the reference of the other variable.

## 3.10 Type never

Type `never` is assignable to any other type (see [Assignability](#)).

Type `never` has no instance. Type `never` is used as one of the following:

- Return type for functions or methods that never return a value, but throw an error when completing an operation.
- Type of variables that can never be assigned.
- Type of parameters of a function or a method to prevent the body of that function or method from being executed.

```

1  function foo (): never {
2      throw new Error("foo() never returns")
3  }
4
5  let x: never = foo() // x will never get a value
6
7  function bar (p: never) { // body of this
8      // function will never be executed
9  }
10
11 bar (foo())

```

### 3.11 Type void

Type `void` has no instance and no value. It is typically used as the return type if a function or a method returns no value:

```

1  function foo (): void {}
2
3  class C {
4      bar(): void {}
5  }
6
7  type FunctionWithNoParametersType = () => void
8
9  let funcTypeVariable: FunctionWithNoParametersType = (): void => {}

```

A compile-time error occurs if:

- Type `void` is used as type annotation;
- Expression of type `void` is used as a value.

```

1  let x: void // compile-time error - void used as type annotation
2
3  function foo (): void {}
4  let y = foo() // compile-time error - void used as a value
5
6  type ErroneousType = void | number
7      // compile-time error - void used as type annotation

```

Type `void` can be used as type argument that instantiates a generic type if a specific value of type argument is irrelevant. In this case, it is a synonym for type `undefined` (see [Type undefined](#)):

```

1  class A<T> {}
2  let a = new A<void>() // ok, type parameter is irrelevant
3  let a = new A<undefined>() // ok, the same
4
5  function foo<T>(x: T) {}
6
7  foo<void>(undefined) // ok
8  foo<void>(void) // compile-time error: void is used as value

```

### 3.12 Type undefined

The only value of type `undefined` is the keyword `undefined` (see [Undefined Literal](#)).

Using type `undefined` as type annotation is not recommended, except in nullish types (see [Nullish Types](#)).

Type `undefined` can be used as type argument to instantiate a generic type if the specific value of type argument is irrelevant:

```

1  class A<T> {}
2  let a = new A<undefined>() // ok, type parameter is irrelevant

```

(continues on next page)

(continued from previous page)

```

3 function foo<T>(x: T) {}
4
5 foo<undefined>(undefined) // ok

```

### 3.13 Type null

The only value of type `null` is the keyword `null` (see *Null Literal*).

Using type `null` as type annotation is not recommended, except in nullish types (see *Nullish Types*).

### 3.14 Type string

Type `string` stores sequences of characters as Unicode UTF-16 code units. Type `string` includes all string literals, e.g., `'abc'`.

A `string` object is immutable, for the value of a `string` object cannot be changed after the object is created. The value of a `string` object can be shared.

Type `string` has dual semantics:

- Type `string` behaves like a reference type (see *Reference Types*) if it is created, assigned, or passed as an argument.
- Type `string` is handled as a value (see *Value Types*) by all `string` operations (see *String Concatenation*, *String Equality Operators*, and *String Relational Operators*).

If the result is not a constant expression (see *Constant Expressions*), then the string concatenation operator `+` (see *String Concatenation*) can implicitly create a new `string` object.

Using `string` is recommended in all cases, although the name `String` also refers to type `string`.

### 3.15 Type bigint

ArkTS has the built-in `bigint` type. Type `bigint` allows handling theoretical arbitrary large integers. Values of type `bigint` can hold numbers which are larger than the maximum value of type `long`. Type `bigint` uses the arbitrary-precision arithmetic. Values of type `bigint` can be created from the following:

- *Bigint literals* (see *Bigint Literals*); or
- Numeric type values, by using a call to the standard library class `BigInt` methods or constructors (see *Standard Library*).

Similarly to `string`, `bigint` type has dual semantics:

- If created, assigned, or passed as an argument, type `bigint` behaves in the same manner as a reference type (see [Reference Types](#)).
- All applicable operations handle type `bigint` as a value type (see [Value Types](#)). Operations are described in [Integer Types and Operations](#).

Using `bigint` is recommended in all cases, although the name `BigInt` also refers to type `bigint`. Using `BigInt` creates new objects and calls to static methods in order to improve TypeScript compatibility.

```
1 let b1: bigint = new BigInt(5) // for Typescript compatibility
2 let b2: bigint = 123n
```

## 3.16 Literal Types

*Literal types* are aligned with some ArkTS literals (see [Literals](#)). Their names are the same as the names of their values, i.e., literals. Only three literal types are supported.

```
1 let a: "string literal" = "string literal"
2 let b: null = null
3 let c: undefined = undefined
4
5 printThem (a, b, c)
6 function printThem (p1: "string literal", p2: null, p3: undefined) {
7     console.log (p1, p2, p3)
8 }
```

### 3.16.1 Supertypes of Literal Types

The supertype for string literals (see [String Literals](#)) is type `string`. This affects overriding as shown in the example below:

```
1 class Base {
2     foo(p: "1"): string { return "666" }
3 }
4 class Derived extends Base {
5     override foo(p: string): "1" { return "1" }
6 }
7 // Type "1" <: string
8
9 let base: Base = new Derived
10 let result: string = base.foo("1")
11 /* Argument "1" (value) is compatible to type "1" and to type string in
12    the overridden method
13    Function result of type string accepts "1" (value) of literal type "1"
14 */
```

`Null` and `undefined` literals (see [Null Literal](#) and [Undefined Literal](#)) have no supertype:

```

1 let o: Object = new Object
2 o = null      // compile-time error
3 o = undefined // compile-time error

```

### 3.16.2 Operations on Literal Types

Operations on variables of literal types are identical to the operations of their supertypes. The resulting operation type is the type specified for the operation in the supertype. In most cases, it is the supertype itself:

```

1 let s0: "string literal" = "string literal"
2 let s1: string = s0 + s0    // + for string returns string

```

## 3.17 Array Types

ArkTS supports the following two predefined array types:

- *Resizable Array Types*; and
- *Fixed Array Types* as an experimental feature.

*Resizable array types* are recommended for most cases. *Fixed array types* can be used where performance is the major requirement.

*Resizable arrays* differ from *fixed arrays* as follows:

- Length of *fixed array* is set once. It can lead to better performance.
- *Fixed arrays* have no methods defined.

**Note.** The term *array type* as used in this document applies to both *resizable array type* and *fixed array type*. The same holds true for *array value* and *array instance*. *Resizable arrays* and *fixed arrays* are not assignable to each other.

### 3.17.1 Resizable Array Types

There are two syntax forms of *resizable array type* with elements of type T as follows:

- T[]
- Array<T>

The first form uses the following rule:

```

arrayType:
  type '[' ' ' ]
  ;

```

**Note.** `T[]` and `Array<T>` specify identical (indistinguishable) types (see [Type Identity](#)).

*Resizable array type* is the built-in type characterized by the following:

- Any object of resizable array type contains elements. The number of elements is known as *array length*.
- Array length is a non-negative integer number.
- Array length can be set and changed at runtime.
- Array element is accessed by its index. The index is an integer number in the range from 0 to *array length minus 1*.
- Accessing an element by its index is a constant-time operation.
- If passed to non-ArkTS environment, an array is represented as a contiguous memory location.
- Type of each array element is assignable to the element type specified in the array declaration (see [Assignability](#)).

Two basic operations with array elements take elements out of, and put elements into an array by using the operator `[]` and index expression.

The same syntax can be used to work with [Indexable Types](#), some of such types are parts of [Standard Library](#).

The number of elements in an array can be obtained by accessing the property `length`.

The length of an array can be set and changed in runtime using methods defined in the standard library (see [Standard Library](#)).

An array can be created by using [Array Literal](#), [Array Creation Expressions](#), or the constructors defined in the standard library (see [Standard Library](#)).

ArkTS allows setting a new value to `length` to shrink an array and provide better TypeScript compatibility. The new value must be less or equal to the previous length. Attempting to increase the length of the array by assignment to `length` causes a compile-time error (if the compiler has the information sufficient to determine this) or a runtime error.

The examples are presented below:

```
1  let a : number[] = [0, 0, 0, 0, 0]
2      /* allocate array with 5 elements of type number */
3  a[1] = 7 /* put 7 as the 2nd element of the array, index of this element is 1 */
4  let y = a[4] /* get the last element of array 'a' */
5  let count = a.length // get the number of array elements
6  a.length = 3 // shrink array
7  y = a[2] // OK, 2 is the index of the last element now
8  y = a[3] // Will lead to runtime error - attempt to access non-existing array element
9
10 let b: Array<number> = a // 'b' points to the same array as 'a'
```

A type alias can set a name for an array type (see [Type Alias Declaration](#)):

```
1  type Matrix = number[][] /* Two-dimensional array */
```

An array as an object is assignable to a variable of type `Object`:

```
1  let a: number[] = [1, 2, 3]
2  let o: Object = a
```



## 3.18 Tuple Types

```
tupleType:
  '[' (type (',' type)* ',' '?' )? ']'
  ;
```

*Tuple type* is a reference type created as a fixed set of other types. The value of a tuple type is a group of values of types that comprise the tuple type. Types are specified in the order as declared within the tuple type declaration. It implies that each element of a tuple has its own type. The operator '[' (square brackets) is used to access the elements of a tuple in a manner similar to how the elements of an array are accessed.

An index expression belongs to an integer type. The index of the first tuple element is 0. Only constant expressions can be used as the index providing access to tuple elements.

```
1 let tuple: [number, number, string, boolean, Object] =
2   [      6,      7,  "abc",    true,    666]
3 tuple[0] = 666
4 console.log (tuple[0], tuple[4]) // `666 666` be printed
```

Any tuple type is assignable (see *Assignability*) to class Object (see *Type Object*).

An empty tuple is a corner case. It is only added to support TypeScript compatibility:

```
1 let empty: [] = [] // empty tuple with no elements in it
```

## 3.19 Function Types

*Function type* can be used to express the expected signature of a function. A function type consists of the following:

- List of parameters (which can be empty);
- Optional return type.

```
functionType:
  '(' ftParameterList? ')' ftReturnType
  ;

ftParameterList:
  ftParameter (',' ftParameter)* (',' ftRestParameter)?
  | ftRestParameter
  ;

ftParameter:
  identifier ('?')? ':' type
  ;

ftRestParameter:
  '...' ftParameter
  ;

ftReturnType:
```

(continues on next page)

(continued from previous page)

```
'=>' type
;
```

The rest parameter is described in *Rest Parameter*.

```
1 let binaryOp: (x: number, y: number) => number
2 function evaluate(f: (x: number, y: number) => number) { }
```

A type alias can set a name for a *function type* (see *Type Alias Declaration*):

```
1 type BinaryOp = (x: number, y: number) => number
2 let op: BinaryOp
```

If a function type has the ‘?’ mark for a parameter name, then this parameter and all parameters that follow (if any) are optional. Otherwise, a compile-time error occurs. The actual type of the parameter is then a union of the parameter type and type undefined. This parameter has no default value.

```
1 type FuncTypeWithOptionalParameters = (x?: number, y?: string) => void
2 let foo: FuncTypeWithOptionalParameters
3   = ():void => {} // CTE as call with more than zero arguments is invalid
4 foo = (p: number):void => {} // CTE as call with zero arguments is invalid
5 foo = (p?: number):void => {} // CTE as call with two arguments is invalid
6 foo = (p1: number, p2?: string):void => {} // CTE as call with zero arguments is_
↪invalid
7 foo = (p1?: number, p2?: string):void => {} // OK
8
9 foo()
10 foo(undefined)
11 foo(undefined, undefined)
12 foo(666)
13 foo(666, undefined)
14 foo(666, "a string")
15
16 type IncorrectFuncTypeWithOptionalParameters = (x?: number, y: string) => void
17   // compile-time error; no mandatory parameter can follow an optional parameter
18
19 function bar (
20   p1?: number,
21   p2: number|undefined
22 ) {
23   p1 = p2 // OK
24   p2 = p1 // OK
25   // Types of p1 and p2 are identical
26 }
```

All function types are subtypes of *Object* (see *Type Object*). More details on function types assignability are provided in *Assignment-like Contexts*, and conversions in *Function Types Conversions*.

## 3.20 Union Types

```
unionType:
  type ('|' type) *
;
```

*Union* type is a reference type created as a combination of other types. The values of a *union* type are valid values of all types the union is created from.

A compile-time error occurs if the type in the right-hand side of a union type declaration leads to a circular reference.

If a *union* contains an enumeration type (see *Enumerations*), then every such type is replaced for a union of enumeration constant values.

If a *union* contains a primitive type (see *Primitive Types*), then every such type is replaced for its boxed version (see *Boxed Types*) to keep the reference nature of the *union* type.

If a *union* type contains more than one numeric type (see *Numeric Types*) or numeric literal (see *Numeric Literals*), then a compile-time error occurs.

Examples of incorrect union types are represented below:

```
1 type BadUnion1 = int | double // Compile-time error
2 type BadUnion2 = Int | Double // Compile-time error
3 type BadUnion3 = int | Double // Compile-time error
4 let x = cond? new Int (): new Double /* Compile-time error as conditional
5     expression contains an invalid union type Int | Double */
6
7 type BadUnion4 = 1 | 2 // Compile-time error: only types allowed
```

Typical usage examples of *union* type are represented below:

```
1 type OperationResult = "Done" | "Not done"
2 function do_action(): OperationResult {
3     if (someCondition) {
4         return "Done"
5     } else {
6         return "Not done"
7     }
8 }
9
10 class Cat {
11     // ...
12 }
13 class Dog {
14     // ...
15 }
16 class Frog {
17     // ...
18 }
19 type Animal = Cat | Dog | Frog | number
20 // Cat, Dog, and Frog are some types (class or interface ones)
21
22 let animal: Animal = new Cat()
23 animal = new Frog()
24 animal = 42
25 // One may assign the variable of the union type with any valid value
26
```

(continues on next page)

(continued from previous page)

```

27 enum NumberEnum {One, Two}
28 enum StringEnum {One = "One", Two = "Two"}
29
30 type Union1 = number | NumberEnum // compile-time error more than one numeric
31 type Union2 = string | StringEnum // OK, will be reduced during normalization

```

Different mechanisms can be used to get values of particular types from a *union*:

```

1 class Cat { sleep () {} ; meow () {} }
2 class Dog { sleep () {} ; bark () {} }
3 class Frog { sleep () {} ; leap () {} }
4
5 type Animal = Cat | Dog | Frog
6
7 let animal: Animal = new Cat()
8 if (animal instanceof Frog) {
9     // animal is of type Frog here, conversion can be used:
10     let frog: Frog = animal as Frog
11     frog.leap()
12 }
13
14 animal.sleep () // Any animal can sleep

```

The following example represents primitive types:

```

1 type Primitive = number | boolean
2 let p: Primitive = 7
3 if (p instanceof Number) { // type of 'p' is Number here
4     let i: number = p as number // Casting conversion from Primitive to number
5 }

```

The following example represents literal types:

```

1 type BMW_ModelCode = "325" | "530" | "735"
2 let car_code: BMW_ModelCode = "325"
3 if (car_code == "325"){
4     car_code = "530"
5 } else if (car_code == "530"){
6     car_code = "735"
7 } else {
8     // pension :-)
9 }

```

**Note.** A compile-time error occurs if an expression of a *union* type is compared to a literal value or constant that does not belong to the values of the *union* type:

```

1 type BMW_ModelCode = "325" | "530" | "735"
2 let car_code: BMW_ModelCode = "325"
3 if (car_code == "666"){ ... }
4 /*
5     compile-time error as "666" does not belong to
6     values of literal type BMW_ModelCode
7 */
8
9 function model_code_test (code: string) {
10     if (car_code == code) { ... }

```

(continues on next page)

(continued from previous page)

```

11 // This test is to be resolved during program execution
12 }

```

### 3.20.1 Union Types Normalization

Union types normalization allows minimizing the number of types within a union type, while keeping type safety. Some types can also be replaced for more general types.

Formally, union type  $T_1 \mid \dots \mid T_N$ , where  $N > 1$ , can be reduced to type  $U_1 \mid \dots \mid U_M$ , where  $M \leq N$ , or even to a non-union type  $V$ . In this latter case  $V$  can be a primitive value type, or a literal type that changes the reference nature of the union type.

The normalization process presumes that the following steps are performed one after another:

1. All nested union types are linearized.
2. All type aliases (if any and except recursive ones) are recursively replaced for non-alias types.
3. Identical types within the union type are replaced for a single type with account to the `readonly` type flag priority.
4. If at least one type in the union is `Object`, then all other non-nullish types are removed.
5. If present among union types, type `never` is removed.
6. If one type in the union is `string`, then all string literal types (if any) are removed.
7. If a primitive type equals another union type after boxing (see [Boxing Conversions](#)), then the initial type is removed.
8. The following procedure is performed recursively until no assignable types remain, or the union type is reduced to a single type:
  - If a union type includes two types  $T_i$  and  $T_j$  ( $i \neq j$ ), and  $T_i$  is assignable to  $T_j$  (see [Assignability](#)), then only  $T_j$  remains in the union type, and  $T_i$  is removed.

The normalization process results in a normalized union type. The process is presented in the examples below:

```

1 ( T1 | T2 ) | ( T3 | T4 ) // normalized as T1 | T2 | T3 | T4. Linearization
2
3 type A = A[] | string // No changes. Recursive type alias is kept
4
5 type B = number
6 type C = string
7 type D = B | C // normalized as number | string. Type aliases are unfolded
8
9 number | number // normalized as number. Identical types elimination
10
11 (number[]) | (readonly number[]) // normalized as readonly number[]. Readonly_
↳ version wins
12
13 "1" | string | number // normalized as string | number. Literal type value belongs_
↳ to another type values
14
15 "1" | Object // normalized as Object. Object always wins

```

(continues on next page)

(continued from previous page)

```

16 AnyNonNullishType | Object // normalized as Object
17
18 enum Strings1 {aa = "AA", bb = "BB"}
19 enum Strings2 {aa = "AA", cc = "CC"}
20 Strings1 | Strings2 | string // normalized as string. string wins over string_
    ↪ literals
21 Strings1 | Strings2 | number // normalized as "AA" | "BB" | "CC" | number
22                               // string enumerations unfolded and merged
23
24 class Base {}
25 class Derived1 extends Base {}
26 class Derived2 extends Base {}
27 Base | Derived1 // normalized as Base. Base wins over Derived.
28 Derived1 | Derived2 // normalized as Derived1 | Derived2. End of normalization

```

The ArkTS compiler applies normalization while processing union types and handling the type inference for array literals (see *Array Type Inference from Types of Elements*).

### 3.20.2 Access to Common Union Members

Where  $u$  is a variable of union type  $T_1 | \dots | T_N$ , ArkTS supports access to a common member of  $u.m$  if the following conditions are fulfilled:

- Each  $T_i$  is an interface or class type;
- Each  $T_i$  has a member with the name  $m$ ; and
- For any  $T_i$ ,  $m$  is one of the following:
  - Method or accessor with an equal signature; or
  - Same-type field.

A compile-time error occurs otherwise:

```

1  class A {
2      n = 1
3      s = "aa"
4      foo() {}
5      goo(n: number) {}
6  }
7  class B {
8      n = 2
9      s = 3.14
10     foo() {}
11     goo() {}
12 }
13
14 let u: A | B = new A
15
16 let x = u.n // ok, common field
17 u.foo() // ok, common method
18

```

(continues on next page)

(continued from previous page)

```

19 console.log(u.s) // compile-time error as field types differ
20 u.goo() // compile-time error as signatures differ

```

## 3.21 Nullish Types

ArkTS has *nullish types* that are in fact a special form of union types (see [Union Types](#)).

`T | null` or `T | undefined` or `T | undefined | null` can be used as the type to specify a nullish version of type `T`.

All predefined and user-defined type declarations create non-nullish types. Non-nullish types cannot have a `null` or `undefined` value at runtime.

A variable declared to have type `T | null` can hold the values of type `T` and its derived types, or the value `null`. Such a type is called a *nullable type*.

A variable declared to have type `T | undefined` can hold the values of type `T` and its derived types, or the value `undefined`.

A variable declared to have type `T | null | undefined` can hold values of type `T` and its derived types, and the values `undefined` or `null`.

*Nullish type* is a reference type (see [Union Types](#)). A reference that is `null` or `undefined` is called a *nullish value*.

An operation that is safe with no regard to the presence or absence of *nullish values* (e.g., re-assigning one nullable value to another) can be used 'as is' for *nullish types*.

The following nullish-safe options exist for dealing with nullish type `T`:

- Using of safe operations:
  - Safe method call (see [Method Call Expression](#) for details);
  - Safe field access expression (see [Field Access Expression](#) for details);
  - Safe indexing expression (see [Indexing Expressions](#) for details);
  - Safe function call (see [Function Call Expression](#) for details);
- Conversion from `T | null` or `T | undefined` to `T`:
  - Cast expression (see [Cast Expressions](#) for details);
  - Ensure-not-nullish expression (see [Ensure-Not-Nullish Expression](#) for details);
- Supplying a value to be used if a *nullish value* is present:
  - Nullish-coalescing expression (see [Nullish-Coalescing Expression](#) for details).

**Note.** *Nullish types* are not compatible with type `Object`:

```

1 function nullish (
2   o: Object, nullish1: null, nullish2: undefined, nullish3: null|undefined,
3   nullish4: AnyClassOrInterfaceType|null|undefined
4 ) {
5   o = nullish1 /* compile-time error - type 'null' is not compatible with
6                Object */

```

(continues on next page)

(continued from previous page)

```

7   o = nullish2 /* compile-time error - type 'undefined' is not compatible
8               with Object */
9   o = nullish3 /* compile-time error - type 'null|undefined' is not
10              compatible with Object */
11  o = nullish4 /* compile-time error - type
12              'AnyClassOrInterfaceType|null|undefined' is not
13              compatible with Object */
14 }

```

## 3.22 Default Values for Types

**Note.** This feature in ArkTS is experimental.

The following types use so-called *default values* for variables that require no explicit initialization (see [Variable Declarations](#)):

- Primitive types (see the table below);
- Literal types;
- All union types that have at least one `undefined`.

All other types, including reference types, enumeration types, and type parameters have no default values. Variables of such types must be initialized explicitly with a value before these variables are used for the first time.

Default values of primitive types are as follows:

| Data Type | Default Value  |
|-----------|----------------|
| number    | 0 as number    |
| byte      | 0 as byte      |
| short     | 0 as short     |
| int       | 0 as int       |
| long      | 0 as long      |
| float     | +0.0 as float  |
| double    | +0.0 as double |
| char      | u0000          |
| boolean   | false          |

Default values of literal types are literals of such types:

```

1  let a: "string literal"
2  let b: null
3  let c: undefined
4
5  printThem (a, b, c)
6  function printThem (p1: "string literal", p2: null, p3: undefined) {
7      console.log (p1, p2, p3)
8      // Output: string literal null undefined
9  }

```

The default value of a union type that contains type `undefined` is `undefined`.



```
1 class A {  
2   f1: string|undefined  
3   f2?: boolean  
4 }  
5 let a = new A()  
6 console.log (a.f1, a.f2)  
7 // Output: undefined, undefined
```

---



---

## Names, Declarations and Scopes

---

This chapter introduces the following three mutually-related notions:

- Names,
- Declarations, and
- Scopes.

Each entity in an ArkTS program—a variable, a constant, a class, a type, a function, a method, etc.—is introduced via a *declaration*. An entity declaration defines a *name* of the entity. The name is used to refer to the entity further in the program text. The declaration binds the entity name with the *scope* (see [Scopes](#)). The scope affects the accessibility of a new entity, and how it can be referred to by its qualified or simple (unqualified) name.

### 4.1 Names

A name is a sequence of one or more identifiers. A name allows referring to any declared entity. Names can have two syntactical forms:

- *Simple name* that consists of a single identifier;
- *Qualified name* that consists of a sequence of identifiers with the token ‘.’ as separator.

Both situations are covered by the below syntax rule:

```
qualifiedName:  
  identifier ( '.' identifier ) *  
  ;
```

In a qualified name  $N.x$  (where  $N$  is a simple name, and  $x$  is an identifier that can follow a sequence of identifiers separated with ‘.’ tokens),  $N$  can name the following:

- Name of a compilation unit (see *Compilation Units*) that is introduced as a result of `import * as N` (see *Bind All with Qualified Access*) with `x` to name the exported entity;
- A class or interface type (see *Classes*, *Interfaces*) with `x` to name its static member;
- A class or interface type variable with `x` to name its instance member.

## 4.2 Declarations

A declaration introduces a named entity in an appropriate *declaration scope* (see *Scopes*).

## 4.3 Distinguishable Declarations

Each declaration in the declaration scope must be *distinguishable*.

A compile-time error occurs otherwise.

Declarations are *distinguishable* if they have:

- Different names,
- Different signatures (see *Declaration Distinguishable by Signatures*).

The examples below represent declarations distinguishable by names:

```
1  const PI = 3.14
2  const pi = 3
3  function Pi() {}
4  type IP = number[]
5  class A {
6      static method() {}
7      method() {}
8      field: number = PI
9      static field: number = PI + pi
10 }
```

If a declaration is not distinguishable by name (except a valid overloading as in *Function, Method and Constructor Overloading* and *Declaration Distinguishable by Signatures*), then a compile-time error occurs:

```
1  // compile-time error: The constant and the function have the same name.
2  const PI = 3.14
3  function PI() { return 3.14 }
4
5  // compile-time error: The type and the variable have the same name.
6  class Person {}
7  let Person: Person
8
9  // compile-time error: The field and the method have the same name.
10 class C {
```

(continues on next page)

(continued from previous page)

```

11     counter: number
12     counter(): number {
13         return this.counter
14     }
15 }
16
17 /* compile-time error: Name of the declaration clashes with the predefined
18    type or standard library entity name. */
19 let number: number = 1
20 let String = true
21 function Record () {}
22 interface Object {}
23 let Array = 666
24
25 // Functions have the same name but they are distinguishable by signatures
26 function foo() {}
27 function foo(p: number) {}

```

## 4.4 Scopes

Different entity declarations introduce new names in different *scopes*. Scope (see [Scopes](#)) is the region of program text where an entity is declared, along with other regions it can be used in. The following entities are always referred to by their qualified names only:

- Class and interface members (both static and instance ones);
- Entities imported via qualified import; and
- Entities declared in namespaces (see [Namespace Declarations](#)).

Other entities are referred to by their simple (unqualified) names.

Entities within the scope are accessible (see [Accessible](#)).

The scope of an entity depends on the context the entity is declared in:

- Name declared on the package level (*package level scope*) is accessible (see [Accessible](#)) throughout the entire package. The name can be accessed (see [Accessible](#)) in other packages or modules if exported.
- *Module level scope* is applicable to separate modules only. A name declared on the module level is accessible (see [Accessible](#)) throughout the entire module. If exported, a name can be accessed in other compilation units.
- *Namespace level scope* is applicable to namespaces only. A name declared in a namespace is accessible (see [Accessible](#)) throughout the entire namespace and in all embedded namespaces. If exported, a name can be accessed outside the namespace with mandatory namespace name qualification.
- A name declared inside a class (*class level scope*) is accessible (see [Accessible](#)) in the class and sometimes, depending on the access modifier (see [Access Modifiers](#)), outside the class, or by means of a derived class.

Access to names inside the class is qualified with one of the following:

- Keywords `this` or `super`;
- Class instance expression for the names of instance entities; or
- Name of the class for static entities.

Outside access is qualified with one of the following:

- The expression the value stores;
  - A reference to the class instance for the names of instance entities; or
  - Name of the class for static entities.
- A name declared inside an interface (*interface level scope*) is accessible (see [Accessible](#)) inside and outside that interface (default `public`).
  - The scope of a type parameter name in a class or interface declaration is that entire declaration, excluding static member declarations.
  - The scope of a type parameter name in a function declaration is that entire declaration (*function parameter scope*).
  - The scope of a name declared immediately inside the body of a function or a method declaration is the body of that declaration from the point of declaration and up to the end of the body (*method or function scope*). This scope is also applied to function or method parameter names.
  - The scope of a name declared inside a statement block is the body of the statement block from the point of declaration and up to the end of the block (*block scope*).

```
1 function foo() {  
2     let x = y // compile-time error - y is not accessible yet  
3     let y = 1  
4 }
```

Scopes of two names can overlap (e.g., when statements are nested). If scopes of two names overlap, then:

- The innermost declaration takes precedence; and
- Access to the outer name is not possible.

Class, interface, and enum members can only be accessed by applying the dot operator `.` to an instance. Accessing them otherwise is not possible.

## 4.5 Accessible

Entity is considered accessible if it belongs to the current scope (see [Scopes](#)) and means that its name can be used for different purposes as follows:

- Type name is used to declare variables, constants, parameters, class fields, or interface properties;
- Function or method name is used to call the function or method;
- Variable name is used to read or change the value of the variable;
- Name of a compilation unit introduced as a result of import with Bind All with Qualified Access (see [Bind All with Qualified Access](#)) is used to deal with exported entities.

## 4.6 Type Declarations

An interface declaration (see *Interfaces*), a class declaration (see *Classes*), an enum declaration (see *Enumerations*), or a type alias (see *Type Alias Declaration*) are type declarations.

```
typeDeclaration:
  classDeclaration
  | interfaceDeclaration
  | enumDeclaration
  | typeAlias
  ;
```

### 4.6.1 Type Alias Declaration

Type aliases enable using meaningful and concise notations by providing the following:

- Names for anonymous types (array, function, and union types); or
- Alternative names for existing types.

Scopes of type aliases are package, module, or namespace level scopes. Names of all type aliases must be follow uniqueness rules of *Distinguishable Declarations* in the current context.

```
typeAlias:
  'type' identifier typeParameters? '=' type
  ;
```

Meaningful names can be provided for anonymous types as follows:

```
1 type Matrix = number[][]
2 type Handler = (s: string, no: number) => string
3 type Predicate<T> = (x: T) => Boolean
4 type NullableNumber = Number | null
```

If the existing type name is too long, then a shorter new name can be introduced by using type alias (particularly for a generic type).

```
1 type Dictionary = Map<string, string>
2 type MapOfString<T> = Map<T, string>
```

A type alias acts as a new name only. It neither changes the original type meaning nor introduces a new type.

```
1 type Vector = number[]
2 function max(x: Vector): number {
3     let m = x[0]
4     for (let v of x)
5         if (v > m) v = m
6     return m
7 }
8
9 function main() {
10     let x: Vector = [3, 2, 1]
```

(continues on next page)

(continued from previous page)

```

11     console.log(max(x)) // ok
12 }

```

Type aliases can be recursively referenced inside the right-hand side of a type alias declaration.

In a type alias defined as `type A = something`, `A` can be used recursively if it is one of the following:

- Array element type: `type A = A[]`; or
- Type argument of a generic type: `type A = C<A>`.

```

1  type A = A[] // ok, used as element type
2
3  class C<T> { /*body*/ }
4  type B = C<B> // ok, used as a type argument
5
6  type D = string | Array<D> // ok

```

Any other use causes a compile-time error, because the compiler does not have enough information about the defined alias:

```

1  type E = E // compile-time error
2  type F = string | E // compile-time error

```

The same rules apply to a generic type alias defined as `type A<T> = something`:

```

1  type A<T> = Array<A<T>> // ok, A<T> is used as a type argument
2  type A<T> = string | Array<A<T>> // ok
3
4  type A<T> = A<T> // compile-time error

```

A compile-time error occurs if a generic type alias is used without a type argument:

```

1  type A<T> = Array<A> // compile-time error

```

**Note.** There is no restriction on using a type parameter *T* in the right side of a type alias declaration. The following code is valid:

```

1  type NodeValue<T> = T | Array<T> | Array<NodeValue<T>>;

```

## 4.7 Variable and Constant Declarations

### 4.7.1 Variable Declarations

A *variable declaration* introduces a new named storage location. The named storage location is assigned an initial value as part of the declaration, or via initialization before the first usage:

```

variableDeclarations:
    'let' variableDeclarationList
;

```

(continues on next page)



(continued from previous page)

```

variableDeclarationList:
    variableDeclaration (',' variableDeclaration) *
    ;

variableDeclaration:
    identifier ('?')? ':' ('readonly')? type initializer?
    | identifier initializer
    ;

initializer:
    '=' expression
    ;

```

When a variable is introduced by a variable declaration, type *T* of the variable is determined as follows:

- *T* is the type specified in a type annotation (if any) of the declaration.
  - If the name of the variable is followed by the ‘?’ sign, then the type of the variable is semantically equivalent to `type | undefined`.
  - If the declaration also has an initializer, then the initializer expression type must be assignable to *T* (see *Assignability with Initializer*).
- If no type annotation is available, then *T* is inferred from the initializer expression (see *Type Inference from Initializer*).

```

1  let a: number // ok
2  let b = 1 // ok, type 'int' is inferred
3  let c: number = 6, d = 1, e = "hello" // ok
4
5  // ok, type of lambda and type of 'f' can be inferred
6  let f = (p: number) => b + p
7  let x // compile-time error -- either type or initializer

```

Every variable in a program must have an initial value before it can be used:

- If the *initializer* of a variable is specified explicitly, then its execution produces the initial value for this variable.
- Otherwise, the following situations are possible:
  - If the type of a variable is *T*, and *T* has a *default value* (see *Default Values for Types*), then the variable is initialized with the default value.
  - If a variable has no default value, then a value must be set by the *Simple Assignment Operator* before attempting to use the variable.

**Note.** A variable of an array type must be initialized as a whole by a single assignment. Otherwise, the variable is not initialized, and a compile-time error occurs.

If an initializer expression is provided, then additional restrictions apply to the content of the expression as described in *Errors and Initialization Expression*. An initializer expression must not lead to cyclic dependencies caused by the use of non-initialized variables. Otherwise, a compile-time error occurs.

```

1  let a = b // a uses b for its initialization
2  let b = a // b uses a for its initialization
3
4  class A {
5      a = this.b // a uses b for its initialization

```

(continues on next page)

(continued from previous page)

```

6   b = this.a // b uses a for its initialization
7 }

```

If the type of a variable declaration has the prefix `readonly`, then the type must be of the *array* or *tuple* kind, and the restrictions on its operations apply to the variable as described in *Readonly Parameters*, and in *Contexts and Conversions*. If the prefix `readonly` is used with a non-array or non-tuple type, then a compile-time error occurs:

```

1  function foo (p: number[]) {
2      let x: readonly number [] = p
3      x[0] = 666 // compile-time error as array itself is readonly
4      console.log (x[0]) // read operation is OK
5  }

```

## 4.7.2 Constant Declarations

A *constant declaration* introduces a named variable with a mandatory explicit value.

The value of a constant cannot be changed by an assignment expression (see *Assignment*). If the constant is an object or array, then its properties or items can be modified.

```

constantDeclarations:
    'const' constantDeclarationList
    ;

constantDeclarationList:
    constantDeclaration (',' constantDeclaration) *
    ;

constantDeclaration:
    identifier (':' type)? initializer
    ;

```

The type *T* of a constant declaration is determined as follows:

- If *T* is the type specified in a type annotation (if any) of the declaration, then the initializer expression must be assignable to *T* (see *Assignability with Initializer*).
- If no type annotation is available, then *T* is inferred from the initializer expression (see *Type Inference from Initializer*).
- If '?' is used after the name of the constant, then the type of the constant is *T* | undefined, regardless of whether *T* is identified explicitly or via type inference.

```

1  const a: number = 1 // ok
2  const b = 1 // ok, int type is inferred
3  const c: number = 1, d = 2, e = "hello" // ok
4  const x // compile-time error -- initializer is mandatory
5  const y: number // compile-time error -- initializer is mandatory

```

Additional restrictions on the content of the initializer expression are described in *Errors and Initialization Expression*.

### 4.7.3 Assignability with Initializer

If a variable or constant declaration contains type annotation  $T$  and initializer expression  $E$ , then the type of  $E$  must be assignable to  $T$  (see *Assignability*).

### 4.7.4 Type Inference from Initializer

The type of a declaration that contains no explicit type annotation is inferred from the initializer expression as follows:

- In a variable declaration (not in a constant declaration, though), if the initializer expression is of a literal type, then the literal type is replaced for its supertype (see *Supertypes of Literal Types*). If the initializer expression is of a union type that contains literal types, then each literal type is replaced for its supertype (see *Supertypes of Literal Types*), and then normalized (see *Union Types Normalization*).
- Otherwise, the type of a declaration is inferred from the initializer expression.

If the type of the initializer expression cannot be inferred, then a compile-time error occurs (see *Object Literal*):

```

1  let a = null           // type of 'a' is null
2  let aa = undefined    // type of 'aa' is undefined
3  let arr = [null, undefined] // type of 'arr' is (null | undefined) []
4
5  let cond: boolean = /*some initialization*/
6
7  let b = cond ? 1 : 2    // type of 'b' is int
8  let c = cond ? 3 : 3.14 // type of 'c' is double
9  let d = cond ? "one" : "two" // type of 'd' is string
10 let e = cond ? 1 : "one" // type of 'e' is Int | string
11
12 const bb = cond ? 1 : 2 // type of 'bb' is int
13 const cc = cond ? 3 : 3.14 // type of 'cc' is double
14 const dd = cond ? "one" : "two" // type of 'dd' is "one" | "two"
15 const ee = cond ? 1 : "one" // type of 'ee' is Int | "one"
16
17 let f = {name: "aa"} // compile-time error
18
19 declare let x1 = 1 // type of 'x1' is int
20 declare const x2 = 1 // type of 'x2' is int
21 let x3 = 1 // type of 'x3' is int
22 const x4 = 1 // type of 'x4' is int
23
24 declare let s1 = "1" // type of 's1' is string
25 declare const s2 = "1" // type of 's2' is "1"
26 let s3 = "1" // type of 's3' is string
27 const s4 = "1" // type of 's4' is "1"

```

## 4.8 Function Declarations

*Function declarations* specify names, signatures, and bodies when introducing *named functions*. An optional function body is a block (see *Block*):

```
functionDeclaration:
  modifiers? 'function' identifier
  typeParameters? signature block?
  ;

modifiers:
  'native' | 'async'
  ;
```

If a function is declared *generic* (see *Generics*), then its type parameters must be specified.

The modifier `native` indicates that the function is a *native function* (see *Native Functions* in Experimental Features). If a *native function* has a body, then a compile-time error occurs.

Functions must be declared on the top level (see *Top-Level Statements*).

### 4.8.1 Signatures

A signature defines parameters and the return type (see *Return Type*) of a function, method, or constructor.

```
signature:
  '(' parameterList? ')' returnType?
  ;

returnType:
  ':' type
  ;
```

Overloading (see *Function, Method and Constructor Overloading*) is supported for functions, methods and constructors. The signatures of overloaded entities are important for their unique identification.

### 4.8.2 Parameter List

A signature may contain a *parameter list* that specifies an identifier of each parameter name, and the type of each parameter. The type of each parameter must be defined explicitly. If the *parameter list* is omitted, then the function or the method has no parameters.

```
parameterList:
  parameter '(' parameter) * '(' restParameter) ? ',' ?
  | restParameter ',' ?
  ;
```

(continues on next page)

(continued from previous page)

```

parameter:
    annotationUsage? (requiredParameter | optionalParameter)
    ;

requiredParameter:
    identifier ':' type
    ;

```

If a parameter is *required*, then each function or method call must contain an argument corresponding to that parameter. The function below has *required parameters*:

```

1  function power(base: number, exponent: number): number {
2      return Math.pow(base, exponent)
3  }
4  power(2, 3) // both arguments are required in the call

```

Several parameters can be *optional*, allowing to omit corresponding arguments in a call (see [Optional Parameters](#)).

A compile-time error occurs if an *optional parameter* precedes a *required parameter* in the parameter list.

The last parameter of a function or a method can be a single *rest parameter* (see [Rest Parameter](#)).

If a parameter type is prefixed with `readonly`, then there are additional restrictions on the parameter as described in [Readonly Parameters](#).

### 4.8.3 Readonly Parameters

If the parameter type is prefixed with `readonly`, then the type must be of array type `T[]` (see [Array Types](#)) or tuple type `[T1, T2, ..., Tn]` (see [Tuple Types](#)). Otherwise, a compile-time error occurs.

No function or method body can modify an array or tuple content that has the *readonly* parameter. A compile-time error occurs if an operation modifies an array or tuple content that has the *readonly* parameter:

```

1  function foo(array: readonly number[], tuple: readonly [number, string]) {
2      let element = array[0] // OK, one can get array element
3      array[0] = element // compile-time error, array is readonly
4
5      element = tuple[0] // OK, one can get tuple element
6      tuple[0] = element // compile-time error, tuple is readonly
7  }

```

This rule applies to variables as discussed in [Variable Declarations](#).

Any assignment of `readonly` parameters and variables must follow the limitations stated in [Contexts and Conversions](#).

### 4.8.4 Optional Parameters

*Optional parameters* can be of two forms as follows:

```
optionalParameter:
  identifier ':' type '=' expression
  | identifier '?' ':' type
  ;
```

The first form contains an expression that specifies a *default value*. It is called a *parameter with default value*. The value of the parameter is set to the *default value* if the argument corresponding to that parameter is omitted in a function or method call:

```
1 function pair(x: number, y: number = 7)
2 {
3   console.log(x, y)
4 }
5 pair(1, 2) // prints: 1 2
6 pair(1) // prints: 1 7
```

The second form is a short-cut notation and identifier '?' ':' type effectively means that identifier has type T | undefined with the default value undefined. If a type is of the *value* kind, then implicit boxing (see [Boxing Conversions](#)) must be applied (as in [Union Types](#)) as follows: identifier '?' ':' valueType is equivalent to identifier ':' referenceTypeForValueType | undefined = undefined.

For example, the following two functions can be used in the same way:

```
1 function hello1(name: string | undefined = undefined) {}
2 function hello2(name?: string) {}
3
4 hello1() // 'name' has 'undefined' value
5 hello1("John") // 'name' has a string value
6 hello2() // 'name' has 'undefined' value
7 hello2("John") // 'name' has a string value
8
9 function fool (p?: number) {}
10 function foo2 (p: Number | undefined = undefined) {}
11
12 fool() // 'p' has 'undefined' value
13 fool(5) // 'p' has an integer value
14 foo2() // 'p' has 'undefined' value
15 foo2(5) // 'p' has an integer value
```

### 4.8.5 Rest Parameter

*Rest parameters* allow functions, methods, constructors, or lambdas to take arbitrary numbers of arguments. *Rest parameters* have the spread operator '...' as prefix before the parameter name:

```
restParameter:
  annotationUsage? '...' identifier ':' type
  ;
```

A compile-time error occurs if a rest parameter:

- Is not the last parameter in a parameter list;
- Has a type that is neither an array type nor a tuple type.

A call of entity with a rest parameter of array type `T[]` (or `FixedArray<T>`) can accept any number of arguments of types that are assignable (see [Assignability](#)) to `T`:

```

1  function sum(...numbers: number[]): number {
2      let res = 0
3      for (let n of numbers)
4          res += n
5      return res
6  }
7
8  sum() // returns 0
9  sum(1) // returns 1
10 sum(1, 2, 3) // returns 6

```

If an argument of array type `T[]` is to be passed to a call of entity with the rest parameter, then the spread expression (see [Spread Expression](#)) must be used with the spread operator `'...'` as prefix before the array argument:

```

1  function sum(...numbers: number[]): number {
2      let res = 0
3      for (let n of numbers)
4          res += n
5      return res
6  }
7
8  let x: number[] = [1, 2, 3]
9  sum(...x) // spread an array 'x'
10           // returns 6

```

A call of entity with a rest parameter of tuple type `[T1, ..., Tn]` can accept only `n` arguments of types that are assignable (see [Assignability](#)) to the corresponding `Ti`:

```

1  function sum(...numbers: [number, number, number]): number {
2      return numbers[0] + numbers[1] + numbers[2]
3  }
4
5  sum()           // compile-time error: wrong number of arguments, 0 instead of 3
6  sum(1)          // compile-time error: wrong number of arguments, 1 instead of 3
7  sum(1, 2, "a")  // compile-time error: wrong type of the 3rd argument
8  sum(1, 2, 3)    // returns 6

```

If an argument of tuple type `[T1, ..., Tn]` is to be passed to a call of entity with the rest parameter, then a spread expression (see [Spread Expression](#)) must have the spread operator `'...'` as a prefix before the tuple argument:

```

1  function sum(...numbers: [number, number, number]): number {
2      return numbers[0] + numbers[1] + numbers[2]
3  }
4
5  let x: [number, number, number] = [1, 2, 3]
6  sum(...x) // spread tuple 'x'
7           // returns 6

```

If an argument of fixed array type `FixedArray<T>` is to be passed to a function or a method with the rest parameter, then the spread expression (see [Spread Expression](#)) must be used with the spread operator `'...'` as prefix before the fixed array argument:

```

1  function sum(...numbers: Array<number>): number {
2      let res = 0

```

(continues on next page)

(continued from previous page)

```
3   for (let n of numbers)
4       res += n
5   return res
6 }
7
8 let x: FixedArray<number> = [1, 2, 3]
9 sum(...x) // spread an fixed array 'x'
10 // returns 6
```

## 4.8.6 Shadowing by Parameter

If the name of a parameter is identical to the name of a top-level variable accessible (see [Accessible](#)) within the body of a function or a method with that parameter, then the name of the parameter shadows the name of the top-level variable within the body of that function or method:

```
1  class T1 {}
2  class T2 {}
3  class T3 {}
4
5  let variable: T1
6  function foo (variable: T2) {
7      // 'variable' has type T2 and refers to the function parameter
8  }
9  class SomeClass {
10     method (variable: T3) {
11         // 'variable' has type T3 and refers to the method parameter
12     }
13 }
```

## 4.8.7 Return Type

Function or method return type defines the static type of the result of the function or method execution (see [Function Call Expression](#) and [Method Call Expression](#)). During the execution, the function or method can produce a value of a type assignable (see [Assignability](#)) to the return type.

If function or method return type is not `void` (see [Type void](#)), and the execution path of the function or method body has no return statement (see [return Statements](#)), then a compile-time error occurs.

If function or method return type is not specified, then it is inferred from its body (see [Return Type Inference](#)). If there is no body, then the function or method return type is `void` (see [Type void](#)).



### 4.8.8 Return Type Inference

An omitted function or method return type can be inferred from the function, or the method body. If the return type is omitted in a native function (see [Native Functions](#)), then a compile-time error occurs.

The current version of ArkTS allows inferring return types at least under the following conditions:

- If there is no return statement, or if all return statements have no expressions, then the return type is `void` (see [Type void](#)).
- If there are  $k$  return statements (where  $k$  is 1 or more) with the same type expression  $R$ , then  $R$  is the return type.
- If there are  $k$  return statements (where  $k$  is 2 or more) with expressions of types  $T_1, \dots, T_k$ , then  $R$  is the *union type* (see [Union Types](#)) of these types ( $T_1 \mid \dots \mid T_k$ ), and its normalized version (see [Union Types Normalization](#)) is the return type.
- If the function is `async`, the return type is inferred by using the rules above, and the type  $T$  is not of type `Promise`, then the return type is `Promise<T>`.

Future compiler implementations are to infer the return type in more cases. The example below represents type inference:

```
// Explicit return type
function foo(): string { return "foo" }

// Implicit return type inferred as string
function goo() { return "goo" }

class Base {}
class Derived1 extends Base {}
class Derived2 extends Base {}

function bar (condition: boolean) {
    if (condition)
        return new Derived1()
    else
        return new Derived2()
}
// Return type of bar will be Derived1|Derived2 union type

function boo (condition: boolean) {
    if (condition) return 1
}
// That is a compile-time error as there is an execution path with no return
```

If the compiler fails to recognize a particular type inference case, then a corresponding compile-time error occurs.



Class, interface, type alias, method, function, and lambda are program entities that can be parameterized in ArkTS by one or several types. An entity so parameterized introduces a *generic declaration* (called a *generic* for brevity).

Types used as generic parameters in a generic are called *type parameters* (see [Type Parameters](#)).

A *generic* must be instantiated in order to be used. *Generic instantiation* is the action that transforms a *generic* into a real program entity (non-generic class, interface, union, array, method, function, or lambda), or into another *generic instantiation*. Instantiation (see [Generic Instantiations](#)) can be performed either explicitly or implicitly.

ArkTS has special types that look similar to generics syntax-wise, and allow creating new types during compilation (see [Utility Types](#)).

## 5.1 Type Parameters

*Type parameter* is declared in the type parameter section. It can be used as an ordinary type inside a *generic*.

Syntax-wise, a *type parameter* is an unqualified identifier with a proper scope (see [Scopes](#) for the scope of type parameters). Each type parameter can have a *constraint* (see [Type Parameter Constraint](#)). A type parameter can have a default type (see [Type Parameter Default](#)), and can specify its *in-* or *out-* variance (see [Type Parameter Variance](#)).

```
typeParameters:
    '<' typeParameterList '>'
    ;

typeParameterList:
    typeParameter (',' typeParameter)*
    ;

typeParameter:
```

(continues on next page)

(continued from previous page)

```

('in' | 'out')? identifier constraint? typeParameterDefault?
;

constraint:
  'extends' type
;

typeParameterDefault:
  '=' typeReference ('[]')?
;

```

A generic class, interface, type alias, method, function, or lambda defines a set of parameterized classes, interfaces, unions, arrays, methods, functions, or lambdas respectively (see *Generic Instantiations*). A single type argument can define only one set for each possible parameterization of the type parameter section.

No type parameter has a default value, and initialization is mandatory for variables and fields of a type parameter (see *Field Initialization*):

```

1  class G<T> {
2      field: T // compile-time error, field is not initialized
3      function foo() {
4          let t: T // compile-time error, variable is not initialized
5      }
6  }

```

### 5.1.1 Type Parameter Constraint

If possible instantiations need to be constrained, then an individual *constraint* can be set for each type parameter.

A constraint of any type parameter can follow the keyword `extends`. The constraint is denoted as any type. If no constraint is declared, then the type parameter is not compatible with `Object`, and has no methods or fields available for use. Lack of constraint effectively means `extends Object|null|undefined`. If type parameter *T* has type constraint *S*, then the actual type of the generic instantiation must be assignable to *S* (see *Assignability*). If the constraint *S* is a non-nullish type (see *Nullish Types*), then *T* is also non-nullish.

```

1  class Base {}
2  class Derived extends Base { }
3  class SomeType { }
4
5  class G<T extends Base> { }
6
7  let x = new G<Base> // OK
8  let y = new G<Derived> // OK
9  let z = new G<SomeType> // Compile-time : SomeType is not compatible with Base
10
11 class H<T extends Base|SomeType> {}
12 let h1 = new H<Base> // OK
13 let h2 = new H<Derived> // OK
14 let h3 = new H<SomeType> // OK
15 let h4 = new H<Object> // Compile-time : Object is not compatible with
    ↪ Base|SomeType

```

(continues on next page)

(continued from previous page)

```

16
17 class Exotic<T extends "aa" | "bb"> {}
18 let e1 = new Exotic<"aa"> // OK
19 let e2 = new Exotic<"cc"> // Compile-time : "cc" is not compatible with "aa" | "bb"
20
21 class A {
22     f1: number = 0
23     f2: string = ""
24     f3: boolean = false
25 }

```

A type parameter of a generic can *depend* on another type parameter of the same generic.

If  $S$  constrains  $T$ , then the type parameter  $T$  *directly depends* on the type parameter  $S$ , while  $T$  directly depends on the following:

- $S$ ; or
- Type parameter  $U$  that depends on  $S$ .

A compile-time error occurs if a type parameter in the type parameter section depends on itself.

```

1 class Base {}
2 class Derived extends Base { }
3 class SomeType { }
4
5 class G<T, S extends T> {}
6
7 let x: G<Base, Derived> // correct: the second argument directly
8                        // depends on the first one
9 let y: G<Base, SomeType> // error: SomeType does not depend on Base
10
11 class A0<T> {
12     data: T
13     constructor (p: T) { this.data = p }
14     foo () {
15         let o: Object = this.data // error: as type T is not compatible with Object
16         console.log (this.data.toString()) // error: type T has no methods or fields
17     }
18 }
19
20 class A1<T extends Object> extends A0<T> {
21     constructor (p: T) { this.data = p }
22     override foo () {
23         let o: Object = this.data // OK!
24         console.log (this.data.toString()) // OK!
25     }
26 }

```

## 5.1.2 Type Parameter Default

Type parameters of generic types can have defaults. This situation allows dropping a type argument when a particular type of instantiation is used. However, a compile-time error occurs if:

- A type parameter without a default type follows a type parameter with a default type in the declaration of a generic type;
- Type parameter default refers to a type parameter defined after the current type parameter.

The application of this concept to both classes and functions is presented in the examples below:

```

1  class SomeType {}
2  interface Interface <T1 = SomeType> { }
3  class Base <T2 = SomeType> { }
4  class Derived1 extends Base implements Interface { }
5  // Derived1 is semantically equivalent to Derived2
6  class Derived2 extends Base<SomeType> implements Interface<SomeType> { }
7
8  function foo<T = number>(): T {
9      // ...
10 }
11 foo() // this call is semantically equivalent to the call below
12 foo<number>()
13
14 class C1 <T1, T2 = number, T3> {}
15 // That is a compile-time error, as T2 has default but T3 does not
16
17 class C2 <T1, T2 = number, T3 = string> {}
18 let c1 = new C2<number> // equal to C2<number, number, string>
19 let c2 = new C2<number, string> // equal to C2<number, string, string>
20 let c3 = new C2<number, Object, number> // all 3 type arguments provided
21
22 function foo <T1 = T2, T2 = T1> () {}
23 // That is a compile-time error,
24 // as T1's default refers to T2, which is defined after the T1
25 // T2's default is valid as it refers to already defined type parameter T1

```

### 5.1.3 Type Parameter Variance

Normally, two different argument types used to instantiate a generic class or interface are handled as different and unrelated types (*invariance*). ArkTS supports type parameter variance that allows such instantiations become base classes and derived classes (*Invariance, Covariance and Contravariance*), or vice versa (*Invariance, Covariance and Contravariance*), depending on the relationship of inheritance between argument types.

Special markers are used to specify *declaration-site variance*. The markers are to be added to generic parameter declarations. The markers are expressed as keywords `in` or `out` (a *variance modifier* that specifies the variance of the type parameter).

Type parameters with the keyword `out` are *covariant* (see *Invariance, Covariance and Contravariance*), and can be used in the out-position only:

- Methods may have `out` type parameters as return types
- Fields of `out` type parameters as type should be `readonly`.

Otherwise a compile-time error occurs.

Type parameters with the keyword `in` are *contravariant* (see *Invariance, Covariance and Contravariance*), and can be used in the in-position only:

- Methods may have `in` type parameters as parameter types

Otherwise a compile-time error occurs.

Type parameters with no variance modifier are implicitly *invariant*, and can occur in any position.

```

1  class X<in T1, out T2, T3> {
2      // T1 can be used in in-position only
3      foo (p: T1) {...}
4
5      // T2 can be used in out-position only
6      bar(): T2 {...}
7      readonly fld1: T2
8
9      // T3 can be used in any position (in-out, write-read)
10     fld2: T3
11     method (p: T3): T3 {...}
12 }

```

In case of function types (see *Function Types*) variance interleaving occurs.

```

1  class X<in T1, out T2> {
2      foo (p: T1): T2 {...}           // in - out
3      foo (p: (p: T2)=> T1) {...}     // out - in
4      foo (p: (p: (p: T1)=>T2)=> T1) {...} // in - out - in
5      foo (p: (p: (p: (p: T2)=> T1)=>T2)=> T1) {...} // out - in - out - in
6      // and further more
7  }

```

A compile-time error occurs if a function, method, or constructor type parameters have a variance modifier specified.

*Variance* is used to describe the subtyping (see *Subtyping*) operation on parameterized types (see *Generics*). The variance of the corresponding type parameter *F* defines the subtyping between  $T\langle A \rangle$  and  $T\langle B \rangle$  (in the case of declaration-site variance with two different types  $A \prec B$ ) as follows:

- Covariant – *Invariance, Covariance and Contravariance* (out *F*):  $T\langle A \rangle \prec T\langle B \rangle$ ;
- Contravariant – *Invariance, Covariance and Contravariance* (in *F*):  $T\langle A \rangle \succ T\langle B \rangle$ ;
- Invariant – default (*F*).

## 5.2 Generic Instantiations

As mentioned before, a generic class, interface, type alias, method, function, or lambda declaration defines a set of corresponding non-generic entities. The process of instantiation is designed to do the following:

- Allow producing new generic or non-generic entities;
- Provide every type parameter with a type argument that can be any kind of type, including the type argument itself.

As a result of the instantiation process, a new class, interface, union, array, method, function, or lambda is created.

If a value type (see *Value Types*) is specified as type argument in a generic instantiation, then the compiler actually replaces it as follows:

- Primitive type (see *Primitive Types*) for its boxed type (see *Boxed Types*);
- Enumeration type (see *Enumerations*) for a union of literal types that correspond to the values of the enumeration type.

```

1  Array<number> // replaced with Array<Number>
2  Array<boolean> // replaced with Array<Boolean>
3  enum Color {Red, Green, Blue}
4  Array<Color> // replaced with Array<Color.Red/Color.Green/Color.Blue>
5  enum Reply {Yes="yes", No="no"}
6  Array<Reply> // replaced with Array<"yes"/"no">
7
8  class A <T> {}
9  class B <U, V> extends A<U> { // Here A<U> is a new generic type
10     field: A<V> // Here A<V> is a new generic type
11     method (p: A<Object>) {} // Here A<Object> is a new non-generic type
12 }

```

**Note.** Built-in arrays are not generic, thus `number [ ]` contains elements of type `number` but not `Number`.

## 5.2.1 Type Arguments

Type arguments are a non-empty list of types that are used for instantiation.

```

typeArguments:
    '<' type (',' type)* '>'
;

```

The example below represents instantiations with different forms of type arguments:

```

1  Array<number> // instantiated with type number
2  Array<0|1> // instantiated with union type
3  Array<number[]> // instantiated with array type
4  Array<[number, string, boolean]> // instantiated with tuple type
5  Array<()=>void> // instantiated with function type

```

## 5.2.2 Explicit Generic Instantiations

An explicit generic instantiation is a language construct, which provides a list of *type arguments* (see *Type Arguments*) that specify real types or type parameters to substitute corresponding type parameters of a generic:

```

1  class G<T> {} // Generic class declaration
2  let x: G<number> // Explicit class instantiation, type argument provided
3
4  class A {
5     method <T> () {} // Generic method declaration
6  }
7  let a = new A()

```

(continues on next page)



(continued from previous page)

```

8  a.method<string> () // Explicit method instantiation, type argument provided
9
10 function foo <T> () {} // Generic function declaration
11 foo <string> () // Explicit function instantiation, type argument provided
12
13 type MyArray<T> = T[] // Generic type declaration
14 let array: MyArray<boolean> = [true, false] // Explicit array instantiation, type_
    ↪argument provided
15
16 class X <T1, T2> {}
17 // Different forms of explicit instantiations of class X producing new generic_
    ↪entities
18 class Y<T> extends X<number, T> { // class Y extends X instantiated with number and T
19     f1: X<Object, T> // X instantiated with Object and T
20     f2: X<T, string> // X instantiated with T and string
21 }
22
23 let lambda = <T> (p: T) => { console.log (p) } // Generic lambda defined
24 lambda<string> ("string argument") // Generic lambda instantiated and called

```

A compile-time error occurs if type arguments are provided for non-generic class, interface, type alias, method, function, or lambda.

In the explicit generic instantiation  $G <T_1, \dots, T_n>$ ,  $G$  is the generic declaration, and  $<T_1, \dots, T_n>$  is the list of its type arguments.

If type parameters  $T_1, \dots, T_n$  of a generic declaration are constrained by the corresponding  $C_1, \dots, C_n$ , then  $T_i$  is assignable to each constraint type  $C_i$  (see [Assignability](#)). All subtypes of the type listed in the corresponding constraint have each type argument  $T_i$  of the parameterized declaration ranging over them.

A generic instantiation  $G <T_1, \dots, T_n>$  is *well-formed* if **all** of the following is true:

- The generic declaration name is  $G$ ;
- The number of type arguments equals the number of type parameters of  $G$ ; and
- All type arguments are assignable to the corresponding type parameter constraint (see [Assignability](#)).

A compile-time error occurs if an instantiation is not well-formed.

Unless explicitly stated otherwise in appropriate sections, this specification discusses generic versions of class type, interface type, or function.

Any two generic instantiations are considered *provably distinct* if:

- Both are parameterizations of distinct generic declarations; or
- Any of their type arguments is provably distinct.

### 5.2.3 Implicit Generic Instantiations

In an *implicit* instantiation, type arguments are not specified explicitly. Such type arguments are inferred (see [Type Inference](#)) from the context in which a generic is referred. It is represented in the example below:

```
1 function foo <G> (x: G, y: G) {} // Generic function declaration
2 foo (new Object, new Object)    // Implicit generic function instantiation
3 // based on argument types the type argument is inferred
4
5 let lambda = <T>(p: T): void => {console.log (p)} // Generic lambda declaration
6 lambda(6) // Implicit generic lambda instantiation
```

Implicit instantiation is only possible for generic functions, methods, and lambdas.

## 5.3 Utility Types

ArkTS supports several embedded types, called *utility* types. Utility types allow constructing new types by adjusting properties of initial types.

### 5.3.1 Partial Utility Type

Type `Partial<T>` constructs a type with all properties of `T` set to optional. `T` must be a class or an interface type. No method (not even any getter or setter) of `T` is part of the `Partial<T>` type. It is represented in the example below:

```
1 interface Issue {
2     title: string
3     description: string
4 }
5
6 function process(issue: Partial<Issue>) {
7     if (issue.title != undefined) {
8         /* process title */
9     }
10 }
11
12 process({title: "aa"}) // description is undefined
```

In the example above, type `Partial<Issue>` is transformed to a distinct but analogous type as follows:

```
1 interface /*some name*/ {
2     title?: string
3     description?: string
4 }
```

Type `T` is not assignable to `Partial<T>` (see [Assignability](#)), and variables of `Partial<T>` are to be initialized with valid object literals.

**Note.** If class `T` has a user-defined getter, setter, or both, then none of those is called when object literal is used with `Partial<T>` variables. Object literal has its own built-in getters and setters to modify its variables. It is represented in the example below:

```

1 interface I {
2   property: number
3 }
4 class A implements I {
5   set property(property: number) { console.log ("Setter called") ... }
6   get property(): number { console.log ("Getter called") ... }
7 }
8 function foo (partial: Partial<A>) {
9   partial.property = 666 // setter to be called
10  console.log(partial.property) // getter to be called
11 }
12 foo ({property: new SomeType}) // No getter or setter from class A is called
13 // 666 is printed as object literal has its own setter and getter

```

### 5.3.2 Required Utility Type

Type `Required<T>` is opposite to `Partial<T>`, and constructs a type with all properties of `T` set to required (i.e., not optional). `T` must be a class or an interface type. No method (not even any getter or setter) of `T` is part of the `Required<T>` type. It is represented in the example below:

```

1 interface Issue {
2   title?: string
3   description?: string
4 }
5
6 let c: Required<Issue> = { // CTE: 'description' should be defined
7   title: "aa"
8 }

```

In the example above, type `Required<Issue>` is transformed to a distinct but analogous type as follows:

```

1 interface /*some name*/ {
2   title: string
3   description: string
4 }

```

Type `T` is not assignable (see [Assignability](#)) to `Required<T>`, and variables of `Required<T>` are to be initialized with valid object literals.

### 5.3.3 Readonly Utility Type

Type `Readonly<T>` constructs a type with all properties of `T` set to `readonly`. It means that the properties of the constructed value cannot be reassigned. `T` must be a class or an interface type. No method (not even any getter or setter) of `T` is part of the `Readonly<T>` type. It is represented in the example below:

```

1 interface Issue {
2     title: string
3 }
4
5 const myIssue: Readonly<Issue> = {
6     title: "One"
7 };
8
9 myIssue.title = "Two" // compile-time error: readonly property

```

Type T is assignable (see [Assignability](#)) to `Readonly<T>`, and allows assignments as a consequence:

```

1 class A {
2     f1: string = ""
3     f2: number = 1
4     f3: boolean = true
5 }
6 let x = new A
7 let y: Readonly<A> = x // OK

```

### 5.3.4 Record Utility Type

Type `Record<K, V>` constructs a container that maps keys (of type K) to values of type V.

Type K is restricted to numeric types (see [Numeric Types](#)), type `string`, string literal types, enum types, and union types constructed from these types.

A compile-time error occurs if any other type, or literal of any other type is used in place of this type:

```

1 type R1 = Record<number, Object> // ok
2 type R2 = Record<boolean, Object> // compile-time error
3 type R3 = Record<"salary" | "bonus", Object> // ok
4 type R4 = Record<"salary" | boolean, Object> // compile-time error
5 type R5 = Record<"salary" | number, Object> // ok
6 type R6 = Record<string | number, Object> // ok
7 enum Strings { A = "AA", B = "BB" }
8 type R7 = Record<Strings, Object> // ok
9 enum Numbers { A, B }
10 type R7 = Record<Numbers, Object> // ok

```

Type V has no restrictions.

A special form of object literals is supported for instances of type `Record` (see [Object Literal of Record Type](#)).

Access to `Record<K, V>` values is performed by an *indexing expression* like `r[index]`, where `r` is an instance of type `Record`, and `index` is the expression of type K. See [Record Indexing Expression](#) for details.

Variables of type `Record<K, V>` can be initialized with help of valid object literals of record type (see [Object Literal of Record Type](#)). Where literal is valid if type of key expression is compatible with key type K and type of value expression is compatible with value type V.

```

1 type Keys = 'key1' | 'key2' | 'key3'
2

```

(continues on next page)

(continued from previous page)

```

3  let x: Record<Keys, number> = {
4      'key1': 1,
5      'key2': 2,
6      'key3': 4,
7  }
8  console.log(x['key2']) // prints 2
9  x['key2'] = 8
10 console.log(x['key2']) // prints 8

```

In the example above, K is a union of literal types and thus the result of an indexing expression is of type V. In this case it is number.

### 5.3.5 Utility Type Private Fields

Utility types are built on top of other types. Private fields of the initial type stay in the utility type but they are not accessible and cannot be accessed in any way. It is represented in the example below:

```

1  function foo(): string { // Potentially some side effect
2      return "private field value"
3  }
4
5  class A {
6      public_field = 444
7      private private_field = foo()
8  }
9
10 function bar (part_a: Readonly<A>) {
11     console.log (part_a)
12 }
13
14 bar ({public_field: 777}) // OK, object literal has no field `private_field`
15 bar ({public_field: 777, private_field: ""}) // compile-time error, incorrect field_
    ↪ name
16
17 bar (new A) // OK, object of type Readonly<A> has field `private_field`

```



---

## Contexts and Conversions

---

Every expression written in the ArkTS programming language has a type that is inferred (see [Type Inference](#)) at compile time.

In most contexts, an expression must be *compatible* with a type expected in that context. This type is called the *target type*.

If no target type is expected in the context, then the expression is called a *standalone expression*:

```
1  let a = expr // no target type is expected
2
3  function foo() {
4      expr // no target type is expected
5  }
```

Otherwise, the expression is *non-standalone*:

```
1  let a: number = expr // target type of 'expr' is number
2
3  function foo(s: string) {}
4  foo(expr) // target type of 'expr' is string
```

The type of some expressions cannot be inferred from the expression itself (see [Object Literal](#) as an example). A compile-time error occurs if such an expression is used as a *standalone expression*.

There are two ways to facilitate the compatibility of a *non-standalone expression* with its surrounding context:

1. The type of some non-standalone expressions can be inferred from the target type (expression types can be different in different contexts).
2. If the inferred expression type is different from the target type, then performing an implicit *conversion* can ensure [Assignability](#). The conversion from type S to type T causes a type S expression to be handled as a type T expression at compile time.

A compile-time error occurs if neither produces an appropriate expression type.

The rules that determine whether a *target type* allows an implicit *conversion* vary for different kinds of contexts and types of expressions. The *target type* can influence not only the type of the expression but also, in some cases, its

runtime behavior.

Some cases of conversion require action at runtime to check the validity of conversion, or to translate the runtime expression value into a form that is appropriate for the new type `T`.

If the type of the expression is `readonly`, then the target type must also be `readonly`. Otherwise, a compile-time error occurs:

```

1  let readonly_array: readonly number[] = [1, 2, 3]
2
3  foo1(readonly_array) // OK
4  foo2(readonly_array) // compile-time error
5
6  function foo1 (p: readonly number[]) {}
7  function foo2 (p: number[]) {}
8
9  let writable_array: number [] = [1, 2, 3]
10 foo1 (writable_array) // OK, as always safe

```

Contexts can be of the following kinds:

- *Assignment-like Contexts* where the expression value is bound to a variable;
- *String Operator Contexts* with string concatenation (operator '+');
- *Numeric Operator Contexts* with all numeric operators ('+', '-', etc.);
- *Casting Contexts and Conversions*, i.e., the conversion of an expression value to a type explicitly specified by a cast expression (see *Cast Expressions*).

## 6.1 Assignment-like Contexts

*Assignment-like contexts* include the following:

- *Declaration contexts* that allow setting an initial value to a variable (see *Variable Declarations*), a constant (see *Constant Declarations*), or a field (see *Field Declarations*) with an explicit type annotation;
- *Assignment contexts* that allow assigning (see *Assignment*) an expression value to a variable;
- *Call contexts* that allow assigning an argument value to a corresponding formal parameter of a function, method, constructor or lambda call (see *Function Call Expression*, *Method Call Expression*, *Explicit Constructor Call*, and *New Expressions*);
- *Composite literal contexts* that allow setting an expression value to an array element (see *Array Literal Type Inference from Context*), a class, or an interface field (see *Object Literal*);

The examples are presented below:

```

1  // declaration contexts:
2  let x: number = 1
3  const str: string = "done"
4  class C {
5      f: string = "aa"
6  }
7
8  // assignment contexts:

```

(continues on next page)



(continued from previous page)

```

9      x = str.length
10     new C().f = "bb"
11     function foo<T1, T2> (p1: T1, p2: T2) {
12         let t1: T1 = p1
13         let t2: T2 = p2
14     }
15
16     // call contexts:
17     function foo(s: string) {}
18     foo("hello")
19
20     // composite literal contexts:
21     let a: number[] = [str.length, 11]

```

In all these cases, the expression type either must be equal to the *target type*, or can be converted to the *target type* by using one of the conversions discussed below. Otherwise, a compile-time error occurs.

Assignment-like contexts allow using of one of the following:

- *Widening Primitive Conversions;*
- *Constant Narrowing Integer Conversions;*
- *Boxing Conversions;*
- *Unboxing Conversions;*
- *Widening Union Conversions;*
- *Widening Reference Conversions;*
- *Character to String Conversions;*
- *Constant String to Character Conversions;*
- *Function Types Conversions;*
- *Enumeration to Constants Type Conversions;*
- *Constant to Enumeration Conversions;*
- *Literal Type to its Supertype Conversions.*

If there is no applicable conversion, then a compile-time error occurs.

## 6.2 String Operator Contexts

*String context* applies only to a non-*string* operand of the binary operator '+' if the other operand is *string*.

*String conversion* for a non-*string* operand is evaluated as follows:

- The operand of a nullish type that has a nullish value is converted as described below:
  - The operand `null` is converted to `string null`;
  - The operand `undefined` is converted to `string undefined`.
- An operand of a reference type or `enum` type is converted by applying the method call `toString()`.

- An operand of an integer type (see *Integer Types and Operations*) is converted to type `string` with a value that represents the operand in the decimal form.
- An operand of a floating-point type (see *Floating-Point Types and Operations*) is converted to type `string` with a value that represents the operand in the decimal form (without the loss of information).
- An operand of type `boolean` is converted to type `string` with the values `true` or `false`.
- An operand of type `char` is converted by using *Character to String Conversions*.
- An operand of enumeration type (see *Enumerations*) is converted to type `string` with the value of the corresponding enumeration constant if values of enumeration are of type `string`.

If there is no applicable conversion, then a compile-time error occurs.

The target type in this context is always `string`:

```
1 console.log("" + null) // prints "null"
2 console.log("value is " + 123) // prints "value is 123"
3 console.log("BigInt is " + 123n) // prints "BigInt is 123"
4 console.log(15 + " steps") // prints "15 steps"
5 let x: string | null = null
6 console.log("string is " + x) // prints "string is null"
7 let c = "X"
8 console.log("char is " + c) // prints "char is X"
```

## 6.3 Numeric Operator Contexts

Numeric contexts apply to the operands of an arithmetic operator. Numeric contexts use combinations of predefined numeric types conversions (see *Primitive Types Conversions*), and ensure that each argument expression can be converted to target type `T` while the arithmetic operation for the values of type `T` is being defined.

An operand of an enumeration type (see *Enumerations*) can be used in the numeric context if values of this enumeration are of type `int`. The type of this operand is assumed to be `int`.

Numeric contexts actually take the following forms:

- *Unary Expressions*;
- *Multiplicative Expressions*;
- *Additive Expressions*;
- *Shift Expressions*;
- *Relational Expressions*;
- *Equality Expressions*;
- *Bitwise and Logical Expressions*;
- *Conditional-And Expression*;
- *Conditional-Or Expression*.

## 6.4 Casting Contexts and Conversions

Every cast expression (*Cast Expressions*) introduces a *casting context*, and relies on the application of different conversions. These conversions cast an operand in a cast expression to an explicitly specified *target type* by using one of the following:

- Identity conversion, if the *target type* is the same as the expression type;
- *Implicit Conversions*;
- *Numeric Casting Conversions*;
- *Class or Interface Casting Conversions*;
- *Casting Conversions from Object*;
- *Casting Conversions from Type Parameter*;
- *Casting Conversions to Type Parameter*;
- *Casting Conversions from Union*;
- *Casting Conversions to Enumeration*.

If there is no applicable conversion, then a compile-time error occurs.

### 6.4.1 Numeric Casting Conversions

A *numeric casting conversion* occurs if the *target type* and the expression type are both `numeric` or `char`:

```

1  function process_int(an_int: int) { ... }
2
3  let pi = 3.14
4  process_int(pi as int)

```

These conversions never cause runtime errors.

Numeric casting conversion of an operand of type `double` to target type `float` is performed in compliance with the IEEE 754 rounding rules. This conversion can lose precision or range, resulting in the following:

- Float zero from a nonzero double; and
- Float infinity from a finite double.

Double NaN is converted to float NaN.

Double infinity is converted to the same-signed floating-point infinity.

A numeric conversion of a floating-point type operand to target types `long` or `int` is performed by the following rules:

- If the operand is NaN, then the result is 0 (zero).
- If the operand is positive infinity, or if the operand is too large for the target type, then the result is the largest representable value of the target type.
- If the operand is negative infinity, or if the operand is too small for the target type, then the result is the smallest representable value of the target type.
- Otherwise, the result is the value that rounds toward zero by using IEEE 754 *round-toward-zero* mode.

A numeric casting conversion of a floating-point type operand to types `short`, `byte`, or `char` is performed in two steps as follows:

- The casting conversion to `int` is performed first (see above);
- Then, the `int` operand is cast to the target type.

A numeric casting conversion from an integer type (or `char`) to a smaller integer type (or `char`) `I` discards all bits except the  $N$  lowest ones, where  $N$  is the number of bits used to represent type `I`. This conversion can lose the information on the magnitude of the numeric value. The sign of the resulting value can differ from that of the original value.

## 6.4.2 Class or Interface Casting Conversions

A *class casting conversion* or *interface casting conversion* occurs if the *target type* and the expression type are both of `class` or `interface` type. This conversion casts an expression of a supertype (superclass or superinterface, see *Supertyping*) to a subclass or subinterface:

```
1  class Base {}
2  class Derived extends Base {}
3
4  let b: Base = new Derived()
5  let d: Derived = b as Derived
```

If *target type* is not assignable (see *Assignability*) to the expression type then a compile-time error occurs.

A runtime error (`ClassCastException`) occurs during this conversion if the type of a converted expression cannot be cast to the *target type*:

```
1  interface I {}
2  class A implements I {}
3  class B implements I {}
4  class C {}
5
6  let a: A = new A
7  let i: I = a
8  i as B // Will trigger a runtime error ``ClassCastException``
9  i as C // Compile-time error as C is not compatible with I
10 a as B // Compile-time error as B is not compatible with A
```

## 6.4.3 Casting Conversions from Object

Casting conversion from “*Object*” attempts to convert an expression of type `Object` to any reference type (see *Reference Types*) which is to be specified as *target type*.

```
1  function check(kind: string, o: Object)
2      switch (kind) {
3          case "bool": console.log(o as boolean); break
4          case "str" : console.log(o as string); break
```

(continues on next page)

(continued from previous page)

```

5     }
6 }

```

This conversion causes a runtime error (`ClassCastException`) if the runtime type of an expression is not the *target type*.

## 6.4.4 Casting Conversions from Type Parameter

*Casting conversion from a type parameter* attempts to convert an expression of the type parameter to any reference type (see *Reference Types*) which is to be specified as *target type*.

```

1  class X<S, T> {
2      method (p: T) {
3          p as Object           // OK
4          p as Object[]        // OK
5          p as [Object, Object] // OK
6          p as () => void       // OK
7          p as T                // OK
8          p as S                // OK
9          p as number // compile-time error, cast to value type
10         p as Number // OK use this cast instead of the cast above
11     }
12 }

```

This conversion causes a runtime error (`ClassCastException`) if the runtime type of an expression is not the *target type*.

## 6.4.5 Casting Conversions to Type Parameter

*Casting conversion to type parameter* attempts to convert an expression of any type to type parameter type (see *Type Parameters*) which is to be specified as *target type*.

```

1  function foo<T> (p: AnyType) {
2      p as T // attempt to convert of any type to type parameter
3  }

```

This conversion causes a runtime error (`ClassCastException`) if the runtime type of an expression is not the *target type*.

## 6.4.6 Casting Conversions from Union

*Casting conversion from union* converts an expression of union type to one of the types of the union, or to a type that is derived from such one type.

For union type  $U = T_1 \mid \dots \mid T_N$ , the *casting conversion from union* converts an expression of type  $U$  to some type  $TT$  (*target type*).

A compile-time error occurs if target type  $TT$  is not one of  $T_i$ , and not derived from one of  $T_i$ .

```

1  class Cat { sleep () {}; meow () {} }
2  class Dog { sleep () {}; bark () {} }
3  class Frog { sleep () {}; leap () {} }
4  class Spitz extends Dog { override sleep() { /* 18-20 hours a day */ } }
5
6  type Animal = Cat | Dog | Frog | number
7
8  let animal: Animal = new Spitz()
9  if (animal instanceof Frog) {
10     let frog: Frog = animal as Frog // Use casting conversion here
11     frog.leap() // Perform an action specific for the particular union type
12 }
13 if (animal instanceof Spitz) {
14     let dog = animal as Spitz // Use casting conversion here
15     dog.sleep()
16     // Perform an action specific for the particular union type derivative
17 }

```

This conversion cause a runtime error (`ClassCastException`) if the runtime type of an expression is not the *target type*.

Another form of *conversion from union* is implicit conversion from union type to the target type. The conversion is only possible if each type in a union is assignable (see [Assignability](#)) to the target type. If so, the conversion never causes a runtime error. The conversion causes a compile-time error if at least one type of a union is not assignable to the target type:

```

1  class Base {}
2  class Derived1 extends Base {}
3  class Derived2 extends Base {}
4
5  let d: Derived1 | Derived2 = ...
6  let b: Base = d // OK, as Derived1 and Derived2 are assignable to Base
7
8  let x: Double | Base = ...
9  let y: double = x // Compile-time error, as Base cannot be converted to double

```

## 6.4.7 Casting Conversions to Enumeration

Casting conversion can be used to convert a value of a numeric type expression into a value of *enumeration* type if the value:

- Can be converted into type `int`;
- Equals the value of an enumeration type constant.

Casting conversion can be used to convert a value of a string or string literal expression type into a value of *enumeration* type if the value:

- Can be converted into type `string`;
- Equals the value of an enumeration type constant.

If an expression is a constant, then rules of *Constant to Enumeration Conversions* apply.

If the expression value can be evaluated at compile-time, then the checks described above are performed at compile time. A compile-time error occurs if a check fails. Otherwise, the checks are performed during program execution. A runtime error occurs if a check fails.

```

1  enum IntegerEnum {a, b, c}
2
3  let e: IntegerEnum = 1 /* ok, e is set to IntegerEnum.b,
4                        constant to enumeration implicit conversion */
5
6  e = (1 + 1 + 1) as IntegerEnum /* compile-time error, there is no constant
7                                with this value */
8
9  let x = 1
10 e = x as IntegerEnum /* OK, as compiler can guarantee that enum
11                      consistency is not violated */
12
13 e = foo(false) as IntegerEnum // runtime check is required
14
15 function foo(some_condition: boolean) {
16     if (some_condition)
17         return 1 // Valid enum constant value
18     else
19         return 666 // Invalid enum constant value - will cause runtime error
20 }
21
22
23 enum StringEnum {a = "AA", b = "BBB", c = "C"}
24
25 let s: StringEnum = "BBB" as StringEnum /* ok, e is set to StringEnum.b,
26                                          constant to enumeration cast */
27
28 s = ("1" + "1" + "1") as StringEnum /* compile-time error, there is no constant
29                                      with this value */
30
31 let y = "C"
32 s = y as StringEnum /* OK, as compiler can guarantee that enum
33                      consistency is not violated */
34
35 s = bar(false) as StringEnum // runtime check is required
36
37 function bar(some_condition: boolean) {
38     if (some_condition)
39         return "AA" // Valid enum constant value
40     else
41         return "666" // Invalid enum constant value - will cause runtime error
42 }

```

## 6.5 Implicit Conversions

This section describes all implicit conversions that are allowed. Each conversion is allowed in a particular context (for example, if an expression that initializes a local variable is subject to *Assignment-like Contexts*, then the rules of this context define what specific conversion is implicitly chosen for the expression).

### 6.5.1 Primitive Types Conversions

*Primitive type conversion* is one of the following:

- *Widening Primitive Conversions*;
- *Constant Narrowing Integer Conversions*;
- *Boxing Conversions*;
- *Unboxing Conversions*.

### 6.5.2 Widening Primitive Conversions

*Widening primitive conversions* convert the following:

- Values of a smaller numeric type to a larger type (see *Numeric Types Hierarchy*);
- Values of type `byte` to type `char` (see *Character Type and Operations*);
- Values of type `char` to types `int`, `long`, `float`, and `double`;
- Values of an *enumeration* type to types `int`, `long`, `float`, and `double` (if enumeration constants of this type are of type `int`).

| From                     | To                                                                                                                          |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>byte</code>        | <code>short</code> , <code>int</code> , <code>long</code> , <code>float</code> , <code>double</code> , or <code>char</code> |
| <code>short</code>       | <code>int</code> , <code>long</code> , <code>float</code> , or <code>double</code>                                          |
| <code>int</code>         | <code>long</code> , <code>float</code> , or <code>double</code>                                                             |
| <code>long</code>        | <code>float</code> or <code>double</code>                                                                                   |
| <code>float</code>       | <code>double</code>                                                                                                         |
| <code>char</code>        | <code>int</code> , <code>long</code> , <code>float</code> , or <code>double</code>                                          |
| <code>enumeration</code> | <code>int</code> , <code>long</code> , <code>float</code> , or <code>double</code>                                          |

These conversions cause no loss of information about the overall magnitude of a numeric value. Some least significant bits of the value can be lost only in conversions from an integer type to a floating-point type if the IEEE 754 *round-to-nearest* mode is used correctly. The resultant floating-point value is properly rounded to the integer value.

*Widening primitive conversions* never cause runtime errors.



### 6.5.3 Constant Narrowing Integer Conversions

*Constant narrowing integer conversion* converts an expression of integer types or of type `char` to a value of a smaller integer type provided that:

- The expression is a constant expression (see *Constant Expressions*);
- The value of the expression fits into the range of the smaller type.

```

1  let b: byte = 127 // ok, int -> byte conversion
2  let c: char = 0x42E // ok, int -> char conversion
3  b = 128 // compile-time-error, value is out of range
4  b = 1.0 // compile-time-error, floating-point value cannot be converted
5
6  function foo (p: byte) {} // Version #1
7  function foo (p: number) {} // Version #2
8
9  foo (100) // Version #1 is called as int is safely narrowed into byte
10 foo (1000) // Version #2 is called as int is safely widened into double/number

```

These conversions never cause runtime errors.

### 6.5.4 Boxing Conversions

*Boxing conversions* handle primitive type expressions as expressions of a corresponding reference type.

If the unboxed *target type* is larger than the expression type, then a *widening primitive conversion* is performed as the first step of a *boxing conversion* of numeric types and type `char`. For example, a *boxing conversion* converts *i* of primitive value type `int` to a reference *n* of class type `Number` as follows:

```

1  let i: int = 1
2  let n: Number = i // int -> number -> Number
3
4  let c: char = 'a'
5  let l: Long = c // char -> long -> Long

```

*Boxing conversions* can cause `OutOfMemoryError` to be thrown if the storage available to create a new instance of the reference type is not sufficient.

### 6.5.5 Unboxing Conversions

*Unboxing conversions* handle reference type expressions as expressions of a corresponding primitive type.

If the *target type* is larger than the unboxed expression type, then a *widening primitive conversion* is performed as the second step of the *unboxing conversion* of numeric types and type `char`. For example, the *unboxing conversion* converts *i* of reference type `Int` to type `long` as follows:

```

1  let i: Int = 1
2  let l: long = i // Int -> int -> long

```

*Unboxing conversions* never cause runtime errors.

## 6.5.6 Widening Union Conversions

*Widening union conversion* can be of the following three options:

- Conversion from a union type to a wider union type;
- Conversion from a non-union type to a union type;
- Conversion from a union type that consists of literals only to a non-union type.

*Widening union conversions* never cause runtime errors.

Union type  $U (U_1 \mid \dots \mid U_n)$  can be converted to a different union type  $V (V_1 \mid \dots \mid V_m)$  if after normalization (see *Union Types Normalization*) the following is true:

**Note.** If union type normalization issues a single type or value, then this type or value is used instead of the initial set of union types or values. This concept is illustrated by the example below:

```

1  let u1: string | number | boolean = true
2  let u2: string | number = 666
3  u1 = u2 // OK
4  u2 = u1 // compile-time error as type of u1 is not compatible with type of u2
5
6  let u3: "1" | "2" | boolean = "3"
7      // compile-time error as there is no value "3" among values of u3 type
8
9  class Base {}
10 class Derived1 extends Base {}
11 class Derived2 extends Base {}
12
13 let u4: Base | Derived1 | Derived2 = new ...
14 let u5: Derived1 | Derived2 = new ...
15 u4 = u5 // OK, u4 type is Base after normalization and Derived1 and Derived2
16         // are compatible with Base as Note states
17 u5 = u4 // compile-time error as Base is not compatible with both
18         // Derived1 and Derived2

```

Non-union type  $T$  can be converted to union type  $U = U_1 \mid \dots \mid U_n$  if  $T$  is compatible with one of  $U_i$  types.

```

1  let u: number | string = 1 // ok
2  u = "aa" // ok
3  u = true // compile-time error

```

Union type  $U (U_1 \mid \dots \mid U_n)$  can be converted to non-union type  $T$  if each  $U_i$  is a literal that can be implicitly converted to type  $T$ .

```

1  let a: "1" | "2" = "1"
2  let b: string = a // ok, literals fit type 'string'

```

### 6.5.7 Widening Reference Conversions

*Widening reference conversion* handles any subtype (see *Subtyping*) as a supertype (see *Supertyping*). It requires no special action at runtime, and never causes an error.

```

1 interface BaseInterface {}
2 class BaseClass {}
3 interface DerivedInterface extends BaseInterface {}
4 class DerivedClass extends BaseClass implements BaseInterface
5 {}
6 function foo (di: DerivedInterface) {
7     let bi: BaseInterface = new DerivedClass() /* DerivedClass
8         is compatible with BaseInterface */
9     bi = di // DerivedInterface is compatible with BaseInterface
10 }

```

The only exception is the cast to type `never` that is forbidden. This cast is a compile-time error as it can cause type-safety violations:

```

1 class A { a_method() {} }
2 let a = new A
3 let n: never = a as never // compile-time error: no object may be assigned
4 // to a variable of the never type
5
6 class B { b_method() {} }
7 let b: B = n // OK as never is compatible with any type
8 b.b_method() /* this breaks type-safety if casting conversion to 'never'
9               is allowed */

```

### 6.5.8 Character to String Conversions

*Character to string conversion* converts a value of type `char` to type `string`. The length of the resultant new string equals 1. The converted `char` is the single element of the new string:

```

1 let c: char = c'X'
2 let s: string = c // s contains "X"

```

This conversion can cause `OutOfMemoryError` thrown if the storage available for the creation of a new string is not sufficient.

### 6.5.9 Constant String to Character Conversions

*Constant string to character conversion* converts an expression of type `string` to type `char`. The initial type string expression must be a constant expression (see *Constant Expressions*). The length of this expression equals 1. The resultant `char` is the first and only character of the converted `string`. This conversion never causes runtime errors.

## 6.5.10 Function Types Conversions

*Function types conversion* is the conversion of one function type to another. A *function types conversion* is valid if the following conditions are met:

- Parameter types are converted by using *contravariance* (*Invariance*, *Covariance* and *Contravariance*);
- Non-optional parameter types can be converted to the type of an optional parameter;
- Return types are converted by using *covariance* (*Invariance*, *Covariance* and *Contravariance*).

```

1  class Base {}
2  class Derived extends Base {}
3
4  type FuncTypeBaseBase = (p: Base) => Base
5  type FuncTypeBaseDerived = (p: Base) => Derived
6  type FuncTypeDerivedBase = (p: Derived) => Base
7  type FuncTypeDerivedDerived = (p: Derived) => Derived
8
9  function (
10     bb: FuncTypeBaseBase, bd: FuncTypeBaseDerived,
11     db: FuncTypeDerivedBase, dd: FuncTypeDerivedDerived
12 ) {
13     bb = bd
14     /* OK: identical (invariant) parameter types, and compatible return type */
15     bb = dd
16     /* Compile-time error: compatible parameter type (covariance), type unsafe */
17     db = bd
18     /* OK: contravariant parameter types, and compatible return type */
19 }
20
21 // Examples with lambda expressions
22 let foo1: (p: Base) => Base = (p: Base): Derived => new Derived()
23 /* OK: identical (invariant) parameter types, and compatible return type */
24
25 let foo2: (p: Base) => Base = (p: Derived): Derived => new Derived()
26 /* Compile-time error: compatible parameter type (covariance), type unsafe */
27
28 let foo3: (p: Derived) => Base = (p: Base): Derived => new Derived()
29 /* OK: contravariant parameter types, and compatible return type */
30
31 let foo4: (p?: Base) => void = (p: Base): void => {}
32 /* OK: Base is compatible with Base|undefined, and identical return type */
33
34 let foo5: (p: Base) => void = (p?: Base): void => {}
35 /* Compile-time error: as Base|undefined is not compatible with Base */

```

A function type with less parameters is compatible with another function type with more parameters.

```

1  let f: (p: number) => void = ():void => {} // OK
2  f(5)

```

Worth to mention that overriding is governed by *Override-Compatible Signatures* and example below leads to compile-time error:

```

1  class Base {
2      foo(p: (p: number)=> void) {}
3  }
4  class Derived extends Base {
5      override foo(p: ()=> void) {} // Compile-time error
6  }

```

### 6.5.11 Tuple Types Conversions

*Tuple types conversion* is the conversion of one tuple type to another.

Tuple type  $T = [T_1, T_2, \dots, T_n]$  can be converted to tuple type  $U = [U_1, U_2, \dots, U_m]$  if the following conditions are met:

- Tuple types have the same number of elements, thus  $n == m$ .
- Every  $T_i$  is identical to  $U_i$  for any  $i$  in  $1 \dots n$ .

### 6.5.12 Enumeration to Constants Type Conversions

A value of an *enumeration* type is converted to type `int` if enumeration constants of this type are of type `int`. This conversion never causes runtime errors.

```

1  enum IntegerEnum {a, b, c}
2  let int_enum: IntegerEnum = IntegerEnum.a
3  let int_value: int = int_enum // int_value will get the value of 0
4  let number_value: number = int_enum
5      /* number_value will get the value of 0 as a result of conversion
6      sequence: enumeration -> int -> number */

```

A value of enumeration type is converted to type `string` if enumeration constants of this type are of type `string`. This conversion never causes runtime errors.

```

1  enum StringEnum {a = "a", b = "b", c = "c"}
2  let string_enum: StringEnum = StringEnum.a
3  let a_string: string = string_enum // a_string will get the value of "a"

```

### 6.5.13 Constant to Enumeration Conversions

A constant expression of some integer type is converted to *enumeration* type if:

- Enumeration constants are of type `int`;
- Value of the constant expression is equal to the value of one of the enumeration type constants.

This conversion never causes runtime errors.

```
1  enum IntegerEnum {a, b, c}
2  let e: IntegerEnum = 1 // ok, e is set to IntegerEnum.b
3  e = 3 // compile-time error, there is no constant with this value
4
5  const one = 2
6  e = one // ok, e is set to IntegerEnum.c
```

Similar conversion of a string type expression is not supported as it is not part of TypeScript.

```
1  enum StringEnum {"a", "b", "c"}
2  let incorrect: StringEnum = "b" // compile-time error
3  let correct: StringEnum = StringEnum.b // OK
```

### 6.5.14 Literal Type to its Supertype Conversions

A value of `literal` type (see *Literal Types*) can always be converted to its supertype (see *Supertypes of Literal Types*). This conversion never causes a runtime error:

```
1  function foo(d: "string literal") {
2      let dd: string = d
3  }
4  foo("string literal")
```

The reverse conversion is not possible.

# CHAPTER 7

---

## Expressions

---

This chapter describes the meanings of expressions and the rules for the evaluation of expressions, except the expressions related to coroutines (see *Create and Launch a Coroutine* for launch expressions, and *Awaiting a Promise* for await expressions) and expressions described as experimental (see *Lambda Expressions with Receiver*).

```
expression:
  primaryExpression
  | castExpression
  | instanceofExpression
  | typeOfExpression
  | nullishCoalescingExpression
  | spreadExpression
  | unaryExpression
  | binaryExpression
  | assignmentExpression
  | conditionalExpression
  | stringInterpolation
  | lambdaExpression
  | lambdaExpressionWithReceiver
  | launchExpression
  | awaitExpression
/
primaryExpression:
  literal
  | namedReference
  | arrayLiteral
  | objectLiteral
  | recordLiteral
  | thisExpression
  | parenthesizedExpression
  | methodCallExpression
  | fieldAccessExpression
  | indexingExpression
  | functionCallExpression
  | newExpression
```

(continues on next page)

(continued from previous page)

```

    | ensureNotNullishExpression
    ;
binaryExpression:
    multiplicativeExpression
    | additiveExpression
    | shiftExpression
    | relationalExpression
    | equalityExpression
    | bitwiseAndLogicalExpression
    | conditionalAndExpression
    | conditionalOrExpression
    ;

```

The grammar rules below introduce several productions to be used by other expression rules:

```

objectReference:
    typeReference
    | 'super'
    | primaryExpression
    ;

```

`objectReference` refers to one of the following three options:

- Class that is to handle static members;
- `super` that is to access constructors declared in the superclass, or the overridden method version of the superclass;
- `primaryExpression` that is to refer to an instance variable of a class, interface, or function type after evaluation, unless the manner of the evaluation is altered by the chaining operator ‘?.’ (see [Chaining Operator](#)).

If the form of `primaryExpression` is `thisExpression`, then the pattern “`this?.`” is handled as a compile-time error.

If the form of `primaryExpression` is `super`, then the pattern “`super?.`” is handled as a compile-time error.

```

arguments:
    '(' argumentSequence? ')'
    ;

argumentSequence:
    restArgument
    | expression (',' expression)* (',' restArgument)? ','?
    ;

restArgument:
    '...'? expression
    ;

```

The `arguments` grammar rule refers to the list of arguments of a call. Only the last argument can have the form of a spread expression (see [Spread Expression](#)).



## 7.1 Evaluation of Expressions

The result of a program expression *evaluation* denotes the following:

- Variable (the term *variable* is used here in the general, non-terminological sense to denote a modifiable lvalue in the left-hand side of an assignment); or
- Value (results found elsewhere).

A variable or a value are equally considered the *value of the expression* if such a value is required for further evaluation.

The type of an expression is inferred at compile time (see *Contexts and Conversions*).

Expressions can contain assignments, increment operators, decrement operators, method calls, and function calls. The evaluation of an expression can produce side effects as a result.

*Constant expressions* (see *Constant Expressions*) are the expressions with values that can be determined at compile time.

### 7.1.1 Normal and Abrupt Completion of Expression Evaluation

Each expression in a normal mode of evaluation requires certain computational steps. Normal modes of evaluation for each kind of expression are described in the following sections.

An expression evaluation *completes normally* if all computational steps are performed without throwing an error.

On the contrary, an expression *completes abruptly* if the expression evaluation throws an error.

The information about the causes of an abrupt completion can be available in the value attached to the error object.

Runtime errors can occur as a result of expression or operator evaluation as follows:

- If an *array reference expression* has the value `null`, then an *array indexing expression* (see *Array Indexing Expression*) throws `NullPointerException`.
- If the value of an array index expression is negative, or greater than, or equal to the length of the array, then an *array indexing expression* (see *Array Indexing Expression*) throws `ArrayIndexOutOfBoundsException`.
- If a conversion cannot be performed at runtime, then a *cast expression* (see *Cast Expressions*) throws `ClassCastException`.
- If the right-hand operand expression has the zero value, then integer division (see *Division*), or integer remainder (see *Remainder*) operators throw `ArithmeticError`.
- If the boxing conversion (see *Boxing Conversions*) occurs while performing an assignment to an array element of a reference type, then a method call expression (see *Method Call Expression*), or prefix/postfix increment/decrement (see *Unary Expressions*) operators can throw `OutOfMemoryError`.
- If the type of an array element is not compatible with the value that is being assigned, then assignment to an array element of a reference type throws `ArrayStoreError`.

Possible hard-to-predict and hard-to-handle linkage and virtual machine errors can cause errors during the evaluation of an expression.

Abrupt completion of the evaluation of a subexpression results in the following:

- Immediate abrupt completion of the expression that contains such a subexpression (if the evaluation of the contained subexpression is required for the evaluation of the entire expression); and

- Cancellation of all subsequent steps of the normal mode of evaluation.

The terms *complete normally* and *complete abruptly* can also denote normal and abrupt completion of the execution of statements (see [Normal and Abrupt Statement Execution](#)). A statement can complete abruptly for a variety of reasons in addition to an error being thrown.

## 7.1.2 Order of Expression Evaluation

The operands of an operator are evaluated from left to right in accordance with the following rules:

- Any right-hand operand is evaluated only after the left-hand operand of a binary operator is fully evaluated.

If using a compound-assignment operator (see [Simple Assignment Operator](#)), the evaluation of the left-hand operand includes the following:

- Remembering the variable denoted by the left-hand operand;
- Fetching the value of that variable for the subsequent evaluation of the right-hand operand; and
- Saving such a value.

If the evaluation of the left-hand operand completes abruptly, then no part of the right-hand operand is evaluated.

- Any part of the operation can be executed only after every operand of an operator (except conditional operators ‘&&’, ‘||’, and ‘?:’) is fully evaluated.

The execution of a binary operator that is an integer division ‘/’ (see [Division](#)), or integer remainder ‘%’ (see [Remainder](#)) can throw `ArithmeticError` only after the evaluations of both operands complete normally.

- The ArkTS programming language follows the order of evaluation as indicated explicitly by parentheses, and implicitly by the precedence of operators. This rule particularly applies for infinity and NaN values of floating-point calculations. ArkTS considers integer addition and multiplication as provably associative; however, floating-point calculations must not be naively reordered because they are unlikely to be computationally associative (even though they appear mathematically associative).

## 7.1.3 Operator Precedence

The table below summarizes all information on the precedence and associativity of operators. Each section on a particular operator also contains detailed information.

| Operator                                      | Precedence                             | Assoc-ty      |
|-----------------------------------------------|----------------------------------------|---------------|
| postfix increment and decrement               | ++ --                                  | left to right |
| prefix increment and decrement, unary, typeof | ++ -- + - ! ~ typeof                   | right to left |
| multiplicative                                | * / %                                  | left to right |
| additive                                      | + -                                    | left to right |
| cast                                          | as                                     | left to right |
| shift                                         | << >> >>>                              | left to right |
| relational                                    | < > <= >= instanceof                   | left to right |
| equality                                      | == !=                                  | left to right |
| bitwise AND                                   | &                                      | left to right |
| bitwise exclusive OR                          | ^                                      | left to right |
| bitwise inclusive OR                          |                                        | left to right |
| logical AND                                   | &&                                     | left to right |
| logical OR                                    |                                        | left to right |
| null-coalescing                               | ??                                     | left to right |
| ternary                                       | ? :                                    | right to left |
| assignment                                    | = += -= %= *= /= &= ^=  = <<= >>= >>>= | right to left |

### 7.1.4 Evaluation of Arguments

An evaluation of arguments always progresses from left to right up to the first error, or through the end of the expression; i.e., any argument expression is evaluated after the evaluation of each argument expression to its left completes normally (including comma-separated argument expressions that appear within parentheses in method calls, constructor calls, class instance creation expressions, or function call expressions).

If the left-hand argument expression completes abruptly, then no part of the right-hand argument expression is evaluated.

## 7.1.5 Evaluation of Other Expressions

These general rules cannot cover the order of evaluation of certain expressions when they from time to time cause exceptional conditions. The order of evaluation of the following expressions requires specific explanation:

- Class instance creation expressions (see *New Expressions*);
- *Array Creation Expressions*;
- *Indexing Expressions*;
- Method call expressions (see *Method Call Expression*);
- Assignments involving indexing (see *Assignment*);
- *Lambda Expressions*.

## 7.2 Literal

Literals (see *Literals*) denote fixed and unchanging values. Type of a literal (see *Literals*) is the type of an expression.

## 7.3 Named Reference

An expression can have the form of a *named reference* as described by the syntax rule as follows:

```
namedReference:  
    qualifiedName typeArguments?  
    ;
```

Type of a *named reference* expression is the type of the entity a *named reference* refers to.

*QualifiedName* (see *Names*) is an expression that consists of dot-separated names. If *qualifiedName* consists of a single identifier, then it is called a *simple name*.

*Simple name* refers to the following:

- Entity declared in the current compilation unit;
- Local variable or parameter of the surrounding function or method.

If not a *simple name*, *qualifiedName* refers to the following:

- Entity imported from a compilation unit,
- Entity exported from a namespace, or
- Member of some class or interface.

If *typeArguments* are provided, then *qualifiedName* is a valid instantiation of the generic method or function. Otherwise, a compile-time error occurs.

A compile-time error also occurs in the following situations:

- If a name referred by *qualifiedName* is undefined or inaccessible; or
- If ambiguity occurs while resolving a name except the function or method overloading case (see *Function, Method and Constructor Overloading*).

Type of a *named reference* is the type of an expression.

**Note.** When a *named reference* is of a function type, then a new function object is always created as specified in *Uniqueness of Functional Objects*.

```

1  import * as compilationUnitName from "someFile"
2
3  class Type {}
4
5
6  function foo (parameter: Type) {
7      let local: Type = parameter /* 'parameter' here is the
8          expression in the form of simple name, type of 'parameter' is the
9          explicitly declared function parameter type */
10     local = new Type () /* 'local' here is the expression in the
11         form of simple name */
12     local = compilationUnitName.someExportedVariable /* qualifiedName here
13         refers to a variable imported from a compilation unit */
14     let func = foo /* foo is a simple name of the function declared in this
15         module, type of 'func' is the function type derived from the function
16         'foo()' signature. func itself is a new function object */
17
18     goo() // goo is an undefined name - compile-time error
19     let bar_ref = bar // bar is an ambiguous reference - compile-time error
20 }
21
22 function bar (p: string) {}
23 function bar (p: number) {}
24
25 function generic_function<T> () {}
26 let instantiation = generic_function<string> /* type of 'instantiation' is
27     a function type derived from the signature of instantiated function
28     'generic_function<string> ()' */

```

## 7.4 Array Literal

*Array literal* is an expression that can be used to create an array or tuple in some cases, and to provide some initial values:

```

arrayLiteral:
    '[' expressionSequence? ']'
    ;

expressionSequence:
    expression (',' expression)* ','?
    ;

```

An *array literal* is a comma-separated list of *initializer expressions* enclosed in '[' and ']'. A trailing comma after the last expression in an array literal is ignored:

```
1 let x = [1, 2, 3] // ok
2 let y = [1, 2, 3,] // ok, trailing comma is ignored
```

The number of initializer expressions enclosed in braces of the array initializer determines the length of the array to be constructed.

If sufficient space is allocated for a new array, then a one-dimensional array of the specified length is created. All elements of the array are initialized to the values specified by initializer expressions.

On the contrary, the evaluation of the array initializer completes abruptly in the following situations:

- If the space allocated for a new array is insufficient, and `OutOfMemoryError` is thrown; or
- If some initialization expression completes abruptly.

Initializer expressions are executed from left to right. The  $n$ 'th expression specifies the value of the  $n$ -1'th element of the array.

Array literals can be nested (i.e., the initializer expression that specifies an array element can be an array literal if that element is of array type).

Type of an *array literal expression* is inferred by the following rules:

- If a context is available, then type is inferred from the context. If successful, then type of an array literal is the inferred type `T[], Array<T>`, or tuple.
- Otherwise, type is to be inferred from the types of array literal elements.

More details of both cases are presented below.

## 7.4.1 Array Literal Type Inference from Context

Type of an array literal can be inferred from the context, including explicit type annotation of a variable declaration, left-hand part of an assignment, call parameter type, or type of a cast expression:

```
1 let a: number[] = [1, 2, 3] // ok, variable type is used
2 a = [4, 5] // ok, variable type is used
3
4 function min(x: number[]): number {
5     let m = x[0]
6     for (let v of x)
7         if (v < m)
8             m = v
9     return m
10 }
11 min([1., 3.14, 0.99]); // ok, parameter type is used
12
13 // Two-dimensional array initialization
14 type Matrix = number[][]
15 let m: Matrix = [[1, 2], [3, 4], [5, 6]]
16
17 class aClass {}
18 let b1: Array<aClass> = [new aClass, new aClass]
```

(continues on next page)

(continued from previous page)

```

19 let b2: Array <Number> = [1, 2, 3]
20 let b3: FixedArray<number> = [1, 2]
21   /* Type of literal is inferred from the context
22      taken from b1, b2 and b3 declarations */

```

All valid conversions are applied to the initializer expression, i.e., each initializer expression type must be assignable (see [Assignability](#)) to the array element type. Otherwise, a compile-time error occurs.

```

1 let value: number = 2
2 let list: Object[] = [1, value, "hello", new Error()] // ok

```

In the example above, the first literal and ‘value’ are implicitly boxed to `Number`, and types of a string literal and the instance of type `Error` are assignable (see [Assignability](#)) to `Object` because the corresponding classes are inherited from `Object`.

**Note.** There are cases when every array or tuple element of primitive type is boxed (see [Boxing Conversions](#)) according to the type of the context that creates a new array or tuple. Such transformation is performed at compile time to ensure that the new array or tuple fits the type of the context.

```

1 let array: Number[] = [1, 2, 3] // assignment context
2 function foo (array: Number[]) {}
3 foo ([1, 2, 3]) // call context
4 [1, 2, 3] as Number[] // casting conversion
5
6 let tuple: [Number, Boolean] = [1, true] // assignment context
7 bar ([1, true]) // call context
8 function bar (tuple: [Number, Boolean]) {}
9 [1, true] as [Number, Boolean] // casting conversion

```

If the type used in the context is a *tuple type* (see [Tuple Types](#)), and types of all literal expressions are compatible with tuple type elements at respective positions, then an array literal is of tuple type.

```

1 let tuple: [number, string] = [1, "hello"] // ok
2
3 let incorrect: [number, string] = ["hello", 1] // compile-time error

```

If the type used in the context is a *union type* (see [Union Types](#)), then it is necessary to try inferring the type of the array literal from its elements (see [Array Type Inference from Types of Elements](#)). If successful, then the type so inferred must be compatible with one of the types that form the union type. Otherwise, a compile-time error occurs:

```

1 let union_of_arrays: number[] | string[] = [1, 2] // OK, type of literal is number[]
2 let incorrect_union_of_arrays: number[] | string[] = [1, 2, "string"]
3   /* compile-time error: (number|string)[] (type of the literal) is not compatible_
4   ↪with
5     number[] | string[] (type of the variable)
6   */

```

If the type used in the context is a *fixed array type* (see [Fixed Array Types](#)), and each initializer expression type is compatible with the array element type, then an array literal is of *fixed array type*.

```

1 let farray: FixedArray<number> = [1, 2]

```

## 7.4.2 Array Type Inference from Types of Elements

When type of an array literal `[ expr1, ..., exprN ]` cannot be inferred from the context, then the following algorithm is used to infer it from initialization expressions:

1. If there are no elements in the array literal ( $N == 0$ ), then type of the array literal cannot be inferred, and a compile-time error occurs.
2. If type of at least one of element expression cannot be determined, then type of the array literal cannot be inferred, and a compile-time error occurs.
3. If each initialization expression is of numeric type (see [Numeric Types](#)), then the type of the array literal is `number[]`.
4. If all initialization expressions are of the same type `T`, then the type of the array literal is `T[]`.
5. Otherwise, the array literal type is an array of elements of the union type which is constructed as a union of  $T_1 \mid \dots \mid T_N$ , where  $T_i$  is the type of `expri`. Union type normalization (see [Union Types Normalization](#)) is applied to this union type.

```

1  let a = []                // compile-time error, type cannot be inferred
2  let b = ["a"]             // type is string[]
3  let c = [1, 2, 3]         // type is number[]
4  let d = ["a", 1, 3.14]    // type is (string | number)[]
5  let e = [(): void => {}, new A()] // type is (() => void | A)[]

```

## 7.5 Object Literal

*Object literal* is an expression that can be used to create a class instance and to provide some initial values. In some cases it is more convenient to use an *object literal* in place of a class instance creation expression (see [New Expressions](#)):

```

objectLiteral:
    '{' valueSequence? '}'
    ;

valueSequence:
    nameValue (',' nameValue)* ','?
    ;

nameValue:
    identifier ':' expression
    ;

```

An *object literal* is written as a comma-separated list of *name-value pairs* enclosed in curly braces `{` and `}`. A trailing comma after the last pair is ignored. Each *name-value pair* consists of an identifier and an expression:

```

1  class Person {
2      name: string = ""
3      age: number = 0
4  }
5  let b : Person = {name: "Bob", age: 25}
6  let a : Person = {name: "Alice", age: 18, } //ok, trailing comma is ignored

```



Type of an *object literal expression* is always some class *C* that is inferred from the context. A type inferred from the context can be either a named class (see *Object Literal of Class Type*), or an anonymous class created for the inferred interface type (see *Object Literal of Interface Type*).

A compile-time error occurs if:

- Type of an object literal cannot be inferred from the context; or
- The inferred type is not a class or interface type.
- The inferred type has abstract methods (see *Abstract Methods*). **Note.** An abstract class without abstract methods can be used.

```
1 let p = {name: "Bob", age: 25}
2      // compile-time error, type cannot be inferred
```

### 7.5.1 Object Literal of Class Type

If class type *C* is inferred from the context, then type of an object literal is *C*:

```
1 class Person {
2     name: string = ""
3     age: number = 0
4 }
5 function foo(p: Person) { /*some code*/ }
6 // ...
7 let p: Person = {name: "Bob", age: 25} /* ok, variable type is
8     used */
9 foo({name: "Alice", age: 18}) // ok, parameter type is used
```

An identifier in each *name-value pair* must name a field of class *C*, or a field of any superclass of class *C*.

A compile-time error occurs if the identifier does not name an *accessible member field* (see *Accessible*) in type *C*:

```
1 class Friend {
2     name: string = ""
3     private nick: string = ""
4     age: number
5     sex?: "male"|"female"
6 }
7 // compile-time error, nick is private:
8 let f: Friend = {name: "Alexander", age: 55, nick: "Alex"}
```

A compile-time error occurs if type of an expression in a *name-value pair* is not assignable (see *Assignability*) to the field type:

```
1 let f: Friend = {name: 123} /* compile-time error - type of right hand-side
2     is not assignable to the type of the left hand-side */
```

If some class fields have default values (see *Default Values for Types*) or explicit initializers (see *Variable and Constant Declarations*), then such fields can be skipped in the object literal.

```

1  let f: Friend = {} /* OK, as name, nick, age, and sex have either default
2                      value or explicit initializer */

```

If an object literal is to use class *C*, then class *C* must have a *parameterless* constructor (explicit or default) that is *accessible* (see [Accessible](#)) in the class-composite context.

A compile-time error occurs if:

- *C* contains no parameterless constructor; or
- No constructor is accessible (see [Accessible](#)).

These situations are presented in the examples below:

```

1  class C {
2      constructor (x: number) {}
3  }
4  // ...
5  let c: C = {} /* compile-time error - no parameterless
6                constructor */

```

```

1  class C {
2      private constructor () {}
3  }
4  // ...
5  let c: C = {} /* compile-time error - constructor is not
6                accessible */

```

If a class has accessors (see [Accessor Declarations](#)) for a property, and its setter is provided, then this property can be used as a part of an object literal. Otherwise, a compile-time error occurs:

```

1  class OK {
2      set attr (attr: number) {}
3  }
4  const a: OK = {attr: 666} // OK, as the setter be called
5
6  class CTE {
7      get attr (): number { return 666 }
8  }
9  const b: CTE = {attr: 666} // compile-time error - no setter for 'attr'

```

## 7.5.2 Object Literal of Interface Type

If the interface type *I* is inferred from the context, then type of an object literal is an anonymous class implicitly created for interface *I*:

```

1  interface Person {
2      name: string
3      age: number
4  }
5  let b: Person = {name: "Bob", age: 25}

```

In the example above, type of *b* is an anonymous class that contains the same fields as the interface *I* properties.

If some interface properties are of an optional type, then such properties can be skipped in an object literal as their default values are *undefined* (see [Default Values for Types](#)) according to the union type nature (see [Variable Declarations](#)):

```

1 interface Person {
2   name: string
3   age: number
4   sex?: "male" | "female"
5 }
6 let b: Person = {name: "Bob", age: 25}
7   // 'sex' field will have 'undefined' value

```

The interface type *I* must contain properties only. A compile-time error occurs if interface type *I* contains a method:

```

1 interface I {
2   name: string
3   foo()
4 }
5 let i : I = {name: "Bob"} // compile-time error, interface has methods

```

If an interface has accessors (see [Accessor Declarations](#)) for some property, and the property is used in an object literal, then a compile-time error occurs:

```

1 interface I1 {
2   set attr (attr: number)
3 }
4 const a: I1 = {attr: 666} /* compile-time error - 'attr' cannot be used
5                           in object literal */
6
7 interface I2 {
8   get attr (): number
9 }
10 const b: I2 = {attr: 666} /* compile-time error - 'attr' cannot be used
11                           in object literal */

```

### 7.5.3 Object Literal of Record Type

Generic type `Record<Key, Value>` (see [Record Utility Type](#)) is used to map properties of a type (type *Key*) to another type (type *Value*). A special form of object literal is used to initialize the value of such type:

```

recordLiteral:
  '{' keyValueSequence? '}'
  ;

keyValueSequence:
  keyValue (',' keyValue)* ','?
  ;

keyValue:
  expression ':' expression
  ;

```

The first expression in `keyValue` denotes a key and must be of type `Key`. The second expression denotes a value and must be of type `Value`:

```
let map: Record<string, number> = {  
    "John": 25,  
    "Mary": 21,  
}  
  
console.log(map["John"]) // prints 25
```

```
interface PersonInfo {  
    age: number  
    salary: number  
}  
  
let map: Record<string, PersonInfo> = {  
    "John": { age: 25, salary: 10},  
    "Mary": { age: 21, salary: 20}  
}
```

If a key is a union type consisting of literals or enum type, then all variants must be listed in the object literal. Otherwise, a compile-time error occurs:

```
let map: Record<"aa" | "bb", number> = {  
    "aa": 1,  
} // compile-time error: "bb" key is missing  
  
enum Enum {A, B}  
const map1: Record<Enum, number> = {  
    Enum.A: 1  
} // compile-time error: key "Enum.b" is missing
```

## 7.5.4 Object Literal Evaluation

The evaluation of an object literal of type `C` (where `C` is either a named class type or an anonymous class type created for the interface) is to be performed by the following steps:

- A parameterless constructor is executed to produce an instance  $x$  of class `C`. The execution of the object literal completes abruptly if so does the execution of the constructor.
- Name-value pairs of the object literal are then executed from left to right in the textual order they occur in the source code. The execution of a name-value pair includes the following:
  - Evaluation of the expression; and
  - Assignment of the value of expression to the corresponding field of  $x$  as its initial value. This rule also applies to *readonly* fields.

The execution of an object literal completes abruptly if so does the execution of a name-value pair.

An object literal completes normally with the value of a newly initialized class instance if so do all name-value pairs.

## 7.6 Spread Expression

```
spreadExpression:
    '...' expression
    ;
```

*Spread expression* can be used only within an array literal (see *Array Literal*) or argument passing. The *expression* must be of array type (see *Array Types*) or tuple type (see *Tuple Types*). Otherwise, a compile-time error occurs.

A *spread expression* for arrays or tuples can be evaluated as follows:

- By the compiler at compile time if *expression* is constant (see *Constant Expressions*);
- At runtime otherwise.

An array or tuple referred by the *expression* is broken by the evaluation into a sequence of values. This sequence is used where a *spread expression* is used. It can be an assignment, a call of a function, method, or constructor. A sequence of types of these values is the type of the *spread expression*.

```
1  let array1 = [1, 2, 3]
2  let array2 = [4, 5]
3  let array3 = [...array1, ...array2] // spread array1 and array2 elements
4      // while building new array literal during compile-time
5  console.log(array3) // prints [1, 2, 3, 4, 5]
6
7  foo (...array2) // spread array2 elements into arguments of the foo() call
8  function foo (...array: number[]) {
9      console.log (array)
10 }
11
12 run_time_spread_application1 (array1, array2) // prints [1, 2, 3, 666, 4, 5]
13 function run_time_spread_application1 (a1: number[], a2: number[]) {
14     console.log ([...a1, 666, ...a2])
15     // array literal will be built at runtime
16 }
17
18 let tuple1: [number, string, boolean] = [1, "2", true]
19 let tuple2: [number, string] = [4, "5"]
20 let tuple3: [number, string, boolean, number, string] = [...tuple1, ...tuple2] //
↳spread tuple1 and tuple2 elements
21     // while building new tuple object during compile-time
22 console.log(tuple3) // prints [1, "2", true, 4, "5"]
23
24 bar (...tuple2) // spread tuple2 elements into arguments of the foo() call
25 function bar (...tuple: [number, string]) {
26     console.log (tuple)
27 }
28
29 run_time_spread_application2 (tuple1, tuple2) // prints [1, "2", true, 666, 4, "5"]
30 function run_time_spread_application2 (a1: [number, string, boolean], a2: [number,
↳string]) {
31     console.log ([...a1, 666, ...a2])
32     // such array literal will be built at runtime
33 }
```

**Note.** If an array is spread while calling a function, then an appropriate parameter must be of the spread array kind. If an array is spread into a sequence of ordinary parameters, then a compile-time error occurs:

```
1 let an_array = [1, 2]
2 bar (...an_array) // compile-time error
3 function bar (n1: number, n2: number) { ... }
```

**Note.** If a tuple is spread while calling a function, then an appropriate parameter must be of the spread tuple kind. If a tuple is spread into a sequence of ordinary parameters, then a compile-time error occurs:

```
1 let a_tuple: [number, string] = [1, "2"]
2 bar (...a_tuple) // compile-time error
3 function bar (n1: number, n2: string) { ... }
```

## 7.7 Parenthesized Expression

```
parenthesizedExpression:
    '(' expression ')'
    ;
```

Type and value of a parenthesized expression are the same as those of the contained expression.

## 7.8 this Expression

```
thisExpression:
    'this'
    ;
```

The keyword `this` can be used as an expression in the body of an instance method of a class (see [Method Body](#)) or interface (see [Default Interface Method Declarations](#)). The type of `this` expression is the appropriate class or interface type.

It can be used in a lambda expression only if it is allowed in the context in which the lambda expression occurs.

The keyword `this` in a *direct call* `this( arguments )` expression can only be used in the explicit constructor call statement (see [Explicit Constructor Call](#)).

The keyword `this` can also be used in the body of a function with receiver (see [Functions with Receiver](#)). The type of `this` expression is the declared type of the parameter `this` in a function.

A compile-time error occurs if the keyword `this` appears elsewhere.

The keyword `this` used as a primary expression denotes a value that is a reference to the following:

- Object for which the instance method is called; or
- Object being constructed.

The parameter `this` in a lambda body and in the surrounding context denote the same value.

The class of the actual object referred to at runtime can be  $T$  if  $T$  is a class type, or a class assignable (see [Assignability](#)) to  $T$ .

## 7.9 Field Access Expression

*Field access expression* can access a field of an object to which an object reference refers. The object reference can have different forms as described in detail in [Accessing Current Object Fields](#) and in [Accessing SuperClass Properties](#).

```
fieldAccessExpression:
  objectReference ('.' | '?.') identifier
  ;
```

A *field access expression* that contains `'?.'` (see [Chaining Operator](#)) is called *safe field access* because it handles nullish object references safely.

If object reference evaluation completes abruptly, then so does the entire field access expression.

An object reference used to access a field must be a non-nullish reference type  $T$ . Otherwise, a compile-time error occurs.

A field access expression is valid if the identifier refers to an accessible member field (see [Accessible](#)) in type  $T$ . A compile-time error occurs otherwise.

Type of a *field access expression* is the type of a member field.

### 7.9.1 Accessing Current Object Fields

The result of a field access expression is computed at runtime as described below.

- a. *Static field access* (*objectReference* is evaluated in the form *typeReference*)

The evaluation of *typeReference* is performed. The result of a *field access expression* of a static field in a class is as follows:

- `variable` if the field is not `readonly`. The resultant value can be changed later.
- `value` if the field is `readonly`, except where *field access* occurs in a class initializer (see [Class Initializer](#)).

- b. *Instance field access* (*objectReference* is evaluated in the form *primaryExpression*)

The evaluation of *primaryExpression* is performed. The result of *field access expression* of an instance field in a class or interface is as follows:

- `variable` if the field is not `readonly`. The resultant value can be changed later.
- `value` if the field is `readonly`, except where *field access* occurs in a constructor (see [Constructor Declaration](#)).

Only the *primaryExpression* type (not class type of an actual object referred at runtime) is used to determine the field to be accessed.

## 7.9.2 Accessing SuperClass Properties

The form `super.identifier` is valid when accessing the superclass property via accessor (see [Accessor Declarations](#)). A compile-time error occurs if `identifier` in ‘`super.identifier`’ denotes a field.

```
1  class Base {  
2      get property(): number { return 1 }  
3      set property(p: number) { }  
4      field = 1234  
5  }  
6  class Derived extends Base {  
7      get property(): number { return super.property } // OK  
8      set property(p: number) { super.property = 666 } // OK  
9      foo () {  
10         super.field = 666           // compile-time error  
11         console.log (super.field)  // compile-time error  
12     }  
13 }
```

## 7.10 Method Call Expression

A *method call expression* calls a static or instance method of a class or an interface.

```
methodCallExpression:  
    objectReference ('.' | '?.') identifier typeArguments? arguments block?  
    ;
```

The syntax form that has a block associated with the method call is a special form called *trailing lambda call* (see [Trailing Lambdas](#) for details).

A method call with ‘`?.`’ (see [Chaining Operator](#)) is called a *safe method call* because it handles nullish values safely.

Resolving a method at compile time is more complicated than resolving a field because method overloading (see [Class Method Overloading](#)) can occur.

There are several steps that determine and check the method to be called at compile time (see [Step 1: Selection of Type to Use](#), [Step 2: Selection of Method](#), and [Step 3: Checking Method Modifiers](#)).

### 7.10.1 Step 1: Selection of Type to Use

The *object reference* is used to determine the type in which to search for the method. Three forms of *object reference* are possible:



| Form of Object Reference    | Type to Use                                                                                                                                                                               |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>typeReference</code>  | Type denoted by <code>typeReference</code> .                                                                                                                                              |
| expression of type <i>T</i> | <i>T</i> if <i>T</i> is a class, interface, or union; <i>T</i> 's constraint ( <i>Type Parameter Constraint</i> ) if <i>T</i> is a type parameter. A compile-time error occurs otherwise. |
| <code>super</code>          | The superclass of the class that contains the method call.                                                                                                                                |

### 7.10.2 Step 2: Selection of Method

After the type to use is known, the method to call must be determined. ArkTS supports overloading, and more than one method can be accessible under the method name used in the call.

All accessible methods are called *potentially applicable candidates*, and *Overload Resolution* is used to select the method to call. If *overload resolution* can definitely select a single method, then this method is called. Otherwise, a compile-time error occurs as more than one applicable method is available (no method to call, or ambiguity).

### 7.10.3 Step 3: Checking Method Modifiers

In this step, the single method to call is known, and the following set of semantic checks must be performed:

- If the method call has the form `typeReference.identifier`, then `typeReference` refers to a class, and the method must be declared `static`. Otherwise, a compile-time error occurs.
- If the method call has the form `expression.identifier`, then the method must not be declared `static`. Otherwise, a compile-time error occurs.
- If the method call has the form `super.identifier`, then the method must not be declared `abstract` or `static`. Otherwise, a compile-time error occurs.

### 7.10.4 Type of Method Call Expression

Type of a *method call expression* is the return type of the method.

```

1  class A {
2      static method() { console.log ("Static method() is called") }
3      method()      { console.log ("Instance method() is called") }
4  }
5
6
7  let x = A.method()    // compile-time error as void cannot be used as type_
↪annotation
8  A.method ()          // OK

```

(continues on next page)

(continued from previous page)

```

9  let y = new A.method() // compile-time error as void cannot be used as type_
   ↪ annotation
10 new A.method()         // OK

```

## 7.11 Function Call Expression

*Function call expression* is used to call a function (see *Function Declarations*), a variable of a function type (*Function Types*), or a lambda expression (see *Lambda Expressions*):

```

functionCallExpression:
    expression ('?.' | typeArguments)? arguments block?
    ;

```

A special syntactic form that contains a block associated with the function call is called *trailing lambda call* (see *Trailing Lambdas* for details).

A compile-time error occurs if the expression type is one of the following:

- Different than the function type;
- Nullish type but without '?.' (see *Chaining Operator*).

If the operator '?.' (see *Chaining Operator*) is present, and the *expression* evaluates to a nullish value, then:

- *Arguments* are not evaluated;
- Call is not performed; and
- Result of *functionCallExpression* is not produced as a consequence.

The function call is *safe* because it handles nullish values properly.

The following important situations depend on the form of expression in a call, and require different semantic checks:

- The form of expression in the call is *qualifiedName*, and *qualifiedName* refers to an accessible function (*Function Declarations*), or to a set of accessible overloaded functions.

In this case, all accessible functions are *potentially applicable candidates*. *Overload Resolution* is used to select the function to call. If *overload resolution* can definitely select a single function, then this function is called. Otherwise (i.e., if there is no function to call, or if there is ambiguity caused where more than one applicable function is available), a compile-time error occurs.

- All other forms of expression.

In this case, *overload resolution* is not required as the expression determines the entity to call unambiguously. Semantic check is performed in accordance with *Compatibility of Call Arguments*.

The example below represents different forms of function calls:

```

1  function foo() { console.log ("Function foo() is called") }
2  foo() // function call uses function name to call it
3
4  call (foo) // top-level function passed
5  call (() : void => { console.log ("Lambda is called") }) // lambda is passed
6  call (A.method) // static method

```

(continues on next page)

(continued from previous page)

```

7  call ((new A).method) // instance method is passed
8
9  class A {
10     static method() { console.log ("Static method() is called") }
11     method() { console.log ("Instance method() is called") }
12 }
13
14 function call (callee: () => void) {
15     callee() // function call uses parameter name to call any functional object
16     ↪passed as an argument
17 }
18
19 (() : void => { console.log ("Lambda is called") }) () // function call uses lambda
20 ↪expression to call it
21
22 let x = foo() // compile-time error as void cannot be used as type annotation

```

Type of a *function call expression* is the return type of the function.

## 7.12 Indexing Expressions

*Indexing expressions* are used to access elements of arrays (see [Array Types](#)) and Record instances (see [Record Utility Type](#)). Indexing expressions can also be applied to instances of indexable types (see [Indexable Types](#)):

```

indexingExpression:
    expression ('?.')? '[' expression ']'
    ;

```

Any *indexing expression* has two subexpressions as follows:

- *Object reference expression* before the left bracket; and
- *Index expression* inside the brackets.

If the operator '?.' (see [Chaining Operator](#)) is present in an indexing expression, then:

- If an object reference expression is not of a nullish type, then the chaining operator has no effect.
- Otherwise, object reference expression must be checked to nullish value. If the value is undefined or null, then the evaluation of the entire surrounding *primary expression* stops. The result of the entire *primary expression* is then undefined.

If no '?.' is present in an indexing expression, then object reference expression must be an array type or the Record type. Otherwise, a compile-time error occurs.

### 7.12.1 Array Indexing Expression

*Index expression* for array indexing must be of a numeric type (see [Numeric Types](#)).

If an *index expression* is of type `number` or other floating-point type, and the fractional part differs from 0, then errors occur as follows:

- A runtime error, if the situation is identified during program execution; and
- A compile-time error, if the situation is detected during compilation.

A numeric types conversion (see *Primitive Types Conversions*) is performed on an *index expression* to ensure that the resultant type is `int`. Otherwise, a compile-time error occurs.

If the chaining operator ‘`?.`’ (see *Chaining Operator*) is present, and after its application the type of *object reference expression* is an *array type*, then it makes a valid *array reference expression*, and the type of the array indexing expression is `T`.

The result of an array indexing expression is a variable of type `T` (i.e., an element of the array selected by the value of that *index expression*).

It is essential that, if type `T` is a reference type, then the fields of array elements can be modified by changing the resultant variable fields:

```

1  let names: string[] = ["Alice", "Bob", "Carol"]
2  console.log(name[1]) // prints Bob
3  string[1] = "Martin"
4  console.log(name[1]) // prints Martin
5
6  class RefType {
7      field: number = 666
8  }
9  const objects: RefType[] = [new RefType(), new RefType()]
10 const object = objects [1]
11 object.field = 777           // change the field in the array element
12 console.log(objects[0].field) // prints 666
13 console.log(objects[1].field) // prints 777
14
15 let an_array = [1, 2, 3]
16 let element = an_array [3.5] // Compile-time error
17 function foo (index: number) {
18     let element = an_array [index]
19     // Runtime-time error if index is not integer
20 }

```

An array indexing expression evaluated at runtime behaves as follows:

- Object reference expression is evaluated first.
- If the evaluation completes abruptly, then so does the indexing expression, and the index expression is not evaluated.
- If the evaluation completes normally, then the index expression is evaluated. The resultant value of the object reference expression refers to an array.
- If the index expression value of an array is less than zero, greater than or equal to that array’s *length*, then the `ArrayIndexOutOfBoundsException` is thrown.
- Otherwise, the result of the array access is a type `T` variable within the array selected by the value of the index expression.

```

1  function setElement(names: string[], i: number, name: string) {
2      names[i] = name // run-time error, if 'i' is out of bounds
3  }

```

## 7.12.2 Record Indexing Expression

*Index expression* for a `Record<Key, Value>` indexing (see *Record Utility Type*) must be of type `Key`.

The following two cases are to be considered separately:

1. Type `Key` is a union that contains literal types only;
2. Other cases.

**Case 1.** If type `Key` is a union that contains literal types only, then an *index expression* can only be one of the literals listed in the type. The result of the indexing expression is of type `Value`.

```
1  type Keys = 'key1' | 'key2' | 'key3'
2
3  let x: Record<Keys, number> = {
4      'key1': 1,
5      'key2': 2,
6      'key3': 4,
7  }
8  let y = x['key2'] // y value is 2
```

A compile-time error occurs if an index expression is not a valid literal:

```
1  console.log(x['key4']) // compile-time error
2  x['another key'] = 5 // compile-time error
```

The compiler guarantees that an object of `Record<Key, Value>` for this type `Key` contains values for all `Key` keys.

**Case 2.** An *index expression* has no restriction. The result of an indexing expression is of type `Value | undefined`.

```
1  let x: Record<number, string> = {
2      1: "hello",
3      2: "buy",
4  }
5
6  function foo(n: number): string | undefined {
7      return x[n]
8  }
9
10 function bar(n: number): string {
11     let s = x[n]
12     if (s == undefined) { return "no" }
13     return s!
14 }
15
16 foo(3) // prints "undefined"
17 bar(3) // prints "no"
18
19 let y = x[3]
```

Type of `y` in the code above is `string | undefined`. The value of `y` is `undefined`.

An indexing expression evaluated at runtime behaves as follows:

- Object reference expression is evaluated first.
- If the evaluation completes abruptly, then so does the indexing expression, and the index expression is not evaluated.
- If the evaluation completes normally, then the index expression is evaluated. The resultant value of the object reference expression refers to a `record` instance.
- If the `record` instance contains a key defined by the index expression, then the result is the value mapped to the key.
- Otherwise, the result is the literal `undefined`.

## 7.13 Chaining Operator

The *chaining operator* `'?.'` is used to effectively access values of nullish types. It can be used in the following contexts:

- *Field Access Expression*,
- *Method Call Expression*,
- *Function Call Expression*,
- *Indexing Expressions*.

If the value of the expression to the left of `'?.'` is undefined or null, then the evaluation of the entire surrounding *primary expression* stops. The result of the entire *primary expression* is then `undefined`. Thus the type of the entire *primary expression* is the union `undefined | non-nullish type of the entire primary expression`:

```
1  class Person {
2      name: string
3      spouse?: Person = undefined
4      constructor(name: string) {
5          this.name = name
6      }
7  }
8
9  let bob = new Person("Bob")
10 console.log(bob.spouse?.name) // prints "undefined"
11    // type of bob.spouse?.name is undefined|string
12
13 bob.spouse = new Person("Alice")
14 console.log(bob.spouse?.name) // prints "Alice"
15    // type of bob.spouse?.name is undefined|string
```

If an expression is not of a nullish type, then the chaining operator has no effect.

A compile-time error occurs if a chaining operator is placed in the context where a variable is expected, e.g., in the left-hand-side expression of an assignment (see *Assignment*) or expression (see *Postfix Increment*, *Postfix Decrement*, *Prefix Increment* or *Prefix Decrement*).

## 7.14 New Expressions

There are two syntactical forms of the *new expression*:

```
newExpression:
  newClassInstance
  | newArrayInstance
  ;
```

Type of a *new expression* is either `class` or `array`.

A *new class instance expression* creates a new object that is an instance of the specified class and it is described in full details below.

The creation of array instances is an experimental feature discussed in *Array Creation Expressions*.

```
newClassInstance:
  'new' typeArguments? typeReference arguments?
  ;
```

*Class instance creation expression* specifies a class to be instantiated. It optionally lists all actual arguments for the constructor.

```
1  class A {
2      constructor(p: number) {}
3  }
4
5  new A(5) // create an instance and call constructor
6  const a = new A(6) /* create an instance, call constructor and store
7                      created and initialized instance in 'a' */
```

*Class instance creation expression* can throw an error (see *Error Handling*, *Constructor Declaration*).

The execution of a class instance creation expression is performed as follows:

- New instance of class is created;
- Constructor of class is called to fully initialize the created instance.

The validity of the constructor call is similar to the validity of the method call as discussed in *Step 2: Selection of Method*, except the cases discussed in *Constructor Body*.

A compile-time error occurs if `typeReference` is a type parameter.

**Note.** If the *class instance creation expression* with no arguments is used as the object reference in the method call expression, then empty parentheses `()` are to be used.

```
1  class A { method() {} }
2
3  new A.method()    // compile-time error
4  new A().method()  // OK
5  let a = new A     // OK
```

## 7.15 Cast Expressions

*Cast expression* applies *cast operator* `as` to an *expression* by issuing a value of a specified type.

```
castExpression:
    expression 'as' type
    ;
```

```
1  class X {}
2
3  let x1 : X = new X()
4  let ob : Object = x1 as Object
5  let x2 : X = ob as X
```

The cast operator converts the value *V* of one type (as denoted by the expression) at runtime to a value of another type.

The cast expression introduces the target type for the casting context (see *Casting Contexts and Conversions*). The target type can be either `type` or `typeReference`.

Cast expression type is always the target type.

The result of a cast expression is a value, not a variable (even if the operand expression is a variable).

A compile-time error occurs if the cast operator cannot convert the compile-time type of the operand to the target type specified by the cast operator.

If the casting conversion cannot be performed during program execution, then `ClassCastError` is thrown.

## 7.16 InstanceOf Expression

```
instanceOfExpression:
    expression 'instanceof' type
    ;
```

Any `instanceof` expression is of type `boolean`.

The expression operand of the operator `instanceof` must be of a reference type. Otherwise, a compile-time error occurs.

A compile-time error occurs if `type` operand of the operator `instanceof` is one of the following:

- Type parameter (see *Type Parameters*),
- Primitive type (see *Primitive Types*),
- Union type that contains type parameter after normalization (see *Union Types Normalization*),
- *Generic type* (see *Generics*)—this temporary limitation is expected to be removed in the future (see *Generic and Function Types Peculiarities*).

If type of expression at compile time is assignable to `type` (see *Assignability*), then the result of an `instanceof` expression is `true`.

Otherwise, an `instanceof` expression checks during program execution whether type of the value the expression successfully evaluates to is assignable to `type` (see *Assignability*). If so, then the result of an `instanceof` expression is `true`. Otherwise, the result is `false`.



If the expression evaluation causes an error, then the execution control is transferred to a proper `catch` section or runtime system, and the result of an `instanceof` expression cannot be determined.

## 7.17 TypeOf Expression

```
typeofExpression:
  'typeof' expression
  ;
```

Any `typeof` expression is of type `string`. Its evaluation starts with the expression evaluation. If this evaluation causes an error, then the result of a `typeof` expression cannot be determined. Otherwise, the value of a `typeof` expression is defined as follows:

1. Types defined at compile time

| Type of Expression | Resulting String | Code Example                                                         |
|--------------------|------------------|----------------------------------------------------------------------|
| number/Number      | "number"         | <pre>let n: number = ... typeof n let N: Number = ... typeof N</pre> |
| string/String      | "string"         | <pre>let s: string = ... typeof s</pre>                              |

(table cont'd)

| Type of Expression                                                                                                                                                | Resulting String                                 | Code Example                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------|
| boolean/Boolean                                                                                                                                                   | "boolean"                                        | <pre>let b: boolean = ... typeof b let B: Boolean = ... typeof B</pre> |
| bigint/BigInt                                                                                                                                                     | "bigint"                                         | <pre>let b: bigint = ... typeof b let B: BigInt = ... typeof B</pre>   |
| any class or interface                                                                                                                                            | "object"                                         | <pre>let a: Object = ... typeof a</pre>                                |
| any function type                                                                                                                                                 | "function"                                       | <pre>let f: () =&gt; void = ... typeof f</pre>                         |
| undefined                                                                                                                                                         | "undefined"                                      | <pre>typeof undefined</pre>                                            |
| null                                                                                                                                                              | "object"                                         | <pre>typeof null</pre>                                                 |
| T   null, when T is a class (but not Object - see next table), interface or array                                                                                 | "object"                                         | <pre>class C {} let x: C   null = ... typeof x</pre>                   |
| enum                                                                                                                                                              | "number" or "string", depending of constant type | <pre>enum C {R, G, B} let c: C = ... typeof c</pre>                    |
| All high-performance numeric value types and their boxed versions: byte, short, int, long, float, double, Byte, Short, Int, long, Long, Float, Double, char, Char | "number"                                         | <pre>let x: byte = ... typeof x ...</pre>                              |

2. All other types evaluated at runtime

| Type of Expression | Code Example                                                                                                                |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Object             | <pre>function f(o: Object) {   typeof o }</pre>                                                                             |
| union type         | <pre>function f(p:A B) {   typeof p }</pre>                                                                                 |
| type parameter     | <pre>class A&lt;T null undefined&gt; {   f: T   m() {     typeof this.f   }   constructor(p:T) {     this.f = p   } }</pre> |

## 7.18 Ensure-Not-Nullish Expression

```
ensureNotNullishExpression:
  expression '!'
;
```

An *ensure-not-nullish expression* is a postfix expression with the operator ‘!’. An *ensure-not-nullish expression* in the expression *e!* checks whether *e* of a nullish type (see [Nullish Types](#)) evaluates to a nullish value.

If the expression *e* is not of a nullish type, then the operator ‘!’ has no effect.

If the result of the evaluation of *e* is not equal to `null` or `undefined`, then the result of *e!* is the outcome of the evaluation of *e*.

If the result of the evaluation of *e* is equal to `null` or `undefined`, then `NullPointerException` is thrown.

Type of *ensure-not-nullish* expression is the non-nullish variant of type of *e*.

## 7.19 Nullish-Coalescing Expression

```
nullishCoalescingExpression:
  expression '??' expression
;
```

*Nullish-coalescing expression* is a binary expression that uses the operator ‘??’, and checks whether the evaluation of the left-hand-side expression equals the *nullish* value:

- If so, then the right-hand-side expression evaluation is the result of a nullish-coalescing expression.
- If not so, then the result of the left-hand-side expression evaluation is the result of a nullish-coalescing expression, and the right-hand-side expression is not evaluated (the operator ‘??’ is thus *lazy*).

If the left-hand-side expression is not of a nullish type, then type of the expression is a nullish-coalescing expression. Otherwise, type of a nullish-coalescing expression is a normalized *union type* (see [Union Types](#)) formed from the following:

- Non-nullish variant of the type of the left-hand-side expression; and
- Type of the right-hand-side expression.

The semantics of a nullish-coalescing expression is represented in the following example:

```

1  let x = expression1 ?? expression2
2
3  let x$ = expression1
4  if (x$ == null) {x = expression2} else x = x$!
5
6  // Type of x is NonNullishType(expression1) | Type(expression2)

```

A compile-time error occurs if the nullish-coalescing operator is mixed with conditional-and or conditional-or operators without parentheses.

## 7.20 Unary Expressions

```

unaryExpression:
    expression '++'
  | expression '--'
  | '++' expression
  | '--' expression
  | '+' expression
  | '-' expression
  | '~' expression
  | '!' expression
;

```

All expressions with unary operators (except postfix increment and postfix decrement operators) group right-to-left for ‘~+x’ to have the same meaning as ‘~(+x)’.

Type of any *unary Expression* is the type of the `expression` provided.

### 7.20.1 Postfix Increment

*Postfix increment expression* is an *expression* followed by the increment operator ‘++’.

The *expression* must be *left-hand-side expression* (see [Left-Hand-Side Expressions](#)), so it denotes a variable.

A compile-time error occurs if type of the *expression* is not convertible (see [Implicit Conversions](#)) to a numeric type (see [Numeric Types](#)).

Type of a *postfix increment expression* is the type of the variable. The result of a *postfix increment expression* is a value, not a variable.

If the evaluation of the operand *expression* completes normally at runtime, then:

- The value *1* is added to the value of the variable by using necessary conversions (see *Primitive Types Conversions*); and
- The sum is stored back into the variable.

Otherwise, the *postfix increment expression* completes abruptly, and no incrementation occurs.

The value of the *postfix increment expression* is the value of the variable *before* the new value is stored.

## 7.20.2 Postfix Decrement

*Postfix decrement expression* is an expression followed by the decrement operator ‘--’.

The *expression* must be *left-hand-side expression* (see *Left-Hand-Side Expressions*). The *expression* denotes a variable.

A compile-time error occurs if type of the *expression* is not convertible (see *Implicit Conversions*) to a numeric type (see *Numeric Types*).

Type of a postfix decrement expression is the type of the variable. The result of a postfix decrement expression is a value, not a variable.

If evaluation of the operand *expression* completes at runtime, then:

- The value *1* is subtracted from the value of the variable by using necessary conversions (see *Primitive Types Conversions*); and
- The sum is stored back into the variable.

Otherwise, the *postfix decrement expression* completes abruptly, and no decrementation occurs.

The value of the *postfix decrement expression* is the value of the variable *before* the new value is stored.

## 7.20.3 Prefix Increment

*Prefix increment expression* is an expression preceded by the operator ‘++’.

The *expression* must be *left-hand-side expression* (see *Left-Hand-Side Expressions*). The *expression* denotes a variable.

A compile-time error occurs if the type of the *expression* is not convertible (see *Implicit Conversions*) to a numeric type (see *Numeric Types*).

Type of a prefix increment expression is the type of the variable. The result of a prefix increment expression is a value, not a variable.

If evaluation of the operand *expression* completes normally at runtime, then:

- The value *1* is added to the value of the variable by using necessary conversions (see *Primitive Types Conversions*); and
- The sum is stored back into the variable.

Otherwise, the *prefix increment expression* completes abruptly, and no incrementation occurs.

The value of the *prefix increment expression* is the value of the variable *before* the new value is stored.

## 7.20.4 Prefix Decrement

*Prefix decrement expression* is an expression preceded by the operator ‘--’.

The *expression* must be *left-hand-side expression* (see [Left-Hand-Side Expressions](#)). The *expression* denotes a variable.

A compile-time error occurs if type of the *expression* is not convertible (see [Implicit Conversions](#)) to a numeric type (see [Numeric Types](#)).

Type of a prefix decrement expression is the type of the variable. The result of a prefix decrement expression is a value, not a variable.

If evaluation of the operand *expression* completes normally at runtime, then:

- The value *1* is subtracted from the value of the variable by using necessary conversions (see [Primitive Types Conversions](#)); and
- The sum is stored back into the variable.

Otherwise, the *prefix decrement expression* completes abruptly, and no decrementation occurs. The value of the *prefix decrement expression* remains the value of the variable *before* a new value is stored.

## 7.20.5 Unary Plus

*Unary plus expression* is an expression preceded by the operator ‘+’.

Type of the operand *expression* with the unary operator ‘+’ must be convertible (see [Implicit Conversions](#)) to a numeric type (see [Numeric Types](#)). Otherwise, a compile-time error occurs.

The numeric types conversion (see [Primitive Types Conversions](#)) is performed on the operand to ensure that the resultant type is that of the unary plus expression. The result of a unary plus expression is always a value, not a variable (even if the result of the operand expression is a variable).

Type of the *unary plus expression* is the type of the expression provided.

## 7.20.6 Unary Minus

*Unary minus expression* is an expression preceded by the operator ‘-’.

Type of the operand *expression* with the unary operator ‘-’ must be convertible (see [Implicit Conversions](#)) to a numeric type (see [Numeric Types](#)). Otherwise, a compile-time error occurs.

The numeric types conversion (see *Primitive Types Conversions*) is performed on the operand to ensure that the resultant type is that of the unary minus expression. The result of a unary minus expression is a value, not a variable (even if the result of the operand expression is a variable).

A unary numeric promotion performs the value set conversion (see *Implicit Conversions*).

The unary negation operation is always performed on, and the result is drawn from the same value set as the promoted operand value.

Type of the *unary minus expression* is the type of the expression provided.

Further value set conversions are then performed on the same result.

The value of a unary minus expression at runtime is the arithmetic negation of the promoted value of the operand.

The negation of integer values is the same as subtraction from zero. The ArkTS programming language uses two's-complement representation for integers. The range of two's-complement value is not symmetric. The same maximum negative number results from the negation of the maximum negative *int* or *long*. In that case, an overflow occurs but throws no error. For any integer value  $x$ ,  $-x$  is equal to  $(\sim x) + 1$ .

The negation of floating-point values is *not* the same as subtraction from zero (if  $x$  is  $+0.0$ , then  $0.0 - x$  is  $+0.0$ , however  $-x$  is  $-0.0$ ).

A unary minus merely inverts the sign of a floating-point number. Special cases to consider are as follows:

- Operand NaN results in NaN (NaN has no sign).
- Operand infinity results in the infinity of the opposite sign.
- Operand zero results in zero of the opposite sign.

## 7.20.7 Bitwise Complement

*Bitwise complement expression* is an expression preceded by the operator ' $\sim$ '.

Type of the operand *expression* with the unary operator ' $\sim$ ' must be convertible (see *Implicit Conversions*) to a primitive integer type. Otherwise, a compile-time error occurs.

Thus the type of the *Bitwise complement expression* is a primitive integer.

The numeric types conversion (see *Primitive Types Conversions*) is performed on the operand to ensure that the resultant type is that of the unary bitwise complement expression.

The result of a unary bitwise complement expression is a value, not a variable (even if the result of the operand expression is a variable).

The value of a unary bitwise complement expression at runtime is the bitwise complement of the promoted value of the operand. In all cases,  $\sim x$  equals  $(-x) - 1$ .

## 7.20.8 Logical Complement

*Logical complement expression* is an expression preceded by the operator ' $!$ '.

Type of the operand *expression* with the unary ‘!’ operator must be `boolean` or `Boolean` or type mentioned in *Extended Conditional Expressions*. Otherwise, a compile-time error occurs.

The unary logical complement expression’s type is `boolean`.

The unboxing conversion (see *Unboxing Conversions*) is performed on the operand at runtime if needed.

The value of a unary logical complement expression is `true` if the (possibly converted) operand value is `false`, and `false` if the operand value (possibly converted) is `true`.

## 7.21 Multiplicative Expressions

Multiplicative expressions use *multiplicative operators* ‘\*’, ‘/’, and ‘%’:

```
multiplicativeExpression:
    expression '*' expression
  | expression '/' expression
  | expression '%' expression
  ;
```

Multiplicative operators group left-to-right.

Type of each operand in a multiplicative operator must be convertible (see *Contexts and Conversions*) to a numeric type (see *Numeric Types*). Otherwise, a compile-time error occurs.

The numeric types conversion (see *Primitive Types Conversions*) is performed on both operands to ensure that the resultant type is the type of the multiplicative expression.

The result of a unary bitwise complement expression is a value, not a variable (even if the operand expression is a variable).

### 7.21.1 Multiplication

The binary operator ‘\*’ performs multiplication, and returns the product of its operands.

Multiplication is a commutative operation if operand expressions have no side effects.

Integer multiplication is associative when all operands are of the same type.

Floating-point multiplication is not associative.

Type of a *multiplication expression* is the ‘heaviest’ (see *Numeric Types Hierarchy*) type of its operands.

If overflow occurs during integer multiplication, then:

- The result is the low-order bits of the mathematical product as represented in some sufficiently large two’s-complement format.
- The sign of the result can be other than the sign of the mathematical product of the two operand values.

A floating-point multiplication result is determined in compliance with the IEEE 754 arithmetic:

- The result is NaN if:



- Either operand is NaN;
- Infinity is multiplied by zero.
- If the result is not NaN, then the sign of the result is as follows:
  - Positive, where both operands have the same sign; and
  - Negative, where the operands have different signs.
- If infinity is multiplied by a finite value, then the multiplication results in a signed infinity (the sign is determined by the rule above).
- If neither NaN nor infinity is involved, then the exact mathematical product is computed.

The product is rounded to the nearest value in the chosen value set by using the IEEE 754 *round-to-nearest* mode. The ArkTS programming language requires gradual underflow support as defined by IEEE 754 (see [Floating-Point Types and Operations](#)).

If the magnitude of the product is too large to represent, then the operation overflows, and the result is an appropriately signed infinity.

The evaluation of a multiplication operator ‘\*’ never throws an error despite possible overflow, underflow, or loss of information.

## 7.21.2 Division

The binary operator ‘/’ performs division and returns the quotient of its left-hand-side and right-hand-side operands (dividend and divisor respectively).

Integer division rounds toward 0, i.e., the quotient of integer operands  $n$  and  $d$ , after a numeric types conversion on both (see [Primitive Types Conversions](#) for details), is the integer value  $q$  with the largest possible magnitude that satisfies  $|d \cdot q| \leq |n|$ .

**Note.** The integer value  $q$  is:

- Positive, where  $|n| \geq |d|$ , and  $n$  and  $d$  have the same sign; but
- Negative, where  $|n| \geq |d|$ , and  $n$  and  $d$  have opposite signs.

Only a single special case does not comply with this rule: the integer overflow occurs, and the result equals the dividend if the dividend is a negative integer of the largest possible magnitude for its type, while the divisor is  $-1$ . No error is thrown in this case despite the overflow. However, if the divisor value is 0 in an integer division, then `ArithmeticError` is thrown.

The result of a floating-point division is determined in compliance with the IEEE 754 arithmetic:

- The result is NaN if:
  - Either operand is NaN;
  - Both operands are infinity; or
  - Both operands are zero.
- If the result is not NaN, then the sign of the result is:
  - Positive, where both operands have the same sign; or
  - Negative, where the operands have different signs.

- Division produces a signed infinity (the sign is determined by the rule above) if:
  - Infinity is divided by a finite value; and
  - A nonzero finite value is divided by zero.
- Division produces a signed zero (the sign is determined by the rule above) if:
  - A finite value is divided by infinity; and
  - Zero is divided by any other finite value.
- If neither NaN nor infinity is involved, then the exact mathematical quotient is computed.

If the magnitude of the product is too large to represent, then the operation overflows, and the result is an appropriately signed infinity.

The quotient is rounded to the nearest value in the chosen value set by using the IEEE 754 *round-to-nearest* mode. The ArkTS programming language requires gradual underflow support as defined by IEEE 754 (see [Floating-Point Types and Operations](#)).

The evaluation of a floating-point division operator ‘/’ never throws an error despite possible overflow, underflow, division by zero, or loss of information.

The type of the *division expression* is an integer or a floating-point number.

### 7.21.3 Remainder

The binary operator ‘%’ yields the remainder of its operands (*dividend* as the left-hand-side, and *divisor* as the right-hand-side operand) from an implied division.

The remainder operator in ArkTS accepts floating-point operands (unlike in C and C++).

The remainder operation on integer operands produces a result value, i.e.,  $(a/b) * b + (a \% b)$  equals  $a$ . The numeric type conversion on remainder operation is discussed in [Primitive Types Conversions](#).

This equality holds even in the special case where the dividend is a negative integer of the largest possible magnitude of its type, and the divisor is  $-1$  (the remainder is then  $0$ ). According to this rule, the result of the remainder operation can only be one of the following:

- Negative if the dividend is negative; or
- Positive if the dividend is positive.

The magnitude of the result is always less than that of the divisor.

If the value of the divisor for an integer remainder operator is  $0$ , then `ArithmeticError` is thrown.

The result of a floating-point remainder operation as computed by the operator ‘%’ is different than that produced by the remainder operation defined by IEEE 754. The IEEE 754 remainder operation computes the remainder from a rounding division (not a truncating division), and its behavior is different from that of the usual integer remainder operator. On the contrary, ArkTS presumes that the operator ‘%’ behaves on floating-point operations in the same manner as the integer remainder operator (comparable to the C library function *fmod*). The standard library (see [Standard Library](#)) routine `Math.IEEEremainder` can compute the IEEE 754 remainder operation.

The result of a floating-point remainder operation is determined in compliance with the IEEE 754 arithmetic:

- The result is NaN if:
  - Either operand is NaN;

- The dividend is infinity;
- The divisor is zero; or
- The dividend is infinity, and the divisor is zero.
- If the result is not NaN, then the sign of the result is the same as the sign of the dividend.
- The result equals the dividend if:
  - The dividend is finite, and the divisor is infinity; or
  - If the dividend is zero, and the divisor is finite.
- If infinity, zero, or NaN are not involved, then the floating-point remainder  $r$  from the division of the dividend  $n$  by the divisor  $d$  is determined by the mathematical relation  $r = n - (d \cdot q)$ , where  $q$  is an integer that is only:
  - Negative if  $n/d$  is negative, or
  - Positive if  $n/d$  is positive.
- The magnitude of  $q$  is the largest possible without exceeding the magnitude of the true mathematical quotient of  $n$  and  $d$ .

The evaluation of the floating-point remainder operator ‘%’ never throws an error, even if the right-hand operand is zero. Overflow, underflow, or loss of precision cannot occur.

The type of the *remainder expression* is an integer or floating-point number.

## 7.22 Additive Expressions

Additive expressions use *additive operators* ‘+’ and ‘-’:

```
additiveExpression:
  expression '+' expression
  | expression '-' expression
  ;
```

Additive operators group left-to-right.

If either operand of the operator is ‘+’ of type `string`, then the operation is a string concatenation (see *String Concatenation*). In all other cases, type of each operand of the operator ‘+’ must be convertible (see *Primitive Types Conversions*) to a numeric type (see *Numeric Types*). Otherwise, a compile-time error occurs.

Type of each operand of the binary operator ‘-’ must be convertible (see *Primitive Types Conversions*) to a numeric type (see *Numeric Types*) in all cases. Otherwise, a compile-time error occurs.

Type of *Additive expression* is `string` or the ‘heaviest’ (see *Numeric Types Hierarchy*) type of its operands.

### 7.22.1 String Concatenation

If one operand of an expression is of type `string`, then the string conversion (see *String Operator Contexts*) is performed on the other operand at runtime to produce a string.

String concatenation produces a reference to a `string` object that is a concatenation of two operand strings. The left-hand-side operand characters precede the right-hand-side operand characters in a newly created string.

If the expression is not a constant expression (see *Constant Expressions*), then a new `string` object is created (see *New Expressions*).

## 7.22.2 Additive Operators for Numeric Types

The primitive types conversion (see *Primitive Types Conversions*) performed on a pair of operands ensures that both operands are of a numeric type. If the conversion fails, then a compile-time error occurs.

The binary operator `+` performs addition and produces the sum of such operands.

The binary operator `-` performs subtraction and produces the difference of two numeric operands.

Type of an additive expression performed on numeric operands is the largest type (see *Numeric Types Hierarchy*) to which operands of that expression are converted.

If the promoted type is `int` or `long`, then integer arithmetic is performed. If the promoted type is `float` or `double`, then floating-point arithmetic is performed.

If operand expressions have no side effects, then addition is a commutative operation.

If all operands are of the same type, then integer addition is associative.

Floating-point addition is not associative.

If overflow occurs on an integer addition, then:

- Result is the low-order bits of the mathematical sum as represented in a sufficiently large two's-complement format.
- Sign of the result is different than that of the mathematical sum of the operands' values.

The result of a floating-point addition is determined in compliance with the IEEE 754 arithmetic as follows:

- The result is NaN if:
  - Either operand is NaN; or
  - The operands are two infinities of the opposite signs.
- The sum of two infinities of the same sign is the infinity of that sign.
- The sum of infinity and a finite value equals the infinite operand.
- The sum of two zeros of opposite sign is positive zero.
- The sum of two zeros of the same sign is zero of that sign.
- The sum of zero and a nonzero finite value is equal to the nonzero operand.
- The sum of two nonzero finite values of the same magnitude and opposite sign is positive zero.
- If infinity, zero, or NaN are not involved, and the operands have the same sign or different magnitudes, then the exact sum is computed mathematically.

If the magnitude of the sum is too large to represent, then the operation overflows. The result is an appropriately signed infinity.

Otherwise, the sum is rounded to the nearest value within the chosen value set by using the IEEE 754 *round-to-nearest* mode. The ArkTS programming language requires gradual underflow support as defined by IEEE 754 (see *Floating-Point Types and Operations*).

When applied to two numeric type operands (see *Numeric Types*), the binary operator ‘-’ performs subtraction, and returns the difference of such operands (minuend as left-hand-side, and subtrahend as the right-hand-side operand).

The result of  $a-b$  is always the same as that of  $a+(-b)$  in both integer and floating-point subtraction.

The subtraction from zero for integer values is the same as negation. However, the subtraction from zero for floating-point operands and negation is *not* the same (if  $x$  is  $+0.0$ , then  $0.0-x$  is  $+0.0$ ; however  $-x$  is  $-0.0$ ).

The evaluation of a numeric additive operator never throws an error despite possible overflow, underflow, or loss of information.

## 7.23 Shift Expressions

*Shift expressions* use *shift operators* ‘<<’ (left shift), ‘>>’ (signed right shift), and ‘>>>’ (unsigned right shift). The value to be shifted is the left-hand-side operand in a shift operator, and the right-hand-side operand specifies the shift distance:

```
shiftExpression:
    expression '<<' expression
  | expression '>>' expression
  | expression '>>>' expression
  ;
```

Shift operators group left-to-right.

Numeric types conversion (see *Primitive Types Conversions*) is performed separately on each operand to ensure that both operands are of primitive integer type.

Thus, *shift expression* is of primitive integer type.

**Note.** If the initial type of one or both operands is `double` or `float`, then such operand or operands are first truncated to the appropriate integer type. If both operands are of type `bigint`, then shift operator is applied to `bigint` operands.

A compile-time error occurs if either operand in a shift operator (after unary numeric promotion) is not a primitive integer or `bigint` type.

The shift expression type is the promoted type of the left-hand-side operand.

If the left-hand-side operand is of the promoted type `int`, then only five lowest-order bits of the right-hand-side operand specify the shift distance (as if using a bitwise logical AND operator ‘&’ with the mask value `0x1f` or `0b11111` on the right-hand-side operand). Thus, it is always within the inclusive range of 0 through 31.

If the left-hand-side operand is of the promoted type `long`, then only six lowest-order bits of the right-hand-side operand specify the shift distance (as if using a bitwise logical AND operator ‘&’ with the mask value `0x3f` or `0b111111` the right-hand-side operand). Thus, it is always within the inclusive range of 0 through 63.

Shift operations are performed on the two’s-complement integer representation of the value of the left-hand-side operand at runtime.

The value of  $n << s$  is  $n$  left-shifted by  $s$  bit positions. It is equivalent to multiplication by two to the power  $s$  even in case of an overflow.

The value of  $n \gg s$  is  $n$  right-shifted by  $s$  bit positions with sign-extension. The resultant value is  $\text{floor}(n/2^s)$ . If  $n$  is non-negative, then it is equivalent to truncating integer division (as computed by the integer division operator by 2 to the power  $s$ ).

The value of  $n \ggg s$  is  $n$  right-shifted by  $s$  bit positions with zero-extension, where:

- If  $n$  is positive, then the result is the same as that of  $n \gg s$ .
- If  $n$  is negative, and type of the left-hand-side operand is `int`, then the result is equal to that of the expression  $(n \gg s) + (2 \ll s)$ .
- If  $n$  is negative, and type of the left-hand-side operand is `long`, then the result is equal to that of the expression  $(n \gg s) + (2L \ll s)$ .

## 7.24 Relational Expressions

Relational expressions use *relational operators* '`<`', '`>`', '`<=`', and '`>=`'.

```
relationalExpression:
    expression '<' expression
  | expression '>' expression
  | expression '<=' expression
  | expression '>=' expression
  ;
```

Relational operators group left-to-right.

A relational expression is always of type `boolean`.

Two kinds of relational expressions are described below. The kind of a relational expression depends on types of operands. It is a compile-time error if at least one type of operands is different from types described below.

### 7.24.1 Numerical Relational Operators

Type of each operand in a numerical relational operator must be convertible (see *Implicit Conversions*) to a numeric type (see *Numeric Types*). Otherwise, a compile-time error occurs.

Numeric types conversions (see *Primitive Types Conversions*) are performed on each operand as follows:

- Signed integer comparison if the converted type of the operand is `int` or `long`.
- Floating-point comparison if the converted type of the operand is `float` or `double`.

The comparison of floating-point values drawn from any value set must be accurate.

A floating-point comparison must be performed in accordance with the IEEE 754 standard specification as follows:

- The result of a floating-point comparison is false if either operand is NaN.
- All values other than NaN must be ordered with the following:
  - Negative infinity less than all finite values; and

- Positive infinity greater than all finite values.
- Positive zero equals negative zero.

Based on the above presumption, the following rules apply to integer operands, or floating-point operands other than NaN:

- The value produced by the operator '`<`' is `true` if the value of the left-hand-side operand is less than that of the right-hand-side operand. Otherwise, the value is `false`.
- The value produced by the operator '`<=`' is `true` if the value of the left-hand-side operand is less than or equal to that of the right-hand-side operand. Otherwise, the value is `false`.
- The value produced by the operator '`>`' is `true` if the value of the left-hand-side operand is greater than that of the right-hand-side operand. Otherwise, the value is `false`.
- The value produced by the operator '`>=`' is `true` if the value of the left-hand-side operand is greater than or equal to that of the right-hand-side operand. Otherwise, the value is `false`.

### 7.24.2 String Relational Operators

Results of all string comparisons are defined as follows:

- Operator '`<`' delivers `true` if the string value of the left-hand-side operand is lexicographically less than the string value of the right-hand-side operand, or `false` otherwise.
- Operator '`<=`' delivers `true` if the string value of the left-hand-side operand is lexicographically less than or equal to the string value of the right-hand-side operand, or `false` otherwise.
- Operator '`>`' delivers `true` if the string value of the left-hand-side operand is lexicographically greater than the string value of the right-hand-side operand, or `false` otherwise.
- Operator '`>=`' delivers `true` if the string value of the left-hand-side operand is lexicographically greater than or equal to the string value of the right-hand operand, or `false` otherwise.

### 7.24.3 Bigint Relational Operators

Type of each operand in a bigint relational operator must be `bigint`. Otherwise, a compile-time error occurs.

All bigint relational operators compare bigint values.

### 7.24.4 Boolean Relational Operators

Results of all boolean comparisons are defined as follows:

- Operator '`<`' delivers `true` if the left-hand-side operand is `false` and the right-hand-side operand is `true`, or `false` otherwise.

- Operator '`<=`' delivers `true` if the left-hand-side operand is `false` and the right-hand-side operand is `true` or `false`, or `false` otherwise.
- Operator '`>`' delivers `true` if the left-hand-side operand is `true` and the right-hand-side operand is `false`, or `false` otherwise.
- Operator '`>=`' delivers `true` if the left-hand-side operand is `true` and the right-hand-side operand is `false` or `true`, or `false` otherwise.

### 7.24.5 Enumeration Relational Operators

If both operands are of the same Enumeration type (see *Enumerations*), then *Numerical Relational Operators* or *String Relational Operators* are used depending on the kind of enumeration constant value ( *Enumeration Integer Values* or *Enumeration String Values*). Otherwise, a compile-time error occurs.

## 7.25 Equality Expressions

Equality expressions use *equality operators* '`==`', '`===`', '`!=`', and '`!==`':

```
equalityExpression:  
  expression ('==' | '===' | '!=' | '!==') expression  
  ;
```

Any equality expression is of type `boolean`. The result of operators '`==`' and '`===`' is `true` if operands are *equal* as shown below. Otherwise, the result is `false`.

Equality operators group left-to-right. Equality operators are commutative if operand expressions cause no side effects.

Equality operators are similar to relational operators, except for their lower precedence (`a < b === c < d` is `true` if both `a < b` and `c < d` have the same `true` value).

In all cases, `a != b` produces the same result as `!(a == b)`, and `a !== b` produces the same result as `!(a === b)`.

The result of operators '`==`' and '`===`' is the same in all cases, except when comparing the values `null` and `undefined` (see *Reference Equality*).

The variant of equality evaluation to be used depends on types of the operands used as follows:

- *Value equality* is applied to entities of primitive types (see *Value Types*), their boxed versions (see *Boxed Types*), type `string` (see *Type string*), type `bigint` (see *Type bigint*), and enumeration types (see *Enumerations*).
- *Reference Equality based on actual (dynamic) type* is applied to values of type `Object` (*Type Object*), values of union types (*Union Types*), and type parameters (*Type Parameters*).
- *Reference equality* is applied to entities of all other reference types (see *Reference Types*).

Operators '`===`' and '`!==`', or '`!=`' and '`!=`' are used for:

- *Numerical Equality Operators* if operands are of numeric types (see *Numeric Types*), type `char`, or the boxed version of numeric types;



- *String Equality Operators* if both operands are of type `string`;
- *Bigint Equality Operators* if both operands are of type `bigint`;
- *Boolean Equality Operators* if both operands are of type `boolean` or `Boolean`;
- *Enumeration Equality Operators* if both operands are of enumeration type;
- *Reference Equality Based on Actual Type* if at least one operand is of `Object` type or union type, or is a type parameter;
- *Reference Equality* if both operands are of compatible reference types, except types `string`, `bigint`, `Object`, union types, and type parameters;
- *Extended Equality with null or undefined* if one operand is `null` or `undefined`.
- Otherwise, a compile-time error occurs.

```

1 // Entities of value types are not comparable between each other
2 5 == "5" // compile-time error
3 5 == true // compile-time error
4 "5" == true // compile-time error

```

### 7.25.1 Numerical Equality Operators

Type of each operand in a numerical equality operator must be convertible (see *Implicit Conversions*) to a numeric type (see *Numeric Types*). Otherwise, a compile-time error occurs.

A widening conversion can occur (see *Widening Primitive Conversions*) if type of one operand is smaller than type of the other operand (see *Numeric Types Hierarchy*).

If the converted type of the operands is `int` or `long`, then an integer equality test is performed.

If the converted type is `float` or `double`, then a floating-point equality test is performed.

The floating-point equality test must be performed in accordance with the following IEEE 754 standard rules:

- The result of `'=='` or `'==='` is `false` but the result of `'!=`' is `true` if either operand is NaN.

The test `x != x` or `x !== x` is `true` only if `x` is NaN.

- Positive zero equals negative zero.
- Equality operators consider two distinct floating-point values unequal in any other situation.

For example, if one value represents positive infinity, and the other represents negative infinity, then each compares equal to itself and unequal to all other values.

Based on the above presumptions, the following rules apply to integer operands or floating-point operands other than NaN:

- If the value of the left-hand-side operand is equal to that of the right-hand-side operand, then the operator `'=='` or `'==='` produces the value `true`. Otherwise, the result is `false`.
- If the value of the left-hand-side operand is not equal to that of the right-hand-side operand, then the operator `'!=`' or `'!=='` produces the value `true`. Otherwise, the result is `false`.

The following example illustrates *numerical equality*:

```
1 5 == 5 // true
2 5 != 5 // false
3
4 5 === 5 // true
5
6 5 == new Number(5) // true
7 5 === new Number(5) // true
8
9 new Number(5) == new Number(5) // true
10 5 == 5.0 // true
```

## 7.25.2 String Equality Operators

Type of one operand must be of type `string`, other operand must be convertible (see *Implicit Conversions*) to `string` type.

Two strings are equal if they represent the same sequence of characters:

```
1 "abc" == "abc" // true
2 "abc" === "ab" + "c" // true
3
4 function foo(s: string) {
5     console.log(s == "hello")
6 }
7 foo("hello") // prints "true"
```

## 7.25.3 Bigint Equality Operators

*Boolean equality* is used for operands of types `bigint` or `BigInt`.

Two `bigints` are equal if they have the same value:

```
1 let x = 2n
2 x == 2n // true
```

## 7.25.4 Boolean Equality Operators

*Boolean equality* is used for operands of types `boolean` or `Boolean`.

If an operand is of type `Boolean`, then the unboxing conversion must be performed (see *Primitive Types Conversions*).

If both operands (after the unboxing conversion is performed if required) are either `true` or `false`, then the result of `'=='` or `'==='` is `true`. Otherwise, the result is `false`.

If both operands are either `true` or `false`, then the result of `'!='` or `'!=='` is `false`. Otherwise, the result is `true`.

### 7.25.5 Enumeration Equality Operators

If both operands are of the same enumeration type (see *Enumerations*), then *Numerical Equality Operators* or *String Equality Operators* are used depending on the kind of enumeration constant value (*Enumeration Integer Values* or *Enumeration String Values*). Otherwise, a compile-time error occurs.

### 7.25.6 Reference Equality Based on Actual Type

If an operand of an equality operator is of type `Object`, or a union type, or is a type parameter, then the operator is evaluated at runtime, and is based on the actual type of this operand. If the other operand is of a type other than that above, then the static type of this operand is used for evaluation.

If actual types of objects are compatible, then the corresponding evaluation of equality operator is used. Otherwise, the result of the operators `'=='` and `'==='` is `false`.

#### Object Type Equality Operators

A value of type `Object` can be compared to a value of any reference type.

The following example illustrates an equality with a value of type `Object`:

```

1  function equToString(a: Object, b: string): boolean {
2      return a == b
3  }
4
5  equToString("aa", "aa") // true, string equality
6  equToString(1, "aa") // false, not compatible types
7
8  function equ(a: Object, b: Object): boolean {
9      return a == b
10 }
11
12 equ(1, 1) // true, numerical equality
13 equ(1, 2) // false, numerical equality
14 equ(1, new Number(1)) // true, numerical equality
15
16 equ("aa", "aa") // true, string equality
17 equ(1, "aa") // false, not compatible types

```

**Note.** The actual type of an `Object` value can be none of the following:

- Union type, as only the current value of a union type variable can be assigned to an `Object` variable;
- Type parameter, if it has no type constraint (see *Type Parameter Constraint*) as in the example below:

```
1  function check(a: Object) {}
2
3  class G<A, B extends Number> {
4      foo(x: A, y: B) {
5          check(x) // compile-type error, A is not assignable to Object
6          check(y) // ok, B is assignable to Object (as it is, at least, Number)
7      }
8  }
```

## Union Equality Operators

Where one operand is of type  $T_1$ , and the other operand is of type  $T_2$ , while  $T_1$ ,  $T_2$ , or both are a union type, then a compile-time error occurs if  $T_1$  and  $T_2$  have no overlap (i.e., if no value belongs to both  $T_1$  and  $T_2$ ).

**Note.** Any union type has an overlap with a value of type `Object`.

The following example illustrates an equality with values of two union types:

```
1  function f1(x: number | string, y: boolean | null): boolean {
2      return x == y // compile-time error, types have no overlap
3  }
4
5  function f2(x: number | string, y: boolean | "abc"): boolean {
6      return x == y // ok, types have overlap - value "abc"
7  }
```

If actual types of values are compatible, then the corresponding evaluation of an equality operator is used. Otherwise, the result of the operators `'=='` and `'==='` is `false`:

```
1  function equ(x: number | string, y: string): boolean {
2      return x == y
3  }
4
5  console.log(equ("aa", "aa")) // string equality: prints true
6  console.log(equ("ab", "aa")) // string equality: prints false
7  console.log(equ(1, "aa")) // different types: prints false
```

## Type Parameter Equality Operators

If one operand is a type parameter, then the other operand can be of any reference type, including type parameter.

If actual object types are compatible, then the corresponding evaluation of an equality operator is used. Otherwise, the result of the operators `'=='` and `'==='` is `false`:

```

1  function equ<A>(x: A, y: A): boolean {
2      return x == y
3  }
4
5  console.log(equ<string>("aa", "aa")) // string equality: prints true
6  console.log(equ<number>(1, 2)) // numerical equality: prints false

```

### 7.25.7 Reference Equality

Reference equality compares operands of two reference types except types `string`, `bigint`, `Object`, union types, and type parameters. The extended semantics is discussed in *Extended Equality with null or undefined*.

A compile-time error occurs if:

- Any operand is not of a reference type;
- There is no implicit conversion (see *Implicit Conversions*) that can convert type of either operand to the type of the other operand.

The result of `'=='` or `'==='` is `true` if both operand values:

- Are `null`;
- Are `undefined`; or
- Refer to the same object, array, or function.

In addition, the result of the `'=='` operator is `true` if one operand value is `null`, and the other operand value is `undefined`. Otherwise, the result is `false`. This semantics is illustrated by the example below:

```

1  class X {}
2  new X() == new X() // false, two different object of class X
3  new X() === new X() // false, the same
4  let x1 = new X()
5  let x2 = x1
6  x1 == x2 // true, as x1 and x2 refer to the same object
7  x1 === x2 // true, the same
8
9  new Number(5) === new Number(5) // true, value equality is used
10 new Number(5) == new Number(6) // false, value equality is used
11
12 null == undefined // true
13 null === undefined // false

```

### 7.25.8 Extended Equality with null or undefined

ArkTS provides extended semantics for equalities with `null` and `undefined` to ensure better alignment with TypeScript.

Any entity can be compared to `null` by using the operators `'=='` and `'==='`. This comparison can return `true` only for the entities of *nullable* types if they actually have the `null` value during program execution. In all other cases the comparison to `null` returns `false`.

Operators `'!='` and `'!=='` return `true` for any entity of *non-nullable* types, and for *nullable* entities if they actually have no `null` value during program execution.

These situations are to be known at compile time.

Similarly, an equality comparison to `undefined` returns `false` if the variable being compared is neither type `undefined` nor a union type with `undefined` as one of its types.

The following comparisons evaluate to `false` at compile time:

```
1 5 == null // false
2 5 == undefined // false
3 (() : void => {}) == null // false
4
5 class X {}
6 new X() == null // false
```

The following comparison is evaluated at runtime:

```
1 function foo<T> (p1: string | null, p2: T) {
2   console.log (p1 == undefined, p1 == null, p2 == undefined, p2 == null)
3 }
4 let nullable: string|null = "a string"
5 let undefinable: Object|undefined = undefined
6
7 foo (nullable, undefinable) // Output: false false true true
```

## 7.26 Bitwise and Logical Expressions

The *bitwise operators* and *logical operators* are as follows:

- AND operator `'&'`;
- Exclusive OR operator `'^'`; and
- Inclusive OR operator `'|'`.

```
bitwiseAndLogicalExpression:
  expression '&' expression
| expression '^' expression
| expression '|' expression
;
```

These operators have different precedence. The operator `'&'` has the highest, while `'|'` has the lowest precedence.

Operators group left-to-right. Each operator is commutative if the operand expressions have no side effects, and associative.

The bitwise and logical operators can compare two operands of a numeric type, or two operands of the `boolean` type. Otherwise, a compile-time error occurs.

### 7.26.1 Integer Bitwise Operators

The numeric types conversion (see *Primitive Types Conversions*) is performed first on the operands of operator ‘&’, ‘^’, or ‘|’ if both operands are of a type convertible (see *Implicit Conversions*) to a primitive integer type.

**Note.** If the initial type of one or both operands is `double` or `float`, then that operand or operands are truncated first to the appropriate integer type. If both operands are of type `bigint`, then no conversion is required.

A bitwise operator expression type is the converted type of its operands.

The resultant value of ‘&’ is the bitwise AND of the operand values.

The resultant value of ‘^’ is the bitwise exclusive OR of the operand values.

The resultant value of ‘|’ is the bitwise inclusive OR of the operand values.

Thus, *integer bitwise expression* is of primitive integer type.

### 7.26.2 Boolean Logical Operators

Type of the bitwise operator expression is `boolean` if both operands of operator ‘&’, ‘^’, or ‘|’ are of type `boolean` or `Boolean`. In any case, the unboxing conversion (see *Unboxing Conversions*) is performed on the operands if required.

If both operand values are `true`, then the resultant value of ‘&’ is `true`. Otherwise, the result is `false`.

If the operand values are different, then the resultant value of ‘^’ is `true`. Otherwise, the result is `false`.

If both operand values are `false`, then the resultant value of ‘|’ is `false`. Otherwise, the result is `true`.

Thus, *boolean logical expression* is of the boolean type.

## 7.27 Conditional-And Expression

The *conditional-and* operator ‘&&’ is similar to ‘&’ (see *Bitwise and Logical Expressions*) but evaluates its right-hand-side operand only if the value of the left-hand-side operand is `true`.

The results of computation of ‘&&’ and ‘&’ on `boolean` operands are the same, but the right-hand-side operand in ‘&&’ cannot be evaluated.

```
conditionalAndExpression:
    expression ' && ' expression
    ;
```

A *conditional-and* operator groups left-to-right.

A *conditional-and* operator is fully associative as regards both the result value and side effects (i.e., the evaluations of the expressions  $((a) \&\& (b)) \&\& (c)$  and  $(a) \&\& ((b) \&\& (c))$  produce the same result, and the same side effects occur in the same order for any  $a$ ,  $b$ , and  $c$ ).

A *conditional-and* expression is always of type `boolean` except the extended semantics (see [Extended Conditional Expressions](#)). A *conditional-and* expression with extended semantics can be of the first expression type.

Each operand of the *conditional-and* operator must be of type `boolean`, `Boolean`, or of a type mentioned in [Extended Conditional Expressions](#). Otherwise, a compile-time error occurs.

The left-hand-side operand expression is first evaluated at runtime. If the result is of type `Boolean`, then the unboxing conversion (see [Unboxing Conversions](#)) is performed as follows:

- If the resultant value is `false`, then the value of the *conditional-and* expression is `false`. The evaluation of the right-hand-side operand expression is omitted.
- If the value of the left-hand-side operand is `true`, then the right-hand-side expression is evaluated. If the result of the evaluation is of type `Boolean`, then the unboxing conversion (see [Unboxing Conversions](#)) is performed. The resultant value is the value of the *conditional-and* expression.

## 7.28 Conditional-Or Expression

The *conditional-or* operator `'||'` is similar to `'|'` (see [Integer Bitwise Operators](#)) but evaluates its right-hand-side operand only if the value of its left-hand-side operand is `false`.

```
conditionalOrExpression:
    expression '||' expression
    ;
```

A *conditional-or* operator groups left-to-right.

A *conditional-or* operator is fully associative as regards both the result value and side effects (i.e., the evaluations of the expressions  $((a) || (b)) || (c)$  and  $(a) || ((b) || (c))$  produce the same result, and the same side effects occur in the same order for any  $a$ ,  $b$ , and  $c$ ).

A *conditional-or* expression is always of type `boolean` except the extended semantics (see [Extended Conditional Expressions](#)). A *conditional-or* expression with extended semantics can be of the first expression type.

Each operand of the *conditional-or* operator must be of type `boolean` or `Boolean` or type mentioned in [Extended Conditional Expressions](#). Otherwise, a compile-time error occurs.

The left-hand-side operand expression is first evaluated at runtime. If the result is of type `Boolean`, then the *unboxing conversion* () is performed as follows:

- If the resultant value is `true`, then the value of the *conditional-or* expression is `true`, and the evaluation of the right-hand-side operand expression is omitted.
- If the resultant value is `false`, then the right-hand-side expression is evaluated. If the result of the evaluation is of type `Boolean`, then the *unboxing conversion* is performed (see [Unboxing Conversions](#)). The resultant value is the value of the *conditional-or* expression.

The computation results of `'||'` and `'|'` on `boolean` operands are the same, but the right-hand-side operand in `'||'` cannot be evaluated.



## 7.29 Assignment

All *assignment operators* group right-to-left (i.e.,  $a = b = c$  means  $a = (b = c)$ ). The value of  $c$  is thus assigned to  $b$ , and then the value of  $b$  to  $a$ ).

```
assignmentExpression:
    lhsExpression assignmentOperator rhsExpression
    ;

assignmentOperator
    : '='
    | '+=' | '-=' | '*=' | '=' | '%='
    | '<=<' | '>>=' | '>>>='
    | '&=' | '|=' | '^='
    ;

lhsExpression:
    expression
    ;

rhsExpression:
    expression
    ;
```

The first operand in an assignment operator represented by *lhsExpression* must be *left-hand-side expression* (see [Left-Hand-Side Expressions](#)). This first operand denotes a variable.

Type of the variable is the type of the assignment expression.

The result of the *assignment expression* at runtime is not a variable itself but the value of a variable after the assignment.

### 7.29.1 Simple Assignment Operator

The form of a simple assignment expression is  $E1 = E2$ .

A compile-time error occurs if type of the right-hand-side operand (*rhsExpression*) is not assignable (see [Assignability](#)) to the type of the variable. Otherwise, the expression is evaluated at runtime in one of the following ways:

1. If the left-hand-side operand *lhsExpression* is a field access expression  $e.f$  (see [Field Access Expression](#)), possibly enclosed in parentheses, then:
  1. *lhsExpression*  $e$  is evaluated: if the evaluation of  $e$  completes abruptly, then so does the assignment expression.
  2. Right-hand-side operand *rhsExpression* is evaluated: if the evaluation completes abruptly, then so does the assignment expression.
  3. Value of the right-hand-side operand as computed above is assigned to the variable denoted by  $e.f$ .
2. If the left-hand-side operand is an array reference expression (see [Array Indexing Expression](#)), possibly enclosed in parentheses, then:

1. Array reference subexpression of the left-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand and the index subexpression are not evaluated, and the assignment does not occur.
2. If the evaluation completes normally, then the index subexpression of the left-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and the assignment does not occur.
3. If the evaluation completes normally, then the right-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression, and the assignment does not occur.
4. If the evaluation completes normally, but the value of the index subexpression is less than zero, or greater than, or equal to the *length* of the array, then `ArrayIndexOutOfBoundsException` is thrown, and the assignment does not occur.
5. Otherwise, the value of the index subexpression is used to select an element of the array referred to by the value of the array reference subexpression.

That element is a variable of type `SC`. If `TC` is type of the left-hand-side operand of the assignment operator determined at compile time, then there are two options:

- If `TC` is a primitive type, then `SC` can only be the same as `TC`.

The value of the right-hand-side operand is converted to the type of the selected array element. The value set conversion (see [Implicit Conversions](#)) is performed to convert it to the appropriate standard value set (not an extended-exponent value set). The result of the conversion is stored into the array element.

- If `TC` is a reference type, then `SC` can be the same as `TC` or a type that extends or implements `TC`.

If the ArkTS compiler cannot guarantee at compile time that the array element is exactly of type `TC`, then a check must be performed at runtime to ensure that class `RC` is assignable to the actual type `SC` of the array element (see [Assignability with Initializer](#)). Class `RC` is the class of the object referred to by the value of the right-hand-side operand at runtime. If class `RC` is not assignable to type `SC`, then `ArrayStoreError` is thrown, and the assignment does not occur. Otherwise, the reference value of the right-hand-side operand is stored in the selected array element.

3. If the left-hand-side operand is a record access expression (see [Record Indexing Expression](#)), possibly enclosed in parentheses, then:
  1. Object reference subexpression of the left-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand and the index subexpression are not evaluated, and the assignment does not occur.
  2. If the evaluation completes normally, the index subexpression of the left-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and the assignment does not occur.
  3. If the evaluation completes normally, the right-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the assignment does not occur.
  4. Otherwise, the value of the index subexpression is used as the *key*. In that case, the right-hand-side operand is used as the *value*, and the key-value pair is stored in the record instance.

If none of the above is true, then the following three steps are required:

1. Left-hand-side operand is evaluated to produce a variable. If the evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and the assignment does not occur.
2. If the evaluation completes normally, then the right-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, the assignment does not occur.

3. If that evaluation completes normally, then the value of the right-hand-side operand is converted to the type of the left-hand-side variable. In that case, the result of the conversion is stored into the variable. A compile-time error occurs if type of the left-hand-side variable is one of the following:
  - `readonly array` (see [Readonly Parameters](#)), while the converted type of the right-hand-side operand is a `non-readonly array`;
  - `readonly tuple` (see [Readonly Parameters](#)), while the converted type of the right-hand-side operand is a `non-readonly tuple`.

## 7.29.2 Compound Assignment Operators

A compound assignment expression in the form  $E1 \text{ op} = E2$  is equivalent to  $E1 = ((E1) \text{ op } (E2))$  as  $T$ , where  $T$  is type of  $E1$ , except that  $E1$  is evaluated only once. This expression can be evaluated at runtime in one of the following ways:

1. If the left-hand-side operand expression is not an indexing expression:
  - The left-hand-side operand is evaluated to produce a variable. If the evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and no assignment occurs.
  - If the evaluation completes normally, then the value of the left-hand-side operand is saved, and the right-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.
  - If the evaluation completes normally, then the saved value of the left-hand-side variable, and the value of the right-hand-side operand are used to perform the binary operation as indicated by the compound assignment operator. If the operation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.
  - If the evaluation completes normally, then the result of the binary operation converts to the type of the left-hand-side variable. The result of such conversion is stored into the variable.
2. If the left-hand-side operand expression is an array reference expression (see [Array Indexing Expression](#)), then:
  - Array reference subexpression of the left-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand and the index subexpression are not evaluated, and no assignment occurs.
  - If the evaluation completes normally, then the index subexpression of the left-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and no assignment occurs.
  - If the evaluation completes normally, the value of the array reference subexpression refers to an array, and the value of the index subexpression is less than zero, greater than, or equal to the *length* of the array, then `ArrayIndexOutOfBoundsError` is thrown. In that case, no assignment occurs.
  - If the evaluation completes normally, then the value of the index subexpression is used to select an array element referred to by the value of the array reference subexpression. The value of this element is saved, and then the right-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.
  - If the evaluation completes normally, consideration must be given to the saved value of the array element selected in the previous step. While this element is a variable of type  $S$ , and  $T$  is type of the left-hand-side operand of the assignment operator determined at compile time:

- If `T` is a primitive type, then `S` is the same as `T`.

The saved value of the array element, and the value of the right-hand-side operand are used to perform the binary operation of the compound assignment operator.

If this operation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.

If this evaluation completes normally, then the result of the binary operation converts to the type of the selected array element. The result of the conversion is stored into the array element.

- If `T` is a reference type, then it must be `string`.

`S` must also be a `string` because the class `string` is the *final* class. The saved value of the array element, and the value of the right-hand operand are used to perform the binary operation (string concatenation) of the compound assignment operator `+=`. If this operation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.

- If the evaluation completes normally, then the `string` result of the binary operation is stored into the array element.

3. If the left-hand-side operand expression is a record access expression (see [Record Indexing Expression](#)):

- The object reference subexpression of the left-hand-side operand is evaluated. If this evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand and the index subexpression are not evaluated, and no assignment occurs.
- If this evaluation completes normally, then the index subexpression of the left-hand operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, the right-hand-side operand is not evaluated, and no assignment occurs.
- If this evaluation completes normally, the value of the object reference subexpression and the value of index subexpression are saved, then the right-hand-side operand is evaluated. If the evaluation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.
- If this evaluation completes normally, the saved values of the object reference subexpression and index subexpression (as the *key*) are used to get the *value* that is mapped to the *key* (see [Record Indexing Expression](#)), then this *value* and the value of the right-hand-side operand are used to perform the binary operation as indicated by the compound assignment operator. If the operation completes abruptly, then so does the assignment expression. In that case, no assignment occurs.
- If the evaluation completes normally, then the result of the binary operation is stored as the key-value pair in the record instance (as in [Simple Assignment Operator](#)).

### 7.29.3 Left-Hand-Side Expressions

*Expression* is a *left-hand-side expression* if it is one of the following:

- Named variable;
- Field or setter resultant from a field access (see [Field Access Expression](#)); or
- Array or record element access (see [Indexing Expressions](#)).

A compile-time error occurs if:

- *Expression* contains the chaining operator `?.` (see [Chaining Operator](#));
- *Expression* result is not a variable.

## 7.30 Conditional Expressions

The conditional expression ‘`? :`’ uses the boolean value of the first expression to decide which of the other two expressions to evaluate:

```
conditionalExpression:
    expression '?' expression ':' expression
    ;
```

The conditional operator ‘`? :`’ groups right-to-left (i.e., the meaning of `a?b : c?f : g` and `a?b : (c?f : g)` is the same).

The conditional operator ‘`? :`’ consists of three operand expressions with the separators ‘`?`’ between the first and the second expression, and ‘`:`’ between the second and the third expression.

A compile-time error occurs if the first expression is not of type `boolean`, `Boolean`, or a type mentioned in *Extended Conditional Expressions*.

Type of the conditional expression is determined as the union of types of the second and the third expressions further normalized in accordance with the process discussed in *Union Types Normalization*. If the second and the third expressions are of the same type, then this is the type of the conditional expression.

The following steps are performed as the evaluation of a conditional expression occurs at runtime:

1. The operand expression of a conditional expression is evaluated first. The unboxing conversion (see *Unboxing Conversions*) is performed on the result if necessary. If the unboxing conversion fails, then so does the evaluation of the conditional expression.
2. If the value of the first operand is `true`, then the second operand expression is evaluated. Otherwise, the third operand expression is evaluated. The result of successful evaluation is the result of the conditional expression.

The examples below represent different scenarios with standalone expressions:

```
1  class A {}
2  class B extends A {}
3
4  condition ? new A() : new B() // A | B => A
5
6  condition ? 5 : 6             // int
7
8  condition ? "5" : 6           // "5" | Int
```

## 7.31 String Interpolation Expressions

‘*String interpolation expression*’ is a multiline string literal (a string literal delimited with backticks, see *Multiline String Literal* for details) that contains at least one *embedded expression*:

```
stringInterpolation:
    '`' (BacktickCharacter | embeddedExpression) * '`'
    ;

embeddedExpression:
    '${' expression '}'
    ;
```

An *embedded expression* is an expression specified inside curly braces preceded by the *dollar sign* ‘\$’. A string interpolation expression is of type `string` (see *Type string*).

When evaluating a *string interpolation expression*, the result of each embedded expression substitutes that embedded expression. An embedded expression must be of type `string`. Otherwise, the implicit conversion to `string` takes place in the same way as with the string concatenation operator (see *String Concatenation*):

```
1 let a = 2
2 let b = 2
3 console.log(`The result of ${a} * ${b} is ${a * b}`)
4 // prints: The result of 2 * 2 is 4
```

The string concatenation operator can be used to rewrite the above example as follows:

```
1 let a = 2
2 let b = 2
3 console.log("The result of " + a + " * " + b + " is " + a * b)
```

An embedded expression can contain nested multiline strings.

## 7.32 Lambda Expressions

*Lambda expression* fully defines an instance of a function type (see *Function Types*) by providing optional `async` mark, type parameters (see *Type Parameters*), mandatory lambda signature, and its body. The definition of *lambda expression* is generally similar to that of a function declaration (see *Function Declarations*), except that a lambda expression has no function name specified, and can have types of parameters omitted:

```
lambdaExpression:
    annotationUsage?
    ('async' | typeParameters)? lambdaSignature '=>' lambdaBody
    ;

lambdaBody:
    expression | block
    ;

lambdaSignature:
    '(' lambdaParameterList? ')' returnType?
    | identifier
    ;

lambdaParameterList:
    lambdaParameter (',' lambdaParameter) * (',' restParameter)? ','?
```

(continues on next page)

(continued from previous page)

```

    | restParameter ',' '?'
    ;

lambdaParameter:
    annotationUsage? (lambdaRequiredParameter | lambdaOptionalParameter)
    ;

lambdaRequiredParameter:
    identifier ':' type?
    ;

lambdaOptionalParameter:
    identifier '?' (':' type)?
    ;

lambdaRestParameter:
    '...' lambdaRequiredParameter
    ;

```

The usage of annotations is discussed in *Using Annotations*.

The examples of usage are presented below:

```

1  (x: number): number => { return Math.sin(x) } // block as lambda body
2  (x: number) => Math.sin(x)                    // expression as lambda body
3  <T> (x: T, y: T) => { let local = x }          // generic lambda
4  e => e                                         // shortest form of lambda

```

A *lambda expression* evaluation creates an instance of a function type (see *Function Types*) as described in detail in *Runtime Evaluation of Lambda Expressions*.

### 7.32.1 Lambda Signature

Similarly to function declarations (see *Function Declarations*), a *lambda signature* is composed of formal parameters and optional return types. Unlike function declarations, type annotations of formal parameters can be omitted.

```

1  function foo<T> (a: (p1: T, ...p2: T[]) => T) {}
2  // All calls to foo pass valid lambda expressions in different forms
3  foo (e => e)
4  foo ((e1, e2) => e1)
5  foo ((e1, e2: Object) => e1)
6  foo ((e1: Object, e2) => e1)
7  foo ((e1: Object, e2, e3) => e1)
8  foo ((e1: Object, ...e2) => e1)
9
10 foo (<Object>(e1: Object, e2: Object) => e1)
11
12 function bar<T> (a: (...p: T[]) => T) {}
13 // Type can be omitted for the rest parameter
14 bar (...e) => e
15
16 function goo<T> (a: (p?: T) => T) {}

```

(continues on next page)

(continued from previous page)

```

17 // Type can be omitted for the optional parameter
18 goo ((e?) => e)

```

The specification of scope is discussed in *Scopes*, and shadowing details of formal parameter declarations in *Shadowing by Parameter*.

A compile-time error occurs if:

- Lambda expression declares two formal parameters with the same name.
- Formal parameter contains no type provided, and type cannot be derived by type inference.

### 7.32.2 Lambda Body

*Lambda body* can be a single expression or a block (see *Block*). Similarly to the body of a method or a function, a lambda body describes the code to be executed when a lambda expression call occurs (see *Function Call Expression*).

The meanings of names, and of the keywords `this` and `super` (along with the accessibility of the referred declarations) are the same as in the surrounding context. However, lambda parameters introduce new names.

If a *lambda body* is a single expression, then it is equivalent to the block with one return statement that returns that single expression *{ return singleExpression }*.

If any local variable or formal parameter of the surrounding context is used but not declared in a lambda body, then the local variable or formal parameter is *captured* by the lambda.

If an instance member of the surrounding type is used in the lambda body defined in a method, then `this` is *captured* by the lambda.

A compile-time error occurs if a local variable is used in a lambda body but is neither declared in nor assigned before it.

If *lambda signature* return type is not `void` (see *Type void*), and the execution path of the lambda body has no return statement (see *return Statements*) or no single expression as a body, then a compile-time error occurs.

### 7.32.3 Lambda Expression Type

*Lambda expression type* is a function type (see *Function Types*) that has the following:

- Lambda parameters (if any) as parameters of the function type; and
- Lambda return type as the return type of the function type.

**Note.** Lambda return type can be inferred from the *lambda body* and thus the return type can be dropped off.

```

1  const lambda = () => { return 123 } // Type of the lambda is () => int
2  const int_var: int = lambda()

```



### 7.32.4 Runtime Evaluation of Lambda Expressions

The evaluation of a lambda expression itself never causes the execution of the lambda body. If completing normally at runtime, the evaluation of a lambda expression produces a reference to an allocated and initialized new instance of a function type (see *Function Types*) that corresponds to the lambda signature. In that case, it is similar to the evaluation of a class instance creation expression (see *New Expressions*).

If the available space is not sufficient for a new instance to be created, then the evaluation of the lambda expression completes abruptly, and `OutOfMemoryError` is thrown.

During a lambda expression evaluation, the captured values of the lambda expression are saved to the internal state of the created instance.

| Source Fragment                                                                                                     | Output |
|---------------------------------------------------------------------------------------------------------------------|--------|
| <pre> 1  function foo() { 2    let y: int = 1 3    let x = () =&gt; { return y+1 } 4    console.log(x()) 5  }</pre> | 2      |

The variable 'y' is *captured* by the lambda.

The captured variable is not a copy of the original variable. If the value of the variable captured by the lambda changes, then the original variable is implied to change:

| Source Fragment                                                                                                                         | Output |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre> 1  function foo() { 2    let y: int = 1 3    let x = () =&gt; { y++ } 4    console.log(y) 5    x() 6    console.log(y) 7  }</pre> | 1<br>2 |

In order to make lambdas behave as required, the language implementation can act as follows:

- Replace primitive type for the corresponding boxed type (x: int to x: Int) if the captured variable is of primitive type;
- Replace the captured variable's type for a proxy class that contains an original reference (x: T for x: Proxy<T>; x.ref = original-ref) if that captured variable is of non-primitive type.

If the captured variable is defined as `const`, then neither boxing nor proxying is required.

If the captured formal parameter can be neither boxed nor proxied, then the implementation can require addition of a local variable as follows:

| Source Code                                                                                        | Pseudo Code                                                                                                              |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <pre> 1  function foo(y: int) { 2  let x = () =&gt; { return y+1 } 3  console.log(x()) 4  } </pre> | <pre> 1  function foo(y: int) { 2  let y\$: Int = y 3  let x = () =&gt; { return y\$+1 } 4  console.log(x()) 5  } </pre> |

## 7.33 Constant Expressions

*Constant expressions* are expressions with values that can be evaluated at compile time.

```

constantExpression:
    expression
    ;

```

A *constant expression* is an expression of a primitive type (see *Primitive Types*), or of type `string` that completes normally while being composed only of the following:

- Literals of a primitive type, and literals of type `string` (see *Literals*);
- Conversions to primitive types and conversions to type `string` (see *Cast Expressions*);
- Unary operators `'+'`, `'-'`, `'~'`, and `'!'`, but not `'++'` or `'--'` (see *Unary Plus*, *Unary Minus*, *Prefix Increment*, and *Prefix Decrement*);
- Multiplicative operators `'*'`, `'/'`, and `'%'` (see *Multiplicative Expressions*);
- Additive operators `'+'` and `'-'` (see *Additive Expressions*);
- Shift operators `'<<'`, `'>>'`, and `'>>>'` (see *Shift Expressions*);
- Relational operators `'<'`, `'<='`, `'>'`, and `'>='` (see *Relational Expressions*);
- Equality operators `'=='` and `'!=='` (see *Equality Expressions*);
- Bitwise and logical operators `'&'`, `'^'`, and `'|'` (see *Bitwise and Logical Expressions*);
- Conditional-and operator `'&&'` (see *Conditional-And Expression*), and conditional-or operator `'||'` (see *Conditional-Or Expression*);
- Ternary conditional operator `'? :'` (see *Conditional Expressions*);
- Parenthesized expressions (see *Parenthesized Expression*) that contain constant expressions;
- Simple names or qualified names that refer to constants (see *Constant Declarations*) with constant expressions as initializers, declared in the same compilation unit.

Statements are designed to control execution:

```
statement :  
    expressionStatement  
    | block  
    | localDeclaration  
    | ifStatement  
    | loopStatement  
    | breakStatement  
    | continueStatement  
    | returnStatement  
    | switchStatement  
    | throwStatement  
    | tryStatement  
    ;
```

### 8.1 Normal and Abrupt Statement Execution

The actions that every statement performs in a normal mode of execution are specific for the particular kind of statement. Normal modes of evaluation for each kind of statement are described in the following sections.

A statement execution is considered to *complete normally* if the desired action is performed without an error being thrown. On the contrary, a statement execution is considered to *complete abruptly* if it causes an error thrown.

## 8.2 Expression Statements

Any expression can be used as a statement:

```
expressionStatement:  
    expression  
    ;
```

The execution of a statement leads to the execution of the expression. The result of such execution is discarded.

## 8.3 Block

A sequence of statements (see *Statements*) enclosed in balanced braces forms a *block*:

```
block:  
    '{' statement* '}'  
    ;
```

The execution of a block means that all block statements, except type declarations, are executed one after another in the textual order of their appearance within the block until an erroneous situation is thrown (see *Errors*), or return (see *return Statements*) occurs.

If a block is the body of a `functionDeclaration` (see *Function Declarations*) or a `classMethodDeclaration` (see *Method Declarations*) declared implicitly or explicitly with return type `void` (see *Type void*), then the block can contain no return statement at all. Such a block is equivalent to one that ends in a `return` statement, and is executed accordingly.

## 8.4 Local Declarations

*Local declarations* define new mutable or immutable variables or types within the enclosing context.

`let` and `const` declarations have the initialization part that presumes execution, and actually act as statements:

```
localDeclaration:  
    annotationUsage?  
    (  
        variableDeclaration  
    |   constantDeclaration  
    |   typeDeclaration  
    )  
    ;
```

The visibility of a local declaration name is determined by the surrounding function or method, and by the block scope rules (see *Scopes*).

The usage of annotations is discussed in *Using Annotations*.

## 8.5 if Statements

An `if` statement allows executing alternative statements (if provided) under certain conditions:

```
ifStatement:
    'if' '(' expression ')' thenStatement
    ('else' elseStatement)?
    ;

thenStatement:
    statement
    ;

elseStatement:
    statement
    ;
```

Type of expression must be `boolean`, `Boolean`, or a type mentioned in *Extended Conditional Expressions*. Otherwise, a compile-time error occurs.

If an expression is successfully evaluated as `true`, then *thenStatement* is executed. Otherwise, *elseStatement* is executed (if provided).

Any `else` corresponds to the first `if` of an `if` statement:

```
1  if (Cond1)
2  if (Cond2) statement1
3  else statement2 // Executes only if: Cond1 && !Cond2
```

A list of statements in braces (see *Block*) is used to combine the `else` part with the first `if`:

```
1  if (Cond1) {
2      if (Cond2) statement1
3  }
4  else statement2 // Executes if: !Cond1
```

## 8.6 Loop Statements

ArkTS has four kinds of loops. A loop of each kind can have an optional loop label that can be used only by `break` and `continue` statements contained in the body of the loop. The label is characterized by an *identifier* as shown below:

```
loopStatement:
    (identifier ':' )?
    whileStatement
    | doStatement
    | forStatement
```

(continues on next page)

(continued from previous page)

```
| forOfStatement  
;
```

An *identifier* at the beginning of a loop statement denotes a *label* that can be used in *break Statements* and *continue Statements*.

A compile-time error occurs if the label *identifier* is not used within *loopStatement*.

## 8.7 while Statements and do Statements

A *while* statement and a *do* statement evaluate an expression and execute the statement repeatedly till the expression value is *true*. The key difference is that *whileStatement* starts from evaluating and checking the expression value, and *doStatement* starts from executing the statement:

```
whileStatement:  
    'while' '(' expression ')' statement  
    ;  
  
doStatement  
    : 'do' statement 'while' '(' expression ')' '  
    ;
```

Type of expression must be *boolean*, *Boolean*, or a type mentioned in *Extended Conditional Expressions*. Otherwise, a compile-time error occurs.

## 8.8 for Statements

```
forStatement:  
    'for' '(' forInit? ';' forContinue? ';' forUpdate? ')' statement  
    ;  
  
forInit:  
    expressionSequence  
    | variableDeclarations  
    ;  
  
forContinue:  
    expression  
    ;  
  
forUpdate:  
    expressionSequence  
    ;
```

Type of *forContinue* expression must be boolean, Boolean, or a type mentioned in *Extended Conditional Expressions*. Otherwise, a compile-time error occurs.

```

1  // existing variable is used as a loop index variable
2  let i: number
3  for (i = 1; i < 10; i++) {
4      console.log(i)
5  }
6
7  // new variable is declared as a loop index variable with its type
8  // explicitly specified
9  for (let i: number = 1; i < 10; i++) {
10     console.log(i)
11 }
12
13 // new variable is declared as loop index variable with its type
14 // inferred from its initialization part of the declaration
15 for (let i = 1; i < 10; i++) {
16     console.log(i)
17 }

```

## 8.9 for-of Statements

A for-of loop iterates elements of array or string, or an instance of *iterable* class or interface (see *Iterable Types*):

```

forOfStatement:
    'for' '(' forVariable 'of' expression ')' statement
    ;

forVariable:
    identifier | ('let' | 'const') identifier (':' type)?
    ;

```

A compile-time error occurs if the type of an expression is not array, string, or an iterable type.

The execution of a for-of loop starts from the evaluation of *expression*. If the evaluation is successful, then the resultant expression is used for loop iterations (execution of the statement). On each iteration, *forVariable* is set to successive elements of the array, string, or the result of class iterator advancing.

If *forVariable* has the modifiers *let* or *const*, then a new variable is used inside the loop. Otherwise, the variable is as declared above. The modifier *const* prohibits assignments into *forVariable*, while *let* allows modifications.

Explicit type annotation of *forVariable* is allowed as an experimental feature (see *For-of Type Annotation*).

```

1  // existing variable 'ch'
2  let ch : char
3  for (ch of "a string object") {
4      console.log(ch)
5  }
6
7  // new variable 'ch', its type is inferred from expression after 'of'

```

(continues on next page)

(continued from previous page)

```
8  for (let ch of "a string object") {  
9      console.log(ch)  
10 }  
11  
12 // new variable 'element', its type is inferred from expression after 'of',  
13 // and it cannot be assigned with a new value in the loop body  
14 for (const element of [1, 2, 3]) {  
15     console.log(element)  
16     element = 66 // Compile-time error as 'element' is 'const'  
17 }
```

## 8.10 break Statements

A `break` statement transfers control out of the enclosing *loopStatement* or *switchStatement*:

```
breakStatement:  
    'break' identifier?  
    ;
```

A `break` statement with the label *identifier* transfers control out of the enclosing statement with the same label *identifier*. If there is no enclosing loop statement with the same label *identifier* (within the body of the surrounding function or method), then a compile-time error occurs.

A statement without a label transfers control out of the innermost enclosing `switch`, `while`, `do`, `for`, or `for-of` statement. If `breakStatement` is placed outside *loopStatement* or *switchStatement*, then a compile-time error occurs.

## 8.11 continue Statements

A `continue` statement stops the execution of the current loop iteration, and transfers control to the next iteration. Appropriate checks of loop exit conditions depend on the kind of the loop.

```
continueStatement:  
    'continue' identifier?  
    ;
```

A `continue` statement with the label *identifier* transfers control to the next iteration of the enclosing loop statement with the same label *identifier*. If there is no enclosing loop statement with the same label *identifier* (within the body of the surrounding function or method), then a compile-time error occurs.



## 8.12 return Statements

A `return` statement can have or not have an expression.

```
returnStatement:
    'return' expression?
    ;
```

A *return expression* statement can only occur inside a function or a method body.

A `return` statement (with no expression) can occur in one of the following situations:

- Inside a class initializer;
- Inside a constructor body; or
- Inside a function or a method body with return type `void` (see *Type void*).

A compile-time error occurs if a `return` statement is found in:

- Top-level statements (see *Top-Level Statements*);
- Class initializers (see *Class Initializer*) and constructors (see *Constructor Declaration*), where it has an expression;
- Functions or methods with return type `void` (see *Type void*) that have an expression;
- Functions or methods with a non-`void` return type that have no expression.

The execution of *returnStatement* leads to the termination of the surrounding function or method. If an *expression* is provided, the resultant value is the evaluated *expression*.

In case of constructors, class initializers, and top-level statements, the control is transferred out of the scope of the construction in question, but no result is required. Other statements of the surrounding function, method body, class initializer, or top-level statement are not executed.

## 8.13 switch Statements

A `switch` statement transfers control to a statement or a block by using the result of successful evaluation of the value of a `switch` expression.

```
switchStatement:
    (identifier ':' )? 'switch' '(' expression ')' switchBlock
    ;

switchBlock
    : '{' caseClause* defaultClause? caseClause* '}'
    ;

caseClause
    : 'case' expression ':' statement*
    ;

defaultClause
```

(continues on next page)

(continued from previous page)

```
: 'default' ':' statement*  
;
```

The switch expression type must be of type `char`, `byte`, `short`, `int`, `long`, `Char`, `Byte`, `Short`, `Int`, `Long`, `string`, or `enum`.

A compile-time error occurs if not **all** of the following is true:

- Every case expression type is assignable (see [Assignability](#)) to the type of the `switch` statement expression.
- In a `switch` statement expression of type `enum`, every case expression associated with the `switch` statement is of type `enum`.
- No two case constant expressions (see [Constant Expressions](#)) have identical values.
- No case expression is `null`.

```
1 let arg = prompt("Enter a value?");  
2 switch (arg) {  
3   case '0':  
4   case '1':  
5     alert('One or zero')  
6     break  
7   case '2':  
8     alert('Two')  
9     break  
10  default:  
11    alert('An unknown value')  
12 }
```

The execution of a `switch` statement starts from the evaluation of the `switch` expression. If the evaluation result is of type `Char`, `Byte`, `Short`, `Int`, or `Long`, then the unboxing conversion (see [Unboxing Conversions](#)) follows.

Otherwise, the value of the `switch` expression is compared repeatedly to the value of each case expression.

If a case expression value equals the value of the `switch` expression in terms of the operator `'=='`, then the case label *matches*.

However, if the expression value is a `string`, then the equality for strings determines the equality.

## 8.14 throw Statements

A `throw` statement causes an *error* object to be created and raised (see [Error Handling](#)). It immediately transfers control, and can exit multiple statements, constructors, functions, and method calls until a `try` statement (see [try Statements](#)) is found that catches the value thrown. If no `try` statement is found, then `UncaughtExceptionError` is thrown.

```
throwStatement:  
  'throw' expression  
;
```

The expression type must be assignable (see [Assignment](#)) to type `Error`. Otherwise, a compile-time error occurs.

This implies that the object thrown is never `null`.

Errors can be thrown at any place in the code.

## 8.15 try Statements

A `try` statement runs blocks of code, and provides sets of catch clauses to handle different errors (see [Error Handling](#)).

```
tryStatement:
    'try' block catchClause? finallyClause?
    ;

catchClause:
    'catch' '(' identifier ')' block
    ;

finallyClause:
    'finally' block
    ;
```

A `try` statement must contain either a `finally` clause, or a `catch` clause. Otherwise, a compile-time error occurs.

If the `try` block completes normally, then no action is taken, and no `catch` clause block is executed.

If an error is thrown in the `try` block directly or indirectly, then the control is transferred to the `catch` clause.

### 8.15.1 catch Clause

A `catch` clause consists of two parts:

- A *catch identifier* that provides access to an object associated with the *error* thrown; and
- A block of code that handles the error.

The type of *catch identifier* inside the block is `Error` (see [Error Handling](#)).

```
1  class ZeroDivisor extends Error {}
2
3  function divide(a: number, b: number): number {
4      if (b == 0)
5          throw new ZeroDivisor()
6      return a / b
7  }
8
9  function process(a: number, b: number): number {
10     try {
11         let res = divide(a, b)
12
13         // further processing ...
14     }
15     catch (e) {
```

(continues on next page)

(continued from previous page)

```
16     return e instanceof ZeroDivisor? -1 : 0
17   }
18 }
```

A catch clause handles all errors at runtime. It returns `'-1'` for the `ZeroDivisor`, and `'0'` for all other errors.

### 8.15.2 finally Clause

A `finally` clause defines the set of actions in the form of a block to be executed without regard to whether a `try-catch` completes normally or abruptly.

```
finallyClause:
  'finally' block
;
```

A `finally` block is executed without regard to how (by reaching `return` or `try-catch` end or raising new *error*) the program control is transferred out. The `finally` block is particularly useful to ensure proper resource management.

Any required actions (e.g., flush buffers and close file descriptors) can be performed while leaving the `try-catch`:

```
class SomeResource {
  // some API
  // ...
  close() {}
}

function ProcessFile(name: string) {
  let r = new SomeResource()
  try {
    // some processing
  }
  finally {
    // finally clause will be executed after try-catch is
    // executed normally or abruptly
    r.close()
  }
}
```

### 8.15.3 try Statement Execution

1. A `try` block and the entire `try` statement complete normally if no `catch` block is executed. The execution of a `try` block completes abruptly if an error is thrown inside the `try` block.
2. The execution of a `try` block completes abruptly if error *x* is thrown inside the `try` block. If the `catch` clause is present and the execution of the body of the `catch` clause completes normally, then the entire `try` statement completes normally. Otherwise, the `try` statement completes abruptly.

3. If no `catch` clause in place then the error is propagated to the surrounding scope until it reaches the scope with the `catch` clause which handles the error. If no such scope exists then `UncaughtExceptionError` is thrown and program execution terminates (see *Program Exit*).
4. If `finally` clause in place and its execution completes abruptly then the `try` statement completes abruptly as well.



Class declarations introduce new reference types and describe the manner of their implementation.

Classes can be *top-level* and local (see *Local Classes and Interfaces*).

A class body contains declarations and class initializers.

Declarations can introduce class members (see *Class Members*) or class constructors (see *Constructor Declaration*).

The body of the declaration of a member comprises the scope of a declaration (see *Scopes*).

Class members include:

- Fields,
- Methods, and
- Accessors.

Class members can be *declared* or *inherited*.

Every member is associated with the class declaration it is declared in.

Field, method, accessor and constructor declarations can have the following access modifiers (see *Access Modifiers*):

- `Public`,
- `Protected`,
- `Internal`, or
- `Private`.

A newly declared method can shadow, overload, implement, or override a method declared in a superclass or superinterface.

Every class defines two class-level scopes (see *Scopes*): one for instance members, and the other for static members. It means that two members of a class can have the same name if one is static while the other is not.

## 9.1 Class Declarations

Every class declaration defines a *class type*, i.e., a new named reference type.

The class name is specified by an *identifier* inside a class declaration.

If `typeParameters` are defined in a class declaration, then that class is a *generic class* (see *Generics*).

```
classDeclaration:
    classModifier? 'class' identifier typeParameters?
    classExtendsClause? implementsClause? classBody
    ;

classModifier:
    'abstract' | 'final'
    ;
```

The scope of a class declaration is specified in *Scopes*.

An example of a class is presented below:

```
1  class Point {
2      public x: number
3      public y: number
4      public constructor(x : number, y : number) {
5          this.x = x
6          this.y = y
7      }
8      public distanceBetween(other: Point): number {
9          return Math.sqrt(
10             (this.x - other.x) * (this.x - other.x) +
11             (this.y - other.y) * (this.y - other.y)
12         )
13     }
14     static origin = new Point(0, 0)
15 }
```

### 9.1.1 Abstract Classes

A class with the modifier `abstract` is known as abstract class. Abstract classes can be used to represent notions that are common to some set of more concrete notions.

A compile-time error occurs if an attempt is made to create an instance of an abstract class:

```
1  abstract class X {
2      field: number
3      constructor (p: number) { this.field = p }
4  }
5  let x = new X (666)
6  // Compile-time error: Cannot create an instance of an abstract class.
```

Subclasses of an abstract class can be abstract or non-abstract. A non-abstract subclass of an abstract superclass can be instantiated. As a result, a constructor for the abstract class, and field initializers for non-static fields of that class are executed:



```

1 abstract class Base {
2     field: number
3     constructor (p: number) { this.field = p }
4 }
5
6 class Derived extends Base {
7     constructor (p: number) { super(p) }
8 }

```

A method with the modifier `abstract` is considered an *abstract method* (see [Abstract Methods](#)). Abstract methods have no bodies, i.e., they can be declared but not implemented.

Only abstract classes can have abstract methods. A compile-time error occurs if a non-abstract class has an abstract method:

```

1 class Y {
2     abstract method (p: string)
3     /* Compile-time error: Abstract methods can only
4        be within an abstract class. */
5 }

```

A compile-time error occurs if an abstract method declaration contains the modifiers `final` or `override`.

## 9.1.2 Final Classes

Final classes are discussed in the chapter [Experimental Features](#) (see [Final Classes](#)).

## 9.1.3 Class Extension Clause

All classes except class `Object` can contain the `extends` clause that specifies the *base class*, or the *direct superclass* of the current class. In this situation, the current class is a *derived class*, or a *direct subclass*. Any class, except class `Object` that has no `extends` clause, is assumed to have the `extends Object` clause.

```

classExtendsClause:
    'extends' typeReference
    ;

```

A compile-time error occurs if:

- An `extends` clause appears in the definition of the class `Object`, which is the top of type hierarchy, and has no superclass.
- Class type named by `typeReference` is not accessible (see [Accessible](#)).
- The `extends` graph has a cycle.
- `typeReference` refers directly to, or is an alias of interface, or of any non-class type, e.g., of primitive, array, string, enumeration, union, function, or utility type.

*Class extension* implies that a class inherits all members of the direct superclass.

**Note.** Private members are inherited from superclasses, but are not accessible (see [Accessible](#)) within subclasses:

```

1  class Base {
2      // All methods are mutually accessible in the class where
3      they were declared
4      public publicMethod () {
5          this.protectedMethod()
6          this.privateMethod()
7      }
8      protected protectedMethod () {
9          this.publicMethod()
10         this.privateMethod()
11     }
12     private privateMethod () {
13         this.publicMethod();
14         this.protectedMethod()
15     }
16 }
17 class Derived extends Base {
18     foo () {
19         this.publicMethod()    // OK
20         this.protectedMethod() // OK
21         this.privateMethod()  // compile-time error:
22                               // the private method is inaccessible
23     }
24 }

```

The transitive closure of a *direct subclass* relationship is the *subclass* relationship. Class A can be a subclass of class C if:

- Class A is the direct subclass of C; or
- Class A is a subclass of some class B, which is in turn a subclass of C (i.e., the definition applies recursively).

Class C is a *superclass* of class A if A is its subclass.

## 9.1.4 Class Implementation Clause

A class can implement one or more interfaces. Interfaces to be implemented by a class are listed in the `implements` clause. Interfaces listed in this clause are *direct superinterfaces* of the class:

```

implementsClause:
    'implements' interfaceTypeList
    ;

interfaceTypeList:
    typeReference (',' typeReference)*
    ;

```

A compile-time error occurs if `typeReference` fails to name an accessible interface type (see [Accessible](#)).

If some interface is repeated as a direct superinterface in a single `implements` clause (even if that interface is named differently), then all repetitions are ignored.

For the class declaration  $C \langle F_1, \dots, F_n \rangle$  ( $n \geq 0, C \neq \text{Object}$ ):

- *Direct superinterfaces* of class type  $C \langle F_1, \dots, F_n \rangle$  are the types specified in the `implements` clause of the declaration of  $C$  (if there is an `implements` clause).

For the generic class declaration  $C \langle F_1, \dots, F_n \rangle$  ( $n > 0$ ):

- *Direct superinterfaces* of the parameterized class type  $C \langle T_1, \dots, T_n \rangle$  are all types  $I \langle U_1\theta, \dots, U_k\theta \rangle$  if:
  - $T_i$  ( $1 \leq i \leq n$ ) is a type;
  - $I \langle U_1, \dots, U_k \rangle$  is the direct superinterface of  $C \langle F_1, \dots, F_n \rangle$ ; and
  - $\theta$  is the substitution  $[F_1 := T_1, \dots, F_n := T_n]$ .

Interface type  $I$  is a superinterface of class type  $C$  if  $I$  is one of the following:

- Direct superinterface of  $C$ ;
- Superinterface of  $J$  which is in turn a direct superinterface of  $C$  (see *Superinterfaces and Subinterfaces* that defines superinterface of an interface); or
- Superinterface of the direct superclass of  $C$ .

A class *implements* all its superinterfaces.

A compile-time error occurs if a class implements two interface types that represent different instantiations of the same generic interface (see *Generics*).

If a class is not declared *abstract*, then:

- Any abstract method of each direct superinterface is implemented (see *Inheritance*) by a declaration in that class.
- The declaration of an existing method is inherited from a direct superclass, or a direct superinterface.

If superinterfaces have default implementations (see *Default Interface Method Declarations*) for some method  $m$ , then:

- The class that implements these interfaces must have method  $m$  declared with an override-compatible signature (see *Override-Compatible Signatures*); or
- All these methods refer to the same implementation, and this default implementation is the current class method.

Otherwise, a compile-time error occurs.

```

1  interface I1 { foo () {} }
2  interface I2 { foo () {} }
3  class C1 implements I1, I2 {
4      foo () {} // foo() from C1 overrides both foo() from I1 and foo() from I2
5  }
6  class C2 implements I1, I2 {
7      // Compile-time error as foo() from I1 and foo() from I2 have different
↪ implementations
8  }
9  interface I3 extends I1 {}
10 interface I4 extends I1 {}
11 class C3 implements I3, I4 {
12     // OK, as foo() from I3 and foo() from I4 refer to the same implementation
13 }

```

A single method declaration in a class is allowed to implement methods of one or more superinterfaces.

A compile-time error occurs if a class field has the same name as a method from one of superinterfaces implemented by the class, except when one is static and the other is not.

## 9.1.5 Implementing Interface Properties

A class must implement all properties from all superinterfaces (see *Interface Properties*) that are always defined as a getter, a setter, or both. Providing implementation for the property in the form of a field is not necessary:

```

1  interface Style {
2      get color(): string
3      set color(s: string)
4  }
5
6  class StyleClassOne implements Style {
7      color: string = ""
8  }
9
10 class StyleClassTwo implements Style {
11     private color_: string = ""
12
13     get color(): string {
14         return this.color_
15     }
16
17     set color(s: string) {
18         this.color_ = s
19     }
20 }

```

If a property is defined in a form that requires a setter, then the implementation of the property in the form of a readonly field causes a compile-time error:

```

1  interface Style {
2      set color(s: string)
3      writable: number
4  }
5
6  class StyleClassTwo implements Style {
7      readonly color: string = "" // compile-time error
8      readonly writable: number = 0 // compile-time error
9  }
10
11 function write_into_read_only (s: Style) {
12     s.color = "Black"
13     s.writable = 666
14 }
15
16 write_into_read_only (new StyleClassTwo)

```

If a property is defined in the readonly form, then the implementation of the property can either keep the readonly form or extend it to a writable form as follows:

```

1  interface Style {
2      get color(): string
3      readonly readable: number
4  }
5

```

(continues on next page)

(continued from previous page)

```

6  class StyleClassThree implements Style {
7      get color(): string { return "Black" }
8      set color(s: string) {} // OK!
9      readable: number = 0 // OK!
10 }
11
12 function how_to_write (s: Style) {
13     s.color = "Black" // compile-time error
14     s.readable = 666 // compile-time error
15     if (s instanceof StyleClassThree) {
16         let s1 = s as StyleClassThree
17         s1.color = "Black" // OK!
18         s1.readable = 666 // OK!
19     }
20 }
21
22 how_to_write (new StyleClassThree)

```

## 9.2 Class Body

A *class body* can contain declarations of the following members:

- Fields,
- Methods,
- Accessors,
- Constructors, and
- Class initializers.

```

classBody:
    '{ '
        classBodyDeclaration* classInitializer? classBodyDeclaration*
    '}'
    ;

classBodyDeclaration:
    annotationUsage?
    accessModifier?
    ( constructorDeclaration
    | classFieldDeclaration
    | classMethodDeclaration
    | classAccessorDeclaration
    )
    ;

```

Declarations can be inherited or immediately declared in a class. Any declaration within a class has a class scope. The class scope is fully defined in *Scopes*.

The usage of annotations is discussed in *Using Annotations*.

## 9.3 Class Members

Class members are as follows:

- Members inherited from their direct superclass (see *Inheritance*), except class `Object` that cannot have a direct superclass.
- Members declared in a direct superinterface (see *Superinterfaces and Subinterfaces*).
- Members declared in the class body (see *Class Body*).

Class members declared `private` are not accessible (see *Accessible*) to all subclasses of the current class.

Class members declared `protected` or `public` are inherited by all subclasses of the class and accessible (see *Accessible*) for all subclasses.

Class members declared `internal` are accessible within the package the current class resides in. They are inherited by all subclasses of the current class.

Constructors and class initializers are not members, and are not inherited.

Members can be as follows:

- Class fields (see *Field Declarations*),
- Methods (see *Method Declarations*), and
- Accessors (see *Accessor Declarations*).

A *method* is defined by the following:

1. *Type parameter*, i.e., the declaration of any type parameter of the method member.
2. *Argument type*, i.e., the list of types of arguments applicable to the method member.
3. *Return type*, i.e., the return type of the method member.

Members can be as follows:

- Static members that are not part of class instances, and can be accessed by using a qualified name notation (see *Names*) anywhere the class name is accessible (see *Accessible*); and
- Non-static, or instance members that belong to any instance of the class.

All names in static and separately in non-static class declaration scopes (see *Scopes*) must be unique, i.e., fields and methods cannot have the same name.

## 9.4 Access Modifiers

Access modifiers define how a class member or a constructor can be accessed. Accessibility in ArkTS can be of the following kinds:

- `Private`,
- `Internal`,

- Protected, or
- Public.

The desired accessibility of class members and constructors can be explicitly specified by the corresponding *access modifiers*:

```
accessModifier:
  'private'
  | 'internal'
  | 'protected'
  | 'public'
  ;
```

If no explicit modifier is provided, then a class member or a constructor is implicitly considered `public` by default.

### 9.4.1 Private Access Modifier

The modifier `private` indicates that a class member or a constructor is accessible (see [Accessible](#)) within its declaring class, i.e., a private member or constructor *m* declared in some class *C* can be accessed only within the class body of *C*:

```
1  class C {
2    private count: number
3    getCount(): number {
4      return this.count // ok
5    }
6  }
7
8  function increment(c: C) {
9    c.count++ // compile-time error - 'count' is private
10 }
```

### 9.4.2 Internal Access Modifier

The modifier `internal` is discussed in the chapter [Experimental Features](#) (see [Internal Access Modifier](#)).

### 9.4.3 Protected Access Modifier

The modifier `protected` indicates that a class member or a constructor is accessible (see [Accessible](#)) only within its declaring class and the classes derived from that declaring class. A protected member *M* declared in some class *C* can be accessed only within the class body of *C* or of a class derived from *C*:

```
1  class C {  
2      protected count: number  
3      getCount(): number {  
4          return this.count // ok  
5      }  
6  }  
7  
8  class D extends C {  
9      increment() {  
10         this.count++ // ok, D is derived from C  
11     }  
12 }  
13  
14 function increment(c: C) {  
15     c.count++ // compile-time error - 'count' is not accessible  
16 }
```

### 9.4.4 Public Access Modifier

The modifier `public` indicates that a class member or a constructor can be accessed everywhere, provided that the member or the constructor belongs to a type that is also accessible (see [Accessible](#)).

## 9.5 Field Declarations

*Field declarations* represent data members in class instances or static data members (see [Static Fields](#)). Syntactically, a field declaration is similar to a variable declaration.

```
classFieldDeclaration:  
    fieldModifier* variableDeclaration  
    ;  
  
fieldModifier:  
    'static' | 'readonly'  
    ;
```

A compile-time error occurs if:

- One and the same field modifier is used more than once in a field declaration.
- Name of a field declared in the body of a class declaration is already used for a method of this class.
- Name of a field declared in the body of a class declaration is already used for another field in the same declaration with the same static or non-static status.

Any static field can be accessed only with the qualification of a superclass name (see [Field Access Expression](#)).

A class can inherit more than one field or property with the same name from its superinterfaces, or from both its superclass and superinterfaces. However, an attempt to refer to such a field or property by its simple name within the



body of the class causes a compile-time error.

The same field or property declaration can be inherited from an interface in more than one way. In that case, the field or property is considered to be inherited only once.

### 9.5.1 Static Fields

There are two categories of class fields as follows:

- Static fields

Static fields are declared with the modifier `static`. A static field is not part of a class instance. There is one copy of a static field irrespective of how many instances of the class (even if zero) are eventually created.

Static fields are always accessed by using a qualified name notation wherever the class name is accessible (see [Accessible](#)).

- Instance, or non-static fields

Instance fields belong to each instance of the class. An instance field is created for, and associated with a newly-created instance of a class, or of its superclass. An instance field is accessible (see [Accessible](#)) via the instance name.

### 9.5.2 Readonly (Constant) Fields

A field with the modifier `readonly` is a *readonly field*. Changing the value of a readonly field after initialization is not allowed. Both static and non-static fields can be declared *readonly fields*.

### 9.5.3 Field Initialization

Any field must be initialized before it is used for the first time (see [Field Access Expression](#)). Initialization is performed by using the result of the evaluation of the following:

- Default values (see [Default Values for Types](#)), or
- A field initializer (see below), and then
- A class initializer of a static field (see [Class Initializer](#)), or
- A class constructor of a non-static field (see [Constructor Declaration](#)).

If none of the above is applicable, then a compile-time error occurs.

*Field initializer* is an expression that is evaluated at compile time or runtime. The result of successful evaluation is assigned into the field. The semantics of field initializers is therefore similar to that of assignments (see [Assignment](#)).

The following rules apply to an initializer in a static field declaration:

- If the initializer uses the keywords `this` or `super` while calling a method (see *Method Call Expression*) or accessing a field (see *Field Access Expression*), then a compile-time error occurs.
- The initializer is evaluated, and the assignment is performed only once before the field is accessed for the first time.

Readonly fields initialization never uses default values (see *Default Values for Types*).

In a non-static field declaration, an initializer is evaluated at runtime. The assignment is performed each time an instance of the class is created.

The instance field initializer expression cannot do the following:

- Call methods that use `this` or `super`;
- Use `this` directly (as an argument of function calls or in assignments);
- Use uninitialized fields of the current object.

If the initializer expression contains one of the above patterns, then a compile-time error occurs.

If allowed in the code, the above restrictions can break the consistency of class instances as shown in the following examples:

```

1  class C {
2      a = this // Compile-time error as 'this' is not fully initialized
3      b = a.c // Refers to a non-initialized 'c' field of the same object
4      c = a.b // Refers to a non-initialized 'b' field of the same object
5
6      f1 = this.foo() // Compile-time error as 'this' is used as an argument
7      f2 = "a string field"
8      foo (): string {
9          console.log (this.f1, this.f2) // Fields are not yet initialized
10         return this.f2
11     }
12
13 }
```

The compiler can determine the right order of field initialization. A compile-time error occurs if the order cannot be determined, or circular dependencies are identified:

```

1  class X {
2      // The initialization order could be determined
3      a = this.b + this.c // 'a' is to be initialized third
4      b = 1               // 'b' is to be initialized first
5      c = this.b + 1      // 'c' is to be initialized second
6
7      // The initialization order can be textually defined by the programmer
8      b = 1               // 'b' is to be initialized first
9      c = this.b + 1      // 'c' is to be initialized second
10     a = this.b + this.c // 'a' is to be initialized third
11
12     // The initialization order cannot be determined
13     f1 = this.f2 + this.f3
14     // Compile-time error: circular dependency between 'f1' and 'f2'
15     f2 = this.f1 + this.f3
16     f3 = 666
17
18 }
```

Additional restrictions (as specified in *Errors and Initialization Expression*) apply to variable initializers that refer to the fields that cannot be initialized yet.

### 9.5.4 Overriding Fields

While extending a class or implementing interfaces, a field declared in a superclass or a superinterface can be overridden by field with the same name and the same static or non-static status. Using the keyword *override* is not required. The new declaration acts as redeclaration. The type of the overriding field is to be the same as the type of the overridden field. Otherwise a compile-time error occurs. Initialization process presumes calls to all initializers starting from the super ones. If the field was not declared in a superclass as *readonly* it is a compile-time error if overriding field is marked as *readonly*.

```

1  class Base1 {
2      field: number = init_in_base_1()
3      private init_in_base_1() {
4          console.log ("Base1 field initialization")
5          return 123
6      }
7  }
8  interface Base2 {
9      field: number
10 }
11 class Derived extends Base1 implements Base2 {
12     field = init_in_derived() // overriding 'field' and providing new initial value
13     private init_in_derived() {
14         console.log ("Derived field initialization")
15         return 666
16     }
17 }
18 new Derived()
19 /* Output:
20     Base1 field initialization
21     Derived field initialization
22 */

```

## 9.6 Method Declarations

*Methods* declare executable code that can be called:

```

classMethodDeclaration:
    methodModifier* identifier typeParameters? signature block?
    ;

methodModifier:
    'abstract'
    | 'static'

```

(continues on next page)

(continued from previous page)

```
| 'final'
| 'override'
| 'native'
| 'async'
;
```

The identifier of `classMethodDeclaration` is the method name that can be used to refer to a method (see *Method Call Expression*).

A compile-time error occurs if:

- The method modifier appears more than once in a method declaration.
- The body of a class declaration declares a method but the name of that method is already used for a field in the same declaration.
- The body of a class declaration declares two same-name methods with overload-equivalent signatures (see *Overload-Equivalent Signatures*) as members of that body of a class declaration.

### 9.6.1 Static Methods

A method declared in a class with the modifier `static` is a *static method*.

A compile-time error occurs if:

- The method declaration contains another modifier (`abstract`, `final`, or `override`) along with the modifier `static`.
- The header or body of a class method includes the name of a type parameter of the surrounding declaration.

Static methods are always called without reference to a particular object. As a result, a compile-time error occurs if the keywords `this` or `super` are used inside a static method.

### 9.6.2 Instance Methods

A method that is not declared static is called *non-static method*, or *instance method*.

An instance method is always called with respect to an object that becomes the current object which the keyword `this` refers to during the execution of the method body.

### 9.6.3 Abstract Methods

An *abstract* method declaration introduces the method as a member along with its signature but without implementation. An abstract method is declared with the modifier `abstract` in the declaration.

Non-abstract methods can be referred to as *concrete methods*.

A compile-time error occurs if:

- An abstract method is declared `private`.
- The method declaration contains another modifier (`static`, `final`, `native`, or `async`) along with the modifier `abstract`.
- The declaration of an abstract method *m* does not appear directly within abstract class A.
- Any non-abstract subclass of A (see [Abstract Classes](#)) does not provide implementation for *m*.

An abstract method declaration provided by an abstract subclass can override another abstract method. A compile-time error occurs if an abstract method overrides a non-abstract instance method.

### 9.6.4 Final Methods

Final methods are discussed in [Final Methods](#).

### 9.6.5 Async Methods

Async methods are discussed in [Experimental Async Methods](#).

### 9.6.6 Overriding Methods

The `override` modifier indicates that an instance method in a superclass is overridden by the corresponding instance method from a subclass (see [Overloading and Overriding](#)).

The usage of the modifier `override` is optional but strongly recommended as it makes the overriding explicit.

A compile-time error occurs if:

- A method marked with the modifier `override` does not override a method from a superclass.
- A method declaration contains modifier `static` along with the modifier `override`.

If the signature of an overridden method contains parameters with default values (see [Optional Parameters](#)), then the overriding method always uses the default parameter values of the overridden method.

A compile-time error occurs if a parameter in the overriding method has a default value.

### 9.6.7 Native Methods

Native methods are discussed in [Native Methods](#).

### 9.6.8 Method Body

*Method body* is a block of code that implements a method. A semicolon or an empty body (i.e., no body at all) indicate the absence of implementation.

An abstract or native method must have an empty body.

In particular, a compile-time error occurs if:

- The body of an abstract or native method declaration is a block.
- The method declaration is neither abstract nor native, but its body is either empty or a semicolon.

The rules that apply to return statements in a method body are discussed in [return Statements](#).

A compile-time error occurs if a method is declared to have a return type, but its body can complete normally (see [Normal and Abrupt Statement Execution](#)).

### 9.6.9 Methods Returning `this`

A return type of an instance method can be `this`. It means that the return type is the class type that the method belongs to. The extended grammar for a method signature (see [Signatures](#)) is as follows:

```
returnType:
    ':' (type | 'this')
    ;
```

The only result that is allowed to be returned from such a method is `this`:

```
1  class C {
2      foo(): this {
3          return this
4      }
5  }
```

The return type of an overridden method in a subclass must also be `this`:

```
1  class D extends C {
2      foo(): this {
3          return this
4      }
5  }
6
7  let x = new C().foo() // type of 'x' is 'C'
8  let y = new D().foo() // type of 'y' is 'D'
```

Otherwise, a compile-time error occurs.

## 9.7 Accessor Declarations

Accessors are often used instead of fields to add additional control for operations of getting or setting a field value. An accessor can be either a getter or a setter.

```
classAccessorDeclaration:
    accessorModifier*
    ( 'get' identifier '(' ' ' ')' returnType block?
    | 'set' identifier '(' parameter ' ' ')' block?
    )
    ;

accessorModifier:
    'abstract'
    | 'static'
    | 'final'
    | 'override'
    | 'native'
    ;
```

Accessor modifiers are a subset of method modifiers. The allowed accessor modifiers have exactly the same meaning as the corresponding method modifiers (see *Abstract Methods* for the modifier `abstract`, *Static Methods* for the modifier `static`, *Final Methods* for the modifier `final`, *Overriding Methods* for the modifier `override`, and *Native Methods* for the modifier `native`).

```
1  class Person {
2      private _age: number = 0
3      get age(): number { return this._age }
4      set age(a: number) {
5          if (a < 0) { throw new Error("wrong age") }
6          this._age = a
7      }
8  }
```

A *get-accessor* (*getter*) must have an explicit return type but no parameters. A *set-accessor* (*setter*) must have a single parameter and no return type. The use of getters and setters looks the same as the use of fields. A compile-time error occurs if:

- Getters or setters are used as methods;
- *Set-accessor* (*setter*) has a single parameter that is optional (see *Optional Parameters*):

```
1  class Person {
2      private _age: number = 0
3      get age(): number { return this._age }
4      set age(a: number) {
5          if (a < 0) { throw new Error("wrong age") }
6          this._age = a
7      }
8  }
```

(continues on next page)

(continued from previous page)

```

9
10 let p = new Person()
11 p.age = 25 // setter is called
12 if (p.age > 30) { // getter is called
13     // do something
14 }
15 p.age(17) // Compile-time error: setter is used as a method
16 let x = p.age() // Compile-time error: getter is used as a method
17
18 class X {
19     set x (p?: Object) {} // Compile-time error: setter has optional parameter
20 }

```

A class can define a getter, a setter, or both with the same name. If both a getter and a setter with a particular name are defined, then both must have the same accessor modifiers. Otherwise, a compile-time error occurs.

Accessors can be implemented by using a private field or fields to store the data (as in the example above).

```

1 class Person {
2     name: string = ""
3     surname: string = ""
4     get fullName(): string {
5         return this.surname + " " + this.name
6     }
7 }
8 console.log (new Person().fullName)

```

A name of an accessor cannot be the same as that of a non-static field, or of a method of class or interface. Otherwise, a compile-time error occurs. Moreover, a name of an accessor cannot be the same as that of another accessor for overloading is not allowed:

```

1 class Person1 {
2     name: string = ""
3     get name(): string { // Compile-time error: getter name clashes with the field name
4         return this.name
5     }
6     set name(a_name: string) { // Compile-time error: setter name clashes with the
↪field name
7         this.name = a_name
8     }
9 }
10
11 class Person2 {
12     set name(name: string) {}
13     set name(name: number) {} // Compile-time error: setters overloading is not
↪permitted
14     get name(): string { return "A name" }
15     get name(): number { return 100 } // Compile-time error: getters overloading is
↪not permitted
16 }

```

In the process of inheriting and overriding (see [Overloading and Overriding](#)), accessors behave as methods. The getter parameter type follows the covariance pattern, and the setter parameter type follows the contravariance pattern (see [Override-Compatible Signatures](#)):

```

1 class Base {

```

(continues on next page)



(continued from previous page)

```

2   get field(): Base { return new Base }
3   set field(a_field: Derived) {}
4   }
5   class Derived extends Base {
6       override get field(): Derived { return new Derived }
7       override set field(a_field: Base) {}
8   }
9   function foo (base: Base) {
10      base.field = new Derived // setter is called
11      let b: Base = base.field // getter is called
12  }
13  foo (new Derived)

```

## 9.8 Class Initializer

When a class is initialized, the *class initializer* declared in the class is executed along with all *class initializers* of all superclasses. The order of execution is from the top superclass to the current class. *Class initializers* (along with field initializers for static fields as described in [Field Initialization](#)) ensure that all static fields receive their initial values before they are used for the first time.

*Class initializer* is syntactically identical to the initializer block and reuses the initializer block semantics (see [Initializer Block](#)):

```

classInitializer:
    initializerBlock
    ;

```

In addition, a compile-time error occurs if a class initializer contains the following:

- Keyword `this` (see [this Expression](#));
- Keyword `super` (see [Method Call Expression](#) and [Field Access Expression](#)); or
- Any type of a variable declared outside the class initializer.

Restrictions of class initializer's ability to refer to static fields (even static fields within a scope) are specified in [Errors and Initialization Expression](#).

## 9.9 Constructor Declaration

*Constructors* are used to initialize objects that are instances of class. A *constructor declaration* starts with the keyword `constructor`, and has no name. In any other syntactical aspect, a constructor declaration is similar to a method declaration with no return type:

```
constructorDeclaration:
  'native'? 'constructor' parameters throwMark? constructorBody?
  ;
```

Constructors are called by the following:

- Class instance creation expressions (see *New Expressions*);
- Conversions and concatenations caused by the string concatenation operator '+' (see *String Concatenation*); and
- Explicit constructor calls from other constructors (see *Constructor Body*).

Access to constructors is governed by access modifiers (see *Access Modifiers* and *Scopes*). Declaring a constructor inaccessible prevents class instantiation from using this constructor. If the only constructor is declared inaccessible, then no class instance can be created.

A compile-time error occurs if two constructors in a class are declared, and have identical signatures.

Constructors marked with the keyword `native` are discussed as an experimental feature in *Native Constructors*.

## 9.9.1 Formal Parameters

The syntax and semantics of a constructor's formal parameters are identical to those of a method.

## 9.9.2 Constructor Body

*Constructor body* is a block of code that implements a constructor. A semicolon, or an empty body (i.e., no body at all) indicates the absence of implementation.

A native constructor (see *Native Constructors*) must have an empty body.

A compile-time error occurs in particular if:

- The body of a native constructor declaration is a block.
- The constructor declaration is not native, but its body is either empty or a semicolon.

The constructor body must provide correct initialization of new class instances. Constructors have two variations:

- *Primary constructor* that initializes its instance own fields<sup>1</sup> directly;
- *Secondary constructor* that uses another same-class constructor to initialize its instance fields.

Both variations have the same syntax:

```
constructorBody:
  '{ ' statement* constructorCall? statement* ' }'
  ;

constructorCall:
  'this' arguments
  | 'super' arguments
  ;
```

The high-level sequence of a *primary constructor* body includes the following:

---

<sup>1</sup> Instance own fields here means fields declared within an instance.

1. Optional arbitrary code that does not use `this` or `super`.
2. Mandatory call to `super ( arguments )` (see [Explicit Constructor Call](#)) if a class has an extension clause (see [Class Extension Clause](#)).
3. Implicitly added compiler-generated code that:
  - Sets default values for instance own fields.
  - Executes instance own field initializers in a valid order determined by the compiler. If the compiler detects a circular reference, then a compile-time error occurs.
4. Arbitrary code that guarantees all remaining instance fields (if any) are initialized but does not:
  - Use the value of an instance field before its initialization.
  - Call any instance method before all instance own fields are initialized.
5. Optional arbitrary code.

The example below represents *primary constructors*:

```

1  class Point {
2      x: number
3      y: number
4      constructor(x: number, y: number) {
5          this.x = x
6          this.y = y
7      }
8  }
9
10 class ColoredPoint extends Point {
11     static readonly WHITE = 0
12     static readonly BLACK = 1
13     color: number
14     constructor(x: number, y: number, color: number) {
15         super(x, y) // calls base class constructor
16         this.color = color
17     }
18 }

```

The high-level sequence of a *secondary constructor* body includes the following:

1. Optional arbitrary code that does not use `this` or `super`.
2. Call to another same-class constructor `this ( arguments )` with arguments.
3. Optional arbitrary code.

The example below represents *primary* and *secondary* constructors:

```

1  class ColoredPoint extends Point {
2      static readonly WHITE = 0
3      static readonly BLACK = 1
4      color: number
5
6      // primary constructor:
7      constructor(x: number, y: number, color: number) {
8          super(x, y) // calls base class constructor as class has 'extends'
9          this.color = color
10     }
11     // secondary constructor:

```

(continues on next page)

(continued from previous page)

```

12  constructor(color: number) {
13      this(0, 0, color)
14  }
15  }

```

A compile-time error occurs if a constructor calls itself, directly or indirectly through a series of one or more explicit constructor calls using `this`.

A constructor body looks like a method body (see [Method Body](#)), except for the semantics as described above. Explicit return of a value (see [return Statements](#)) is prohibited. On the opposite, a constructor body can use a return statement without an expression.

A constructor body must not use fields of a created object before the fields are initialized: `this` can be passed as an argument only after each object field receives an initial value. The checks are performed by the compiler. If the compiler finds a violation, then a compile-time error occurs.

A constructor body can have no more than one call to the current class or direct superclass constructor. Otherwise, a compile-time error occurs.

### 9.9.3 Explicit Constructor Call

There are two kinds of *explicit constructor call* statements:

- *Alternate constructor calls* that begin with the keyword `this`, and can be prefixed with explicit type arguments (see [Type Arguments](#)) (used to call an alternate same-class constructor).
- *Superclass constructor calls* (used to call a constructor from the direct superclass) called *unqualified superclass constructor calls* that begin with the keyword `super`, and can be prefixed with explicit type arguments.

A compile-time error occurs if the constructor body of an explicit constructor call statement:

- Refers to any non-static field or instance method; or
- Uses the keywords `this` or `super` in any expression.

An ordinary method call evaluates an alternate constructor call statement left-to-right. The evaluation starts from arguments, proceeds to constructor, and then the constructor is called.

The process of evaluation of a superclass constructor call statement is performed as follows:

1. If instance *i* is created, then the following procedure is used to determine *i*'s immediately enclosing instance with respect to *S* (if available):
  - If the declaration of *S* occurs in a static context, then *i* has no immediately enclosing instance with respect to *S*.
  - If the superclass constructor call is unqualified, then *S* must be a local class.  
If *S* is a local class, then the immediately enclosing type declaration of *S* is *O*.  
If *n* is an integer ( $n \geq 1$ ), and *O* is the *n*'th lexically enclosing type declaration of *C*, then *i*'s immediately enclosing instance with respect to *S* is the *n*'th lexically enclosing instance of *this*.
2. After *i*'s immediately enclosing instance with respect to *S* (if available) is determined, the evaluation of the superclass constructor call statement continues left-to-right. The arguments to the constructor are evaluated, and then the constructor is called.

3. If the superclass constructor call statement completes normally after all, then all non-static field initializers of *C* are executed. *I* is executed before *J* if a non-static field initializer *I* textually precedes another non-static field initializer *J*.

Non-static field initializers are executed if the superclass constructor call:

- Has an explicit constructor call statement; or
- Is implicit.

An alternate constructor call does not perform the implicit execution.

### 9.9.4 Default Constructor

If a class contains no constructor declaration, then a default constructor is implicitly declared. This guarantees that every class effectively has at least one constructor. The form of a default constructor is as follows:

- Default constructor has modifier `public` (see [Access Modifiers](#)).
- The default constructor body contains a call to a superclass constructor with no arguments except the primordial class `Object`. The default constructor body for the primordial class `Object` is empty.

A compile-time error occurs if a default constructor is implicit, but the superclass has no accessible constructor without parameters (see [Accessible](#)).

```

1  // Class declarations without constructors
2  class Object {}
3  class Base {}
4  class Derived extends Base {}
5
6  // Class declarations with default constructors declared implicitly
7  class Object {
8      constructor () {} // Empty body - as there is no superclass
9  }
10 // Default constructors added
11 class Base { constructor () { super () } }
12 class Derived extends Base { constructor () { super () } }
13
14 // Example of an error case
15 class A {
16     private constructor () {}
17 }
18 class B extends A {} // No constructor in B
19 // During compilation of B
20 class B extends A { constructor () { super () } } // Default constructor added
21 // that leads to compile-time error as default constructor calls super()
22 // which is private and inaccessible

```

## 9.10 Inheritance

Class *C* inherits all accessible members from its direct superclass and direct superinterfaces (see *Accessible*), and optionally overrides or hides some of the inherited members.

An accessible member is a public, protected, or internal member in the same package as *C*.

If *C* is not abstract, then it must implement all inherited abstract methods. The method of each inherited abstract method must be defined with *override-compatible* signatures (see *Override-Compatible Signatures*).

Semantic checks for inherited method and accessors are described in *Overloading and Overriding in Classes*.

Constructors from the direct superclass of *C* are not subject of overloading and overriding because such constructors are not accessible (see *Accessible*) in *C* directly, and can only be called from a constructor of *C* (see *Constructor Body*).

If *C* defines a static or instance field *F* with the same name as that of a field accessible from its direct superclass (see *Accessible*), then *F* hides the inherited field:

```

1  interface Interface {
2      foo()
3  }
4  class Base {
5      foo() { /* Base class method body */ }
6      // foo() is declared in class Base
7
8      static foo () { /* Base class static method body */ }
9  }
10 class Derived extends Base implements Interface {
11     override foo() { /* Derived class method body */ }
12     // foo() is both
13     //   - overridden in class Derived, and
14     //   - implements foo() from the Interface
15     static foo () { /* Derived class static method body */ }
16 }
17
18 let target: Interface = new Derived
19 target.foo() // this is a call to an instance method foo() overridden in class_
20             ↳ Derived
21
22 Base.foo()   // this is a call to a static method foo() declared in Base
23 Derived.foo() // this is a call to a static method foo() declared in Derived

```

## 9.11 Local Classes and Interfaces

Local classes and interfaces (see *Interfaces*) can be declared within any block (see *Block*) for example body of a function, method, constructor, or any embedded block.

Names of local classes and interfaces are visible only within the scope in which they are declared. Local classes and interfaces have access to, and can capture surrounding function/method parameters and local variables.

The example below represents local classes and interfaces in a top-level function:

```

1  function foo (parameter: number) {
2      let local: string = "function local"
3      interface LocalInterface { // Local interface in a top-level function
4          method ()           // It has a method
5          field: string // and a property
6      }
7      class LocalClass implements LocalInterface { // Class implements interface
8          // Local class in a top-level function
9          method () { console.log ("Instance field = ", this.field, " par = ", parameter,
10             ↪ " loc = ", local) }
11          field: string = "`instance field value`"
12          static method () { console.log ("Static field = ", LocalClass.field) }
13          static field: string = "`class/static field value`"
14      }
15      let lc: LocalInterface = new LocalClass
16      // Both local types can be freely used in the top-level function scope
17      lc.method()
18      LocalClass.method()
19  }

```

The example below represents local classes and interfaces in a class method. The algorithm is similar to that in a top-level function. However, the surrounding class members are not accessible from local classes (see [Accessible](#)):

```

1  class A_class {
2      field: number = 1234 // Not visible for the local class
3      method (parameter: number) {
4          let local: string = "instance local"
5          interface LocalInterface {
6              method ()
7              field: string
8          }
9          class LocalClass implements LocalInterface {
10             method () { console.log ("Instance field = ", this.field, " par = ", ↪
11             ↪ parameter, " loc = ", local) }
12             field: string = "`instance field value`"
13             static method () { console.log ("Static field = ", LocalClass.field) }
14             static field: string = "`class/static field value`"
15         }
16         let lc: LocalInterface = new LocalClass
17         lc.method()
18         LocalClass.method()
19     }
20     static method (parameter: number) {
21         let local: string = "class/static local"
22         interface LocalInterface {
23             method ()
24             field: string
25         }
26         class LocalClass implements LocalInterface {
27             method () { console.log ("Instance field = ", this.field, " par = ", ↪
28             ↪ parameter, " loc = ", local) }
29             field: string = "`instance field value`"
30             static method () { console.log ("Static field = ", LocalClass.field) }
31             static field: string = "`class/static field value`"
32         }
33         let lc: LocalInterface = new LocalClass
34         lc.method()
35     }
36 }

```

(continues on next page)

(continued from previous page)

```
33     LocalClass.method()  
34 }  
35 }
```

Note: Local classes and interfaces are not nested ones which are not part of the language. Thus the code below will lead to a compile-time error.

```
1  class A {  
2      class B {} // That is not a local class - compile-time error  
3  }
```

---



An interface declaration declares an *interface type*, i.e., a reference type that:

- Includes properties and methods as its members;
- Has no instance variables (fields);
- Usually declares one or more methods;
- Allows otherwise unrelated classes to provide implementations for the methods, and so implement the interface.

Creating an instance of interface type is not possible.

Interfaces can be *top-level* and local (see [Local Classes and Interfaces](#)).

An interface can be declared *direct extension* of one or more other interfaces. In that case the interface inherits all members from the interfaces it extends. Inherited members can be optionally overridden or hidden.

A class can be declared to *directly implement* one or more interfaces. Any instance of a class implements all methods specified by its interface(s). A class implements all interfaces that its direct superclasses and direct superinterfaces implement. Interface inheritance allows objects to support common behaviors without sharing a superclass.

The value of a variable declared *interface type* can be a reference to any instance of class that implements the specified interface. However, it is not enough for a class to implement all methods of an interface. A class or one of its superclasses must be actually declared to implement an interface. Otherwise, the class is not considered to implement the interface.

Interfaces are assignable to the class `Object` (see [Assignability](#)).

## 10.1 Interface Declarations

*Interface declaration* specifies a new named reference type:

```

interfaceDeclaration:
  'interface' identifier typeParameters?
  interfaceExtendsClause? '{ ' interfaceMember* ' }'
  ;

interfaceExtendsClause:
  'extends' interfaceTypeList
  ;

interfaceTypeList:
  typeReference (',' typeReference)*
  ;

```

The *identifier* in an interface declaration specifies the interface name.

An interface declaration with `typeParameters` introduces a new generic interface (see *Generics*).

The scope of an interface declaration is defined in *Scopes*.

## 10.2 Superinterfaces and Subinterfaces

An interface declared with an `extends` clause extends all other named interfaces, and thus inherits all their members. Such other named interfaces are *direct superinterfaces* of a declared interface. A class that *implements* the declared interface also implements all interfaces that the interface *extends*.

A compile-time error occurs if:

- Interface type named by `typeReference` in the `extends` clause of an interface declaration is not accessible (see *Accessible*).
- Type arguments (see *Type Arguments*) of `typeReference` denote a parameterized type that is not well-formed (see *Generic Instantiations*).
- The `extends` graph has a cycle.
- At least one `typeReference` is an alias of one of primitive, enumeration, union, or function types.

Each `typeReference` in the `extends` clause of an interface declaration must name an accessible interface type (see *Accessible*). Otherwise, a compile-time error occurs.

If an interface declaration (possibly generic)  $I \langle F_1, \dots, F_n \rangle (n \geq 0)$  contains an `extends` clause, then the *direct superinterfaces* of the interface type  $I \langle F_1, \dots, F_n \rangle$  are the types given in the `extends` clause of the declaration of  $I$ .

All *direct superinterfaces* of the parameterized interface type  $I \langle T_1, \dots, T_n \rangle$  are types  $J \langle U_1\theta, \dots, U_k\theta \rangle$ , if:

- $T_i (1 \leq i \leq n)$  is the type of a generic interface declaration  $I \langle F_1, \dots, F_n \rangle (n > 0)$ ;
- $J \langle U_1, \dots, U_k \rangle$  is a direct superinterface of  $I \langle F_1, \dots, F_n \rangle$ ; and
- $\theta$  is a substitution  $[F_1 := T_1, \dots, F_n := T_n]$ .

The transitive closure of the direct superinterface relationship results in the *superinterface* relationship.

Interface  $I$  is a *subinterface* of  $K$  wherever  $K$  is a superinterface of  $I$ . Interface  $K$  is a superinterface of  $I$  if:

- $I$  is a direct subinterface of  $K$ ; or

- $K$  is a superinterface of some interface  $J$  of which  $I$  is, in turn, a subinterface.

There is no single interface to which all interfaces are extensions (unlike class `Object` to which every class is an extension).

A compile-time error occurs if an interface depends on itself.

If superinterfaces have default implementations (see *Default Interface Method Declarations*) for some method  $m$ , then the following occurs:

- Method  $m$  with an override-compatible signature (see *Override-Compatible Signatures*) must be declared within the current interface that extends these interfaces; or
- All these methods refer to the same implementation, and this default implementation is the current class method.

Otherwise, a compile-time error occurs.

```

1  interface I1 { foo () {} }
2  interface I2 { foo () {} }
3  interface C1 extends I1, I2 {
4      foo () {} // foo() from C1 overrides both foo() from I1 and foo() from I2
5  }
6  interface C2 implements I1, I2 {
7      // Compile-time error as foo() from I1 and foo() from I2 have different
↪ implementations
8  }
9  interface I3 extends I1 {}
10 interface I4 extends I1 {}
11 interface C3 extends I3, I4 {
12     // OK, as foo() from I3 and foo() from I4 refer to the same implementation
13 }

```

## 10.3 Interface Body

*Interface body* can declare members of an interface, i.e., its properties (see *Interface Properties*) and methods (see *Interface Method Declarations*).

```

interfaceMember
: annotationUsage?
( interfaceProperty
| interfaceMethodDeclaration
)
;

```

The scope of declaration of a member  $m$  that the interface type  $I$  declares or inherits is specified in *Scopes*.

The usage of annotations is discussed in *Using Annotations*.

## 10.4 Interface Members

*Interface type members* are as follows:

- Members declared in the interface body (see *Interface Body*);
- Members inherited from a direct superinterface (see *Superinterfaces and Subinterfaces*).

A compile-time error occurs if the names of the method explicitly declared by the interface, and of the `Object`'s `public` method are the same, but their signatures are different.

An interface inherits all members of the interfaces it extends (see *Interface Inheritance*).

A name in a declaration scope must be unique, i.e., the names of properties and methods of an interface type must not be the same (see *Interface Declarations*).

## 10.5 Interface Properties

*Interface property* can be defined in the form of a field or an accessor (a getter or a setter):

```
interfaceProperty:
  'readonly'? identifier '??' ':' type
  | 'get' identifier '(' ')' returnType
  | 'set' identifier '(' parameter ')'
  ;
```

A property defined in the form of a field implicitly defines the following:

- A getter, if the property is marked as `readonly`;
- Otherwise, both a getter and a setter with the same name.

If `'?'` is used after the name of the property, then the property type is semantically equivalent to `type | undefined`. As a result, the following definitions have the same effect:

```
1  interface Style {
2      color: string
3  }
4  // is the same as
5  interface Style {
6      get color(): string
7      set color(s: string)
8  }
```

A class that implements an interface with properties can also use a field or an accessor notation (see *Implementing Interface Properties*).

## 10.6 Interface Method Declarations

An ordinary *interface method declaration* specifies the method's name and signature, and is called *abstract*.

An interface method can have a body (see *Default Interface Method Declarations*) as an experimental feature.

```
interfaceMethodDeclaration:
  identifier signature
  | interfaceDefaultMethodDeclaration
  ;
```

The methods declared within interface bodies are implicitly `public`.

A compile-time error occurs if the body of an interface declares a method with a name that is already used for a property in this declaration.

### 10.6.1 Interface Method Overloading

ArkTS allows specifying several interface methods with a single name. A compile-time error occurs if signatures of these methods are overload-equivalent (see *Overload-Equivalent Signatures*).

Overloading methods used in a class are represented in the example below:

```
1  interface I {
2      foo()           // 1st method
3      foo(x: string) // 2st method
4  }
5  class C implements I {
6      foo()           { /*1st method body*/ }
7      foo(x: string)  { /*2nd method body*/ }
8  }
9
10 function demo(i: I) {
11     i.foo()           // ok, 1st method is called
12     i.foo("aa")       // ok, 2nd method is called
13 }
```

## 10.7 Interface Inheritance

Interface *I* inherits all properties and methods from its direct superinterfaces. Semantic checks are described in *Overloading and Overriding in Interfaces*.

**Note.** The semantic rules of methods apply to properties because any interface property implicitly defines a getter, a setter, or both.

Private methods defined in superinterfaces are not accessible (see *Accessible*) in the interface body.

A compile-time error occurs if:

- Interface *I* declares a `private` method *m*;
- Signature of *m* is compatible with the `public` instance method *m'* in a superinterface of *I* (see *Override-Compatible Signatures*); and
- *m'* is otherwise accessible (see *Accessible*) to code in *I*.

---

Enumerations

---

Enumeration type `enum` specifies a distinct user-defined type with an associated set of named constants that define its possible values:

```
enumDeclaration:
  'const'? 'enum' identifier '{' enumConstantList '}'
  ;

enumConstantList:
  enumConstant (',' enumConstant)* ','?
  ;

enumConstant:
  identifier ('=' constantExpression)?
  ;
```

Type `const enum` is supported for source-level compatibility with TypeScript. Type `const` is skipped as it has no impact on `enum` semantics in ArkTS.

Qualification by type is mandatory to access the enumeration constant:

```
1  enum Color { Red, Green, Blue }
2  let c: Color = Color.Red
```

If enumeration type is exported, then all enumeration constants are exported along with the mandatory qualification.

For example, if `Color` is exported, then all constants like `Color.Red` are exported along with the mandatory qualification `Color`.

The value of an `enum` constant can be set as follows:

- Explicitly to a numeric constant expression (expression of type `int`) or to a constant expression of type `string`; or
- Implicitly by omitting the constant expression.

If constant expression is omitted, then the value of the `enum` constant is set implicitly to a numeric value (see *Enumeration Integer Values*).

A compile-time error occurs if `integer` and `string` type enumeration constants are combined in one enumeration.

Any enumeration constant is of type `enumeration`. Implicit conversion (see [Enumeration to Constants Type Conversions](#)) of an enumeration constant to integer types or type `string` depends on the type of constants.

In addition, all enumeration constant names must be unique. Otherwise, a compile-time error occurs.

```
1 enum E1 { A, B = "hello" } // compile-time error
2 enum E2 { A = 5, B = "hello" } // compile-time error
3 enum E3 { A = 5, A = 77 } // compile-time error
4 enum E4 { A = 5, B = 5 } // OK! values can be the same
```

## 11.1 Enumeration Integer Values

The integer value of an `enum` constant is set implicitly if an enumeration constant specifies no value.

A constant expression of type `int` is a signed 32-bit integer (see [Integer Types and Operations](#) for details). A constant expression of type `int` can be used to set the value explicitly:

```
1 enum Background { White = 0xFF, Grey = 0x7F, Black = 0x00 }
```

If all constants have no value, then the first constant is assigned the value zero. The other constant is assigned the value of the immediately preceding constant plus one.

If some but not all constants have their values set explicitly, then the values of the constants are set by the following rules:

- The value of the first constant without an explicit value is assigned to zero.
- A constant with an explicit value has that explicit value.
- A constant that is not the first and has no explicit value takes the value of the immediately preceding constant plus one.

In the example below, the value of `Red` is 0, of `Blue`, 5, and of `Green`, 6:

```
1 enum Color { Red, Blue = 5, Green }
```

## 11.2 Enumeration String Values

A string value for enumeration constants must be set explicitly:

```
1 enum Commands { Open = "fopen", Close = "fclose" }
```



## 11.3 Enumeration Operations

The value of an enumeration constant can be converted to type `string` by using the method `toString`:

```
1 enum Color { Red, Green = 10, Blue }
2 let c: Color = Color.Green
3 console.log(c.toString()) // prints: 10
```

The name of enumeration type can be indexed by the value of this enumeration type to get the name of the constant:

```
1 enum Color { Red, Green = 10, Blue }
2 let c: Color = Color.Green
3 console.log(Color[c]) // prints: Green
```

Additional methods available for enumeration types and constants are discussed in [Enumeration Methods](#) in the chapter Experimental Features.



# CHAPTER 12

## Error Handling

ArKTS is designed to provide first-class support in responding to, and recovering from different error situations in a program. Normal program execution can be interrupted by the occurrence of situations of two kinds:

- Runtime errors (e.g., null pointer dereferencing, array bounds checking, or division by zero);
- Operation completion failures (e.g., the task of reading and processing data from a file on disk can fail if the file does not exist on a specified path, read permissions are not available, or else).

The term *error* in this specification denotes all kinds of error situations.

### 12.1 Errors

*Error* is the base class of all error situations. Defining a new error class is normally not required because essential error classes for various cases (e.g., `ArrayIndexOutOfBoundsException`) are defined in the standard library (see *Standard Library*).

However, a developer can handle a new error situation by using `Error` class itself, or by a subclass of `Error`. An example of error handling is provided below:

```
1 class UnknownError extends Error { // User-defined error class
2     error: Error
3     constructor (error: Error) {
4         super()
5         this.error = error
6     }
}
```

(continues on next page)

(continued from previous page)

```
7  }
8
9  function get_array_element<T>(array: T[], index: number): T|null {
10     try {
11         return array[index] // access array
12     }
13     catch (error) {
14         if (error instanceof ArrayIndexOutOfBoundsException) // invalid index detected
15             return null
16         throw new UnknownError (error) // Unknown error occurred
17     }
18 }
```

In most cases, errors are raised by the ArkTS runtime system, or by the standard library (see *Standard Library*) code.

New error situations can be created and raised by `throw` statements (see *throw Statements*) .

Errors are handled by using `try` statements (see *try Statements*).

**Note.** Not every error can be recovered.

```
1  function handleAll(
2      actions : () => void,
3      handling_actions : () => void)
4  {
5      try {
6          actions()
7      }
8      catch (x) { // Type of x is Error
9          handling_actions()
10     }
11 }
```

---

## Compilation Units

---

Programs are structured as sequences of elements ready for compilation, i.e., compilation units. Each compilation unit creates its own scope (see *Scopes*). The compilation unit's variables, functions, classes, interfaces, or other declarations are only accessible (see *Accessible*) within such scope if not explicitly exported.

A variable, function, class, interface, or other declarations exported from some compilation unit must be imported first by the compilation unit that needs to use them.

There are three kinds of compilation units:

- *Separate modules* (discussed below),
- *Declaration modules* (discussed in detail in *Declaration Modules*), and
- *Packages* (discussed in detail in *Packages*).

```
compilationUnit:  
    separateModuleDeclaration  
    | packageDeclaration  
    | declarationModule  
    ;  
  
packageDeclaration:  
    packageModule+  
    ;
```

All modules (both separate modules and packages) are stored in a file system or a database (see *Compilation Units in Host System*).

## 13.1 Separate Modules

*Separate module* is a module without a package header. A *separate module* can optionally consist of the following four parts:

1. Import directives that enable referring imported declarations in a module;
2. Top-level declarations;
3. Top-level statements; and
4. Re-export directives.

```
separateModuleDeclaration:  
  importDirective* (topDeclaration | topLevelStatements | exportDirective)*  
  ;
```

Every module can directly use all exported entities from the core packages of the standard library (see *Standard Library Usage*).

```
1 // Hello, world! module  
2 function main() {  
3   console.log("Hello, world!") // console is defined in the standard library  
4 }
```

## 13.2 Separate Module\_INITIALIZER

*Separate module* used for import is initialized only once with the details listed in *Compilation Unit Initialization*. The initialization process is performed in the following steps:

- If the separate module has variable or constant declarations (see *Variable and Constant Declarations*), then declaration initializers are executed to ensure that all declarations have valid initial values;
- If the separate module has top-level statements (see *Top-Level Statements*), then the statements are also executed.

## 13.3 Import Directives

*Import directives* make entities exported from other compilation units (see also *Declaration Modules*) available for use in the current compilation unit by using different binding forms.

An import declaration has the following two parts:

- Import path that determines which compilation unit to import from;
- Import binding that defines what entities, and in what form—qualified or unqualified—the current compilation unit can use.

Alternatively, a module can be imported without binding simply in order to run the initialization code.

```

importDirective:
  'import' allBinding | selectiveBindings | defaultBinding | typeBinding
  'from' importPath
  ;

allBinding:
  '*' bindingAlias
  ;

selectiveBindings:
  '{' nameBinding (',' nameBinding)* ','? '}'
  ;

defaultBinding:
  identifier | ( '{' 'default' 'as' identifier '}' )
  ;

typeBinding:
  'type' selectiveBindings
  ;

nameBinding:
  identifier bindingAlias?
  ;

bindingAlias:
  'as' identifier
  ;

importPath:
  StringLiteral
  ;

```

Each binding adds a declaration or declarations to the scope of a module or a package (see *Scopes*). Any declaration added so must be distinguishable in the declaration scope (see *Distinguishable Declarations*). A compile-time error occurs if:

- A declaration added to the scope of a module or a package by a binding is not distinguishable;
- `importPath` refers to the file in which the current module is stored.

**Note:** Import directives are handled by the compiler during compilation, and have no effect during program execution. Though they ensure that imported entities are initialized before use in the current compilation unit.

### 13.3.1 Bind All with Qualified Access

Import binding `* as A` binds the single named entity *A* to the declaration scope of the current module.

A qualified name consisting of *A* and the name of entity *A.name* is used to access any entity exported from the compilation unit as defined by the *import path*.

| Import                                   |  | Usage                            |
|------------------------------------------|--|----------------------------------|
| <pre>import * as Math from "... ↪"</pre> |  | <pre>let x = Math.sin(1.0)</pre> |

This form of import is recommended because it simplifies the reading and understanding of the source code when all exported entities are prefixed with the name of the imported compilation unit.

### 13.3.2 Simple Name Binding

The import binding `identifier` binds an exported entity with the name `identifier` to the declaration scope of the current module. The name `identifier` can only correspond to multiple entities with `identifier` denoting several overloaded functions (see [Function, Method and Constructor Overloading](#)).

The import binding `identifier as A` binds an exported entity (entities) with the name `A` to the declaration scope of the current module. The bounded entity is no longer accessible (see [Accessible](#)) under the name `identifier`. It is shown in the following examples:

```
1 export const PI = 3.14
2 export function sin(d: number): number {}
```

The import path of the module is now irrelevant:

| Import                                         |  | Usage                                                                                        |
|------------------------------------------------|--|----------------------------------------------------------------------------------------------|
| <pre>import {sin} from "..."</pre>             |  | <pre>let x = sin(1.0) let f: float = 1.0</pre>                                               |
| <pre>import {sin as Sine} from ↪"   ..."</pre> |  | <pre>let x = Sine(1.0) // OK let y = sin(1.0) /* Error_ ↪ 'sin'   is not accessible */</pre> |

A single import statement can list several names:



| Import                                           |  | Usage                         |
|--------------------------------------------------|--|-------------------------------|
| <code>import {sin, PI} from "..."</code>         |  | <code>let x = sin(PI)</code>  |
| <code>import {sin as Sine, PI} from "..."</code> |  | <code>let x = Sine(PI)</code> |

Complex cases with several bindings mixed on one import path are discussed below in *Several Bindings for One Import Path*.

### 13.3.3 Several Bindings for One Import Path

The same bound entities can use the following:

- Several import bindings,
- One import directive, or several import directives with the same import path:

|                              |                                                                              |
|------------------------------|------------------------------------------------------------------------------|
| In one import directive      | <code>import {sin, cos} from "..."</code>                                    |
| In several import directives | <code>import {sin} from "..."</code><br><code>import {cos} from "..."</code> |

No conflict occurs in the above example, because the import bindings define disjoint sets of names.

The order of import bindings in an import declaration has no influence on the outcome of the import.

The rules below prescribe what names must be used to add bound entities to the declaration scope of the current module if multiple bindings are applied to a single name:

| Case                                                                | Sample                                                                            | Rule                                                                                                                                                                        |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A name is explicitly used without an alias in several bindings.     | <pre>import {sin, sin} from "..."</pre>                                           | OK. The compile-time warning is recommended.                                                                                                                                |
| A name is used explicitly without alias in one binding.             | <pre>import {sin} from "..."</pre>                                                | OK. No warning.                                                                                                                                                             |
| A name is explicitly used without alias, and implicitly with alias. | <pre>import {sin} from "..."</pre><br><pre>import * as M from "..."</pre>         | OK. Both the name and qualified name can be used:<br>sin and M.sin are accessible.                                                                                          |
| A name is explicitly used with alias.                               | <pre>import {sin as Sine} from "..."</pre>                                        | OK. Only alias is accessible for the name, but not the original name: <ul style="list-style-type: none"> <li>Sine is accessible;</li> <li>sin is not accessible.</li> </ul> |
| A name is explicitly used with alias, and implicitly with alias.    | <pre>import {sin as Sine} from "..."</pre><br><pre>import * as M from "..."</pre> | OK. Both options can be used: <ul style="list-style-type: none"> <li>Sine is accessible;</li> <li>M.sin is accessible.</li> </ul>                                           |
| A name is explicitly used with alias several times.                 | <pre>import {sin as Sine, sin as SIN} from "..."</pre>                            | OK. Both aliases are accessible. But warning can be displayed.                                                                                                              |

### 13.3.4 Default Import Binding

Default import binding allows importing a declaration exported from some module as default export. Knowing the actual name of a declaration is not required as the new name is given at importing. A compile-time error occurs if another form of import is used to import an entity initially exported as default.

```

1  import DefaultExportedItemBindedName from ".../someFile"
2  import {default as DefaultExportedItemNewName} from ".../someFile"
3  function foo () {
4      let v1 = new DefaultExportedItemBindedName()
5      // instance of class 'SomeClass' to be created here
6      let v2 = new DefaultExportedItemNewName()
7      // instance of class 'SomeClass' to be created here
8  }
9
10 // SomeFile
11 export default class SomeClass {}

```

(continues on next page)

(continued from previous page)

```

12
13 // Or
14 class SomeClass {}
15 export default SomeClass

```

### 13.3.5 Type Binding

*Type import binding* allows importing only the type declarations exported from a module or a package. These declarations can be exported normally, or by using the *export type* form. The difference between *import* and *import type* is that *import* imports all exported top-level declarations, while *import type* imports only exported types.

```

1 // File module.sts
2 console.log ("Module initialization code")
3
4 export class Class1 { /*body*/ }
5
6 class Class2 {}
7 export type {Class2}
8
9 // MainProgram.sts
10
11 import {Class1} from "./module.sts"
12 import type {Class2} from "./module.sts"
13
14 let c1 = new Class1() // OK
15 let c2 = new Class2() // OK, the same

```

### 13.3.6 Import Path

*Import path* is a string literal—represented as a combination of the slash character ‘/’ and a sequence alpha-numeric characters—that determines how an imported compilation unit must be placed.

The slash character ‘/’ is used in import paths irrespective of the host system. The backslash character is not used in this context.

In most file systems, an import path looks like a file path. *Relative* (see below) and *non-relative* import paths have different *resolutions* that map the import path to a file path of the host system.

The compiler uses the following rule to define the kind of imported compilation units, and the exact placement of the source code:

- If import path refers to a folder denoted by the last name in the resolved file path, then the compiler imports the package that resides in the folder. The source code of the package is comprised of all the ArkTS source files in the folder.
- Otherwise, the compiler imports the module that the import path refers to. The source code of the module is the file with the extension provided within the import path, or—if none is so provided—appended by the compiler.

A *relative import path* starts with `'./'` or `'../'` as in the following examples:

```
1  "./components/entry"
2  "../constants/http"
```

Resolving *relative import* is relative to the importing file. *Relative import* is used on compilation units to maintain their relative location.

```
1  import * as Utils from "./mytreeutils"
```

Other import paths are *non-relative* as in the examples below:

```
1  "/net/http"
2  "std/components/treemap"
```

Resolving a *non-relative path* depends on the compilation environment. The definition of the compiler environment can be particularly provided in a configuration file or environment variables.

The *base URL* setting is used to resolve a path that starts with `'/'`. *Path mapping* is used in all other cases. Resolution details depend on the implementation. For example, the compilation configuration file can contain the following lines:

```
1  "baseUrl": "/home/project",
2  "paths": {
3      "std": "/arkts/stdlib"
4  }
```

In the example above, `/net/http` is resolved to `/home/project/net/http`, and `std/components/treemap` to `/arkts/stdlib/components/treemap`.

File name, placement, and format are implementation-specific.

## 13.4 Standard Library Usage

All entities exported from the core packages of the standard library (see *Standard Library*) are accessible as simple names (see *Accessible*) in any compilation unit across all its scopes. Using these names as programmer-defined entities causes to a compile-time error in accordance to *Distinguishable Declarations*.

```
1  console.log("Hello, world!")
2      // variable 'console' is defined in the standard library
```

## 13.5 Declaration Modules

*Declaration module* is a special kind of compilation units that can be imported by using *Import Directives*. A declaration module contains *Ambient Declarations* and *Type Alias Declaration* only. An ambient declaration declared in a declaration module must be fully defined elsewhere.

```

declarationModule:
  importDirective*
  ( 'export'? ambientDeclaration
  | 'export'? typeAlias
  | selectiveExportDirective
  ) *
  ;

```

The following example shows how ambient functions can be declared and exported:

```

1 declare function foo()
2 export declare function goo()
3 export { foo }

```

Optional usage of the keyword `export` means that a particular declaration is used by other exported declarations. However, it is not exported on its own, and cannot be used by modules that import this declaration module:

```

1 // module with implementation
2 class A {} // It is not exported
3 export class B {
4   public a: A = new A // the field is exported but its type is not
5 }
6 export function process_field (p: A) {}
7
8 // declaration module should look like
9 declare class A {}
10 export declare class B {
11   public a: A // the field is exported but its type is not
12 }
13 export function process_field (p: A)
14
15 // Module which uses B and process_field
16 import * as m from "path_to_declaration_module"
17
18 let b = new m.B // B instance is created
19 m.process_field (b.a) // exported field is passed to function as an argument
20
21 let a = new m.A // compile-time error as A is not exported

```

How declaration modules are stored in the file system and if the storage scheme of declaration modules differs from the way other modules are stored is determined by the particular implementation.

## 13.6 Compilation Unit Initialization

*Compilation unit* is a separate module (see *Separate Module Initializer*) or a package (see *Package Initializer*) that is initialized once before an entity (function, variable, or type), exported from the compilation unit, is used for the first time. If a *compilation unit* has an import directive (see *Import Directives*) but the imported entities are not actually used, then the imported compilation unit (separate or package) is initialized before the entry point code (see *Program Entry Point*) starts. If different compilation units are not connected by import, then the order of compilation unit initialization is not determined. If there is a cyclic dependency between top-level variable declarations, then a compile-time error occurs.

```

1 // Source file 1
2 import {x} from "Source file 2"
3 let y = x // y uses x for its initialization
4
5 // Source file 2
6 import {y} from "Source file 1"
7 let x = y // x uses y for its initialization

```

## 13.7 Top-Level Declarations

*Top-level declarations* declare top-level types (class, interface, or enum see *Type Declarations*), top-level variables (see *Variable Declarations*), constants (see *Constant Declarations*), functions (see *Function Declarations*), or namespaces (see *Namespace Declarations*). Top-level declarations can be exported.

```

topDeclaration:
  ('export' 'default'?)?
  annotationUsage?
  (
    typeDeclaration
  | variableDeclarations
  | constantDeclarations
  | functionDeclaration
  | functionWithReceiverDeclaration
  | accessorWithReceiverDeclaration
  | namespaceDeclaration
  | ambientDeclaration
  )
;

```

```

1 export let x: number[], y: number

```

The usage of annotations is discussed in *Using Annotations*.

### 13.7.1 Exported Declarations

Top-level declarations can use export modifiers that make the declarations accessible (see *Accessible*) in other compilation units by using import (see *Import Directives*). The declarations not marked as exported can be used only inside the compilation unit they are declared in.

```

1 export class Point {}
2 export let Origin = new Point(0, 0)
3 export function Distance(p1: Point, p2: Point): number {
4   // ...
5 }

```

In addition, only one top-level declaration can be exported by using the default export scheme. It allows specifying no declared name when importing (see *Default Import Binding* for details). A compile-time error occurs if more than one

top-level declaration is marked as default.

```
1 export default let PI = 3.141592653589
```

## 13.8 Namespace Declarations

*Namespace declaration* introduces the qualified name to be used as a qualifier for access to each exported entity of the namespace. The appropriate syntax is presented below:

```
namespaceDeclaration:
  'namespace' qualifiedName
  '{' topDeclaration* initializerBlock? topDeclaration* '}'
  ;
```

Namespace can have an initializer block to ensure that all namespace variables receive initial values. Initialization details are based on *Compilation Unit Initialization*, except the parts related to import which is not applicable to namespaces and *Initializer Block*.

An usage example is presented below:

```
1 namespace NS1 {
2   export function foo() { ... }
3   export let variable = 1234
4   export const constant = 1234
5   export let someVar: SomeType
6   static {
7     someVar = new SomeType
8   }
9 }
10
11 if (NS1.variable == NS1.constant) {
12   NS1.variable = 4321
13 }
```

**Note.** A namespace must be exported to be used in another compilation unit.

```
1 // File1
2 namespace Space1 {
3   export function foo() { ... }
4   export let variable = 1234
5   export const constant = 1234
6 }
7 export namespace Space2 {
8   export function foo(p: number) { ... }
9   export let variable = "1234"
10 }
11
12 // File2
13 import {Space2 as Space1} from "File1"
14 if (Space1.variable == Space1.constant) { // compile-time error - there is no
15   ↪ variable or constant called 'constant'
16   Space1.variable = 4321 // compile-time error - incorrect assignment as type
17   ↪ 'number' is not compatible with type 'string'
```

(continues on next page)

(continued from previous page)

```

16  }
17  Space1.foo()      // compile-time error - there is no function 'foo()'
18  Space1.foo(1234) // OK

```

**Note.** Embedded namespaces are allowed.

```

1  namespace ExternalSpace {
2      export function foo() { ... }
3      export let variable = 1234
4      export namespace EmbeddedSpace {
5          export const constant = 1234
6      }
7  }
8
9  if (ExternalSpace.variable == ExternalSpace.EmbeddedSpace.constant) {
10     ExternalSpace.variable = 4321
11 }

```

**Note.** Namespaces with identical namespace names in a single compilation unit merge their exported declarations into a single namespace. A duplication causes a compile-time error. Exported and non-exported declarations having the same name is also considered a compile-time error. Only one of merging namespaces can have an initializer. Otherwise, a compile-time error occurs.

```

1  // One source file
2  namespace A {
3      export function foo() { console.log ("1st A.foo() exported") }
4      function bar() { }
5      export namespace C {
6          export function too() { console.log ("1st A.C.too() exported") }
7      }
8  }
9
10 namespace B { }
11
12 namespace A {
13     export function goo() {
14         A.foo() // calls exported foo()
15         foo()  /* calls exported foo() as well as all A namespace
16                  declarations are merged into one */
17         A.C.moo()
18     }
19     //export function foo() { }
20     // Compile-time error as foo() was already defined
21
22     // function foo() { console.log ("2nd A.foo() non-exported") }
23     // Compile-time error as foo() was already defined as exported
24 }
25
26 namespace A.C {
27     export function moo() {
28         too() // too() accessible when namespace C and too() are both exported
29         A.C.too()
30     }
31 }
32 }
33

```

(continues on next page)



(continued from previous page)

```

34  A.goo()
35
36  // File1
37  package P
38  namespace A {
39      export function foo() { ... }
40      export function bar() { ... }
41  }
42
43  // File2
44  package P
45  namespace A {
46      function goo() { bar() } // exported bar() is accessible in the same_
↪namespace
47      export function foo() { ... } // Compile-time error as foo() was already_
↪defined
48  }
49
50  namespace X {
51      static {}
52  }
53  namespace X {
54      static {} // Compile-time error as only one initializer allowed
55  }

```

**Note.** A namespace name can be a qualified name. It is a shortcut notation of embedded namespaces as represented below:

```

1  namespace A.B {
2      /*some declarations*/
3  }

```

The code above is the shortcut version of the following code:

```

1  namespace A {
2      export namespace B {
3          /*some declarations*/
4      }
5  }

```

This code illustrates the usage of declarations in the following case:

```

1  namespace A.B.C {
2      export function foo() { ... }
3  }
4
5  A.B.C.foo() // Valid function call, as 'B' and 'C' are implicitly exported

```

If an ambient namespace (see [Ambient Namespace Declarations](#)) belongs to a separate module (see [Separate Modules](#)) then all ambient namespace declarations are accessible across all declarations and top-level statements of the separate module.

```

1  declare namespace A {
2      function foo(): void
3      type X = Array<Number>
4  }

```

(continues on next page)

(continued from previous page)

```

5
6  A.foo() // Valid function call, as 'foo' is accessible for top-level statements
7  function foo () {
8      A.foo() // Valid function call, as 'foo' is accessible here as well
9  }
10 class C {
11     method () {
12         A.foo() // Valid function call, as 'foo' is accessible here too
13         let x: A.X = [] // Type A.X can be used
14     }
15 }

```

## 13.9 Export Directives

*Export directive* allows the following:

- Specifying a selective list of exported declarations with optional renaming; or
- Specifying a name of one declaration; or
- Exporting a type; or
- Re-exporting declarations from other compilation units.

```

exportDirective:
  selectiveExportDirective
  | singleExportDirective
  | exportTypeDirective
  | reExportDirective
  ;

```

### 13.9.1 Selective Export Directive

Top-level declarations can be made *exported* by using a selective export directive. The selective export directive provides an explicit list of names of the declarations to be exported. Optional renaming allows having the declarations exported with new names.

```

selectiveExportDirective:
  'export' selectiveBindings
  ;

```

A selective export directive uses the same *selective bindings* as an import directive:

```

export { d1, d2 as d3}

```

The above directive exports 'd1' by its name, and 'd2' as 'd3'. The name 'd2' is not accessible (see [Accessible](#)) in the modules that import this module.

## 13.9.2 Single Export Directive

*Single export directive* allows specifying the declaration to be exported from the current compilation unit by using the declaration's own name. The directive in the example below exports variable 'v' by its name:

```
singleExportDirective:
  'export' 'default'? identifier
  ;
```

If `default` is added, then only one such export directive is possible in the current compilation unit. Otherwise, a compile-time error occurs.

```
1  export v
2  let v = 1
3
4  class A {}
5  export default A
```

*Single export directive* works as re-export when declaration referred by *identifier* was imported.

```
1  import {v} from "some location"
2  export v
```

## 13.9.3 Export Type Directive

In addition to export that is attached to some declaration, the *export type* directive can be used in order to do the following:

- Export *as a type* a particular class or interface already declared; or
- Export an already declared type under a different name.

The appropriate syntax is presented below:

```
exportTypeDirective:
  'export' 'type' selectiveBindings
  ;
```

If a class or an interface is exported in this manner, then its usage is limited similarly to the limitations described for *import type* directives (see *Type Binding*).

If a class or interface is declared exported, but *export type* is applied to the same class or interface name, then a compile-time error occurs. This situation is represented in the following example:

```
1  class A {}
2
3  export type {A} // export already declared class type
4
5  export type MyA = A // name MyA is declared and exported
```

(continues on next page)

(continued from previous page)

```

6
7 export type {MyA} // compile-time error as MyA was already exported

```

### 13.9.4 Re-Export Directive

In addition to exporting what is declared in the module, it is possible to re-export declarations that are part of other modules' export. A particular declaration or all declarations can be re-exported from a module. When re-exporting, new names can be given. This action is similar to importing but has the opposite direction. The appropriate grammar is presented below:

```

reExportDirective:
  'export' ( '*' | selectiveBindings ) 'from' importPath
  ;

```

An importPath cannot refer to the file the current module is stored in. Otherwise, a compile-time error occurs.

The re-exporting practice is represented in the following examples:

```

1 export * from "path_to_the_module" // re-export all exported declarations
2 export { d1, d2 as d3 } from "path_to_the_module"
3 // re-export particular declarations some under new name

```

## 13.10 Top-Level Statements

A separate module can contain sequences of statements that logically comprise one sequence of statements:

```

topLevelStatements:
  statement*
  ;

```

A module can contain any number of top-level statements that logically merge into a single sequence in the textual order:

```

1 statements_1
2 /* top-declarations except constant and variable declarations */
3 statements_2

```

The sequence above is equal to the following:

```

1 /* top-declarations except constant and variable declarations */
2 statements_1; statements_2

```

It is represented by the example below:

```

1 // The actual text mixture of the statements and declarations
2 console.log ("Start of top-level statements")
3 type A = number | string
4 let a: A = 56

```

(continues on next page)

(continued from previous page)

```

5  function foo () {
6      console.log (a)
7  }
8  a = "a string"
9
10
11 // The logically ordered text - declarations then statements
12 type A = number | string
13 function foo () {
14     console.log (a)
15 }
16 console.log ("Start of top-level statements")
17 let a: A = 56
18 a = "a string"

```

- If a separate module is imported by some other module, then the semantics of top-level statements is to initialize the imported module. It means that all top-level statements are executed only once before a call to any other function, or before the access to any top-level variable of the separate module.
- If a separate module is used as a program, then top-level statements are used as a program entry point (see *Program Entry Point*). The set of top-level statements being empty implies that the program entry point is also empty and does nothing. If a separate module has the `main` function, then it is executed after the execution of the top-level statements.

```

1  // Source file A
2  { // Block form
3      console.log ("A.top-level statements")
4  }
5
6  // Source file B
7  import * as A from "Source file A "
8  function main () {
9      console.log ("B.main")
10 }

```

The output is as follows:

- A. Top-level statements,
- B. Main.

```

1  // One source file
2  console.log ("A.Top-level statements")
3  function main () {
4      console.log ("B.main")
5  }

```

A compile-time error occurs if top-level statements contain a return statement (*Expression Statements*).

The execution of top-level statements means that all statements, except type declarations, are executed one after another in the textual order of their appearance within the module until an erroneous situation is thrown (see *Errors*), or last statement is executed.

## 13.11 Program Entry Point

Separate modules or packages can act as programs (applications). Program execution starts from the execution of a *program entry point* which can be of the following two kinds:

- Top-level statements for separate modules (see *Top-Level Statements*); or
- Entry point function (see below).

A separate module can have the following forms of entry point:

- Sole entry point function (`main` or other as described below);
- Sole top-level statement (the first statement in the top-level statements acts as the entry point);
- Both top-level statement and entry point function (same as above, plus the function called after the top-level statement execution is completed).

A package can have a sole entry point function (`main` or other as described below).

Entry point functions have the following features:

- Any exported top-level function can be used as an entry point. An entry point is selected by the compiler, the execution environment, or both;
- Entry point function must either have no parameters, or have one parameter of type `string[]` that provides access to the arguments of a program command line;
- Entry point function return type is either `void` (see *Type void*) or `int`;
- Entry point function cannot have overloading;
- Entry point function is called `main` by default.

The example below represents different forms of valid and invalid entry points:

```

1  function main() {
2      // Option 1: a return type is inferred from the body of main().
3      // It will be 'int' if the body has 'return' with the integer expression
4      // and 'void' if no return at all in the body
5  }
6
7  function main(): void {
8      // Option 2: explicit :void - no return in the function body required
9  }
10
11 function main(): int {
12     // Option 3: explicit :int - return is required
13     return 0
14 }
15
16 function main(): string { // compile-time error: incorrect main signature
17     return ""
18 }
19
20 function main(p: number) { // compile-time error: incorrect main signature
21 }
22
23 // Option 4: top-level statement is the entry point
24 console.log ("Hello, world!")
25
26 // Option 5: top-level exported function

```

(continues on next page)

(continued from previous page)

```
27 export function entry() {}
28
29 // Option 5: top-level exported function with command-line arguments
30 export function entry(cmdLine: string[]) {}
31
32 // Package example - outputs "Package init" then "Package main"
33 package P
34 function main () { console.log ("Package main")}
35 static { console.log ("Package init") }
```

## 13.12 Program Exit

Separate modules and packages can act as programs (applications) and their entry point is described above (see *Program Entry Point*).

*Program exit* takes place when:

- All top-level statements (for separate modules) and statements of the entry point function body, if any, complete normally.
- An unhandled error (see *Error Handling*) occurs during the program execution.

In both cases, the control is transferred to the ArkTS runtime system, which ensures that all coroutines (see *Coroutines*) created during the program execution are terminated.

If an error occurs, then the appropriate diagnostics is displayed. This is the end of the *program exit* process.





---

Ambient Declarations

---

*Ambient declaration* specifies an entity that is declared somewhere else. Ambient declarations:

- Provide type information for entities included in a program from an external source.
- Introduce no new entities like regular declarations do.
- Cannot include executable code and thus have no initializers.
- Can be specified in *Declaration Modules* only.

Ambient functions, methods, and constructors have no bodies.

```
ambientDeclaration:  
  'declare'  
  (  
    ambientConstantDeclaration  
  | ambientFunctionDeclaration  
  | ambientClassDeclaration  
  | ambientInterfaceDeclaration  
  | ambientNamespaceDeclaration  
  | 'const'? enumDeclaration  
  )  
;
```

An ambient enumeration type declaration can be prefixed by the keyword `const` for TypeScript compatibility. It has no influence on the declared type.

A compile-time error occurs if the modifier `declare` is used in a context that is already ambient:

```
1 declare namespace A{  
2   declare function foo(): void // compile-time error  
3 }
```

## 14.1 Ambient Constant Declarations

```
ambientConstantDeclaration:
    'const' ambientConstList ';'
    ;

ambientConstList:
    ambientConst (',' ambientConst)*
    ;

ambientConst:
    identifier ((':' type) | ('=' initializer))
    ↪ (IntegerLiteral | FloatLiteral | StringLiteral | MultilineStringLiteral) )
    ;
```

The initializer expression for an ambient constant must be a numeric or string literal.

## 14.2 Ambient Function Declarations

```
ambientFunctionDeclaration:
    'function' identifier
    typeParameters? signature
    ;
```

A compile-time error occurs if explicit return type for an ambient function declaration is not specified.

```
1 declare function foo(x: number): void // ok
2 declare function bar(x: number) // compile-time error
```

Ambient functions cannot have parameters with default values but can have optional parameters.

Ambient function declarations cannot specify function bodies.

```
1 declare function foo(x?: string): void // ok
2 declare function bar(y: number = 1): void // compile-time error
```

**Note.** The modifier `async` cannot be used in an ambient context.

## 14.3 Ambient Class Declarations

```
ambientClassDeclaration:
    'class' identifier typeParameters?
    classExtendsClause? implementsClause?
    '{' ambientClassBodyDeclaration* '}'
    ;
```

(continues on next page)

(continued from previous page)

```
ambientClassBodyDeclaration:
  ambientAccessModifier?
  ( ambientFieldDeclaration
  | ambientConstructorDeclaration
  | ambientMethodDeclaration
  | ambientAccessorDeclaration
  | ambientIndexerDeclaration
  | ambientCallSignatureDeclaration
  | ambientIterableDeclaration
  )
  ;

ambientAccessModifier:
  'public' | 'protected'
  ;
```

Ambient field declarations have no initializers:

```
ambientFieldDeclaration:
  ambientFieldModifier* identifier ':' type
  ;

ambientFieldModifier:
  'static' | 'readonly'
  ;
```

Ambient constructor, method, and accessor declarations have no bodies:

```
ambientConstructorDeclaration:
  'constructor' parameters throwMark?
  ;

ambientMethodDeclaration:
  ambientMethodModifier* identifier signature
  ;

ambientMethodModifier:
  'static'
  ;

ambientAccessorDeclaration:
  ambientMethodModifier*
  ( 'get' identifier '(' ')' returnType
  | 'set' identifier '(' parameter ')'
  )
  ;
```

### 14.3.1 Ambient Indexer

*Ambient indexer declarations* specify the indexing of a class instance in an ambient context. The feature is provided for TypeScript compatibility:

```
ambientIndexerDeclaration:
  'readonly'? '[' identifier ':' indexType ']' returnType
  ;
```

**Restriction:** *indexType* must be number.

```
1 declare class C {
2   [index: number]: number
3 }
```

**Note.** *Ambient indexer declaration* is supported in ambient contexts only. If written in ArkTS, ambient class implementation must conform to *Indexable Types*.

### 14.3.2 Ambient Call Signature

*Ambient call signature* declarations are used to specify *callable types* in an ambient context. The feature is provided for TypeScript compatibility:

```
ambientCallSignatureDeclaration:
  signature
  ;
```

```
1 declare class C {
2   (someArg: number): boolean
3 }
```

**Note.** *Ambient class signature declaration* is supported in ambient contexts only. If written in ArkTS, ambient class implementation must conform to *Callable Types with \$invoke Method*.

### 14.3.3 Ambient Iterable

*Ambient iterable declaration* indicates that a class instance is iterable in an ambient context. The feature is provided for TypeScript compatibility:

```
ambientIterableDeclaration:
  '[Symbol.iterator]' '(' ')' returnType
  ;
```

**Restriction:** *returnType* must be a type that implements `Iterator` interface defined in the standard library (see *Standard Library*).

```
1 declare class C {
2   [Symbol.iterator]: CIterator
3 }
```

**Note.** *Ambient iterable declaration* is supported in ambient contexts only. If written in ArkTS, ambient class implementation must conform to *Iterable Types*.

## 14.4 Ambient Interface Declarations

```
ambientInterfaceDeclaration:
  'interface' identifier typeParameters?
  interfaceExtendsClause?
  '{' ambientInterfaceMember* '}'
  ;

ambientInterfaceMember
: interfaceProperty
| interfaceMethodDeclaration
| ambientIndexerDeclaration
| ambientCallSignatureDeclaration
| ambientIterableDeclaration
;
```

*Ambient interface* can contain additional members in the same manner as an ambient class (see *Ambient Indexer*, *Ambient Call Signature*, and *Ambient Iterable*).

## 14.5 Ambient Namespace Declarations

Namespaces are used to logically group multiple entities. ArkTS supports *ambient namespaces* for better TypeScript compatibility. TypeScript often uses ambient namespaces to specify the platform API or a third-party library API.

```
ambientNamespaceDeclaration:
  'namespace' identifier '{' ambientNamespaceElement* '}'
  ;

ambientNamespaceElement:
  ambientNamespaceElementDeclaration | selectiveExportDirective
;

ambientNamespaceElementDeclaration:
  'export'?
  (
    ambientConstantDeclaration
  | ambientFunctionDeclaration
  | ambientClassDeclaration
  | ambientInterfaceDeclaration
  | ambientNamespaceDeclaration
  | 'const'? enumDeclaration
  | typeAlias
  )
;
```

An *enumeration type declaration* can be prefixed with the keyword `const` for TypeScript compatibility. The prefix has no influence on the declared type. Only exported entities can be accessed outside a namespace.

Namespaces can be nested:

```
1 declare namespace A {  
2     export namespace B {  
3         export function foo(): void;  
4     }  
5 }
```

A namespace is not an object but merely a scope for entities that can be accessed by using qualified names only.

If an ambient namespace is imported from the declaration module then all ambient namespace declarations are accessible across all declarations and top-level statements of the current module.

```
1 // File1.d.sts  
2 export declare namespace A { // namespace itself must be exported  
3     function foo(): void  
4     type X = Array<Number>  
5 }  
6  
7 // File2.sts  
8 import {A} from 'File1.d.sts'  
9  
10 A.foo() // Valid function call, as 'foo' is accessible for top-level statements  
11 function foo () {  
12     A.foo() // Valid function call, as 'foo' is accessible here as well  
13 }  
14 class C {  
15     method () {  
16         A.foo() // Valid function call, as 'foo' is accessible here too  
17         let x: A.X = [] // Type A.X can be used  
18     }  
19 }
```

### 14.5.1 Implementing Ambient Namespace Declaration

If an *ambient namespace* is implemented in ArkTS, a namespace with the same name must be declared (see [Namespace Declarations](#)) as the top-level declaration of a compilation unit. All namespace names of a nested namespace (i.e. a namespace embedded into another namespace) must be the same as in ambient context.

A compilation unit that implements a namespace is the unit for which the declaration module is built (see [Declaration Modules](#)).

This Chapter contains semantic rules to be used throughout the Specification document. The description of the rules is more or less informal. Some details are omitted to simplify the understanding.

### 15.1 Subtyping

*Subtype* relationship between types  $S$  and  $T$ , where  $S$  is a subtype of  $T$  (recorded as  $S <: T$ ), means that any object of type  $S$  can be safely used in any context to replace an object of type  $T$ . The opposite is called *supertype* relationship (see *Supertyping*).

By the definition of  $S <: T$ , type  $T$  belongs to the set of *supertypes* of type  $S$ . The set of *supertypes* includes all *direct supertypes* (see below), and all their respective *supertypes*. More formally speaking, the set is obtained by reflexive and transitive closure over the direct supertype relation.

Terms *subclass*, *subinterface*, *superclass*, and *superinterface* are used when considering class or interface types.

*Direct supertypes* of a non-generic class, or of the interface type  $C$  are **all** of the following:

- Direct superclass of  $C$  (as mentioned in its extension clause, see *Class Extension Clause*) or type `Object` if  $C$  has no extension clause specified;
- Direct superinterfaces of  $C$  (as mentioned in the implementation clause of  $C$ , see *Class Implementation Clause*); and
- Class `Object` if  $C$  is an interface type with no direct superinterfaces (see *Superinterfaces and Subinterfaces*).

*Direct supertypes* of the generic type  $C <F_1, \dots, F_n>$  (for a generic class or interface type declaration  $C <F_1, \dots, F_n>$  with  $n > 0$ ) are **all** of the following:

- Direct superclass of  $C <F_1, \dots, F_n>$ ;
- Direct superinterfaces of  $C <F_1, \dots, F_n>$ , and

- Type `Object` if  $C <F_1, \dots, F_n>$  is a generic interface type with no direct superinterfaces.

The direct supertype of a type parameter is the type specified as the constraint of that type parameter.

### 15.1.1 Supertyping

*Supertype* relationship between types  $T$  and  $S$ , where  $T$  is a supertype of  $S$  (recorded as  $T > S$ ), is opposite to subtyping (see [Subtyping](#)). *Supertyping* means that any object of type  $S$  can be safely used in any context to replace an object of type  $T$ .

## 15.2 Type Identity

*Identity* relation between two types means that types are indistinguishable. This relation is commutative and transitive.

Types  $A$  and  $B$  are *identical*, if

- $A$  is subtype of  $B$  ( $A < B$ ) and  $B$  is subtype of  $A$  ( $A > B$ ); or
- $A$  is defined as `T[]` and  $B$  is defined as `Array<T>`, meaning that two syntax form defines identical types.

**Note.** *Type Alias Declaration* does not create new type, but only new name to the existing type, so an alias and its base type are also indistinguishable.

## 15.3 Assignability

Type  $T_1$  is assignable to type  $T_2$  if:

- $T_1$  is subtype of  $T_2$ , or
- There is an *implicit conversion* (see [Implicit Conversions](#)) that allows converting value of type  $T_1$  to type  $T_2$ .

*Assignability* relationship is asymmetric, i.e., that  $T_1$  is assignable to  $T_2$  does not imply that  $T_2$  is assignable to type  $T_1$ .

## 15.4 Invariance, Covariance and Contravariance

*Variance* is how subtyping between class types relates to subtyping between class member signatures (types of parameters, return type). Variance can be of three kinds:



- Invariance,
- Covariance, and
- Contravariance.

*Invariance* refers to the ability to use the originally-specified type as a derived one.

*Covariance* is the ability to use a type that is more specific than originally specified.

*Contravariance* is the ability to use a type that is more general than originally specified.

The examples below illustrate valid and invalid usages of variance. If class `Base` is defined as follows:

```
1 class Base {
2     method_one(p: Base): Base {}
3     method_two(p: Derived): Base {}
4     method_three(p: Derived): Derived {}
5 }
```

—then the code below is valid:

```
1 class Derived extends Base {
2     // invariance: parameter type and return type are unchanged
3     override method_one(p: Base): Base {}
4
5     // covariance for the return type: Derived is a subtype of Base
6     override method_two(p: Derived): Derived {}
7
8     // contravariance for parameter types: Base is a supertype for Derived
9     override method_three(p: Base): Derived {}
10 }
```

On the contrary, the following code causes compile-time errors:

```
1 class Derived extends Base {
2
3     // covariance for parameter types is prohibited
4     override method_one(p: Derived): Base {}
5
6     // contravariance for the return type is prohibited
7     override method_tree(p: Derived): Base {}
8 }
```

## 15.5 Compatibility of Call Arguments

The term *assignable* is defined in [Assignability](#).

The following semantic check must be performed for any function, method, or constructor call:

- Type of any argument (except arguments of a `rest` parameter) must be assignable to the type of the corresponding parameter;
- Type of each argument corresponding to the `rest` parameter without the spread operator (*Spread Expression*) must be assignable to the element type of the array `rest` type parameter. If the `rest` parameter is a tuple, then

the number of arguments must be equal to the number of tuple elements, and argument types must be assignable to the appropriate tuple types;

- If a single argument corresponding to the `rest` parameter has the spread operator (*Spread Expression*), then the *expression* that follows the operator must refer to one of the following:
  - An array of a type assignable to the type of the array `rest` parameter; or
  - A tuple of types assignable to the proper types of the tuple `rest` parameter.

## 15.6 Type Inference

ArkTS supports strong typing but allows not to burden a programmer with the task of specifying type annotations everywhere. A smart compiler can infer types of some entities from the surrounding context. This technique called *type inference* allows keeping type safety and program code readability, doing less typing, and focusing on business logic. Type inference can be applied by the compiler in the following contexts:

- Variable and constant declarations (see *Type Inference from Initializer*);
- Implicit generic instantiations (see *Implicit Generic Instantiations*);
- Function or method return type (see *Return Type Inference*);
- Lambda expression parameter type (see *Lambda Signature*);
- Array literal type inference (see *Array Literal Type Inference from Context*, and *Array Type Inference from Types of Elements*);
- Smart types (see *Smart Types*).

### 15.6.1 Smart Types

Data entities include the following:

- Variable (see *Variable and Constant Declarations*),
- Class field (see *Field Declarations*), and
- Parameter of a function or method (see *Parameter List*).

Every data entity has its static type, which is specified explicitly or inferred at the point of declaration. This type defines the set of operations that can be applied to the entity (namely, what methods can be called, and what other entities can be accessed if the entity acts as a receiver of the operation):

```
1 let a = new Object
2 a.toString() // entity 'a' has method toString()
```

If an entity is class type (see *Classes*), interface type (see *Interfaces*), or union type (see *Union Types*), then the compiler can narrow (smart cast) a static type to a more precise type (smart type), and allow operations that are specific to the type so narrowed:

```

1  let a: number | string = 666
2  a++ /* Here we know for sure that type of 'a' is number and number-specific
3      operations are type-safe */
4
5  class Base {}
6  class Derived extends Base { method () {} }
7  let b: base = new Derived
8  b.method () /* Here we know for sure that type of 'b' is Derived and Derived-specific
9      operations are type-safe */

```

Other examples are explicit calls to `instanceof` (see *InstanceOf Expression*) or checks against `null` (see *Reference Equality*) as part of `if` statements (see *if Statements*) or conditional expressions (see *Conditional Expressions*):

```

1  function foo (b: Base, d: Derived|null) {
2      if (b instanceof Derived) {
3          b.method()
4      }
5      if (d != null) {
6          d.method()
7      }
8  }

```

In like cases, a smart compiler can deduce the smart type of an entity without requiring unnecessary casting conversions (see *Cast Expressions*).

Overloading (see *Function, Method and Constructor Overloading*) can cause tricky situations when a smart type leads to the call of a function or a method (see *Overload Resolution*) that suits smart rather than static type of an argument:

```

1  function foo (p: Base) {}
2  function foo (p: Derived) {}
3
4  let b: Base = new Derived
5  foo (b) // potential ambiguity in case of smart type, foo(p:Base) is to be called
6  foo (b as Derived) // no ambiguity, foo(p:Derived) is to be called

```

Particular cases supported by the compiler are determined by the compiler implementation.

## 15.7 Overloading and Overriding

Two important concepts apply to different contexts and entities throughout this specification as follows:

1. *Overloading* allows defining and using functions (in general sense, including methods and constructors) with the same name but different signatures. The actual function to be called is determined at compile time. Thus, *overloading* is related to compile-time polymorphism.
2. *Overriding* is closely connected with inheritance. It is used on methods but not on functions. Overriding allows a subclass to offer a specific implementation of a method already defined in its parent class. The actual method to be called is determined at runtime based on object type. Thus, overriding is related to runtime polymorphism.

ArkTS uses two semantic rules related to these concepts:

- *Overload-equivalence* rule: the *overloading* of two entities is correct if their signatures are **not** *overload-equivalent* (see *Overload-Equivalent Signatures*).

- *Override-compatibility* rule: the *overriding* of two entities is correct if their signatures are *override-compatible* (see *Override-Compatible Signatures*).

See *Overloading for Functions*, *Overloading and Overriding in Classes*, and *Overloading and Overriding in Interfaces* for details.

### 15.7.1 Overload-Equivalent Signatures

Signatures  $S_1$  with  $n$  parameters, and  $S_2$  with the same number of parameters are *overload-equivalent* if:

1. Parameter type at some position in  $S_1$  is a *type parameter* (see *Type Parameters*), and a parameter type at the same position in  $S_2$  is any non-generic reference type (including *union type*) or *type parameter*;
2. Parameter type at some position in  $S_1$  is *generic type*  $G \langle T_1, \dots, T_n \rangle$ , where there is at least one  $T_i$  which is a type parameter (see *Type Parameters*), and a parameter type at the same position in  $S_2$  is also  $G$  with any list of *Type Arguments* or a *union type* that contains  $G$ ;
3. Parameter types at some position in  $S_1$  and  $S_2$  are *union types* that contain types falling under the cases 1. or 2. above;
4. All other parameter types in  $S_1$  are identical (see *Type Identity*) to parameter types in the same positions in  $S_2$  and both parameters are of the same kind, so they both are *required* or *optional* or *rest*.

Parameter names and return types do not influence *overload equivalence*. Signatures are *overload-equivalent* in the following examples:

```
1 (x: number): void
2 (y: number): void
```

```
1 (x: number): void
2 (y: number): number
```

```
1 class G<T>
2 (y: Number): void
3 (x: T): void
```

```
1 class G<T>
2 (y: G<Number>): void
3 (x: G<T>): void
```

```
1 class G<T, S>
2 (y: T): void
3 (x: S): void
```

Signatures are not *overload-equivalent* in the following examples:

```
1 (x: number): void
2 (y: string): void
```

```
1 class A { /*body*/ }
2 class B extends A { /*body*/ }
3 (x: A): void
```

(continues on next page)

(continued from previous page)

```

4  (y: B): void
5
6  class A<T> {
7      foo(p: T) {}
8      foo(p: T[]) {}
9  }

```

```

1  class Base {}
2  class Derived1 extends Base {}
3  class Derived2 extends Base {}
4  (p: Derived1 | Derived2 ): void
5  (p: Base): void

```

```

1  class G<T>
2  (y: G<Number>): void
3  (x: G<String>): void

```

## 15.7.2 Override-Compatible Signatures

If there are two classes `Base` and `Derived`, and class `Derived` overrides the method `foo()` of `Base`, then `foo()` in `Base` has signature  $S_1 \langle V_1, \dots, V_k \rangle (U_1, \dots, U_n) : U_{n+1}$ , and `foo()` in `Derived` has signature  $S_2 \langle W_1, \dots, W_l \rangle (T_1, \dots, T_m) : T_{m+1}$  as in the example below:

```

1  class Base {
2      foo <V1, ... Vk> (p1: U1, ... pn: Un): Un+1
3  }
4  class Derived extends Base {
5      override foo <W1, ... Wl> (p1: T1, ... pm: Tm): Tm+1
6  }

```

The signature  $S_2$  is override-compatible with  $S_1$  only if **all** of the following conditions are met:

1. Number of parameters of both methods is the same, i.e.,  $n = m$ .
2. Each type  $T_i$  is override-compatible with type  $U_i$  for  $i$  in  $1..n+1$ . Type override compatibility is defined below.
3. Number of type parameters of either method is the same, i.e.,  $k = l$ .
4. Constraints of  $W_1, \dots, W_l$  are to be contravariant (see *Invariance, Covariance and Contravariance*) to the appropriate constraints of  $V_1, \dots, V_k$ .

There are two cases of type override-compatibility, as types are used as either parameter types, or return types. Each case has the following kinds of types:

- Class/interface type;
- Function type;
- Primitive type;
- Enum types;
- Union types;
- Array type;

- Tuple type; and
- Type parameter.

Each type is override-compatible with itself (see *Invariance, Covariance and Contravariance*).

Mixed override-compatibility between types of different kinds is always false, except the compatibility with class type `Object` as any type is a subtype of `Object`.

The following rule applies to generics:

- Derived class must have type parameter constraints to be assignable (see *Assignability*) to the respective type parameter constraint in the base type;
- Otherwise, a compile-time error occurs.

```

1 class Base {}
2 class Derived extends Base {}
3 class A1 <CovariantTypeParameter extends Base> {}
4 class B1 <CovariantTypeParameter extends Derived> extends A1<CovariantTypeParameter>
  ↳ {}
5      // OK, derived class may have type compatible constraint of type parameters
6
7 class A2 <ContravariantTypeParameter extends Derived> {}
8 class B2 <ContravariantTypeParameter extends Base> extends A2
  ↳ <ContravariantTypeParameter> {}
9      // Compile-time error, derived class cannot have non-compatible constraints of
  ↳ type parameters

```

Variances to be used for types that can be override-compatible in different positions are represented in the table below:

|   | Positions ==>             | Parameter Types   | Return Types      |
|---|---------------------------|-------------------|-------------------|
|   | Type Kinds                |                   |                   |
| 1 | Class/interface types     | Contravariance >: | Covariance <:     |
| 2 | Function types            | Covariance <:     | Contravariance >: |
| 3 | Primitive types           | Invariance        | Invariance        |
| 4 | Union types               | Contravariance >: | Contravariance >: |
| 5 | Enum types                | Invariance        | Invariance        |
| 6 | Array types               | Invariance        | Invariance        |
| 7 | Tuple types               | Invariance        | Invariance        |
| 8 | Type parameter constraint | Contravariance >: | Contravariance >: |

The semantics is illustrated by the example below:

```

1 enum E1 {Red, Green, Blue}
2 enum E2 {A, B, C}
3 interface BaseSuperType {}
4 interface Base extends BaseSuperType {
5     // Overriding for parameters
6     kinds_of_parameters01 <T extends Derived, U extends Base> (p: Derived): void
7     kinds_of_parameters02 <T extends Derived, U extends Base> (p: (q: Base)=>Derived):
  ↳ void
8     kinds_of_parameters03 <T extends Derived, U extends Base> (p: number): void
9     kinds_of_parameters04 <T extends Derived, U extends Base> (p: Number): void
10    kinds_of_parameters05 <T extends Derived, U extends Base> (p: T | U): void
11    kinds_of_parameters06 <T extends Derived, U extends Base> (p: E1): void
12    kinds_of_parameters07 <T extends Derived, U extends Base> (p: Base[]): void
13    kinds_of_parameters08 <T extends Derived, U extends Base> (p: [Base, Base]): void

```

(continues on next page)

(continued from previous page)

```

14 kinds_of_parameters09 <T extends Derived, U extends Base>(p: T): void
15 kinds_of_parameters10 <T extends Derived, U extends Base>(p10: U): void
16
17 // Overriding for return type
18 kinds_of_return_type01(): Base
19 kinds_of_return_type02(): (q: Derived)=> Base
20 kinds_of_return_type03(): number
21 kinds_of_return_type04(): Number
22 kinds_of_return_type05<T extends Derived, U extends Base>(): T | U
23 kinds_of_return_type06(): E1
24 kinds_of_return_type07(): Base[]
25 kinds_of_return_type08(): [Base, Base]
26 kinds_of_return_type09 <T extends Derived>(): T
27 }
28
29 interface Derived extends Base {
30     // Overriding kinds for parameters, Derived <: Base
31     kinds_of_parameters01 <T extends Base, U extends Object>(
32         p: Base // contravariant parameter type
33     ): void
34     kinds_of_parameters02 <T extends Base, U extends Object>(
35         p: (q: Derived)=>Base // Covariant parameter type, contravariant return type
36     ): void
37     kinds_of_parameters03 <T extends Base, U extends Object>(
38         p: Number // Compile-time error: parameter type is not override-compatible
39     ): void
40     kinds_of_parameters04 <T extends Base, U extends Object>(
41         p: number // Compile-time error: parameter type is not override-compatible
42     ): void
43     kinds_of_parameters05 <T extends Base, U extends Object>(
44         p: Base | BaseSuperType // contravariant parameter type: Derived | Base <:
↳Base | BaseSuperType
45     ): void
46     kinds_of_parameters06 <T extends Base, U extends Object>(
47         // p: E2 // Compile-time error: parameter type is not override-compatible
48         p: E1 // Invariance parameter type
49     ): void
50     kinds_of_parameters07 <T extends Base, U extends Object>(
51         p: Base[] // Invariant array element type
52     ): void
53     kinds_of_parameters08 <T extends Base, U extends Object>(
54         p: [Derived, Derived] // Compile-time error: parameter type is not override-
↳compatible
55     ): void
56     kinds_of_parameters09 <T extends Base, U extends Object>(
57         p: T // Contravariance for constraints of type parameters
58     ): void
59     kinds_of_parameters10 <T extends Base, U extends Object>(
60         p: U // Contravariance for constraints of type parameters
61     ): void
62
63     // Overriding kinds for return type
64     kinds_of_return_type01(): Derived // Covariant return type
65     kinds_of_return_type02(): (q: Base)=> Derived // Contravariant parameter type,
↳covariant return type
66     //kinds_of_return_type03(): Number // Compile-time error: return type is not
↳override-compatible

```

(continues on next page)

(continued from previous page)

```

67 //kinds_of_return_type04(): number // Compile-time error: return type is not_
↳override-compatible
68 kinds_of_return_type05<T extends Base, U extends BaseSuperType>(): T | U
69 //kinds_of_return_type06(): E2 // CTE
70 kinds_of_return_type06(): E1 // OK
71 //kinds_of_return_type07(): Derived[] // Compile-time error
72 //kinds_of_return_type08(): [Derived, Derived] // Compile-time error
73 kinds_of_return_type09 <T extends Base> (): T // OK, contravariance for_
↳constraints of the return type
74 kinds_of_return_type09 <T extends Base> (): T // OK, contravariance for_
↳constraints of the return type
75 }

```

The example below illustrates override-compatibility with Object:

```

1 interface Base {
2   kinds_of_parameters<T extends Derived, U extends Base>( // It represents all_
↳possible kinds of parameter type
3     p01: Derived,
4     p02: (q: Base)=>Derived,
5     p03: number,
6     p04: Number,
7     p05: T | U,
8     p06: E1,
9     p07: Base[],
10    p08: [Base, Base]
11  ): void
12  kinds_of_return_type(): Object // It can be overridden by all subtypes except_
↳primitive ones
13 }
14 interface Derived extends Base {
15   kinds_of_parameters( // Object is a supertype for all types except primitive ones
16     p1: Object,
17     p2: Object,
18     p3: Object, // Compile-time error: number and Object are not override-
↳compatible
19     p4: Object,
20     p5: Object,
21     p6: Object,
22     p7: Object,
23     p8: Object
24   ): void
25 }
26
27
28 interface Derived1 extends Base {
29   kinds_of_return_type(): Base // Valid overriding
30 }
31 interface Derived2 extends Base {
32   kinds_of_return_type(): (q: Derived)=> Base // Valid overriding
33 }
34 interface Derived3 extends Base {
35   kinds_of_return_type(): number // Compile-time error: number and Object are not_
↳override-compatible
36 }
37 interface Derived4 extends Base {
38   kinds_of_return_type(): Number // Valid overriding

```

(continues on next page)



(continued from previous page)

```

39  }
40  interface Derived5 extends Base {
41      kinds_of_return_type(): number | string // Valid overriding
42  }
43  interface Derived6 extends Base {
44      kinds_of_return_type(): E1 // Valid overriding
45  }
46  interface Derived7 extends Base {
47      kinds_of_return_type(): Base[] // Valid overriding
48  }
49  interface Derived8 extends Base {
50      kinds_of_return_type(): [Base, Base] // Valid overriding
51  }

```

### 15.7.3 Overloading for Functions

*Overloading* must only be considered for functions because inheritance for functions is not defined.

The correctness check for functions overloading is performed if two or more functions with the same name are accessible (see [Accessible](#)) in a scope (see [Scopes](#)).

A function can be declared in, or imported to a scope.

The semantic check for overloading functions is as follows:

- If function signatures are *overload-equivalent*, then a compile-time error occurs.
- Otherwise, *overloading* is valid.

It is discussed in detail in [Function Overloading](#) and [Import and Overloading of Function Names](#).

### 15.7.4 Overloading and Overriding in Classes

Both *overloading* and *overriding* must be considered in case of classes for methods and partly for constructors.

**Note.** Only accessible (see [Accessible](#)) methods are subjected to overloading and overriding. For example, neither overriding nor overloading is considered if a superclass contains a `private` method, and a subclass has a method with the same name. The same rules also apply to accessors in case of overriding.

An overriding member can keep or extend an access modifier (see [Access Modifiers](#)) of a member that is inherited or implemented. Otherwise, a compile-time error occurs:

```

1  class Base {
2      public public_member() {}
3      protected protected_member() {}
4      internal internal_member() {}
5      private private_member() {}
6  }
7

```

(continues on next page)

(continued from previous page)

```

8 interface Interface {
9     public_member() // All members are public in interfaces
10 }
11
12 class Derived extends Base implements Interface {
13     public override public_member() {}
14     // Public member can be overridden and/or implemented by the public one
15     public override protected_member() {}
16     // Protected member can be overridden by the protected or public one
17     internal internal_member() {}
18     // Internal member can be overridden by the internal one only
19     override private_member() {}
20     // A compile-time error occurs if an attempt is made to override private member
21 }

```

The table below represents semantic rules that apply in various contexts:

| Context                                                                                                                               | Semantic Check                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Two <i>instance methods</i> , two <i>static methods</i> with the same name, or two <i>constructors</i> are defined in the same class. | If signatures are <i>overload-equivalent</i> , (see <i>Overload-Equivalent Signatures</i> ), then a compile-time error occurs. Otherwise, <i>overloading</i> is used. |

```

1 class aClass {
2
3     instance_method_1() {}
4     instance_method_1() {} // compile-time error: instance method duplication
5
6     static static_method_1() {}
7     static static_method_1() {} // compile-time error: static method duplication
8
9     instance_method_2() {}
10    instance_method_2(p: number) {} // valid overloading
11
12    static static_method_2() {}
13    static static_method_2(p: string) {} // valid overloading
14
15    constructor() {}
16    constructor() {} // compile-time error: constructor duplication
17
18    constructor(p: number) {}
19    constructor(p: string) {} // valid overloading
20
21 }

```

|                                                                                                                       |                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| An <i>instance method</i> is defined in a subclass with the same name as the <i>instance method</i> in a super-class. | If signatures are <i>override-compatible</i> (see <i>Override-Compatible Signatures</i> ), then <i>overriding</i> is used. Otherwise, <i>overloading</i> is used. |
|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

1 class Base {
2     method_1() {}
3     method_2(p: number) {}
4 }
5 class Derived extends Base {

```

(continues on next page)

(continued from previous page)

```
6  override method_1() {} // overriding
7  method_2(p: string) {} // overloading
8  }
```

|                                                                                                                 |                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A <i>static method</i> is defined in a subclass with the same name as the <i>static method</i> in a superclass. | If signatures are <i>overload-equivalent</i> (see <a href="#">Overload-Equivalent Signatures</a> ), then the static method in the subclass <i>hides</i> the previous static method. Otherwise, <i>overloading</i> is used. |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
1  class Base {
2      static method_1() {}
3      static method_2(p: number) {}
4  }
5  class Derived extends Base {
6      static method_1() {} // hiding
7      static method_2(p: string) {} // overloading
8  }
```

|                                                |                                                                                       |
|------------------------------------------------|---------------------------------------------------------------------------------------|
| A <i>constructor</i> is defined in a subclass. | All base class constructors are available for call in all derived class constructors. |
|------------------------------------------------|---------------------------------------------------------------------------------------|

```
1  class Base {
2      constructor() {}
3      constructor(p: number) {}
4  }
5  class Derived extends Base {
6      constructor(p: string) {
7          super()
8          super(5)
9      }
10 }
```

15.7.5 Overloading and Overriding in Interfaces

| Context                                                                                       | Semantic Check                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A method is defined in a subinterface with the same name as the method in the superinterface. | If signatures are <i>override-compatible</i> (see <a href="#">Override-Compatible Signatures</a> ), then <i>overriding</i> is used. Otherwise, <i>overloading</i> is used. |

```
1  interface Base {
2      method_1()
3      method_2(p: number)
4  }
5  interface Derived extends Base {
6      method_1() // overriding
7      method_2(p: string) // overloading
8  }
```

Two methods with the same name are defined in the same interface.

A compile-time error occurs. Otherwise, *overloading* is used.

```

1 interface anInterface {
2     instance_method_1()
3     instance_method_1() // Compile-time error: instance method duplication
4
5     instance_method_2()
6     instance_method_2(p: number) // Valid overloading
7 }

```

## 15.8 Overload Resolution

*Overload resolution* is used to select one entity to call from a set of *potentially applicable candidates* in a function, method, or constructor call. Overload resolution is performed in two steps as follows:

1. Select *applicable candidates* from *potentially applicable candidates*;
2. If there is more than one *applicable candidate*, then select the best candidate.

**Note.** The first step is performed in all cases, even if there is only one *applicable candidate* to check *call signature compatibility*.

### 15.8.1 Selection of Applicable Candidates

The selection of *applicable candidates* is the process of checking *Compatibility of Call Arguments* for all entities from the set of *potentially applicable candidates*. If any argument is not compatible with the corresponding parameter type, then the entity is deleted from the set.

**Note.** Compile-time errors are not reported at this stage.

After processing all entities, one of the following results is achieved:

- Set is empty (all entities are deleted). A compile-time error occurs, and the *overload resolution* is completed.
- Only one entity is left in the set. This is the entity to call, and the *overload resolution* is completed.
- More than one entity is left in the set. The next step of the *overload resolution* is to be performed.

Two overloaded functions are considered in the following example:

```

1 class Base { }
2 class Derived extends Base { }
3
4 function foo(p: Base) { ... } // #1
5 function foo(p: Derived) { ... } // #2
6
7 foo(new Derived) // two applicable candidates for this call

```

(continues on next page)

(continued from previous page)

```

8           // next step of overload resolution is required
9
10 foo(new Base)    // one applicable candidate
11                // overload resolution is completed
12                // #1 will be called
13
14 foo(new Base, 5) // no candidates, compile-time error

```

## 15.8.2 Selection of Best Candidate

If the set of *applicable candidates* has two or more candidates, then the best candidate for the given list of arguments is to be identified, if possible.

The selection of the best candidate is based on the following:

- There are no candidates with the same list of parameters, as this situation is already forbidden by the compiler (at the place of declaration or import) (see *Overload-Equivalent Signatures*);
- If several candidates can be called correctly by using the same argument list, then at least one implicit argument transformation must be applied to make the call.

Possible argument transformations are listed below:

- *Implicit Conversions*;
- Passing default values to fill any missing arguments (*Optional Parameters*);
- Passing the empty array to replace a `rest` parameter that has no argument;
- Folding several arguments to the array for a `rest` parameter.

The examples of transformations are presented below:

```

1 function foo1(x: number) {}
2 foo1(1) // implicit conversion int -> double
3
4 function foo2(x: Int) {}
5 foo2(1) // implicit boxing
6
7 function foo3(x?: string) {}
8 foo3() // passing default value -> foo(undefined)
9
10 function foo4(...x: int[]) {}
11 foo4()    // passing empty array -> foo([])
12 foo4(1, 2) // folding to array -> foo(...[1, 2])

```

The candidate that does not require transformations for all arguments is the *best candidate*. Other candidates are not considered in this case.

The examples below represent the best candidate selected without transformation:

```

1 function foo(i: int)    // #1
2 function foo(n: number) // #2
3

```

(continues on next page)

(continued from previous page)

```

4 function max(a: number, b: number) // #1
5 function max(...args: number[]) // #2
6
7 max(1, 2) // #1 - is the best candidate, no transformations

```

If there is no *best candidate* on this step, then candidates are compared for each argument (taking default values for optional parameters and arguments of `rest` parameter into the account) by calculating partial *better* relation.

If for the argument, parameter types and parameter kinds (optional or `rest`) are the same, this argument is skipped from comparison. Otherwise:

**Case 1.** No argument is *better* than default value for optional parameter and empty list for `rest` parameter.

**Case 2.** If an argument or several arguments correspond to non-optional parameters in the first candidate, and the other candidate has a `rest` parameter for these arguments, then the first one is *better*.

**Case 3.** No transformation for an argument is *better* than any transformation.

**Case 4.** If argument type is of a numeric type (see *Numeric Types*), `char`, or its boxed counterpart, then the candidate with a *shorter* conversion is *better*. E.g., the conversion of `int` to `float` is *better* than `int` to `double`, and `int` to `Int` is *better* than `int` to `Long`.

**Case 5.** If transformations are applied to the argument for both first and second candidates, then both candidates are not *best candidates*.

```

1 // Case 1:
2 function foo(n: number, s?: string) // #1
3 function foo(n: number) // #2
4
5 foo(1) // #2 is better, less parameters
6
7 function bar(...args: number[]) // #1
8 function bar() // #2
9
10 bar(1) // #2 is better, less parameters
11
12 // Case 2:
13 function foo(sum: number, a: number, b: number) // #1
14 function foo(sum: number, ...x: number[]) // #2
15
16 foo(1, 2, 3) // #1 is better, non-rest parameters
17
18 // Case 3:
19 function foo(n: number, s: string|null) // #1
20 function foo(n: number, s: string) // #2
21
22 goo(1, "abc") // #2 is better, no transformation for 2nd argument
23
24 // Case 4:
25 function foo(i: long) // #1
26 function foo(n: float) // #2
27
28 let x: int = 1
29 foo(x) // #1 is better, conversion is shorter
30
31 // Case 5:
32 function foo(n: number) // #1
33 function foo(x: number | string) // #2

```

(continues on next page)

(continued from previous page)

```

34
35 foo(1) // different non-compared transformations - compile-time error

```

If there is exactly one candidate that is *better* than others for at least one argument or several arguments and other arguments were skipped from comparison, then this is the *best candidate*.

A index:*compile-time error* occurs:

- if no candidate is the *best candidate*.
- if according to some argument the first candidate is better and for other argument the other candidate is better.

Example of the last case is presented below:

```

1 function goo(a: int; b: float) // #1
2 function goo(a: float, b: int) // #2
3
4 goo(1, 1) // compile-time error, as
5           // #1 is better for 1st argument,
6           // #2 is better for 2nd argument.

```

## 15.9 Initializer Block

*Initializer block* is used in classes (see *Class Initializer*), packages (see *Package Initializer*), and namespaces (see *Namespace Declarations*) to ensure that all their variables (see *Variable and Constant Declarations*) and fields (see *Field Declarations*) have valid initial values.

Appropriate syntax is presented below:

```

initializerBlock:
    'static' block
    ;

```

If an *initializer block* contains a return <expression> statement (see *return Statements*), then a compile-time error occurs. If the code of an *initializer block* contains an unhandled throw statement (see *throw Statements*), then a program terminates (see *Program Exit*).

These situations are represented in the examples below:

```

1 static {
2
3     throw new Error // No surrounding 'try'
4
5     try { throw new Error } // 'try' has no catch
6     finally {}
7
8     try { throw new Error }
9     catch (e) { throw new Error } // No surrounding 'try' in 'catch'
10
11     foo () // Function call throws an error
12
13 }

```

(continues on next page)

(continued from previous page)

```
14
15 function foo() {
16     throw new Error // No surrounding 'try'
17 }
```

## 15.10 Compatibility Features

Some features are added to ArkTS in order to support smooth TypeScript compatibility. Using these features while doing the ArkTS programming is not recommended in most cases.

### 15.10.1 Extended Conditional Expressions

ArkTS provides extended semantics for conditional expressions to ensure better TypeScript alignment. It affects the semantics of the following:

- Conditional expressions (see *Conditional Expressions*, *Conditional-And Expression*, *Conditional-Or Expression*, and *Logical Complement*);
- `while` and `do` statements (see *while Statements and do Statements*);
- `for` statements (see *for Statements*);
- `if` statements (see *if Statements*).

**Note.** The extended semantics is to be deprecated in one of the future versions of ArkTS.

The extended semantics approach is based on the concept of *truthiness* that extends the Boolean logic to operands of non-Boolean types.

Depending on the type kind of a valid expression, the value of such valid expression can be handled as `true` or `false` as described in the table below:



| Value Type Kind                                     | When false                        | When true                        | ArkTS Code Example to Check                                                    |
|-----------------------------------------------------|-----------------------------------|----------------------------------|--------------------------------------------------------------------------------|
| string                                              | empty string                      | non-empty string                 | <code>s.length == 0</code>                                                     |
| boolean                                             | false                             | true                             | <code>x</code>                                                                 |
| enum                                                | enum constant<br>handled as false | enum constant<br>handled as true | <code>x.valueOf()</code>                                                       |
| number<br>(double/float)                            | 0 or NaN                          | any other number                 | <code>n != 0 &amp;&amp; !isNaN(n)</code>                                       |
| any integer type                                    | <code>== 0</code>                 | <code>!= 0</code>                | <code>i != 0</code>                                                            |
| bigint                                              | <code>== 0n</code>                | <code>!= 0n</code>               | <code>i != 0n</code>                                                           |
| char                                                | <code>== 0</code>                 | <code>!= 0</code>                | <code>c != c'\u0000'</code>                                                    |
| null or undefined                                   | always                            | never                            | <code>x != null or x != undefined</code>                                       |
| Union types                                         | when value is<br>falsy            | when value is<br>truthy          | <code>x != null or x != undefined</code><br>for union types with nullish types |
| Boxed primitive type<br>(Boolean, Char, Int<br>...) | primitive type is<br>falsy        | primitive type is<br>truthy      | <code>new Boolean(true) == true</code><br><code>new Int (0) == 0</code>        |
| any other nonNullish type                           | never                             | always                           | <code>new SomeType != null</code>                                              |

Extended semantics of *Conditional-And Expression* and *Conditional-Or Expression* affects the resultant type of expressions as follows:

- A *conditional-and* expression `A && B` is of type B if the result of A is handled as `true`. Otherwise, it is of type A.
- A *conditional-or* expression `A || B` is of type B if the result of A is handled as `false`. Otherwise, it is of type A.

The example below illustrates the way this approach works in practice. Any nonzero number is handled as `true`. The loop continues until it becomes zero that is handled as `false`:

```

1  for (let i = 10; i; i--) {
2      console.log (i)
3  }
4  /* And the output will be
5      10
6      9
7      8
8      7
9      6
10     5
11     4
12     3
13     2
14     1
15  */

```



## CHAPTER 16

---

### Support for GUI Programming

---

This Chapter is deprecated. See ArkUI documentation.



---

## Experimental Features

---

This Chapter introduces the ArkTS features that are considered parts of the language, but have no counterpart in TypeScript, and are therefore not recommended to those who seek a single source code for TypeScript and ArkTS.

Some features introduced in this Chapter are still under discussion. They can be removed from the final version of the ArkTS specification. Once a feature introduced in this Chapter is approved and/or implemented, the corresponding section is moved to the body of the specification as appropriate.

The *array creation* feature introduced in *Array Creation Expressions* enables users to dynamically create objects of array type by using runtime expressions that provide the array size. This addition is useful to other array-related features of the language, such as array literals.

The construct can also be used to create multidimensional arrays.

The feature *function and method overloading* is supported in many (if not all) modern programming languages. Overloading functions/methods is a practical and convenient way to write program actions that are similar in logic but different in implementation. It is discussed in detail in section *Function, Method and Constructor Overloading*.

Section *Native Functions and Methods* introduces practically important and useful mechanisms for the inclusion of components written in other languages into a program written in ArkTS.

Section *Final Classes and Methods* discusses the well-known feature that in many OOP languages provides a way to restrict class inheritance and method overriding. Making a class *final* prohibits defining classes derived from it, whereas making a method *final* prevents it from overriding in derived classes.

Section *Adding Functionality to Existing Types* discusses the way to add new functionality to an already defined type. This feature can be used for GUI programming (*Support for GUI Programming*).

Section *Enumeration Methods* adds methods to declarations of the enumeration types. Such methods can help in some kinds of manipulations with `enums`.

The ArkTS language supports writing concurrent applications in the form of *coroutines* (see *Coroutines*) that allow executing functions concurrently.

There is a basic set of language constructs that support concurrency. A function to be launched asynchronously is marked by adding the modifier `async` to its declaration. In addition, any function—or lambda expression—can be launched as a separate thread explicitly by using the `launch` expression.

The `await` statement is introduced to synchronize functions launched as threads. The generic class `Promise<T>` from the standard library (see [Standard Library](#)) is used to exchange information between threads. The class can be handled as an implementation of the channel mechanism. The class provides a number of methods to manipulate the values produced by threads.

Section [Packages](#) discusses a well-known and proven language feature intended to organize large pieces of software that typically consist of many components. *Packages* allow developers to construct a software product as a composition of subsystems, and organize the development process in a way that is appropriate for independent teams to work in parallel.

*Package* is the language construct that combines a number of declarations, and makes them parts of an independent compilation unit.

The *export* and *import* features are used to organize communication between *packages*. An entity exported from one package becomes known to and accessible (see [Accessible](#)) in another package which imports that feature. Various options are provided to simplify export/import, e.g., by defining non-exported, i.e., *internal* declarations that are not accessible (see [Accessible](#)) from the outside of the package.

In addition, ArkTS supports the *package* initialization semantics that makes a *package* even more independent from its environment.

## 17.1 Character Type and Literals

### 17.1.1 Character Literals

*Character literal* represents the following:

- Value consisting of a single character; or
- Single escape sequence preceded by the characters *single quote* (U+0027) and *'c'* (U+0063), and followed by a *single quote* (U+0027).

```
CharLiteral:
    'c\' ' SingleQuoteCharacter '\ '
    ;

SingleQuoteCharacter:
    ~['\\\'r\n]
    | '\\' EscapeSequence
    ;
```

The examples are presented below:

```
1  c'a'
2  c'\n'
3  c'\x7F'
4  c'\u0000'
```

*Character literals* are of type `char`.

## 17.1.2 Character Type and Operations

| Type           | Type's Set of Values                                                                     | Corresponding Class Type |
|----------------|------------------------------------------------------------------------------------------|--------------------------|
| char (32-bits) | Symbols with codes from U+0000 to U+10FFFF (maximum valid uni-code code point) inclusive | Char                     |

ArkTS provides a number of operators to act on character values as discussed below.

All character operators are identical to integer operators (see *Integer Types and Operations*) for they handle character values as integers of type `int` (see *Widening Primitive Conversions*).

The class `Char` provides constructors, methods, and constants that are parts of the ArkTS standard library (see *Standard Library*).

## 17.2 Fixed Array Types

*Fixed array type* is the built-in type characterized by the following:

- Any object of array type contains elements, and the number of such elements is known as *array length*.
- Array length is a non-negative integer number.
- Array length is set once at runtime and can not be changed after that.
- Array element is accessed by its index. *Index* is an integer number starting from 0 to *array length minus 1*.
- Accessing an element by its index is a constant-time operation.
- If passed to non-ArkTS environment, an array is represented as a contiguous memory location.
- Type of each array element is assignable to the element's type specified in the array declaration (see *Assignability*).

*Fixed arrays* differ from *resizable arrays* as follows:

- *Fixed array* length is set once to achieve better performance;
- *Fixed arrays* have no methods defined;
- *Fixed arrays* are not compatible with *resizable arrays*.

*Fixed array* can be created by using *Array Literal*.

The examples are presented below:

```

1  let a : FixedArray<number> = [1, 2, 3]
2      /* allocate array with 3 elements of type number */
3  a[1] = 7 /* put 7 as the 2nd element of the array, index of this element is 1 */
4  let y = a[2] /* get the last element of array 'a' */
5  let count = a.length // get the number of array elements
6  y = a[3] // Will lead to runtime error - attempt to access non-existing array element

```

TBD: example for incompatibility between array and fixed array.

## 17.3 Array Creation Expressions

*Array creation expression* creates new objects that are instances of arrays. The *array literal* expression is used to create an array instance, and to provide initial values (see [Array Literal](#)).

```

newArrayInstance:
    'new' arrayElementType dimensionExpression+ (arrayElement)?
    ;

arrayElementType:
    typeReference
    | '(' type ')'
    ;

dimensionExpression:
    '[' expression ']'
    ;

arrayElement:
    '(' expression ')'
    ;

```

```

let x = new number[2][2] // create 2x2 matrix

```

*Array creation expression* creates an object that is a new array with the elements of the type specified by `arrayElementType`.

The type of each *dimensionExpression* must be convertible (see [Primitive Types Conversions](#)) to an integer type. Otherwise, a compile-time error occurs.

A numeric conversion (see [Primitive Types Conversions](#)) is performed on each *dimensionExpression* to ensure that the resultant type is `int`. Otherwise, a compile-time error occurs.

A compile-time error occurs if any *dimensionExpression* is a constant expression that is evaluated to a negative integer value at compile time.

If the type of any *dimensionExpression* is `number` or other floating-point type, and its fractional part is different from '0', then errors occur as follows:

- Compile-time error, if the situation is identified during compilation; and
- Runtime error, if the situation is identified during program execution.

If `arrayElement` is provided, then the type of the expression can be as follows:

- Type of array element denoted by `arrayElementType`, or
- Lambda function with the return type equal to the type of array element denoted by `arrayElementType` and the parameters of type `int`, and the number of parameters equal to the number of array dimensions.

Otherwise, a compile-time error occurs.



```

1  let x = new number[-3] // compile-time error
2
3  let y = new number[3.141592653589] // compile-time error
4
5  foo (3.141592653589)
6  function foo (size: number) {
7      let y = new number[size] // runtime error
8  }

```

A compile-time error occurs if `arrayElementType` refers to a class that does not contain an accessible (see *Accessible*) parameterless constructor, or constructor with all parameters of the second form of optional parameters (see *Optional Parameters*), or if type has no default value:

```

1  let x = new string[3] // compile-time error: string has no default value
2
3  class A {
4      constructor (p1?: number, p2?: string) {}
5  }
6  let y = new A[2] // OK, as all 3 elements of array will be filled with
7  // new A() objects

```

A compile-time error occurs if `arrayElementType` is a type parameter:

```

1  class A<T> {
2      foo() {
3          new T[2] // compile-time error: cannot create an array of type parameter_
↪elements
4      }
5  }

```

The creation of an array with a known number of elements is presented below:

```

1  class A {}
2  // It has no default value or parameterless constructor defined
3
4  let array_size = 5
5
6  let array1 = new A[array_size] (new A)
7  /* Create array of 'array_size' elements and all of them will have
8  initial value equal to an object created by new A expression */
9
10 let array2 = new A[array_size] ((index): A => { return new A })
11 /* Create array of 'array_size' elements and all of them will have
12 initial value equal to the result of lambda function execution with
13 different indices */
14
15 let array2 = new A[2][3] ((index1, index2): A => { return new A })
16 /* Create two-dimensional array of 6 elements total and all of them will
17 have initial value equal to the result of lambda function execution with
18 different indices */

```

The creation of exotic arrays with different kinds of element types is presented below:

```

1  let array_of_union = new (Object|null) [5] // filled with null
2  let array_of_functor = new (() => void) [5] ( (): void => {})
3  type aliasTypeName = number []
4  let array_of_array = new aliasTypeName [5] ( [3.141592653589] )

```

The creation of fixed arrays is presented below:

```
1 let fixed_array : FixedArray<number> = new number[5]
```

### 17.3.1 Runtime Evaluation of Array Creation Expressions

The evaluation of an array creation expression at runtime is performed as follows:

1. The dimension expressions are evaluated. The evaluation is performed left-to-right. If any expression evaluation completes abruptly, then the expressions to the right of it are not evaluated.
2. The values of dimension expressions are checked. If the value of any `dimExpr` expression is less than zero, then `NegativeArraySizeError` is thrown.
3. Space for the new array is allocated. If the available space is not sufficient to allocate the array, then `OutOfMemoryError` is thrown, and the evaluation of the array creation expression completes abruptly.
4. When a one-dimensional array is created, each element of that array is initialized to its default value if type default value is defined (*Default Values for Types*). If the default value for an element type is not defined, but the element type is a class type, then its *parameterless* constructor is used to create the value of each element.
5. When a multidimensional array is created, the array creation effectively executes a set of nested loops of depth *n-1*, and creates an implied array of arrays.

## 17.4 Indexable Types

If a class or an interface declares one or two functions with names `$_get` and `$_set`, and signatures *(index: Type1): Type2* and *(index: Type1, value: Type2)* respectively, then an indexing expression (see *Indexing Expressions*) can be applied to variables of such types:

```
1 class SomeClass {
2     $_get (index: number): SomeClass { return this }
3     $_set (index: number, value: SomeClass) { }
4 }
5 let x = new SomeClass
6 x = x[1] // This notation implies a call: x = x.$_get (1)
7 x[1] = x // This notation implies a call: x.$_set (1, x)
```

If only one function is present, then only the appropriate form of index expression (see *Indexing Expressions*) is available:

```
1 class ClassWithGet {
2     $_get (index: number): ClassWithGet { return this }
3 }
4 let getClass = new ClassWithGet
5 getClass = getClass[0]
6 getClass[0] = getClass // Error - no $_set function available
7
```

(continues on next page)

(continued from previous page)

```

8  class ClassWithSet {
9      $_set (index: number, value: ClassWithSet) { }
10 }
11 let setClass = new ClassWithSet
12 setClass = setClass[0] // Error - no $_get function available
13 setClass[0] = setClass

```

Type string can be used as a type of the index parameter:

```

1  class SomeClass {
2      $_get (index: string): SomeClass { return this }
3      $_set (index: string, value: SomeClass) { }
4  }
5  let x = new SomeClass
6  x = x["index string"]
7      // This notation implies a call: x = x.$_get ("index string")
8  x["index string"] = x
9      // This notation implies a call: x.$_set ("index string", x)

```

Functions `$_get` and `$_set` are ordinary functions with compiler-known signatures. The functions can be used like any other function. The functions can be abstract, or defined in an interface and implemented later. The functions can be overridden and provide a dynamic dispatch for the indexing expression evaluation (see [Indexing Expressions](#)). The functions can be used in generic classes and interfaces for better flexibility. A compile-time error occurs if these functions are marked as `async`.

```

1  interface ReadonlyIndexable<K, V> {
2      $_get (index: K): V
3  }
4
5  interface Indexable<K, V> extends ReadonlyIndexable<K, V> {
6      $_set (index: K, value: V)
7  }
8
9  class IndexableByNumber<V> extends Indexable<number, V> {
10     private data: V[] = []
11     $_get (index: number): V { return this.data [index] }
12     $_set (index: number, value: V) { this.data[index] = value }
13 }
14
15 class IndexableByString<V> extends Indexable<string, V> {
16     private data = new Map<string, V>
17     $_get (index: string): V { return this.data [index] }
18     $_set (index: string, value: V) { this.data[index] = value }
19 }
20
21 class BadClass extends IndexableByNumber<boolean> {
22     override $_set (index: number, value: boolean) { index / 0 }
23 }
24
25 let x: IndexableByNumber<boolean> = new BadClass
26 x[666] = true // This will be dispatched at runtime to the overridden
27 // version of the $_set method
28 x.$_get (15) // $_get and $_set can be called as ordinary
29 // methods

```

## 17.5 Iterable Types

A class or an interface can be made *iterable*. It means that instances of the class or the interface can be used in `for-of` statements (see *for-of Statements*).

Some type `C` is *iterable* if it declares a parameterless function with name `$_iterator` and return type that is assignable (see *Assignability*) to type `Iterator` as defined in the standard library (see *Standard Library*). It guarantees that the object returned is of the class type which implements `Iterator` and allows traversing an object of class type `C`. The example below defines *iterable* class `C`:

```

1  class C {
2      data: string[] = ['a', 'b', 'c']
3      $_iterator() { // Function type is inferred from its body
4          return new CIterator(this)
5      }
6  }
7
8  class CIterator implements Iterator<string> {
9      index = 0
10     base: C
11     constructor (base: C) {
12         this.base = base
13     }
14     next(): IteratorResult<string> {
15         return {
16             done: this.index >= this.base.data.length,
17             value: this.index >= this.base.data.length ? undefined : this.base.data[this.
↪index++]
18         }
19     }
20 }
21
22 let c = new C()
23 for (let x of c) {
24     console.log(x)
25 }

```

In the example above, class `C` function `$_iterator` returns `CIterator<string>` that implements `Iterator<string>`. If executed, this code prints out the following:

```

"a"
"b"
"c"

```

The function `$_iterator` is an ordinary function with a compiler-known signature. The function can be used like any other function. It can be abstract or defined in an interface to be implemented later. A compile-time error occurs if this function is marked as `async`.

**Note.** To support the code compatible with TypeScript, the name of the function `$_iterator` can be written as `[Symbol.iterator]`. In this case, the class *iterable* looks as follows:

```

1  class C {
2      data: string[] = ['a', 'b', 'c'];

```

(continues on next page)

(continued from previous page)

```

3     [Symbol.iterator]() {
4         return new CIterator(this)
5     }
6 }

```

The use of the name `[Symbol.iterator]` is considered deprecated. It can be removed in the future versions of the language.

## 17.6 Callable Types

A type is *callable* if the name of the type can be used in a call expression. A call expression that uses the name of a type is called a *type call expression*. Only class type can be callable. To make a type callable, a static method with the name `$_invoke` or `$_instantiate` must be defined or inherited:

```

1 class C {
2     static $_invoke() { console.log("invoked") }
3 }
4 C() // prints: invoked
5 C.$_invoke() // also prints: invoked

```

In the above example, `C()` is a *type call expression*. It is the short form of the normal method call `C.$_invoke()`. Using an explicit call is always valid for the methods `$_invoke` and `$_instantiate`.

**Note.** Only a constructor—not the methods `$_invoke` or `$_instantiate`—is called in a *new expression*:

```

1 class C {
2     static $_invoke() { console.log("invoked") }
3     constructor() { console.log("constructed") }
4 }
5 let x = new C() // constructor is called

```

The methods `$_invoke` and `$_instantiate` are similar but have differences as discussed below.

A compile-time error occurs if a callable type contains both methods `invoke` and `$_instantiate`.

### 17.6.1 Callable Types with `$_invoke` Method

The method `$_invoke` can have an arbitrary signature. The method can be used in a *type call expression* in either case above. If the signature has parameters, then the call must contain corresponding arguments.

```

1 class Add {
2     static $_invoke(a: number, b: number): number {
3         return a + b
4     }
5 }
6 console.log(Add(2, 2)) // prints: 4

```

## 17.6.2 Callable Types with `$_instantiate` Method

The method `$_instantiate` can have an arbitrary signature by itself. If it is to be used in a *type call expression*, then its first parameter must be a *factory* (i.e., it must be a *parameterless function type returning some class type*). The method can have or not have other parameters, and those parameters can be arbitrary.

In a *type call expression*, the argument corresponding to the *factory* parameter is passed implicitly:

```

1  class C {
2      static $_instantiate(factory: () => C): C {
3          return factory()
4      }
5  }
6  let x = C() // factory is passed implicitly
7
8  // Explicit call of '$_instantiate' requires explicit 'factory':
9  let y = C.$_instantiate(() => { return new C() })

```

If the method `$_instantiate` has additional parameters, then the call must contain corresponding arguments:

```

1  class C {
2      name = ""
3      static $_instantiate(factory: () => C, name: string): C {
4          let x = factory()
5          x.name = name
6          return x
7      }
8  }
9  let x = C("Bob") // factory is passed implicitly

```

A compile-time error occurs in a *type call expression* with type *T*, if:

- *T* has neither method `$_invoke` nor method `$_instantiate`; or
- *T* has the method `$_instantiate` but its first parameter is not a *factory*.

```

1  class C {
2      static $_instantiate(factory: string): C {
3          return factory()
4      }
5  }
6  let x = C() // compile-time error, wrong '$_instantiate' 1st parameter

```

## 17.7 Statements

### 17.7.1 For-of Type Annotation

An explicit type annotation is allowed for a *for-variable*:

```

1  // explicit type is used for a new variable,
2  let x: number[] = [1, 2, 3]
3  for (let n: number of x) {
4      console.log(n)
5  }
```

## 17.8 Function, Method and Constructor Overloading

As many other languages do, ArkTS supports overloading. Overloading allows declaring several functions or methods with the same name but different signatures. ArkTS does not support TypeScript overload signatures that allow specifying several headers for a function or method with a single body but different signatures (see [TypeScript Overload Signatures](#)).

The ArkTS approach delivers better performance because no extra checks are performed during the execution of a specific body at runtime.

### 17.8.1 Function Overloading

If a declaration scope declares and/or imports two or more functions with the same name but different signatures that are not *overload-equivalent* (see [Overload-Equivalent Signatures](#)), then such functions are *overloaded*. Function overload declarations cause no compile-time error on their own.

No specific relationship is required between the return types of the two functions with the same name but different signatures that are not *overload-equivalent* (see [Overload-Equivalent Signatures](#)).

When calling an overloaded function, the number of actual arguments (and any explicit type arguments) and compile-time argument types are used at compile time to determine exactly which one is to be called (see [Function Call Expression](#)).

### 17.8.2 Class Method Overloading

If two or more methods within a class have the same name, and their signatures are not *overload-equivalent* (see [Overload-Equivalent Signatures](#)), then such methods are considered *overloaded*.

Method overloading declarations cause no compile-time error on their own, except where possible instantiation causes an *overload-equivalent* (see [Overload-Equivalent Signatures](#)) method in the instantiated class or interface:

```

1  class Template<T> {
2      foo (p: number) { ... }
3      foo (p: T) { ... }
4  }
5  let instantiation: Template<number>
6      // Leads to two *overload-equivalent* methods
7
8  interface ITemplate<T> {
9      foo (p: number)
10     foo (p: T)
11 }
12 function foo (instantiation: ITemplate<number>) { ... }
13     // Leads to two *overload-equivalent* methods

```

If signatures of two or more methods with the same name are not *overload-equivalent* (see [Overload-Equivalent Signatures](#)), then return types of those methods can have any kind of relationship.

When calling an overloaded method, the number of actual arguments (and any explicit type arguments) and compile-time argument types are used at compile time to determine exactly which one is to be called (see [Method Call Expression](#) and [Step 2: Selection of Method](#)).

### 17.8.3 Constructor Overloading

Constructor overloading behavior is identical to that of method overloading (see [Class Method Overloading](#)). Each class instance creation expression (see [New Expressions](#)) resolves the constructor overloading call if any at compile time.

### 17.8.4 Declaration Distinguishable by Signatures

Same-name declarations are distinguishable by signatures if such declarations are one of the following:

- Functions with the same name and signatures that are not *overload-equivalent* (see [Overload-Equivalent Signatures](#) and [Function Overloading](#)).
- Methods with the same name and signatures that are not *overload-equivalent* (see [Overload-Equivalent Signatures](#), [Class Method Overloading](#), and [Interface Method Overloading](#)).
- Constructors of the same class and signatures that are not *overload-equivalent* (see [Overload-Equivalent Signatures](#) and [Constructor Overloading](#)).

The example below represents the functions distinguishable by signatures:

```

1  function foo() {}
2  function foo(x: number) {}
3  function foo(x: number[]) {}
4  function foo(x: string) {}

```

The following example represents the functions indistinguishable by signatures that cause a compile-time error:



```

1 // Functions have overload-equivalent signatures
2 function foo(x: number) {}
3 function foo(y: number) {}
4
5 // Functions have overload-equivalent signatures
6 function foo(x: number) {}
7 type MyNumber = number
8 function foo(x: MyNumber) {}

```

## 17.9 Native Functions and Methods

### 17.9.1 Native Functions

*Native function* is a function marked with the keyword `native` (see [Function Declarations](#)).

*Native function* implemented in a platform-dependent code is typically written in another programming language (e.g., C). A compile-time error occurs if a native function has a body.

### 17.9.2 Native Methods

*Native method* is a method marked with the keyword `native` (see [Method Declarations](#)).

*Native methods* are the methods implemented in a platform-dependent code written in another programming language (e.g., C).

A compile-time error occurs if:

- Method declaration contains the keyword `abstract` along with the keyword `native`.
- *Native method* has a body (see [Method Body](#)) that is a block instead of a simple semicolon or empty body.
- *Native method* is defined in a local class (see [Local Classes and Interfaces](#)).

### 17.9.3 Native Constructors

*Native constructor* is a constructor marked with the keyword `native` (see [Constructor Declaration](#)).

*Native constructors* are the constructors implemented in a platform-dependent code written in another programming language (e.g., C).

A compile-time error occurs if a *native constructor*:

- Has a non-empty body (see *Constructor Body*).
- Is defined in a local class (see *Local Classes and Interfaces*).

## 17.10 Final Classes and Methods

### 17.10.1 Final Classes

A class can be declared `final` to prevent extension, i.e., a class declared `final` cannot have subclasses. No method of a `final` class can be overridden.

If a class type `F` expression is declared *final*, then only a class `F` object can be its value.

A compile-time error occurs if the `extends` clause of a class declaration contains another class that is `final`.

### 17.10.2 Final Methods

A method can be declared `final` to prevent it from being overridden (see *Overloading and Overriding*) in subclasses.

A compile-time error occurs if:

- The method declaration contains the keyword `abstract` or `static` along with the keyword `final`.
- A method declared `final` is overridden.

## 17.11 Default Interface Method Declarations

```
interfaceDefaultMethodDeclaration:  
    'private'? identifier signature block  
    ;
```

A default method can be explicitly declared `private` in an interface body.

A block of code that represents the body of a default method in an interface provides a default implementation for any class if such class does not override the method that implements the interface.

## 17.12 Adding Functionality to Existing Types

ArkTS supports adding functions and accessors to already defined types. The usage of functions so added looks the same as if they are methods and accessors of these types. The mechanism is called *Functions with Receiver* and *Accessors with Receiver*. This feature is often used to add new functionality to a class or interface without having to inherit from this class or implement this interface. However, it can be used not only for classes and interfaces but also for other types.

Moreover, *Function Types with Receiver* and *Lambda Expressions with Receiver* can be defined and used to make the code more flexible.

### 17.12.1 Functions with Receiver

*Function with receiver* declaration is a top-level declaration (see *Top-Level Declarations*) that looks almost the same as *Function Declarations*, except that the first parameter is mandatory, and the keyword `this` is used as its name:

```
functionWithReceiverDeclaration:
    'function' identifier typeParameters? signatureWithReceiver block
    ;

signatureWithReceiver:
    '(' receiverParameter (' ' parameterList)? ')' returnType? throwMark?
    ;

receiverParameter:
    annotationUsage? 'this' ':' type
    ;
```

A *function with receiver* can be called in the following two ways:

- Making a function call (see *Function Call Expression*), and passing the first parameter in the usual way;
- Making a method call (see *Method Call Expression*) with no argument provided for the first parameter, and using the `objectReference` before the function name as the first argument.

```
1  class C {}
2
3  function foo(this: C) {}
4  function bar(this: C, n: number): void {}
5
6  let c = new C()
7
8  // as a function call:
9  foo(c)
10 bar(c, 1)
11
12 // as a method call:
13 c.foo()
14 c.bar(1)
15
16 interface D {}
17 function fool(this: D) {}
18 function bar1(this: D, n: number): void {}
```

(continues on next page)

(continued from previous page)

```

19
20 function demo (d: D) {
21     // as a function call:
22     foo(d)
23     bar(d, 1)
24
25     // as a method call:
26     d.foo()
27     d.bar(1)
28 }

```

The keyword `this` can be used inside a *function with receiver*. It corresponds to the first parameter. The type of this parameter is called the *receiver type* (see [Receiver Type](#)).

If the *receiver type* is a class or interface type, then private or protected members are not accessible (see [Accessible](#)) within the body of a *function with receiver*. Only public and internal members can be accessed. The internal members can be accessed only when *function with receiver* and class or interface type are declared in the same compilation unit:

```

1 class A {
2     foo () { ... this.bar() ... }
3         // function bar() is accessible here
4     protected member_1 ...
5     private member_2 ...
6 }
7 function bar(this: A) { ...
8     this.foo() // Method foo() is accessible as it is public
9     this.member_1 // Compile-time error as member_1 is not accessible
10    this.member_2 // Compile-time error as member_2 is not accessible
11    ...
12 }
13 let a = new A()
14 a.foo() // Ordinary class method is called
15 a.bar() // Function with receiver is called

```

A compile-time error occurs if the name of a *function with receiver* is the same as the name of an accessible instance method or field of the receiver type. It means that a *function with receiver* cannot overload a method defined for the receiver type.

```

1 class A {
2     foo () { ... }
3 }
4
5 function foo(this: A) { ... } // Compile-time error

```

A function with receiver can overload a global function with the same name. A compile-time error occurs if signatures of a function with receiver is overload-equivalent (see [Overload-Equivalent Signatures](#)) to such overloaded function.

```

1 function foo(this: A) { ... }
2 function foo(param: A) { ... } /* Compile-time error as it is a
3                                 non-distinguishable declaration */

```

*Function with receiver* can be generic as in the following example:

```

1 function foo<T>(this: B<T>, p: T) {
2     console.log (p)

```

(continues on next page)

(continued from previous page)

```

3  }
4  function demo (p1: B<SomeClass>, p2: B<BaseClass>) {
5      p1.foo(new SomeClass())
6      // Type inference should determine the instantiating type
7      p2.foo<BaseClass>(new DerivedClass())
8      // Explicit instantiation
9  }

```

*Functions with receiver* are dispatched statically. What function is being called is known at compile time based on the receiver type specified in the declaration. A *function with receiver* can be applied to the receiver of any derived class until it is redefined within the derived class:

```

1  class Base { ... }
2  class Derived extends Base { ... }
3
4  function foo(this: Base) { console.log ("Base.foo is called") }
5  function foo(this: Derived) { console.log ("Derived.foo is called") }
6
7  let b: Base = new Base()
8  b.foo() // `Base.foo is called` to be printed
9  b = new Derived()
10 b.foo() // `Base.foo is called` to be printed
11 let d: Derived = new Derived()
12 d.foo() // `Derived.foo is called` to be printed

```

As illustrated by the following examples, a *function with receiver* can be defined in a compilation unit other than the one that defines the receiver type:

```

1  // file a.sts
2  class A {
3      foo() { ... }
4  }
5
6  // file ext.sts
7  import {A} from "a.sts" // name 'A' is imported
8  function bar(this: A) () {
9      this.foo() // Method foo() is called
10 }

```

## 17.12.2 Receiver Type

*Receiver type* is the type of the *receiver parameter* in a function, function type, and lambda with receiver. A *receiver type* may be an interface type, a class type, an array type, or a type parameter. Otherwise, a compile-time error occurs.

Using an array type as *receiver type* is illustrated by the example below:

```

1  function addElements(this: number[], ...s: number[]) {
2      ...
3  }
4
5  let x: number[] = [1, 2]
6  x.addElements(3, 4)

```

### 17.12.3 Accessors with Receiver

*Accessor with receiver* declaration is a top-level declaration (see *Top-Level Declarations*) that can be used as class or interface accessor (see *Accessor Declarations*) for a specified receiver type:

```
accessorWithReceiverDeclaration:
    'get' identifier '(' receiverParameter ')' returnType block
  | 'set' identifier '(' receiverParameter ',' parameter ')' block
  ;
```

A get-accessor (getter) must have a single *receiver parameter* and an explicit return type.

A set-accessor (setter) must have a *receiver parameter*, one other parameter, and no return type.

The use of getters and setters looks the same as the use of fields:

```
1  class Person {
2      firstName: string
3      lastName: string
4      constructor (first: string, last: string) {...}
5      ...
6  }
7
8  get fullName(this: C): string {
9      return this.LastName + ' ' + this.FirstName
10 }
11
12 let c = new C("John", "Doe")
13
14 // as a method call:
15 console.log(c.fullName) // output: 'Doe John'
16 c.fullName = "new name" // compile-time error, as setter is not defined
```

A compile-time error occurs if an accessor is used in the form of a function or a method call.

### 17.12.4 Function Types with Receiver

*Function type with receiver* specifies the signature of a function or lambda with receiver. It is almost the same as *function type* (see *Function Types*), except that the first parameter is mandatory, and the keyword `this` is used as its name:

```
functionTypeWithReceiver:
    '(' receiverParameter (',' ftParameterList)? ')' ftReturnType
  ;
```

The type of a *receiver parameter* is called the *receiver type* (see *Receiver Type*).

```
1  class A {...}
2
```

(continues on next page)

(continued from previous page)

```

3  type FA = (this: A) => boolean
4  type FN = (this: number[], max: number) => number

```

Function type with receiver can be generic as in the following example:

```

1  class B<T> {...}
2
3  type FB<T> = (this: B<T>, x: T): void
4  type FBS = (this: B<string>, x: string): void

```

The usual rule of function type compatibility (see *Function Types Conversions*) is applied to *function type with receiver*, and parameter names are ignored.

```

1  class A {...}
2
3  type F1 = (this: A) => boolean
4  type F2 = (a: A) => boolean
5
6  function foo(this: A): boolean {}
7  function goo(a: A): boolean {}
8
9  let f1: F1 = foo // ok
10 f1 = goo // ok
11
12 let f2: F2 = goo // ok
13 f2 = foo // ok
14 f1 = f2 // ok

```

The sole difference is that only an entity of *function type with receiver* can be used in *Method Call Expression*. The definitions from the previous example are reused in the example below:

```

1  let a = new A()
2  a.f1() // ok, function type with receiver
3  f1(a) // ok
4
5  a.f2() // compile-time error
6  f2(a) // ok

```

### 17.12.5 Lambda Expressions with Receiver

*Lambda expression with receiver* defines an instance of a *function type with receiver* (see *Function Types with Receiver*). It looks almost the same as an ordinary lambda expression (see *Lambda Expressions*), except that the first parameter is mandatory, and the keyword `this` is used as its name:

```

lambdaExpressionWithReceiver:
  annotationUsage? typeParameters?
  '(' receiverParameter (',' lambdaParameterList)? ')'
  returnType? throwMark? '=>' lambdaBody
  ;

```

The usage of annotations is discussed in *Using Annotations*.

The keyword `this` can be used inside a *lambda expression with receiver*. It corresponds to the first parameter:

```

1  class A { name = "Bob" }
2
3  let show = (this: A): void {
4      console.log(this.name)
5  }

```

The use of lambda is illustrated by the example below:

```

1  class A {
2      name: string
3      constructor (n: string) {
4          this.name = n
5      }
6  }
7
8  function apply(aa: A[], f: (this: A) => void) {
9      for (let a of aa) {
10         a.f()
11     }
12 }
13
14 let aa: A[] = [new A("aa"), new A("bb")]
15 foo(aa, (this: A) => { console.log(this.name)} ) // output: "aa" "bb"

```

**Note.** If *lambda expression with receiver* is declared in a class or interface, then the use of `this` in a lambda body always refers to the first lambda parameter, but not to `this` of the surrounding class or interface.

```

1  class B {
2      foo() { console.log ("foo() from B is called") }
3  }
4  class A {
5      foo() { console.log ("foo() from A is called") }
6      bar() {
7          let lambda = (this: B): void => { this.foo() }
8          new B().lambda()
9      }
10 }
11 new A().bar() // Output is 'foo() from B is called'

```

### 17.12.6 Implicit `this` in Lambda with Receiver Body

Implicit `this` can be used in the body of *lambda expression with receiver* when accessing the following:

- Instance methods, fields, and accessories of lambda receiver type (see *Receiver Type*); or
- Functions with receiver (see *Functions with Receiver*) of the same receiver type.

In other words, prefix `this.` in such cases can be omitted. This feature is added to ArkTS to improve DSL support. It is illustrated by the following examples:



```

1  class C {
2      name: string = ""
3      foo(): void {}
4  }
5
6  function process(context: (this: C) => void) {}
7
8  process(
9      (this: C): void => {
10         this.foo()    // ok - normal call
11         foo()         // ok - implicit 'this'
12         name = "Bob" // ok - implicit 'this'
13     }
14 )

```

The same applies if *lambda expression with receiver* is defined as *trailing lambda* (see [Trailing Lambdas](#)). In this case, lambda signature is inferred from the context:

```

1  process() {
2      this.foo() // ok - normal call
3      foo()     // ok - implicit 'this'
4  }

```

The example above represents the use of implicit `this` when calling a function with receiver:

```

1  function bar(this: C) {}
2  function otherBar(this: OtherClass) {}
3
4  process() {
5      bar() // ok - implicit 'this'
6      otherBar() // compile-time error, wrong type of implicit 'this'
7  }

```

If a simple name used in a lambda body can be resolved as instance method, field or accessor of the receiver type, and as another entity in the current scope at the same time, then a compile-time error occurs to prevent ambiguity and improve readability.

## 17.13 Trailing Lambdas

The *trailing lambda* mechanism allows using a special form of function or method call when the last parameter of a function or a method is of function type, and the argument is passed as a lambda using the `{ }` notation. The *trailing lambda* syntactically looks as follows:

```

1  class A {
2      foo (f: ()=>void) { ... }
3  }
4
5  let a = new A()
6  a.foo() { console.log ("method lambda argument is activated") }
7  // method foo receives last argument as an inline lambda

```

The formal syntax of the *trailing lambda* is presented below:

```
trailingLambdaCall:
  ( objectReference '.' identifier typeArguments?
  | expression ('?.' | typeArguments)?
  )
  arguments block
;
```

Currently, no parameter can be specified for the trailing lambda, except a receiver parameter (see *Lambda Expressions with Receiver*). Otherwise, a compile-time error occurs.

**Note.** If a call is followed by a block, and the function or method being called has no last function type parameter, then such block is handled as an ordinary block of statements but not as a lambda function.

In case of other ambiguities (e.g., when a function or method call has the last parameter, which can be optional, of a function type), a syntax production that starts with ‘{’ following the function or method call is handled as the *trailing lambda*. If other semantics is needed, then the semicolon ‘;’ separator can be used. It means that the function or the method is to be called without the last argument (see *Optional Parameters*).

```
1  class A {
2      foo (p?: ()=>void) { ... }
3  }
4
5  const a = new A()
6
7  a.foo() { console.log ("method lambda argument is activated") }
8  // method 'foo' receives last argument as an inline lambda
9
10 a.foo(); { console.log ("that is the block code") }
11 // method 'foo' is called with 'p' parameter set to 'undefined'
12 // ';' allows to specify explicitly that '{' starts the block
13
14 function bar(f?: ()=>void) { ... }
15
16 bar() { console.log ("function lambda argument is activated") }
17 // function 'bar' receives last argument as an inline lambda,
18 bar(); { console.log ("that is the block code") }
19 // function 'bar' is called with 'f' parameter set to 'undefined'
```

```
1  function foo (f: ()=>void) { ... }
2  function bar (n: number) { ... }
3
4  foo() { console.log ("function lambda argument is activated") }
5  // function foo receives last argument as an inline lambda,
6
7  bar(5) { console.log ("after call of 'bar' this block is executed") }
8
9  foo(() => { console.log ("function lambda argument is activated") })
10 { console.log ("after call of 'foo' this block is executed") }
11 /* here, function foo receives lambda as an argument and a block after
12    the call is just a block, not a trailing lambda. */
```

## 17.14 Enumeration Types Conversions

Every *enum* type is subtype of type `Object` (see *Subtyping*). Every variable of type *enum* can thus be assigned into a variable of type `Object`. The `instanceof` check can be used to get an enumeration variable back by applying the casting conversion as follows:

```

1  enum Commands { Open = "fopen", Close = "fclose" }
2  let c: Commands = Commands.Open
3  let o: Object = c // Autoboxing of enum type to its reference version
4  // Such reference version type has no name, but can be detected by instanceof
5  if (o instanceof Commands) {
6      c = o as Commands // And cast back by cast operator
7  }

```

## 17.15 Enumeration Methods

Several static methods are available to handle each enumeration type as follows:

- Method `values()` returns an array of enumeration constants in the order of declaration.
- Method `getValueOf(name: string)` returns an enumeration constant with the given name, or throws an error if no constant with such name exists.

```

1  enum Color { Red, Green, Blue }
2  let colors = Color.values()
3  //colors[0] is the same as Color.Red
4  let red = Color.valueOf("Red")

```

There are additional methods for instances of any enumeration type:

- Method `valueOf()` returns an `int` or `string` value of an enumeration constant depending on the type of the enumeration constant.
- Method `getName()` returns the name of an enumeration constant.

```

1  enum Color { Red, Green = 10, Blue }
2  let c: Color = Color.Green
3  console.log(c.valueOf()) // prints 10
4  console.log(c.getName()) // prints Green

```

**Note.** Methods `c.toString()` and `c.valueOf().toString()` return the same value.

### 17.15.1 Errors and Initialization Expression

If code of an *initialization expression* of *variable declaration* (see *Variable Declarations*), *constant declaration* (see *Constant Declarations*), or *class field initializer* (see *Field Initialization*) contains unhandled `throw` statement (see *throw Statements*), then the program terminates (see *Program Exit*).

These situations are represented by the examples below:

```
1  const const_decl = foo(true)
2  class A {
3      field_decl = foo(false)
4  }
5
6
7  function foo(toggle: boolean) {
8      if (toggle)
9          throw new Error // No surrounding 'try'
10     return true
11 }
```

## 17.16 Coroutines

A function or lambda can be a *coroutine*. ArkTS supports *basic coroutines*, *structured coroutines*. Basic coroutines are used to create and launch a coroutine; the result is then to be awaited.

### 17.16.1 Create and Launch a Coroutine

The following expression is used to create and launch a coroutine based on a function call, a lambda call (see [Function Call Expression](#)), or a method call (see [Method Call Expression](#)):

```
launchExpression:
    'launch' functionCallExpression | methodCallExpression;
```

```
1  let res = launch cof(10)
2
3  // where 'cof' can be defined as:
4  function cof(a: int): int {
5      let res: int
6      // Do something
7      return res
8  }
```

Lambda can be used in a launch expression as a part of a function call:

```
1  let res = launch ((n: int) => { /* lambda body */ }) (7)
```

The result of the launch expression is of type `Promise<T>`, where `T` is the return type of the function being called:

```
1  function foo(): int { return 0 }
2  function bar() {}
3  let foo_result = launch foo()
4  let bar_result = launch bar()
```

In the example above the type of `foo_result` is `Promise<int>`, and the type of `bar_result` is `Promise<void>`.

Similarly to TypeScript, ArkTS supports the launching of a coroutine by calling the function `async` (see [Async Functions](#)). No restrictions apply as to from what scope to call the function `async`:

```
1  async function foo(): Promise<int> {}
2
3  // This will create and launch coroutine
4  let foo_result = foo()
```

## 17.16.2 Awaiting a Promise

The `await` expressions are used while an `async` function called previously finishes and returns a value:

```
awaitExpression:
  'await' expression
  ;
```

A compile-time error occurs if `await` is called not from the `async` function. A compile-time error occurs if the expression type is not `Promise<T>`.

```
1  let promise = launch (): int { return 1 } ()
2  console.log(await promise) // output: 1
```

If the coroutine result is ignored, then the expression statement `await` is used:

```
1  function foo() { /* do something */ }
2  let promise = launch foo()
3  await promise
```

The `await` never returns `Promise<T>` or union type that contains `Promise<T>`. If the actual type argument of `T` in `Promise<T>` contains `Promise`, then the compiler eliminates any such usage.

Return types of `await` expressions are represented in the example below:

```
1  // if p has type Promise<Promise<string>>,
2  // await p returns string
3  let x : string = await p;
4
5  // if p2 has type Promise<Promise<string> | number>,
6  // await p2 returns string | number
7  let y : string | number = await p2;
8
9  // if p3 has type Promise<string>|Promise<number>,
10 // await p2 returns string | number
11 let z : string | number = await p3;
```

### 17.16.3 Promise<T> Class

The class `Promise<T>` represents the values returned by launch expressions (see [Create and Launch a Coroutine](#)). It belongs to the core packages of the standard library (see [Standard Library](#)), and can be used without any qualification.

The methods are used as follows:

- `then` takes two arguments (the first argument is used as callback if the promise is fulfilled, and the second if it is rejected), and returns `Promise<U>`.

```
Promise<U> Promise<T>::then<U>(fulfillCallback :  
    function  
<T>(val: T) : Promise<U>, rejectCallback : (err: Object)  
: Promise<U>)
```

- `catch` is the alias for `Promise<T>.then<U>((value: T) : U => {}, onRejected)`.

```
Promise<U> Promise<T>::catch<U>(rejectCallback : (err:  
    Object) : Promise<U>)
```

- `finally` takes one argument (the callback called after promise is either fulfilled or rejected) and returns `Promise<T>`.

```
Promise<U> Promise<T>::finally<U>(finallyCallback : (  
    Object:  
T) : Promise<U>)
```

## 17.17 Async Functions and Methods

### 17.17.1 Async Functions

Async functions are implicit coroutines that can be called as regular functions. Async functions can be neither abstract nor native.

The return type of an async function must be `Promise<T>` (see [Promise<T> Class](#)). Returning values of types `Promise<T>` and `T` from async functions is allowed.

Using `return` statement without an expression is allowed if the return type is `Promise<void>`. *No-argument* return statement can be added implicitly as the last statement of the function body if there is no explicit return statement in a function with the return `Promise<void>`.

**Note.** Using type `Promise<void>` is not recommended as this type is supported for the sake of backward TypeScript compatibility only.

### 17.17.2 Experimental Async Methods

The method `async` is an implicit coroutine that can be called as a regular method. Async methods can be neither abstract nor native.

The return type of an `async` method must be `Promise<T>` (see [Promise<T> Class](#)). Returning values of types `Promise<T>` and `T` from `async` methods is allowed.

Using `return` statement without an expression is allowed if the return type is `Promise<void>`. *No-argument* `return` statement can be added implicitly as the last statement of the methods body if there is no explicit `return` statement in a method with return type `Promise<void>`.

**Note.** Using this annotation is not recommended as this type of methods is supported for the sake of backward TypeScript compatibility only.

## 17.18 DynamicObject Type

The interface `DynamicObject` is used to provide seamless interoperability with dynamic languages (e.g., JavaScript and TypeScript). This interface (defined in [Standard Library](#)) is common for a set of wrappers (also defined in [Standard Library](#)) that provide access to underlying objects.

An instance of `DynamicObject` cannot be created directly. Only an instance of a specific wrapper object can be instantiated.

`DynamicObject` is a predefined type. The following operations applied to an object of type `DynamicObject` are handled by the compiler in a special manner:

- Field access;
- Method call;
- Indexing access;
- New; and
- Cast.

### 17.18.1 DynamicObject Field Access

The field access expression `D.F`, where `D` is of type `DynamicObject`, is handled as an access to a property of an underlying object.

If the value of a field access is used, then it is wrapped in the instance of `DynamicObject`, since the actual type of the field is not known at compile time.

```

1 function foo(d: DynamicObject) {
2     console.log(d.f1) // access of the property named "f1" of underlying object
3     d.f1 = 5 // set a value of the property named "f1"
4     let y = d.f1 // 'y' is of type DynamicObject
5 }

```

The wrapper can raise an error if:

- No property with the specified name exists in the underlying object; or

- Field access is in the right-hand-side part of the assignment, and the type of the assigned value is not assignable to the type of the property (see [Assignability](#)).

### 17.18.2 `DynamicObject` Method Call

The method call expression  $D.F(arguments)$ , where  $D$  is of type `DynamicObject`, is handled as a call of the instance method of an underlying object.

If the result of a method call is used, then it is wrapped in the instance of `DynamicObject`, since the actual type of the returned value is not known at compile time.

```
1 function foo(d: DynamicObject) {  
2   d.foo() // call of a method "foo" of underlying object  
3   let y = d.foo() // 'y' is of type DynamicObject  
4 }
```

The wrapper must raise an error if:

- No method with the specified name exists in the underlying object; or
- Signature of the method is not compatible with the types of call arguments.

### 17.18.3 `DynamicObject` Indexing Access

The indexing access expression  $D[index]$ , where  $D$  is of type `DynamicObject`, is handled as an indexing access to an underlying object.

```
1 function foo(d: DynamicObject) {  
2   let x = d[0]  
3 }
```

The wrapper must raise an error if:

- Indexing access is not supported by the underlying object;
- Type of the *index* expression is not supported by the underlying object.

### 17.18.4 `DynamicObject` New Expression

The new expression  $new D(arguments)$  (see [New Expressions](#)), where  $D$  is of type `DynamicObject`, is handled as a new expression (constructor call) applied to the underlying object.

The result of the expression is wrapped in an instance of `DynamicObject`, as the actual type of the returned value is not known at compile time.



```

1 function foo(d: DynamicObject) {
2     let x = new d()
3 }

```

The wrapper must raise an error if:

- New expression is not supported by the underlying object; or
- Signature of the constructor of the underlying object is not compatible with the types of call arguments.

### 17.18.5 DynamicObject Cast Expression

The cast expression *D as T* (see *Cast Expressions*), where *D* is of type `DynamicObject`, is handled as an attempt to cast the underlying object to a static type *T*.

A compile-time error occurs if *T* is not a class or interface type.

The result of a cast expression is an instance of type *T*.

```

1 interface I {
2     bar()
3 }
4
5 function foo(d: DynamicObject) {
6     let x = d as I
7     x.bar() // a call of interface method (not dynamic)
8 }

```

The wrapper must raise an error if an underlying object cannot be cast to the target type specified by the cast expression.

## 17.19 Packages

One or more *package modules* form a package:

```

packageDeclaration:
    packageModule+
;

```

*Packages* are stored in a file system or a database (see *Compilation Units in Host System*).

A *package* can consist of several package modules if all such modules have the same *package header*:

```

packageModule:
    packageHeader packageModuleDeclaration
;

packageHeader:
    'package' qualifiedName

```

(continues on next page)

(continued from previous page)

```

;

packageModuleDeclaration:
    importDirective* packageTopDeclaration*
;

packageTopDeclaration:
    topDeclaration | initializerBlock
;

```

A compile-time error occurs if:

- *Package module* contains no package header; or
- Package headers of two package modules in the same package have different identifiers.

Every *package module* can directly use all exported entities from the core packages of the standard library (see [Standard Library Usage](#)).

A *package module* can directly access all top-level entities declared in all modules that constitute the package.

If a top declaration in any package module contains an initializer for a variable or a constant, then the form of the initializer must be *constantExpression*. Otherwise, a compile-time error occurs. Initializer block is to be used for initialization to ensure an explicit order of initialization.

```

1 package P
2   let v1 = foo() // Compile-time error as call to foo() is not a constant expression
3   function foo() { return 1 }
4
5   let v2 = 2 + 3 * 4 // OK
6
7   let v2: number
8   static {
9     v2 = foo() // OK
10  }

```

### 17.19.1 Internal Access Modifier

The modifier `internal` indicates that a class member, a constructor, or an interface member is accessible (see [Accessible](#)) within its compilation unit only. If the compilation unit is a package (see [Packages](#)), then `internal` members can be used in any *package module*. If the compilation unit is a separate module (see [Separate Modules](#)), then `internal` members can be used within this module.

```

1 class C {
2   internal count: int
3   getCount(): int {
4     return this.count // ok
5   }
6 }
7
8 function increment(c: C) {
9   c.count++ // ok
10 }

```

### 17.19.2 Package Initializer

Among all *package modules* there can be one to contain a code that performs initialization actions (e.g., setting initial values for variables across all package modules) as described in detail in [Compilation Unit Initialization](#) and in [Initializer Block](#).

A compile-time error occurs if a package contains *package initializer* in more than one source file.

```

1  // Source file 1
2  package P
3      static { // P initializer part one
4      }
5      function foo() {}
6      static { // P initializer part two
7      }
8
9
10 // Source file 2
11 package P
12     static {} // compile-time error as initializer in a different source file

```

### 17.19.3 Import and Overloading of Function Names

While importing functions, the following situations can occur:

- Different imported functions have the same name but different signatures, or a function (functions) of the current module and an imported function (functions) have the same name but different signatures. This situation is called *overloading*. All such functions are accessible (see [Accessible](#)).
- A function (functions) of the current module and an imported function (functions) have the same name and an overload-equivalent signature (see [Overload-Equivalent Signatures](#)). This situation causes a compile-time error as declarations are duplicated. Qualified import or alias in import can be used to access the imported entity.

The two situations are illustrated by the examples below:

```

1  // Overloading case
2  package P1
3      function foo(p: int) {}
4
5  package P2
6      function foo(p: string) {}
7
8  // Main module
9  import {foo} from "path_to_file_with_P1"
10 import {foo} from "path_to_file_with_P2"
11
12 function foo (p: double) {}
13
14 function main() {
15     foo(5) // Call to P1.foo(int)

```

(continues on next page)

(continued from previous page)

```

16     foo("A string") // Call to P2.foo(string)
17     foo(3.141592653589) // Call to local foo(double)
18 }
19
20 // Declaration duplication case
21 package P1
22     function foo() {}
23 package P2
24     function foo() {}
25 // Main program
26 import {foo} from "path_to_file_with_P1"
27 import {foo} from "path_to_file_with_P2" /* Error: duplicating
28     declarations imported*/
29 function foo() {} /* Error: duplicating declaration identified
30     */
31 function main() {
32     foo() // Error: ambiguous function call
33     // But not a call to local foo()
34     // foo() from P1 and foo() from P2 are not accessible
35 }

```

## 17.20 Generics Experimental

### 17.20.1 NonNullish Type Parameter

If some generic class has a type parameter with a nullish union type constraint, then special syntax can be used for type annotation to get a non-nullish version of the type parameter variable. The example below illustrates this possibility:

```

1     class A<T> { // in fact it extends Object|null|undefined
2         foo (p: T): T! { // foo returns non-nullish value of p
3             return p!
4         }
5     }
6
7     class B<T extends SomeType | null> {
8         foo (p: T): T! { // foo returns non-nullish value of p
9             return p!
10        }
11    }
12
13    class C<T extends SomeType | undefined> {
14        foo (p: T): T! { // foo returns non-nullish value of p
15            return p!
16        }
17    }
18
19    let a = new A<Object>

```

(continues on next page)

(continued from previous page)

```
20 let b = new B<SomeType>
21 let c = new C<SomeType>
22
23 let result: Object = new Object // Type of result is non-nullish !
24 result = a.foo(result)
25 result = b.foo(new SomeType)
26 result = c.foo(new SomeType)
27
28 // All such assignments are type-valid as well
29 result = a.foo(void) // void! => never
30 result = b.foo(null) // null! => never
31 result = c.foo(undefined) // undefined! => never
```



---

## Annotations

---

*Annotation* is a special language element that changes the semantics of the declaration to which it is applied by adding metadata.

The example below illustrates how an annotation is declared and used:

```
1 // Annotation declaration:
2 @interface ClassAuthor {
3     authorName: string
4 }
5
6 // Annotation use:
7 @ClassAuthor({authorName: "Bob"})
8 class MyClass { /*body*/ }
```

The annotation *ClassAuthor* in the example above adds metadata to the class declaration.

An annotation must be placed immediately before the declaration to which it is applied. An annotation can include arguments as in the example above.

For an annotation to be used, the name of the annotation must be prefixed with the character '@' (e.g., @MyAnno). No spaces and line separators are allowed between the character '@' and the name:

A compile-time error occurs if the annotation name is not accessible at the place of usage. An annotation declaration can be exported and used in other compilation units.

Multiple annotations can be applied to a single declaration:

```
1 @MyAnno ()
2 @ClassAuthor({authorName: "John Smith"})
3 class MyClass { /*body*/ }
```

## 18.1 Declaring Annotations

Declaring an *annotation* is similar to declaring an interface where the keyword `interface` is prefixed with the character '@':

```
annotationDeclaration:
    '@interface' identifier '{' annotationField* '}'
    ;
annotationField:
    identifier ':' type constInitializer?
    ;
constInitializer:
    '=' constantExpression
    ;
```

As any other declared entity, an annotation can be exported by using the keyword `export`.

*Type* in the annotation field is restricted (see *Types of Annotation Fields*).

The default value of an *annotation field* can be specified by using *initializer* as *constant expression*. A compile-time error occurs if the value of this expression cannot be evaluated at compile time.

*Annotation* must be defined at the top level. Otherwise, a compile-time error occurs.

*Annotation* cannot be extended as inheritance is not supported.

The name of an *annotation* cannot coincide with the name of another entity:

```
1  @interface Position { /*properties*/ }
2
3  class Position { /*body*/ } // compile-time error: duplicate identifier
```

An annotation declaration does not define a type, and a type alias can be neither applied to the annotation, nor used as an interface:

```
1  @interface Position {}
2  type Pos = Position // compile-time error
3
4  class A implements Position {} // compile-time error
```

### 18.1.1 Types of Annotation Fields

The choice of *types for annotation fields* is limited to the following:

- *Numeric Types*;
- Type `boolean` (see *Boolean Types and Operations*);
- *Type string*;
- Enumeration types (see *Enumerations*);
- Array of the above types (e.g., `string[]`), including multidimensional arrays (e.g., `string[][]`).

A compile-time error occurs if any other type is used as the type of an *annotation field*.



## 18.2 Using Annotations

The following syntax is used to apply an annotation to a declaration, and to define the values of annotation fields:

```
annotationUsage:
    '@' qualifiedName annotationValues?
    ;
annotationValues:
    '(' (objectLiteral | constantExpression)? ')'
    ;
```

An annotation declaration is presented in the example below:

```
1  @interface ClassPreamble {
2      authorName: string
3      revision: number = 1
4  }
5  @interface MyAnno{}
```

In general, annotation field values are set by an *object literal*. In a special case, annotation field values are set by using an expression (see [Using Single Field Annotations](#)).

All values in an *object literal* must be constant expressions. Otherwise, a compile-time error occurs.

The usage of annotation is presented in the example below. The annotations in this example are applied to class declarations:

```
1  @ClassPreamble({authorName: "John", revision: 2})
2  class C1 { /*body*/ }
3
4  @ClassPreamble({authorName: "Bob"}) // default value for revision = 1
5  class C2 { /*body*/ }
6
7  @MyAnno ()
8  class C3 { /*body*/ }
```

Annotations can be applied to the following:

- *Top-Level Declarations*, except abstract class declarations;
- Class members (see *Class Body*) or interface members (see *Interface Body*), except members of abstract class declarations;
- Type usage (see *Using Types*);
- Parameters (see *Parameter List* and *Optional Parameters*);
- Lambda expression (see *Lambda Expressions* and *Lambda Expressions with Receiver*);
- *Local Declarations*.

Otherwise, a compile-time error occurs:

```
1  @MyAnno ()
2  abstract class A {} // compile-time error
```

Repeatable annotations are not supported, i.e., an annotation cannot be applied to an entity more than once:

```

1 @ClassPreamble({authorName: "John"})
2 @ClassPreamble({authorName: "Bob"}) // compile-time error
3 class C { /*body*/ }

```

When using an annotation, the order of values has no significance:

```

1 @ClassPreamble({authorName: "John", revision: 2})
2 // the same as:
3 @ClassPreamble({revision: 2, authorName: "John"})

```

When using an annotation, all fields without default values must be listed. Otherwise, a compile-time error occurs:

```

1 @ClassPreamble() // compile-time error, authorName is not defined
2 class C1 { /*body*/ }

```

If a field of an array type for an annotation is defined, then its value is set by using the array literal syntax:

```

1 @interface ClassPreamble {
2     authorName: string
3     revision: number = 1
4     reviewers: string[]
5 }
6
7 @ClassPreamble(
8     {authorName: "Alice",
9     reviewers: ["Bob", "Clara"]}
10 )
11 class C3 { /*body*/ }

```

If setting annotation properties is not required, then parentheses can be omitted after the annotation name:

```

1 @MyAnno
2 class C4 { /*body*/ }

```

## 18.2.1 Using Single Field Annotations

If annotation declaration defines only one field, then it can be used with a short notation to specify just one expression instead of an object literal:

```

1 @interface deprecated{
2     fromVersion: string
3 }
4
5 @deprecated("5.18")
6 function foo() {}
7
8 @deprecated({fromVersion: "5.18"})
9 function goo() {}

```

A short notation and a notation with an object literal behave in exactly the same manner.

## 18.3 Exporting and Importing Annotations

An annotation can be exported and imported. However, a few forms of export and import directives are supported.

To export an annotation, the annotation declaration must be marked with the keyword `export` as follows:

```
1 // a.sts
2 export @interface MyAnno {}
```

If an annotation is imported as a part of an imported module, then the annotation is accessed by its qualified name:

```
1 // b.sts
2 import * as ns from "./a"
3
4 @ns.MyAnno
5 class C { /*body*/ }
```

Unqualified import is also allowed:

```
1 // b.sts
2 import { MyAnno } from "./a"
3
4 @MyAnno
5 class C { /*body*/ }
```

An annotation is not a type. Using `export type` or `import type` notations to export or import annotations is forbidden:

```
1 import type { MyAnno } from "./a" // compile-time error
```

Annotations are forbidden in the following cases:

- Export default,
- Import default,
- Rename in export, and
- Rename in import.

```
1 import {MyAnno as Anno} from "./a" // compile-time error
```

## 18.4 Ambient Annotations

*Ambient annotations* can be specified in *Declaration Modules* only.

```
ambientAnnotationDeclaration:
  'declare' annotationDeclaration
  ;
```

Such a declaration does not introduce a new annotation but provides type information that is required to use the annotation. The annotation itself must be defined elsewhere. A runtime error occurs if no annotation corresponds to the ambient annotation used in the program.

An ambient annotation and the annotation that implements it must be exactly identical, including field initialization:

```
1 // a.d.sts
2 export declare @interface NameAnno{name: string = ""}
3
4 // a.sts
5 export @interface NameAnno{name: string = ""} // ok
```

The code in the example below is incorrect because the ambient declaration is not identical to the annotation declaration:

```
1 // a.d.sts
2 export declare @interface VersionAnno{version: number} // initialization is missing
3
4 // a.sts
5 export @interface VersionAnno{version: number = 1}
```

An ambient declaration can be imported and used in exactly the same manner as a regular annotation:

```
1 // a.d.sts
2 export declare @interface MyAnno {}
3
4 // b.sts
5 import { MyAnno } from "../a"
6
7 @MyAnno
8 class C { /*body*/}
```

If an annotation is applied to an ambient declaration in the *.d.sts* file (see the example below), then the annotation is to be applied to the implementation declaration manually, because the annotation is not automatically applied to the declaration that implements the ambient declaration:

```
1 // a.d.sts
2 export declare @interface MyAnno {}
3
4 @MyAnno
5 declare class C {}
```

## 18.5 Standard Annotations

*Standard annotation* is an annotation that is defined in *Standard Library*, or implicitly defined in the compiler (*built-in annotation*). *Standard annotation* is usually known to the compiler. It modifies the semantics of the declaration it is applied to.

### 18.5.1 Retention Annotation

`@Retention` is a standard annotation that is used to annotate a declaration of another annotation. A compile-time error occurs if it is used in other places.

The annotation has a single field `policy` of type `string`. It is typically used as follows:

```
1 @Retention({policy: "RUNTIME"})
2 @interface MyAnno {} // this annotation uses "RUNTIME" policy
3
4 @MyAnno //
5 class C {}
```

The value of this field determines at which point an annotation is used, and discarded after use. The retention policies can be of three types:

- “SOURCE”

Annotations that use “SOURCE” policy are processed at compile time, and are discarded after compilation;

- “BYTECODE”

Metadata specified in annotations that use “BYTECODE” policy are saved into the bytecode file, but are discarded at runtime.

- “RUNTIME”

Metadata specified in annotations that use “RUNTIME” policy are saved into the bytecode file, are retained and can be accessed at runtime.

The default retention policy is “BYTECODE”.

A compile-time error occurs if any other string literal is used as the value of `policy` field.

As `@Retention` has a single field, it can be used with a short notation (see [Using Single Field Annotations](#)) as follows:

```
1 @Retention("SOURCE")
2 @interface Author {name: string} // this annotation uses "SOURCE" policy
```



## CHAPTER 19

---

### Standard Library

---

The Standard Library of the ArkTS language defines the required set of types, variables, constants, functions, and annotations that provide APIs for effective and convenient programming.

The Standard Library has two parts: the common part provides TypeScript compatibility, and the ArkTS-specific part adds more advanced features.

A detailed description of all elements of the standard library is covered in a separate document that is part of the ArkTS distribution package.





---

## Implementation Details

---

Important implementation details are discussed in this section.

### 20.1 Import Path Lookup

If an import path `<some path>/name` is resolved to a path in the folder *'name'*, then the compiler executes the following lookup sequence:

- If the folder contains the file `index.sts`, then this file is imported as a separate module written in ArkTS;
- If the folder contains the file `index.ts`, then this file is imported as a separated module written in TypeScript;
- Otherwise, the compiler imports the package constituted by files `name/*.sts`.

### 20.2 Compilation Units in Host System

Modules and packages are created and stored in a manner determined by the host system. The exact manner modules and packages are stored in a file system is determined by a particular implementation of the compiler and other tools.

As an example, a simple implementation is characterised by the following:

- Module (package module) is stored in a single file.
- All package modules are stored in files of the same folder.
- Folder that can store several separate modules (one source file to contain a separate module or a package module).
- Folder that stores a single package must not contain separate module files or package modules from other packages.

## 20.3 How to Get Type Via Reflection

The ArkTS standard library (see [Standard Library](#)) provides a pseudo generic static method `Type.for<T>()` to be processed by the compiler in a specific way during compilation. A call to this method allows getting a variable of type `Type` that represents type `T` at runtime. Type `T` can be any valid type.

```
1 let type_of_int: Type = Type.for<int>()
2 let type_of_string: Type = Type.for<String>()
3 let type_of_array_of_int: Type = Type.for<int[]>()
4 let type_of_number: Type = Type.for<number>()
5 let type_of_Object: Type = Type.for<Object>()
6
7 class UserClass {}
8 let type_of_user_class: Type = Type.for<UserClass>()
9
10 interface SomeInterface {}
11 let type_of_interface: Type = Type.for<SomeInterface>()
```

## 20.4 Methods for `T[]` Types

Some methods defined for `Array<T>` can be used for `T[]` (e.g., `at`). It does not mean that `T[]` is a class type, but rather that the compiler supports the syntactical form of the method call for the `T[]` variables. The list of supported methods is defined by the compiler implementation.

```
1 let built_in_array: number[] = [1,2,3]
2 built_in_array.at (0) // That will be a valid call
```

## 20.5 Generic and Function Types Peculiarities

Current compiler and runtime implementations use type erasure, and thus affect the manner of behavior of generics and function types. It is expected to change in the future. The compiler applies boxing (see [Boxing Conversions](#)) to any parameter and return type of primitive types when dealing with variables of function types. A particular example can be found under the last bullet of the compile-time errors list in [InstanceOf Expression](#).

## 20.6 Keyword `struct` and ArkUI

The current compiler reserves the keyword `struct` because it is used in legacy ArkUI code. This keyword can be used as a replacement for the keyword `class` in *Class Declarations*. Class declarations marked with the keyword `struct` are processed by the ArkUI plugin and replaced with class declarations that use specific ArkUI types.



---

## ArkTS-TypeScript compatibility

---

This section discusses all issues related to different aspects of compatibility between ArkTS and TypeScript.

### 21.1 Reserved Names of TypeScript Types

The following tables list words that are reserved and cannot be used as user-defined type names, but are not otherwise restricted.

1. The following names of TypeScript utility types are not supported by ArkTS:

|                       |                   |                   |
|-----------------------|-------------------|-------------------|
|                       |                   |                   |
| Awaited               | NoInfer           | Pick              |
| ConstructorParameters | NonNullable       | ReturnType        |
| Exclude               | Omit              | ThisParameterType |
| Extract               | OmitThisParameter | ThisType          |
| InstanceType          | Parameters        |                   |

2. The following names of TypeScript utility string types are not supported by ArkTS:

|            |              |
|------------|--------------|
|            |              |
| Capitalize | Uncapitalize |
| Lowercase  | Uppercase    |

3. The following class names from TypeScript standard library that are not supported by ArkTS standard library:

|                        |                     |                         |
|------------------------|---------------------|-------------------------|
|                        |                     |                         |
| ArrayBufferTypes       | Function            | Proxy                   |
| AsyncGenerator         | Generator           | ProxyHandler            |
| AsyncGeneratorFunction | GeneratorFunction   | Symbol                  |
| AsyncIterable          | IArguments          | TemplateStringsArray    |
| AsyncIterableIterator  | IteratorYieldResult | TypedPropertyDescriptor |
| AsyncIterator          | NewableFunction     |                         |
| CallableFunction       | PropertyDescriptor  |                         |

## 21.2 Undefined is Not a Universal Value

ArkTS raises a compile-time or a runtime error in many cases, in which TypeScript uses undefined as runtime value.

```
1  let array = new Array<number>
2  let x = array [1234]
3      // TypeScript: x will be assigned with undefined value !!!
4      // ArkTS: compile-time error if analysis may detect array out of bounds
5      //          violation or runtime error ArrayOutOfBounds
6  console.log(x)
```

## 21.3 Numeric Semantics

TypeScript has a single numeric type `number` that handles both integer and real numbers.

ArkTS interprets `number` as a variety of ArkTS types. Calculations depend on the context and can produce different results:

```
1  let n = 1
2      // TypeScript: treats 'n' as having type number
3      // ArkTS: treats 'n' as having type int to reach max code performance
4
5  console.log(n / 2)
6      // TypeScript: will print 0.5 - floating-point division is used
7      // ArkTS: will print 0 - integer division is used
```

## 21.4 Covariant Overriding

TypeScript object runtime model enables TypeScript to handle situations where a non-existing property is accessed from an object during program execution.

ArkTS allows generating highly efficient code that relies on an objects' layout known at compile time. Covariant overriding (see *Invariance, Covariance and Contravariance*) is prohibited because it violates type-safety:

```

1  class Base {
2      foo (p: Base) {}
3  }
4  class Derived extends Base {
5      override foo (p: Derived)
6          // ArkTS will issue a compile-time error - incorrect overriding
7      {
8          console.log ("p.field unassigned = ", p.field)
9          // TypeScript will print 'p.field unassigned = undefined'
10         p.field = 666 // Access the field
11         console.log ("p.field assigned = ", p.field)
12         // TypeScript will print 'p.field assigned = 666'
13         p.method() // Call the method
14         // TypeScript will generate runtime error: p.method is not a function
15     }
16     method () {}
17     field: number = 0
18 }
19
20 let base: Base = new Derived
21 base.foo (new Base)

```

## 21.5 Function Types Compatibility

TypeScript allows more relaxed assignments into variables of function type. ArkTS follows stricter rules as stated in *Function Types Conversions*.

```

1  type FuncType = (p: string) => void
2  let f1: FuncType = (p: string): number => { return 0 } // compile-time error in ArkTS
3  let f1: FuncType = (p: string): string => { return "" } // compile-time error in ArkTS
↪ArkTS

```

## 21.6 Compatibility for Utility Types

Utility type `Partial<T>` in ArkTS is not assignable to `T` (see *Assignability*). Variables of this type are to be initialized with object literals only.

```
1 function foo<T>(t: T, part_t: Partial<T>) {  
2     part_t = t // compile-time error in ArkTS  
3 }
```

## 21.7 TypeScript Overload Signatures

ArkTS does not support overload signatures in TypeScript-style where several function headers are followed by a single body. Each overloaded function, method, or constructor is required to have a separate body.

The following code is valid in TypeScript but causes a compile-time error in ArkTS:

```
1 function foo(): void  
2 function foo(x: string): void  
3 function foo(x?: string): void {  
4     /*body*/  
5 }
```

The following code is valid in ArkTS (see *Function, Method and Constructor Overloading*):

```
1 function foo(): void {  
2     /*body1*/  
3 }  
4 function foo(x: string): void {  
5     /*body2*/  
6 }
```

## 21.8 Class Fields While Inheriting

TypeScript allows overriding a class field with a field in a subclass of invariant or covariant type. ArkTS supports overriding a class field with a field in a subclass of invariant type only. In both languages, an overriding field can have a new initial value.

The situations are illustrated by the following examples:

```
1 // Both TypeScript and ArkTS do the same  
2 class Base {  
3     field: number = 123  
4     foo () {  
5         console.log (this.field)  
6     }  
7 }  
8 class Derived extends Base {  
9     field: number = 456  
10    foo () {  
11        console.log (this.field)
```

(continues on next page)



(continued from previous page)

```

12     }
13 }
14 let b: Base = new Derived()
15 b.foo() // 456 is printed
16
17
18 // That will be a compile-time error in ArkTS as type of 'field' in Child
19 // differs from 'field' type in Parent
20 class Parent {
21     field: Object
22 }
23 class Child extends Parent {
24     field: Number
25 }

```

## 21.9 Overriding for Primitive Types

TypeScript allows overriding class type version of a primitive type into a pure primitive type. ArkTS does not allow such overriding. This situation is represented by the example below:

```

1 class Base {
2     foo(): Number { return 5 }
3 }
4 class Derived extends Base {
5     foo(): number { return 5 } // Such overriding is prohibited
6 }

```

## 21.10 Type void Compatibility

TypeScript allows using type void in union types. ArkTS does not allow void in union types. This situation is represented by the example below:

```

1 type UnionWithVoid = void | number
2 // Such type is OK for Typescript, but leads to a compile-time error for ArkTS

```

## 21.11 Invariant Array Assignment

TypeScript allows covariant array assignment. ArkTS allows invariant array assignment only:

```
1 // Typescript
2 let a: Object[] = [1, 2, 3]
3 let b = [1, 2, 3] // type of 'b' is inferred as number[]
4 a = b // That works well for the Typescript
5
6 // ArkTS
7 let a: Object[] = [1, 2, 3]
8 let b = [1, 2, 3] // type of 'b' is inferred as double[]
9 a = b // compile-time error
10
11 let a: Object[] = ["a", "b", "c"]
12 let b: string[] = ["a", "b", "c"]
13 a = b // compile-time error
```

## 21.12 Tuples and Arrays

TypeScript allows assignments of tuples into arrays. ArkTS handles arrays and tuples as different types, and does not allow assignment of tuples into arrays. This situation is represented by the following example:

```
1 const tuple: [number, number, boolean] = [1, 3.14, true]
2
3 // Typescript accepts such assignment while ArkTS reports an error
4 const array: (number|boolean) [] = tuple
```

## 21.13 Extending Class Object

TypeScript forbids using `super` and `override` if class `Object` is not listed explicitly in the `extends` clause of a class. ArkTS allows this as `Object` is a superclass for any class without an explicit `extends` clause:

```
1 // Typescript reports an error while ArkTS compiles with no issues
2 class A {
3     override toString() { // compile-time error
4         return super.toString() // compile-time error
5     }
6 }
7
8 class A extends Object { // That is the form supported by TypeScript
9     override toString() {
10         return super.toString()
11     }
12 }
```

## 21.14 Syntax of `extends` and `implements` Clauses

TypeScript handles entities listed in `extends` and `implements` clauses as expressions. ArkTS handles such clauses at compile time, and allows no expressions but *type references*:

```
1  class B {}
2  class A extends (B) {} // compile-time error for ArkTS while accepted by TypeScript
```

## 21.15 Uniqueness of Functional Objects

TypeScript and ArkTS handle function objects differently, and the equality test can perform differently. The difference can be eliminated in the future versions of ArkTS.

```
1  function foo() {}
2  foo == foo // true in Typescript while may be false in ArkTS
3  const f1 = foo
4  const f2 = foo
5  f1 == f2 // true in Typescript while may be false in ArkTS
```

## 21.16 Functional Objects for Methods

TypeScript allows creating function objects for methods. ArkTS does not allow creating function objects for methods. This difference can be eliminated in the future versions of ArkTS.

```
1  class C {
2      method() {
3          let method_ref = this.method // compile-time error in ArkTS
4      }
5  }
6
7  interface I { method(): void }
8  function bar (i: I) {
9      let method_ref = i.method // compile-time error in ArkTS
10 }
```

## 21.17 Differences in Namespaces

TypeScript allows having non-exported entities with the same name in two or more different declarations of a namespace, because these entities are local to a particular declaration of the namespace. Such situations are forbidden in

ArkTS, because this language merges all declarations into one:

```
1  // Typescript accepts such code, while ArkTS will report a compile-time error
2  namespace A {
3      function foo() { console.log ("foo() from the 1st namespace A declaration") }
4      export bar () { foo() }
5  }
6  namespace A {
7      function foo() { console.log ("foo() from the 2nd namespace A declaration") }
8      export bar_bar() { foo() }
9  }
10 A.bar()
11 A.bar_bar()
```

## 21.18 Differences in Math.pow

The function `Math.pow` in ArkTS conforms to the latest IEEE 754-2019 standard. The following calls produce the result *1* (one):

- `Math.pow(1, Infinity)`,
- `Math.pow(-1, Infinity)`,
- `Math.pow(1, -Infinity)`,
- `Math.pow(-1, -Infinity)`.

The function `Math.pow` in TypeScript conforms to the outdated 2008 version of the IEEE 754-2019 standard. The same calls as listed above produce NaN in TypeScript.

```
literal: Literal;

identifier: Identifier;

indexType: 'number';

type:
    annotationUsage?
    ( typeReference
    | 'readonly'? arrayType
    | 'readonly'? tupleType
    | functionType
    | functionTypeWithReceiver
    | unionType
    | StringLiteral
    )
    | '(' type ')'
    ;

typeReference:
    typeReferencePart ('.' typeReferencePart)*
    | identifier '!'
    ;

typeReferencePart:
    identifier typeArguments?
    ;

arrayType:
    type '[' ']'
    ;

functionType:
    '(' ftParameterList? ')' ftReturnType
```

(continues on next page)

(continued from previous page)

```

/

ftParameterList:
    ftParameter (',' ftParameter)* (',' ftRestParameter)?
    | ftRestParameter
/

ftParameter:
    identifier ('?')? ':' type
/

ftRestParameter:
    '...' ftParameter
/

ftReturnType:
    '=>' type
/

tupleType:
    '[' (type (',' type)* ',' '?')? ']'
/

unionType:
    type ('|' type)*
/

qualifiedName:
    identifier ('.' identifier)*
/

typeDeclaration:
    classDeclaration
    | interfaceDeclaration
    | enumDeclaration
    | typeAlias
/

typeAlias:
    'type' identifier typeParameters? '=' type
/

variableDeclarations:
    'let' variableDeclarationList
/

variableDeclarationList:
    variableDeclaration (',' variableDeclaration)*
/

variableDeclaration:
    identifier ('?')? ':' type initializer?
    | identifier initializer
/

initializer:
    '=' expression

```

(continues on next page)

(continued from previous page)

```

    /

constantDeclarations:
    'const' constantDeclarationList
    /

constantDeclarationList:
    constantDeclaration (',' constantDeclaration)*
    /

constantDeclaration:
    identifier (':' type)? initializer
    /

functionDeclaration:
    modifiers? 'function' identifier
    typeParameters? signature block?
    /

modifiers:
    'native' | 'async'
    /

signature:
    '(' parameterList? ')' returnType?
    /

returnType:
    ':' type
    /

parameterList:
    parameter (',' parameter)* (',' restParameter)? ','?
    | restParameter ','?
    /

parameter:
    annotationUsage? (requiredParameter | optionalParameter)
    /

requiredParameter:
    identifier ':' type
    /

optionalParameter:
    identifier ':' type '=' expression
    | identifier '?' ':' type
    /

restParameter:
    annotationUsage? '...' identifier ':' type
    /

typeParameters:
    '<' typeParameterList '>'
    /

```

(continues on next page)

(continued from previous page)

```

typeParameterList:
    typeParameter (',' typeParameter)*
    ;

typeParameter:
    ('in' | 'out')? identifier constraint? typeParameterDefault?
    ;

constraint:
    'extends' type
    ;

typeParameterDefault:
    '=' typeReference ( '[' ']' )?
    ;

typeArguments:
    '<' type (',' type)* '>'
    ;

expression:
    primaryExpression
    | castExpression
    | instanceofExpression
    | typeofExpression
    | nullishCoalescingExpression
    | spreadExpression
    | unaryExpression
    | binaryExpression
    | assignmentExpression
    | conditionalExpression
    | stringInterpolation
    | lambdaExpression
    | lambdaExpressionWithReceiver
    | launchExpression
    | awaitExpression
    ;

primaryExpression:
    literal
    | namedReference
    | arrayLiteral
    | objectLiteral
    | recordLiteral
    | thisExpression
    | parenthesizedExpression
    | methodCallExpression
    | fieldAccessExpression
    | indexingExpression
    | functionCallExpression
    | newExpression
    | ensureNotNullishExpression
    ;

binaryExpression:
    multiplicativeExpression
    | additiveExpression

```

(continues on next page)



(continued from previous page)

```

    | shiftExpression
    | relationalExpression
    | equalityExpression
    | bitwiseAndLogicalExpression
    | conditionalAndExpression
    | conditionalOrExpression
    ;

objectReference:
    typeReference
    | 'super'
    | primaryExpression
    ;

arguments:
    '(' argumentSequence? ')'
    ;

argumentSequence:
    restArgument
    | expression (',' expression)* (',' restArgument)? ','?
    ;

restArgument:
    '...'? expression
    ;

namedReference:
    qualifiedName typeArguments?
    ;

arrayLiteral:
    '[' expressionSequence? ']'
    ;

expressionSequence:
    expression (',' expression)* ','?
    ;

objectLiteral:
    '{' valueSequence? '}'
    ;

valueSequence:
    nameValue (',' nameValue)* ','?
    ;

nameValue:
    identifier ':' expression
    ;

recordLiteral:
    '{' keyValueSequence? '}'
    ;

keyValueSequence:
    keyValue (',' keyValue)* ','?

```

(continues on next page)

(continued from previous page)

```
    /

keyValue:
    expression ':' expression
    /

spreadExpression:
    '...' expression
    /

parenthesizedExpression:
    '(' expression ')'
    /

thisExpression:
    'this'
    /

fieldAccessExpression:
    objectReference ('.' | '?.') identifier
    /

methodCallExpression:
    objectReference ('.' | '?.') identifier typeArguments? arguments block?
    /

functionCallExpression:
    expression ('?.' | typeArguments)? arguments block?
    /

indexingExpression:
    expression ('?.')? '[' expression ']'
    /

newExpression:
    newClassInstance
    | newArrayInstance
    /

newClassInstance:
    'new' typeArguments? typeReference arguments?
    /

castExpression:
    expression 'as' type
    /

instanceOfExpression:
    expression 'instanceof' type
    /

typeofExpression:
    'typeof' expression
    /

ensureNotNullishExpression:
    expression '!'
```

(continues on next page)

(continued from previous page)

```

/

nullishCoalescingExpression:
    expression '??' expression
/

unaryExpression:
    expression '++'
    | expression '--'
    | '++' expression
    | '--' expression
    | '+' expression
    | '-' expression
    | '~' expression
    | '!' expression
/

multiplicativeExpression:
    expression '*' expression
    | expression '/' expression
    | expression '%' expression
/

additiveExpression:
    expression '+' expression
    | expression '-' expression
/

shiftExpression:
    expression '<<' expression
    | expression '>>' expression
    | expression '>>>' expression
/

relationalExpression:
    expression '<' expression
    | expression '>' expression
    | expression '<=' expression
    | expression '>=' expression
/

equalityExpression:
    expression ('==' | '===' | '!=' | '!==') expression
/

bitwiseAndLogicalExpression:
    expression '&' expression
    | expression '^' expression
    | expression '|' expression
/

conditionalAndExpression:
    expression '&&' expression
/

conditionalOrExpression:
    expression '||' expression

```

(continues on next page)

(continued from previous page)

```

/

assignmentExpression:
    lhsExpression assignmentOperator rhsExpression
/

assignmentOperator
: '='
| '+=' | '-=' | '*=' | '=' | '%='
| '<=<' | '>=>' | '>>=>'
| '&=' | '|=' | '^='
/

lhsExpression:
    expression
/

rhsExpression:
    expression
/

conditionalExpression:
    expression '?' expression ':' expression
/

stringInterpolation:
    '`' (BacktickCharacter | embeddedExpression) * '`'
/

embeddedExpression:
    '${' expression '}'
/

lambdaExpression:
    annotationUsage? ('async' | typeParameters)? lambdaSignature '=>' lambdaBody
/

lambdaBody:
    expression | block
/

lambdaSignature:
    '(' lambdaParameterList? ')' returnType?
    | identifier
/

lambdaParameterList:
    lambdaParameter (',' lambdaParameter) * (',' restParameter)? ',' '?'
    | restParameter ',' '?'
/

lambdaParameter:
    annotationUsage? (lambdaRequiredParameter | lambdaOptionalParameter)
/

lambdaRequiredParameter:
    identifier (':' type)?

```

(continues on next page)

(continued from previous page)

```

    /

lambdaOptionalParameter:
    identifier '?' (':' type)?
    /

lambdaRestParameter:
    '...' lambdaRequiredParameter
    /

constantExpression:
    expression
    /

statement:
    expressionStatement
    | block
    | localDeclaration
    | ifStatement
    | loopStatement
    | breakStatement
    | continueStatement
    | returnStatement
    | switchStatement
    | throwStatement
    | tryStatement
    /

expressionStatement:
    expression
    /

block:
    '{' statement* '}'
    /

localDeclaration:
    annotationUsage?
    ( variableDeclaration
    | constantDeclaration
    | typeDeclaration
    )
    /

ifStatement:
    'if' '(' expression ')' thenStatement
    ('else' elseStatement)?
    /

thenStatement:
    statement
    /

elseStatement:
    statement
    /
```

(continues on next page)

(continued from previous page)

```
loopStatement:
    (identifier ':'?)?
    whileStatement
    | doStatement
    | forStatement
    | forOfStatement
    ;

whileStatement:
    'while' '(' expression ')' statement
    ;

doStatement
    : 'do' statement 'while' '(' expression ')'
    ;

forStatement:
    'for' '(' forInit? ';' expression? ';' forUpdate? ')' statement
    ;

forInit:
    expressionSequence
    | variableDeclarations
    ;

forUpdate:
    expressionSequence
    ;

forOfStatement:
    'for' '(' forVariable 'of' expression ')' statement
    ;

forVariable:
    identifier | ('let' | 'const') identifier (':' type)?
    ;

breakStatement:
    'break' identifier?
    ;

continueStatement:
    'continue' identifier?
    ;

returnStatement:
    'return' expression?
    ;

switchStatement:
    (identifier ':'?)? 'switch' '(' expression ')' switchBlock
    ;

switchBlock
    : '{' caseClause* defaultClause? caseClause* '}'
    ;
```

(continues on next page)

(continued from previous page)

```

caseClause
    : 'case' expression ':' statement*
    ;

defaultClause
    : 'default' ':' statement*
    ;

throwStatement:
    'throw' expression
    ;

tryStatement:
    'try' block catchClauses finallyClause?
    ;

catchClauses:
    typedCatchClause* catchClause?
    ;

catchClause:
    'catch' '(' identifier ')' block
    ;

typedCatchClause:
    'catch' '(' identifier ':' typeReference ')' block
    ;

finallyClause:
    'finally' block
    ;

classDeclaration:
    classModifier? ('class' | 'struct') identifier typeParameters?
    classExtendsClause? implementsClause? classBody
    ;

classModifier:
    'abstract' | 'final'
    ;

classExtendsClause:
    'extends' typeReference
    ;

implementsClause:
    'implements' interfaceTypeList
    ;

interfaceTypeList:
    typeReference (',' typeReference)*
    ;

classBody:
    '{'
    classBodyDeclaration* globalInitializer? classBodyDeclaration*
    '}'

```

(continues on next page)

(continued from previous page)

```

    /

classBodyDeclaration:
    annotationUsage?
    accessModifier?
    ( constructorDeclaration
    | classFieldDeclaration
    | classMethodDeclaration
    | classAccessorDeclaration
    )
    /

accessModifier:
    'private'
    | 'internal'
    | 'protected'
    | 'public'
    /

classFieldDeclaration:
    fieldModifier* variableDeclaration
    /

fieldModifier:
    'static' | 'readonly'
    /

classMethodDeclaration:
    methodModifier* typeParameters? identifier signature block?
    /

methodModifier:
    'abstract'
    | 'static'
    | 'final'
    | 'override'
    | 'native'
    | 'async'
    /

classAccessorDeclaration:
    accessorModifier*
    ( 'get' identifier '(' ' ' ')' returnType block?
    | 'set' identifier '(' parameter ' ' ')' block?
    )
    /

accessorModifier:
    'abstract'
    | 'static'
    | 'final'
    | 'override'
    /

constructorDeclaration:
    'constructor' parameters throwMark? constructorBody
    /

```

(continues on next page)



(continued from previous page)

```

constructorBody:
    '{' statement* constructorCall? statement* '}'
    ;

constructorCall:
    'this' arguments
    | 'super' arguments
    ;

interfaceDeclaration:
    'interface' identifier typeParameters?
    interfaceExtendsClause? '{' interfaceMember* '}'
    ;

interfaceExtendsClause:
    'extends' interfaceTypeList
    ;

interfaceMember:
    annotationUsage?
    ( interfaceProperty
    | interfaceMethodDeclaration
    )
    ;

interfaceProperty:
    'readonly'? identifier '?:' type
    | 'get' identifier '(' ')' returnType
    | 'set' identifier '(' parameter ')'
    ;

interfaceMethodDeclaration:
    identifier signature
    | interfaceDefaultMethodDeclaration
    ;

enumDeclaration:
    'const'? 'enum' identifier '{' enumConstantList '}'
    ;

enumConstantList:
    enumConstant (',' enumConstant)* ','?
    ;

enumConstant:
    identifier ('=' constantExpression)?
    ;

compilationUnit:
    separateModuleDeclaration
    | packageDeclaration
    | declarationModule
    ;

packageDeclaration:
    packageModule+

```

(continues on next page)

(continued from previous page)

```

/

separateModuleDeclaration:
    importDirective* (topDeclaration | topLevelStatements | exportDirective)*
/

importDirective:
    'import'
    (allBinding | selectiveBindings | defaultBinding | typeBinding 'from')?
    importPath
/

allBinding:
    '*' bindingAlias
/

selectiveBindings:
    '{' (nameBinding (',' nameBinding)*)? '}'
/

defaultBinding:
    identifier | ( '{' 'default' 'as' identifier '}' )
/

typeBinding:
    'type' selectiveBindings
/

nameBinding:
    identifier bindingAlias?
/

bindingAlias:
    'as' identifier
/

importPath:
    StringLiteral
/

declarationModule:
    importDirective*
    ( 'export'? ambientDeclaration
    | 'export'? typeAlias
    | selectiveExportDirective
    )*
/

topDeclaration:
    ('export' 'default'?)?
    annotationUsage?
    ( typeDeclaration
    | variableDeclarations
    | constantDeclarations
    | functionDeclaration
    | functionWithReceiverDeclaration
    | accessorWithReceiverDeclaration

```

(continues on next page)

(continued from previous page)

```

    | namespaceDeclaration
    | ambientDeclaration
  )
;

namespaceDeclaration:
  'namespace' qualifiedName '{' topDeclaration* '}'
;

exportDirective:
  selectiveExportDirective
  | singleExportDirective
  | exportTypeDirective
  | reExportDirective
;

selectiveExportDirective:
  'export' selectiveBindings
;

singleExportDirective:
  'export' 'default'? identifier
;

exportTypeDirective:
  'export' 'type' selectiveBindings
;

reExportDirective:
  'export' ('*' | selectiveBindings) 'from' importPath
;

topLevelStatements:
  statement*
;

ambientDeclaration:
  'declare'
  ( ambientConstantDeclaration
  | ambientFunctionDeclaration
  | ambientClassDeclaration
  | ambientInterfaceDeclaration
  | ambientNamespaceDeclaration
  | 'const'? enumDeclaration
  )
;

ambientConstantDeclaration:
  'const' ambientConstList ';'
;

ambientConstList:
  ambientConst (',' ambientConst)*
;

ambientConst:
  identifier ((':' type) | ('=' 


```

↪ (IntegerLiteral | FloatLiteral | StringLiteral | MultilineStringLiteral) ) (continues on next page)

(continued from previous page)

```

/

ambientFunctionDeclaration:
  'function' identifier typeParameters? signature
/

ambientClassDeclaration:
  'class' identifier typeParameters? classExtendsClause? implementsClause?
  '{' ambientClassBodyDeclaration* '}'
/

ambientClassBodyDeclaration:
  ambientAccessModifier?
  (
    ambientFieldDeclaration
  | ambientConstructorDeclaration
  | ambientMethodDeclaration
  | ambientAccessorDeclaration
  | ambientIndexerDeclaration
  | ambientCallSignatureDeclaration
  | ambientIterableDeclaration
  )
/

ambientAccessModifier:
  'public' | 'protected'
/

ambientFieldDeclaration:
  ambientFieldModifier* identifier ':' type
/

ambientFieldModifier:
  'static' | 'readonly'
/

ambientConstructorDeclaration:
  'constructor' parameters throwMark?
/

ambientMethodDeclaration:
  ambientMethodModifier* identifier signature
/

ambientMethodModifier:
  'static'
/

ambientAccessorDeclaration:
  ambientMethodModifier*
  ( 'get' identifier '(' ')' returnType
  | 'set' identifier '(' parameter ')'
  )
/

ambientIndexerDeclaration:
  'readonly'? '[' identifier ':' indexType ']' returnType
/

```

(continues on next page)

(continued from previous page)

```

ambientCallSignatureDeclaration:
    signature
    ;

ambientIterableDeclaration:
    '[Symbol.iterator]' '(' ')' returnType
    ;

ambientInterfaceDeclaration:
    'interface' identifier typeParameters?
    interfaceExtendsClause?
    '{' ambientInterfaceMember* '}'
    ;

ambientInterfaceMember
    : interfaceProperty
    | interfaceMethodDeclaration
    | ambientIndexerDeclaration
    | ambientCallSignatureDeclaration
    | ambientIterableDeclaration
    ;

ambientNamespaceDeclaration:
    'namespace' qualifiedName '{' ambientNamespaceElement* '}'
    ;

ambientNamespaceElement:
    ambientNamespaceElementDeclaration | selectiveExportDirective
    ;

ambientNamespaceElementDeclaration:
    'export'?
    (
        ambientConstantDeclaration
    | ambientFunctionDeclaration
    | ambientClassDeclaration
    | ambientInterfaceDeclaration
    | ambientNamespaceDeclaration
    | 'const'? enumDeclaration
    | typeAlias
    )
    ;

newArrayInstance:
    'new' arrayElementType dimensionExpression+ (arrayElement)?
    ;

arrayElementType:
    typeReference
    | '(' type ')'
    ;

dimensionExpression:
    '[' expression ']'
    ;

arrayElement:

```

(continues on next page)

(continued from previous page)

```

    '(' expression ')'
    ;

interfaceDefaultMethodDeclaration:
    'private'? identifier signature block
    ;

functionWithReceiverDeclaration:
    'function' identifier typeParameters? signatureWithReceiver block
    ;

signatureWithReceiver:
    '(' receiverParameter (',' parameterList)? ')' returnType? throwMark?
    ;

receiverParameter:
    annotationUsage? 'this' ':' type
    ;

accessorWithReceiverDeclaration:
    'get' identifier '(' receiverParameter ')' returnType block
    | 'set' identifier '(' receiverParameter ',' parameter ')' block
    ;

functionTypeWithReceiver:
    '(' receiverParameter (',' ftParameterList)? ')' ftReturnType
    ;

lambdaExpressionWithReceiver:
    annotationUsage? typeParameters? '(' receiverParameter (',' lambdaParameterList)?
    ↪ ')'
    returnType? throwMark? '=>' lambdaBody
    ;

trailingLambdaCall:
    ( objectReference '.' identifier typeArguments?
    | expression ('?.' | typeArguments)?
    )
    arguments block
    ;

launchExpression:
    'launch' functionCallExpression | methodCallExpression | lambdaExpression;

awaitExpression:
    'await' expression
    ;

packageModule:
    packageHeader packageModuleDeclaration
    ;

packageHeader:
    'package' qualifiedName
    ;

packageModuleDeclaration:

```

(continues on next page)

(continued from previous page)

```

importDirective* packageTopDeclaration*
;

packageTopDeclaration:
    topDeclaration | initializerBlock
;

initializerBlock:
    'static' block
;

annotationDeclaration:
    '@interface' identifier '{' annotationField* '}'
;

annotationField:
    identifier ':' type constInitializer?
;

constInitializer:
    '=' constantExpression
;

annotationUsage:
    '@' qualifiedName annotationValues?
;

annotationValues:
    '(' (objectLiteral | constantExpression)? ')'
;

ambientAnnotationDeclaration:
    'declare' annotationDeclaration
;

Identifier:
    IdentifierStart IdentifierPart*
;

IdentifierStart:
    UnicodeIDStart
    | '$'
    | '_'
    | '\\\' EscapeSequence
;

IdentifierPart:
    UnicodeIDContinue
    | '$'
    | ZWNJ
    | ZWJ
    | '\\\' EscapeSequence
;

ZWJ:
    '\u200C'
;

```

(continues on next page)

(continued from previous page)

```

ZWNJ:
  '\u200D'
;

UnicodeIDStart
: Letter
| ['$']
| '\\ ' UnicodeEscapeSequence;

UnicodeIDContinue
: UnicodeIDStart
| UnicodeDigit
| '\u200C'
| '\u200D';

UnicodeEscapeSequence:
'u' HexDigit HexDigit HexDigit HexDigit
| 'u' '{' HexDigit HexDigit+ '}'
;

Letter
: UNICODE_CLASS_LU
| UNICODE_CLASS_LL
| UNICODE_CLASS_LT
| UNICODE_CLASS_LM
| UNICODE_CLASS_LO
;

UnicodeDigit
: UNICODE_CLASS_ND
;

Literal:
  IntegerLiteral
| FloatLiteral
| BigIntLiteral
| BooleanLiteral
| StringLiteral
| MultilineStringLiteral
| NullLiteral
| UndefinedLiteral
| CharLiteral
;

IntegerLiteral:
  DecimalIntegerLiteral
| HexIntegerLiteral
| OctalIntegerLiteral
| BinaryIntegerLiteral
;

DecimalIntegerLiteral:
'0'
| DecimalDigitNotNull ('_'? DecimalDigit)*
;

```

(continues on next page)



(continued from previous page)

```

DecimalDigit:
    [0-9]
    ;

DecimalDigitNotNull:
    [1-9]
    ;

HexIntegerLiteral:
    '0' [xX] ( HexDigit
    | HexDigit (HexDigit | '_' ) * HexDigit
    )
    ;

HexDigit:
    [0-9a-fA-F]
    ;

OctalIntegerLiteral:
    '0' [oO] ( OctalDigit
    | OctalDigit (OctalDigit | '_' ) * OctalDigit )
    ;

OctalDigit:
    [0-7]
    ;

BinaryIntegerLiteral:
    '0' [bB] ( BinaryDigit
    | BinaryDigit (BinaryDigit | '_' ) * BinaryDigit )
    ;

BinaryDigit:
    [0-1]
    ;

FloatLiteral:
    DecimalIntegerLiteral '.' FractionalPart? ExponentPart? FloatTypeSuffix?
    | '.' FractionalPart ExponentPart? FloatTypeSuffix?
    | DecimalIntegerLiteral ExponentPart FloatTypeSuffix?
    ;

ExponentPart:
    [eE] [+-]? DecimalIntegerLiteral
    ;

FractionalPart:
    DecimalDigit
    | DecimalDigit (DecimalDigit | '_' ) * DecimalDigit
    ;

FloatTypeSuffix:
    'f'
    ;

BigIntLiteral:
    '0n'

```

(continues on next page)

(continued from previous page)

```

| [1-9] ('_'? [0-9]) * 'n'
;

BooleanLiteral:
  'true' | 'false'
;

StringLiteral:
  '"' DoubleQuoteCharacter* '"'
| '\'' SingleQuoteCharacter* '\''
;

DoubleQuoteCharacter:
  ~["\\r\n]
| '\\' EscapeSequence
;

SingleQuoteCharacter:
  ~['\\r\n]
| '\\' EscapeSequence
;

EscapeSequence:
  ["bfprt0\\]
| 'x' HexDigit HexDigit
| 'u' HexDigit HexDigit HexDigit HexDigit
| 'u' '{' HexDigit+ '}'
| ~[1-9xu\r\n]
;

MultilineStringLiteral:
  '`' (BacktickCharacter) * '`'
;

BacktickCharacter:
  ~['\\r\n]
| '\\' EscapeSequence
| LineContinuation
;

LineContinuation:
  '\\' [\r\n\u2028\u2029]+
;

NullLiteral:
  'null'
;

UndefinedLiteral:
  'undefined'
;

CharLiteral:
  'c\' ' SingleQuoteCharacter '\''
;

```

---

### Contributors

---

Language design lead:

- Nedoria Aleksei

Contributors:

- Bronnikov Georgy
- Gavrin Evgeny
- Huo Qingyi
- Kanatov Alexey
- Nedoria Aleksei
- Pavlyuk Alexander
- Pei Jiajun
- Polyakov Alexander
- Qiu Yu
- Rubanov Vladimir
- Soldatov Anton
- Solomennikov Dmitry
- Trubenkov Dmitrii
- Velikanov Michael
- Xian Yuqiang
- Zouev Evgeniy

Authors of the language specification:

- Kanatov Alexey
- Nedoria Aleksei

- Trubenkov Dmitrii
- Zouev Evgeniy

Technical writer:

- Baranov Dmitry

## A

- abrupt completion, 93–95, 98, 104, 107, 112, 114, 121, 122, 141–144, 149, 151, 160, 161, 254
- abstract class, 101, 164, 167, 177
- abstract concept, 2
- abstract data structure, 2
- abstract declaration, 4
- abstract method, 101, 165, 167, 177, 178, 186, 275
- abstract method call, 109
- abstract method declaration, 177
- abstract modifier, 165, 176, 177
- abstract notion, 2
- abstract signature, 193
- abstract symbol, 3
- abstraction, 1
- access, 36, 43, 49, 50, 60, 73, 107–109, 111, 114, 141, 145, 159, 172–174, 182, 186, 190, 193–195, 201, 204, 206, 208, 211, 214, 218, 226, 230, 237, 250, 252, 264, 275, 276, 278, 279, 283, 287, 299
- access declaration, 179
- access expression, 142
- access modifier, 50, 163, 171, 172, 185, 237, 278
- accessibility, 47, 49, 50, 60, 102, 110, 148, 171–173, 185–187, 190, 204, 208, 211, 253, 278, 279
- accessible function, 110
- accessible interface type, 166
- accessible member field, 101, 107
- accessible method, 109
- accessor, 43, 102, 103, 163, 169, 179–181, 192, 223
- accessor declaration, 163
- accessor modifier, 179, 180
- accessor notation, 192
- addition, 11, 128
- additive expression, 28, 127
- additive operator, 27, 28, 78, 127–129
- alias, 33, 36, 41, 51, 52, 68, 206, 274
- allocation, 149
- alpha-numeric character, 208
- alternate constructor call statement, 185
- ambient, 221
- ambient annotation, 288
- ambient call signature, 224
- ambient call signature declaration, 224
- ambient class, 224, 225
- ambient constant, 222
- ambient context, 222, 224
- ambient declaration, 209, 221, 288
- ambient field declaration, 223
- ambient function, 221
- ambient function declaration, 222
- ambient indexer declaration, 224
- ambient interface declaration, 225
- ambient iterable declaration, 224, 226
- ambient namespace declaration, 226
- ambiguity, 97
- AND operator, 138
- annotation, 33, 53, 55, 147, 148, 155, 230, 274, 275, 280, 283–288
- annotation declaration, 288
- annotation field, 284, 285
- anonymous class, 101, 102, 104
- anonymous type, 51
- applicable candidate, 240, 241
- argument, 57–60, 92, 95, 105, 110, 218, 230, 242, 243, 257, 259, 274, 283
- argument expression, 78, 148
- argument transformation, 241, 242
- argument type, 66, 170, 260
- argument value, 53, 54, 76
- arithmetic operator, 78
- ArkUI, 295
- array, 31, 36, 51, 54, 57, 63, 64, 93, 98, 99, 106, 112, 137, 144, 155, 249, 252–254, 284, 294, 302
- array access expression, 93
- array argument, 59, 60
- array bounds checking, 199

- array creation, 249
- array creation expression, 96, 115, 252
- array declaration, 36
- array element, 31, 36, 37, 52–54, 76, 93, 98–100, 111, 142, 144, 145, 252
- array element type, 93, 99
- array indexing, 112
- array indexing expression, 93, 112, 142, 144
- array initializer, 98
- array instance, 115
- array length, 35, 36, 142, 144
- array literal, 42, 43, 76, 93, 98–100, 105, 230, 249, 286
- array literal expression, 98
- array reference expression, 93, 112, 142, 144
- array type, 24, 30, 36, 37, 57, 59, 98, 105, 111, 112, 234, 286
- assignability, 37, 38, 41, 60, 99, 107, 117, 141, 142, 228
- assignable, 230
- assignable class, 142
- assignable type, 117, 141
- assigned value, 276
- assignment, 29, 31, 36, 43, 57, 76, 77, 89, 93–96, 98, 104, 105, 114, 136, 141–144, 148, 155, 159, 174, 189, 196, 271, 276, 299, 302
- assignment context, 76
- assignment expression, 54, 141–144
- assignment operator, 141, 143, 144
- assignment subexpression, 144
- assignment-like context, 76
- assignment-like contexts, 55
- associativity, 94, 124, 128, 138, 140
- asymmetric relationship, 228
- async function, 255, 256, 273, 274
- async mark, 147
- async method, 275
- async modifier, 249
- automatic transition, 2
- await expression, 91, 273
- await statement, 250
- B**
  - backslash, 16, 17
  - backspace, 17
  - backtick, 17, 146
  - backward compatibility, 274
  - balanced brace, 152
  - balanced braces, 186
  - base class, 66, 165
  - base URL, 208
  - basic coroutine, 272
  - best candidate, 241–243
  - BigInt, 27, 29
  - bigint, 27, 29, 129, 131, 132, 134
  - bigint comparison, 131
  - bigint equality operator, 134
  - bigint literal, 15, 34
  - bigint type, 31
  - binary, 12
  - binary expression, 120
  - binary operation, 143, 144
  - binary operator, 29, 94, 124–129
  - bind all, 50
  - binding, 202–205, 210
  - bitwise AND operand, 139
  - bitwise complement expression, 123
  - bitwise complement operator, 27, 28
  - bitwise exclusive OR operand, 139
  - bitwise expression, 78, 138, 140
  - bitwise inclusive OR operand, 139
  - bitwise logical AND operator, 129
  - bitwise operator, 78, 95, 138, 150
  - bitwise operator expression, 139
  - block, 56, 108, 152, 157, 159, 160, 178, 182, 186, 261, 270
  - block scope, 50, 152
  - block statement, 152
  - body, 50, 56
  - Boolean, 30, 244
  - boolean, 27, 28, 30, 45, 78, 251, 284
  - boolean comparison, 132
  - boolean equality, 134, 135
  - boolean equality operator, 135
  - Boolean literal, 16
  - boolean logical expression, 139
  - boolean operand, 140
  - Boolean operator, 139
  - boolean operator, 139
  - Boolean type, 22, 124, 135, 140, 145
  - boolean type, 26, 28, 29, 124, 130, 132, 133, 135, 138–140, 145
  - boolean value, 145
  - bound, 228
  - bound entity, 204, 205
  - boxed type, 68, 99, 149
  - boxing, 41, 58, 99, 149, 242, 294
  - boxing conversion, 28, 29, 77, 84, 85, 93, 124
  - brace, 98
  - bracket, 111
  - break statement, 153, 156
  - built-in annotation, 288
  - built-in array, 68
  - Byte, 27, 28, 158
  - byte, 27, 29, 45, 158
- C**
  - call, 105, 110, 270, 274

- call argument, 230, 240, 277
- call context, 76
- call expression, 257
- call method, 174
- call parameter type, 98
- call signature compatibility, 240
- callable type, 224, 257
- callback, 274
- caller context, 161
- captured by lambda, 149
- captured value, 149
- captured variable, 149
- captured variable type, 149
- carriage return character, 8
- cast, 28, 29, 93, 275
- cast expression, 14, 44, 76, 79, 93, 98, 116, 150, 277
- cast operator, 27, 28, 95, 116, 277
- casting context, 79, 116
- casting conversion, 4
- casting conversion, 116, 231
- catch clause, 159–161
- catch identifier, 159
- catch section, 117
- chaining operator, 92, 110–112, 114, 145
- channel, 249, 250
- Char, 158, 251
- char, 12, 14, 45, 78, 80, 158, 242, 251
- char literal, 250
- character, 3, 7
- character literal, 250
- character to string conversion, 87
- character type, 22, 26
- circular reference, 39
- class, 29, 31, 37, 47, 54, 63, 64, 66, 68, 70, 71, 80, 92, 101, 106–108, 157, 165–170, 172–174, 178, 180, 181, 186, 189, 191, 199, 201, 210, 216, 224, 233, 249, 256, 257, 259, 262, 274
- class body, 163, 169–173
- class constructor, 173, 239
- class declaration, 25, 51, 163, 164, 166, 167, 172, 176, 262, 283
- class declaration body, 176
- class declaration scope, 170
- class extension, 166, 227, 262
- class field, 101, 167, 170, 301
- class fields, 173
- class initializer, 107, 157, 163, 169, 173, 181
- class instance, 53, 100, 104, 170, 173, 182, 224, 260
- class instance creation expression, 95, 96, 115
- class instance field, 172
- class instance variable, 172
- class level scope, 4
- class level scope, 50
- class member, 50, 171, 278
- class member signature, 229
- class method, 180, 187
- class method overloading, 259
- class name, 190
- class object, 302
- class type, 23, 30, 43, 69, 70, 101, 107, 164, 166, 167, 231, 234, 254, 257, 294, 301
- class variable, 230
- class-level scope, 163
- clause, 165
- closure, 227
- code readability, 230
- comma, 11
- comma-separated argument expression, 95
- comma-separated list, 100
- comment, 4
- comment, 3, 7, 8, 18
- common subset, 2
- communication channel, 272
- commutative operation, 124, 128
- commutative operator, 138
- comparison, 11, 30, 131, 133, 138
- comparison operator, 27, 28
- compatibility, 53–55, 59, 69, 75, 86, 99, 101, 110, 135–137, 158, 161, 167, 194, 221, 224, 226, 240, 244, 256, 257, 271, 274, 275, 277, 297, 299, 300
- compatible type, 75, 84, 88, 135
- compilation, 112, 201, 252
- compilation environment, 208
- compilation unit, 2, 3, 49, 50, 97, 150, 201, 202, 204, 208–211, 214, 215, 237, 250, 279, 283, 293
- compile time, 21, 75, 93, 108, 117, 174, 260, 276, 284, 299
- compile-time error, 238, 240
- compile-time error, 14, 15, 32, 36, 38–40, 42, 48, 49, 52–58, 60, 61, 65–67, 69, 72, 75–80, 82, 83, 87, 89, 92, 96, 99–116, 120–124, 127–133, 135–138, 140, 141, 143–145, 148, 153–159, 164–168, 172–185, 188, 190–193, 196, 200, 203, 206, 208–210, 212, 215–217, 221, 222, 234, 237, 243, 252, 253, 255–262, 264–266, 269, 270, 273, 277–279, 283–286, 289
- compile-time feature, 2
- compile-time polymorphism, 232
- compiler, 21, 52, 105, 113, 175, 183, 184, 231, 241, 255, 256, 288, 293–295
- compiler environment, 208
- compiler plugin, 295
- complement operator, 30, 123

- completion, 121, 199
- completion failure, 199
- compliance, 80
- component programming, 2
- composite literal context, 76
- compound assignment operator, 143, 144
- compound-assignment operator, 94
- concatenation, 76, 146, 182
- concatenation operator, 33
- concurrency, 249
- conditional evaluation, 140
- conditional expression, 145, 231, 244
- conditional operator, 27, 28, 30, 94, 145, 150
- conditional-and expression, 78, 140, 244, 245
- conditional-and operator, 30, 78, 120, 140, 150
- conditional-or expression, 78, 140, 244, 245
- conditional-or operator, 30, 78, 120, 140, 150
- configuration file, 208
- const declaration, 152
- const enum, 195
- const modifier, 155
- constant, 4
- constant, 28, 29, 47, 49, 54, 76, 77, 85, 89, 158, 195–197, 210, 230, 251, 271, 272
- constant declaration, 4
- constant declaration, 14, 15, 54, 55, 202, 272
- constant expression, 33, 37, 93, 128, 150, 158, 196, 252, 284
- constant field, 173
- constant value, 132, 135
- constant variable, 150
- constraint, 63–65, 69, 109, 136, 233, 280
- construct, 2, 24, 249, 250
- constructor, 28, 29, 53, 56, 59, 67, 70, 76, 92, 104–107, 115, 157, 159, 161, 163, 164, 169, 171, 172, 182–186, 221, 223, 232, 238, 239, 251, 253, 254, 260, 277, 278
- constructor body, 182, 184, 185
- constructor call, 76, 95, 184, 230, 277
- constructor call statement, 106
- constructor declaration, 157, 163, 182, 185
- constructor overloading, 260
- constructor parameter, 53, 54
- contained expression, 106
- container, 72
- context, 10, 38, 51, 54, 70, 75, 76, 98–102, 124, 221, 228, 230–232, 298
- context-free grammar, 4
- context-free grammar, 3
- continue statement, 153, 156
- contravariance, 66, 67, 88, 229, 234
- contravariance pattern, 181
- contravariant, 67, 233

- control, 179, 219
- control transfer, 156, 157, 159
- conversion, 29, 30, 38, 44, 54, 75–80, 82, 84–90, 99, 116, 121–124, 126–128, 130, 133, 139, 142–145, 150, 182, 197, 228, 231, 242, 252, 271, 277, 294, 299
- conversion from union, 82
- converted type, 130, 133, 143
- convertibility, 121, 123, 124, 134, 139
- convertible expression, 121
- core package, 278
- coroutine, 91, 219, 249, 272–275
- covariance, 66, 67, 88, 229, 234
- covariance pattern, 181
- covariant, 67, 301
- covariant array assignment, 302
- covariant overriding, 299
- creation, 184
- creation expression, 53, 100, 182, 260
- cross-platform development, 2
- curly brace, 100, 146

## D

- data type, 45
- database, 277
- deallocation, 2
- decimal, 12, 15, 78
- decimal form, 28
- decimal number, 15
- declaration, 19, 47, 49, 50, 53, 55, 67, 167, 169, 184, 193, 201, 203, 207, 209–211, 214–216, 221, 222, 230, 250, 283, 285, 288
- declaration context, 76
- declaration module, 201, 209
- declaration scope, 48, 192, 203–205, 210, 259
- declaration-site variance, 67
- declared entity, 48
- declared interface, 190
- declared name, 211
- declaring class, 171
- decrement, 114
- decrement expression, 121, 122
- decrement operator, 27–29, 93, 95, 120–122, 150
- decrementation, 121, 122
- default export scheme, 211, 215
- default implementation, 262
- default method, 262
- default method body, 262
- default type, 63
- default value, 44, 54, 58, 64, 177, 254
- delimiter, 3
- denormalized number, 29
- denormalized value, 29
- derived class, 50, 66, 165, 172, 199, 249



- derived type, 43
- direct call expression, 106
- direct extension, 189
- direct implementation, 189
- direct subclass, 165, 166
- direct subinterface, 191
- direct superclass, 165, 170, 186, 227
- direct superclass constructor, 184
- direct superinstance, 170
- direct superinterface, 166, 167, 186, 190–193, 227, 228
- direct supertype, 227, 228
- directive, 214
- distinct argument, 69
- distinct generic declaration, 69
- distinguishable declaration, 48, 49, 260
- distinguishable function, 261
- dividend, 125–127
- division, 126
- division operator, 28, 125, 126
- divisor, 125–127
- do statement, 154, 156, 244
- dot operator, 50
- dot-separated name, 96
- Double, 29
- double, 28, 29, 45
- double quotes, 16
- dynamically created object, 2
- dynamically dispatched overriding, 1
- DynamicObject, 276

## E

- element type, 36, 252
- embedded expression, 146
- embedded type, 70
- empty body, 178, 261
- enclosing type declaration, 184
- end marker, 3
- ensure-not-nullish expression, 44, 119
- entity, 47–50, 63, 96, 97, 137, 138, 186, 202, 204, 205, 207, 208, 210, 211, 221, 226, 230–232, 240, 275, 284
- entry point, 210, 217, 218
- enum, 210, 249
- enum constant, 158, 195, 196
- enum declaration, 51
- enum member, 50
- enum type, 31, 166, 190, 271
- enumeration, 26, 77, 89, 133, 195, 196
- enumeration constant, 84, 132, 195–197, 271
- enumeration constant value, 135
- enumeration declaration, 25
- enumeration integer value, 196
- enumeration method, 271

- enumeration operator, 132
- enumeration type, 23, 26, 44, 68, 78, 132, 135, 190, 195, 197, 271, 284
- environment variable, 208
- equality, 126, 138
- equality expression, 78, 132
- equality operator, 27, 33, 78, 95, 132–137, 150, 251
- equality test, 303
- error, 28, 29, 31, 93–95, 99, 125, 127, 129, 149, 161, 199, 200, 271
- error handling, 199
- escape character, 17
- escape sequence, 16, 17, 250
- evaluation, 91–96, 98, 104, 105, 107, 110, 112, 114, 117, 119–122, 125, 135–138, 140–145, 148, 153, 155, 157, 158, 173, 174, 185, 254, 255
- exclusive OR operator, 138
- executable code, 176, 221
- execution, 94, 98, 104, 117, 148, 149, 151, 152, 154, 156–158, 160, 161, 185, 199
- exit, 219
- exit condition, 156
- explicit constructor call, 54, 184
- explicit initialization, 44
- explicit instantiation, 63
- explicit type argument, 184
- export, 50, 202, 204, 207–211, 214–216, 250, 275, 287
- export directive, 214–216
- export modifier, 211
- export status, 186
- export type, 207, 216
- exported declaration, 211
- exported entity, 278
- expression, 4
- expression, 3, 21, 50, 58, 61, 75, 76, 78, 80, 82, 85–87, 91–101, 104–106, 110, 111, 114–116, 119–124, 128, 130, 140, 141, 143–146, 152–155, 157, 158, 174, 179, 184, 196, 252, 254, 257, 272, 277, 285, 286
- expression await, 273
- expression statement, 273
- expression type, 77, 79, 110, 115, 139, 158, 273
- expression value, 76, 154
- extended conditional expression, 145, 244
- extended equality, 137, 138
- extended exponent, 142
- extended functionality, 70
- extended semantics, 33, 140
- extends clause, 165, 166, 190, 262, 302, 303
- extends graph, 166
- extension, 189, 191, 208
- extension clause, 183, 227

## F

- factory, 258
- failure, 199
- field, 48, 92, 101, 102, 104, 107, 108, 145, 163, 168–170, 174, 179, 180, 183, 192, 286
- field access, 107, 114, 141, 145, 275, 276
- field access expression, 44, 107, 141, 276
- field annotation, 286
- field declaration, 163, 172, 173, 175
- field initialization, 64, 173, 181, 288
- field initializer, 164, 173–175, 272
- field modifier, 172
- field overriding, 175
- field resolution, 108
- field type, 101
- field value, 179
- file, 293
- file path, 208
- file system, 208, 277, 293
- filename, 208
- final, 165
- final class, 249, 262
- final method, 177, 179, 262
- final modifier, 165, 176
- finally block, 160
- finally clause, 159, 160
- finite value, 125, 126, 128, 131
- fit into (v.), 4
- fixed array, 252
- fixed array type, 30, 99, 252
- flexibility, 255
- Float, 29
- float, 28, 29, 45
- floating-point addition, 128
- floating-point arithmetic, 128
- floating-point calculation, 94
- floating-point comparison, 130, 131
- floating-point division, 125, 126
- floating-point equality test, 133
- floating-point expression, 30
- floating-point infinity, 80
- floating-point literal, 15
- floating-point number, 28, 29, 123
- floating-point operand, 126, 129, 131, 134
- floating-point operation, 29, 125, 126, 129
- floating-point operator, 29
- floating-point remainder, 127
- floating-point remainder operation, 126, 127
- floating-point subtraction, 129
- floating-point type, 26, 28, 29, 80, 84, 112, 125, 126, 129, 252
- floating-point value, 29, 123, 131, 134
- floating-type multiplication, 124

- flush to zero, 29
- folder, 208, 293
- for statement, 155, 156, 244
- for-of loop, 155
- for-of statement, 155, 156, 256
- for-of type annotation, 155, 259
- for-variable, 259
- form feed, 8, 17
- formal parameter, 76
- function, 31, 32, 37, 47, 51, 53, 56–61, 63, 64, 66–70, 76, 105, 110, 111, 152, 156, 157, 159, 161, 201, 210, 217, 218, 221, 230–232, 237, 249, 250, 255–257, 259–261, 270, 272, 279, 285
- function async, 274
- function body, 56, 60, 148, 186, 219, 222, 261, 274, 275
- function body declaration, 50
- function call, 44, 49, 56, 58, 76, 93, 106, 110, 111, 114, 230, 259, 269, 272, 273
- function call expression, 95, 110, 111, 272
- function caller, 54
- function declaration, 4
- function declaration, 50, 56, 69, 147, 148
- function name, 50, 279
- function object, 303
- function overloading, 56, 249, 259
- function parameter, 53, 54, 186
- function parameter scope, 4
- function parameter scope, 50
- function return type, 61
- function scope, 4
- function scope, 50
- function type, 23, 24, 30, 31, 37, 38, 67, 88, 92, 110, 147–149, 190, 234, 269, 270, 294, 299
- function type with receiver, 24
- function types conversion, 4
- function types conversion, 88
- function with receiver, 264

## G

- generic, 4
- generic, 2, 63, 65, 67, 69, 190, 294
- generic class, 25, 64, 66, 68, 164, 200, 228, 250, 255, 280
- generic class declaration, 167
- generic declaration, 63–65, 69, 190
- generic entity, 69
- generic function, 56, 64
- generic instantiation, 63, 64, 68, 69, 230
- generic interface, 64, 167, 190
- generic lambda, 64
- generic method, 97
- generic parameter, 63, 67
- generic type, 4

generic type, 25, 32, 52, 66, 103, 116, 228, 232  
 genericity, 2  
 get-accessor, 180  
 getter, 70–72, 168, 169, 179, 180, 192, 193  
 getting, 179  
 goal symbol, 4  
 goal symbol, 3  
 gradual underflow, 29  
 grammar, 4  
 grammar production, 3  
 grammar rule, 3, 92  
 GUI programming, 247

## H

hard keyword, 10  
 header, 259  
 hexadecimal, 12, 17  
 hiding, 186, 239  
 high-level language, 2  
 horizontal tab, 17  
 horizontal tabulation, 8  
 host system, 208, 293

## I

identification, 56  
 identifier, 3, 9–11, 25, 48, 57, 58, 100, 101, 108, 153, 156, 164, 176, 190, 204, 278  
 identity conversion, 84  
 IEEE 754, 28, 29, 80, 84, 124–129, 131, 133, 304  
 if statement, 153, 231, 244  
 immutable variable, 152  
 implementation, 29, 61, 149, 163, 166, 167, 178, 189–193, 208, 209, 224, 231, 249, 250, 255, 261, 262, 293, 294  
 implementation clause, 227  
 implementation method, 178  
 implementing, 163  
 implements clause, 166, 167, 303  
 implicit boxing, 58  
 implicit conversion, 14, 15, 79, 146, 228  
 implicit instantiation, 70  
 import, 50, 202, 206, 207, 209, 211, 217, 237, 250, 275, 278, 279, 287, 288  
 import binding, 202, 204, 205, 207, 208  
 import declaration, 202, 205  
 import directive, 202, 203, 205, 210, 214, 287  
 import expression, 274  
 import outcome, 205  
 import path, 202, 204, 205, 208, 293  
 import site, 241  
 import statement, 205  
 import type, 207, 216  
 imported declaration, 202  
 imported file, 208  
 imported function, 279  
 imported module, 287  
 in-place type definition, 24  
 in-position, 67  
 in-variance, 63  
 inclusive OR operator, 138  
 increment, 114  
 increment expression, 121, 122  
 increment operator, 27–29, 93, 95, 120, 121, 150  
 incrementation, 121  
 index expression, 37, 73, 111–114, 255, 276  
 index parameter, 255  
 index subexpression, 142, 144  
 indexable type, 111  
 indexing, 96, 224  
 indexing access, 275  
 indexing access expression, 276  
 indexing expression, 44, 73, 96, 111, 113, 114, 143–145, 255  
 inference, 53, 54, 61, 75, 148  
 inference type, 103  
 inferred type, 101  
 infinite operand, 128  
 infinite value, 128  
 infinity, 94, 123, 125–128, 131  
 inheritance, 1, 31, 66, 99, 166, 169, 170, 173, 186, 189–193, 232, 237, 249, 301  
 inherited field, 186  
 inheriting, 181  
 initial value, 54  
 initialization, 44, 53, 54, 71, 98, 103, 104, 115, 148, 149, 152, 173, 182, 183, 202, 210, 217, 221, 250, 254, 272, 279, 300  
 initialization expression, 98, 100, 272  
 initializer, 54, 55, 101, 107, 150, 157, 174, 175, 202, 221, 223, 284  
 initializer expression, 53–55, 98, 99, 222  
 innermost declaration, 50  
 instance, 31, 32, 50, 54, 73, 95, 99, 100, 104, 115, 147, 149, 163, 173, 174, 177, 183–185, 224, 256, 271, 276, 277  
 instance creation expression, 54, 149  
 instance field, 173, 183  
 instance field access, 107  
 instance field initializer, 174  
 instance initializer, 107  
 instance member, 48, 148  
 instance method, 106–108, 176, 178, 183, 184, 186, 194, 238, 239, 260, 262  
 instance method call, 107  
 instance name, 173  
 instance variable, 92, 189  
 instanceof expression, 116, 117  
 instanceof operator, 116

instantiation, 25, 32, 63, 64, 66, 68–70, 115, 167, 173, 189, 257–259, 275

Int, 27, 28, 158

int, 14, 27–29, 45, 158

integer, 12, 13, 15, 29, 34, 36, 84, 127, 128, 184, 196, 252

integer addition, 94

integer arithmetic, 128

integer bitwise expression, 140

integer bitwise operator, 27, 28, 139

integer conversion, 85

integer division, 93, 94, 125, 129, 130

integer equality test, 133

integer literal, 14

integer multiplication, 94, 124

integer operand, 125, 126, 131, 134

integer overflow, 125

integer remainder, 93, 94, 126

integer subtraction, 129

integer type, 26, 28, 29, 123, 129, 139, 196

integer value, 15, 27, 123, 125, 129, 196

interface, 31, 50, 63, 64, 68, 70, 80, 92, 102–104, 107, 108, 166, 167, 169, 180, 186, 189–194, 201, 210, 216, 224, 249, 255, 256, 259, 275, 278, 284

interface body, 191–193, 262

interface declaration, 25, 51, 189–191

interface field, 76

interface level scope, 4

interface level scope, 50

interface member, 50, 191

interface method, 180, 193

interface property, 103, 168

interface type, 23, 25, 27, 30, 43, 69–72, 101–103, 106, 167, 189, 190, 192, 231, 234, 277

interface type declaration, 228

interleaving, 67

internal declaration, 250

internal member, 186

internal modifier, 163, 278

internal state, 149

interoperability, 275

invariance, 66, 67, 229, 234

invariant, 67, 301

invariant array assignment, 302

iterable class, 256

iterable type, 256

iterator, 256, 257

## K

key, 72, 104, 113, 114, 142, 144

key-value pair, 142, 144

keyword, 4

keyword, 3, 9–11, 16, 18

keyword abstract, 261, 262

keyword const, 221

keyword constructor, 182

keyword export, 209, 284

keyword extends, 64

keyword final, 262

keyword in, 67

keyword interface, 284

keyword native, 182, 261, 262

keyword null, 33

keyword out, 67

keyword static, 262

keyword super, 50, 148, 176, 181, 184

keyword this, 50, 106, 107, 148, 176, 181, 184, 264

keyword undefined, 33

## L

label, 156

lambda, 59, 63, 64, 96, 149, 269, 270, 272

lambda body, 107, 148, 149

lambda call, 76, 148

lambda declaration, 68

lambda expression, 96, 106, 110, 147–149, 230, 249

lambda expression evaluation, 149

lambda expression type, 148

lambda function, 252, 270

lambda parameter, 148

lambda return type, 148

lambda signature, 147–149

launch, 249, 250, 272, 273

launch expression, 91, 249, 272–274

lazy operator, 120

left shift, 129

length of array, 98

let declaration, 152

let modifier, 155

lexical element, 3

lexical grammar, 3

lexical input, 7, 8

lexical input element, 9

lexical notation, 3

lexical structure, 3

line separator, 3, 7, 9, 19, 283

line separator character, 8

linearization, 4

linearization, 41

linkage, 94

literal, 4

literal, 9, 12, 13, 16, 17, 33, 41, 71–73, 86, 90, 96, 99, 104, 113, 114, 150

literal expression, 99

literal type, 17, 23, 30, 34, 35, 39–41, 44, 55, 77, 90, 104, 113

literal value, 16, 41  
 local class, 186, 187  
 local declaration, 152  
 local interface, 186, 187  
 local variable, 54, 149, 186, 230, 231  
 logical complement operator, 124  
 logical expression, 78, 138, 140  
 logical operator, 30, 78, 95, 138, 139, 150  
 Long, 27, 28, 158  
 long, 14, 27–29, 45, 158  
 lookup sequence, 293  
 loop, 153, 155, 245  
 loop iterations, 155  
 loop label, 153  
 loop statement, 153, 156  
 loss of information, 125, 126, 129  
 loss of precision, 127  
 low-level representation, 2  
 low-order bit, 124, 128  
 lvalue, 93

## M

magnitude, 126–128  
 maintainability, 2  
 mapped value, 114  
 mask value, 129  
 match (v.), 4  
 member, 163, 169–171, 189, 278  
 member field, 107  
 metadata, 283  
 metasympol, 5  
 metasympol, 3  
 method, 5  
 method, 28, 29, 31, 32, 36, 43, 47, 48, 50, 53, 56, 57, 59, 60, 63, 64, 67, 68, 70, 71, 76, 92, 103, 105, 109, 152, 156, 157, 159, 161, 163, 165, 169, 170, 172, 177, 180, 181, 189, 191–193, 197, 221, 223, 230–233, 237, 238, 240, 251, 257–259, 262, 269, 270, 274, 276, 285, 294  
 method body, 57, 60, 61, 106, 148, 157, 176, 178, 184, 186, 193, 261  
 method body declaration, 50  
 method call, 44, 58, 76, 93, 95, 108, 109, 114, 176, 230, 257, 260, 269, 275, 294  
 method call expression, 93, 96, 108, 110, 115  
 method caller, 54  
 method declaration, 163, 165, 167, 176, 178, 182, 191, 261, 262  
 method final, 249  
 method modifier, 176, 179  
 method name, 50  
 method overloading, 56, 108, 193, 249, 259, 260  
 method parameter, 53, 54, 186  
 method resolution, 108

method return type, 61, 110  
 method scope, 5  
 method scope, 50  
 method signature, 177, 178  
 migration, 2  
 modification, 155  
 modifier, 50, 155, 165, 173, 278  
 modifier abstract, 177, 179  
 modifier async, 177, 222  
 modifier declare, 221  
 modifier final, 177  
 modifier native, 177  
 modifier override, 177  
 modifier private, 186  
 modifier protected, 186  
 modifier public, 185, 186  
 modifier static, 177  
 modularity, 2  
 module, 2, 3, 49, 202–204, 207, 208, 210, 214, 216–219, 221, 277–279, 293  
 module level scope, 5  
 module level scope, 49, 51  
 multiline comment, 18  
 multiline string, 16, 17, 146  
 multiline string literal, 17  
 multiplication, 124, 125, 129  
 multiplication operator, 125  
 multiplicative expression, 124, 150  
 multiplicative operator, 27, 28, 78, 124, 150  
 multitargeting, 2  
 mutable variable, 152

## N

name, 48–53, 56, 60, 204, 206, 216, 259  
 name-value pair, 100, 101, 104  
 named class, 101, 104  
 named entity, 48  
 named function, 56  
 named reference, 96, 97  
 named type, 24, 25  
 named variable, 145  
 namespace, 97, 211, 226  
 namespace declaration, 211  
 NaN, 29, 80, 123, 125–128, 131, 134, 245  
 NaN value, 94  
 narrowing, 77, 84, 85, 231  
 narrowing conversion, 5  
 native constructor, 262  
 native function, 56, 61, 261  
 native method, 178, 261, 275  
 negation, 123, 129  
 negative infinity, 80, 131, 134  
 negative integer, 125, 126  
 negative zero, 128, 131, 134

- nested loop, 254
- nested multiline string, 146
- nested statement, 50
- nested union type, 41
- newline character, 8
- no-argument return statement, 274, 275
- no-break space, 8
- non-abstract class, 164, 165, 285
- non-abstract instance method, 177
- non-abstract method, 177
- non-alias, 41
- non-aliased type, 26
- non-empty body, 262
- non-exported declaration, 250
- non-generic, 5
- non-generic class, 227
- non-generic entity, 68, 69
- non-generic type, 5
- non-generic type, 25
- non-initialized variable, 53
- non-nullable type, 138
- non-nullish type, 41, 43, 107, 114, 120, 280
- non-nullish variant, 119
- non-nullish-type, 64
- non-parameterized entity, 70
- non-primitive type, 149
- non-relative import, 208
- non-relative import path, 208
- non-standalone expression, 75
- non-static class, 170
- non-static field, 164, 172, 173, 175, 180, 184
- non-static field declaration, 174
- non-static field initializer, 185
- non-static method, 176
- non-union type, 41, 86
- nonterminal, 5
- nonterminal, 3
- nonterminal symbol, 5
- nonterminal symbol, 3
- nonzero operand, 128
- nonzero value, 30
- normal completion, 93–95, 104, 112, 114, 121–123, 142–144, 149–151, 159–161, 178, 199, 219
- normal execution, 199
- normalization, 41, 42, 61, 86, 116, 145
- notation, 58, 170, 173, 286, 287
- notion, 164
- null, 45, 119, 137, 138
- null expression, 158
- null literal, 18, 33, 35
- null pointer dereferencing, 199
- null reference, 18
- null safety, 43

- null-coalescing operator, 95
- nullable reference type, 44
- nullable type, 5
- nullable type, 43, 138
- nullish expression, 245
- nullish object reference, 107
- nullish type, 18, 33, 43, 110, 111, 114, 119, 120
- nullish union type, 280
- nullish value, 5
- nullish value, 43, 44, 108, 110, 111, 114, 119, 120
- nullish-coalescing expression, 44, 120
- nullish-coalescing operator, 120
- nullish-safe option, 43
- nullish-type, 64
- number, 3, 15, 45, 59, 73
- numeric, 80
- numeric constant expression, 196
- numeric context, 78
- numeric conversion, 252
- numeric expression, 245
- numeric literal, 3, 222
- numeric literal type, 41
- numeric operand, 128
- numeric operation, 29
- numeric operator, 76
- numeric promotion, 28, 29
- numeric type, 22, 27–29, 41, 72, 84, 100, 112, 121–130, 133, 138–140, 242, 284, 298
- numeric type operand, 129
- numeric types conversion, 29, 112, 121–126, 129, 130, 133, 139, 140
- numeric union type, 41
- numeric value, 84, 196
- numerical equality, 133, 134
- numerical equality operator, 27, 28
- numerical operator, 27–29
- numerical relational operator, 27, 28, 130

## O

- Object, 159, 165, 166, 170, 185, 192, 227, 228, 271
- object, 1, 31, 36, 37, 54, 64, 71, 73, 107, 113, 128, 132, 133, 135, 189, 191, 227, 228, 234, 299
- object literal, 73, 76, 100–104, 285, 286, 300
- object literal expression, 101
- object orientation, 1
- object reference, 107, 109
- object reference expression, 107, 111, 112, 114
- object reference subexpression, 144
- Object type, 41
- object type, 135, 136, 232
- object-oriented, 1
- object-oriented programming, 1
- octal, 12



one-dimensional array, 98  
 OOP, 1  
 OOP (*object-oriented programming*), 249  
 operand, 5  
 operand, 27–29, 80, 94, 116, 121–137, 141–145  
 operand expression, 116, 124, 128, 138, 144, 145  
 operand null, 78  
 operand string, 128  
 operand value, 116, 124, 128, 137, 139  
 operation, 5  
 operation, 29, 123, 128  
 operation overflow, 128  
 operation sign, 5  
 operator, 9, 11, 17, 28, 76, 93, 94, 111, 116, 119, 120, 122–124, 127, 130–133, 135–138, 141, 158  
 operator (*in programming languages*), 5  
 operator context, 128  
 operator precedence, 94  
 operator sign, 3  
 operator undefined, 78  
 optional parameter, 58  
 ordinary type, 64  
 original variable, 149  
 out-position, 67  
 out-variance, 63  
 overflow, 28, 29, 123–129  
 overlap, 50  
 overload, 264  
 overload equivalence, 232, 238, 259, 260  
 overload resolution, 109, 110, 240  
 overload-equivalence, 232, 259, 260  
 overload-equivalent method, 193  
 overload-equivalent signature, 176, 232, 237, 238, 259, 260  
 overloaded function, 110, 204, 240  
 overloaded function name, 259  
 overloaded header, 259  
 overloading, 5  
 overloading, 49, 97, 180, 193, 218, 232, 237–239, 259, 260, 279  
 overloading signature, 176  
 overridden method, 177  
 override, 35  
 override compatibility, 233, 234, 240  
 override method, 179  
 override modifier, 165, 176  
 override-compatibility, 232, 239  
 override-compatible signature, 167, 186, 194, 233, 239, 240  
 overriding, 1, 92, 163, 167, 175, 177, 181, 186, 189, 232, 237, 239, 255, 262, 301  
 overriding method, 177  
 own (*adj.*), 5

## P

package, 2, 48, 49, 186, 201–203, 207, 208, 210, 250, 274, 277, 278, 293  
 package header, 202, 277, 278  
 package initializer, 279  
 package level scope, 6  
 package level scope, 49, 51  
 package member, 48  
 package module, 277–279, 293  
 paragraph separator character, 8  
 parameter, 31, 37, 56, 58, 60, 106, 180, 185, 218, 222, 229, 232, 233, 241, 242, 252, 258, 269, 270, 280  
 parameter declaration, 148  
 parameter list, 57, 59  
 parameter name, 38, 50, 57, 59, 232  
 parameter type, 38, 57, 88, 230, 234, 240  
 parameter with default values, 58  
 parameterization, 63, 64, 69  
 parameterized class, 64  
 parameterized class type, 167  
 parameterized function, 64  
 parameterized interface, 64, 190  
 parameterized type, 67, 190  
 parameterless constructor, 102, 104, 254  
 parameterless function, 256  
 parameterless function type, 258  
 parent class, 232  
 parenthesis, 11, 24, 94, 115, 142  
 parenthesized expression, 106, 150  
 path, 199  
 path mapping, 208  
 pattern, 92  
 platform API, 226  
 platform-dependent code, 261, 262  
 point of declaration, 50  
 positive infinity, 80, 131, 134  
 positive zero, 128, 131, 134  
 postfix, 27, 28, 93, 114, 120, 121  
 postfix expression, 119  
 postfix increment expression, 121  
 postfix operator, 95  
 potentially applicable candidate, 109, 110  
 precedence, 24, 50, 94, 95  
 predefined class, 22  
 predefined numeric types conversion, 28, 29, 84, 124, 129, 130, 140  
 predefined operator, 93  
 predefined reference type, 16  
 predefined type, 21, 22, 26, 33, 122, 275  
 predefined type declaration, 43  
 predefined value, 22

- prefix, 13, 15, 27, 28, 59, 60, 93, 114, 121, 122, 150, 184, 221, 283, 284
- prefix expression, 119
- prefix operator, 95
- prefix readonly, 54, 57
- primary expression, 107, 111, 114
- primitive conversion, 86, 133
- primitive type, 6
- primitive type, 22, 39–41, 44, 68, 78, 84, 86, 116, 123, 128, 129, 132, 144, 149, 150, 166, 190, 234, 294, 301
- primitive type conversion, 84
- primitive types conversion, 128
- primordial class, 185
- private, 170, 171, 177, 237, 262
- private access modifier, 171
- private field, 73, 180
- private member, 264
- private method, 194
- private modifier, 163
- production, 6
- production, 3
- program entity, 2, 63
- promoted operand value, 123
- promoted type, 128, 129
- promoted value, 123
- propagation, 161
- property, 70–72, 103, 168, 169, 189, 191–193, 276, 277, 299
- property declaration, 173
- protected, 170, 172, 186
- protected member, 186, 264
- protected modifier, 163, 172
- provably distinct instantiation, 69
- proxy class, 149
- proxying, 149
- pseudo generic static method, 294
- public, 170–172, 186
- public member, 186, 264
- public method, 192, 193, 264
- public modifier, 163, 172
- punctuator, 6
- punctuator, 9, 11

## Q

- qualification, 195
- qualified access, 50
- qualified form, 202
- qualified name, 6
- qualified name, 48, 49, 110, 150, 170, 173, 204, 226, 287
- qualified type name, 25, 27
- qualifier, 211

## R

- radix, 12, 15
- re-assignment, 43
- re-export, 214, 216
- re-export declaration, 216
- re-export directive, 202, 216
- read permission, 199
- readability, 15
- readonly, 76, 107, 169, 192
- readonly array, 143
- readonly field, 104, 168, 173
- readonly modifier, 173
- readonly parameter, 57
- readonly tuple, 143
- receiver, 230
- receiver type, 264
- record access expression, 142, 144
- record element, 145
- record indexing expression, 144
- record instance, 114, 142, 144
- Record type, 73
- record type, 103, 111
- record utility type, 72
- recursive reference, 52
- reference, 31
- reference equality, 132, 135, 137
- reference expression, 111, 112
- reference subexpression, 142, 144
- reference type, 21, 22, 30, 33, 37, 39, 43, 44, 48, 78, 85, 93, 107, 112, 114, 116, 133, 137, 142, 163, 164, 189, 232
- reference type conversion, 84
- reflexive closure, 227
- relational expression, 78, 130
- relational operator, 30, 33, 78, 95, 130–132, 150
- relative import, 208
- relative import path, 208
- relative location, 208
- remainder operation, 126, 127
- remainder operator, 28, 29, 93, 126, 127
- renaming, 214
- required parameter, 57
- reserved names of |TS| types, 297
- reserved word, 10
- resolution, 208
- resolving, 208
- rest parameter, 38, 57, 59, 230, 241
- restriction, 54, 113, 181, 273
- return, 31, 152, 160
- return expression, 157
- return statement, 60, 61, 148, 157, 178, 181, 217, 274



- return type, 31, 32, 37, 56, 60, 61, 88, 111, 148, 152, 157, 170, 178, 180, 218, 222, 224, 229, 230, 232, 259, 273–275
  - returned value, 277
  - right shift, 130
  - round to nearest, 29
  - round toward zero, 29
  - round-to-nearest, 125, 126
  - round-to-nearest mode, 84, 129
  - round-toward-zero, 125
  - rounding, 80, 84, 125, 126, 129
  - rounding mode, 29
  - rounding rules, 80
  - rounding toward zero, 29
  - routine, 126
  - runtime, 43, 76, 93, 105, 107, 112, 116, 121–123, 128, 129, 138, 140–143, 148, 160, 161, 174, 199, 232, 254, 259, 294
  - runtime behavior, 76
  - runtime error, 80, 82, 84–87, 89, 90, 93, 112, 199, 252, 288, 298
  - runtime evaluation, 149
  - runtime expression, 249
  - runtime implementation, 294
  - runtime model, 299
  - runtime polymorphism, 232
  - runtime system, 200, 219
  - runtime value, 298
- ## S
- safe field access, 44, 107
  - safe indexing expression, 44
  - safe method call, 44, 108
  - scope, 47, 49–51, 63, 148, 163, 164, 169, 181, 186, 190, 191, 201, 237
  - scope declaration, 186
  - scope of a name, 6
  - selective binding, 214
  - selective export directive, 214
  - semantic check, 109, 110, 230, 238, 240
  - semantic correctness check, 110
  - semantics, 23, 33, 137, 138, 140, 174, 184, 195, 217, 234, 250, 283, 288, 298, 301
  - semi-automatic transition, 2
  - semicolon, 11, 19, 178, 182, 261, 270
  - separate module, 201, 202, 217, 293
  - separator, 3, 48, 145
  - sequence, 3, 217
  - set-accessor, 180
  - setter, 70–72, 102, 145, 168, 169, 179, 180, 192, 193
  - setting, 179
  - shadow parameter, 148
  - shadowing, 6
  - shadowing, 60, 148, 163, 173, 189, 301
  - shift, 129
  - shift distance, 129
  - shift expression, 129
  - shift operation, 129
  - shift operator, 78, 95, 129, 150
  - Short, 27, 28, 158
  - short, 27, 29, 45, 158
  - side effect, 93, 124, 128, 138, 140
  - sign-extension, 130
  - signature, 37, 43, 48, 56, 57, 177, 178, 186, 192–194, 232, 233, 255–261, 279
  - signed infinity, 29, 125, 126, 128
  - signed integer comparison, 130
  - signed right shift, 129
  - signed shift operator, 27, 28
  - signed zero, 29
  - simple assignment operator, 94, 141
  - simple name, 6
  - simple name, 47–49, 96, 97, 150, 204, 208, 278
  - simple type name, 25, 27
  - single quote, 16, 250
  - slash character, 208
  - smart cast, 231
  - smart type, 230, 231
  - soft keyword, 10, 11
  - source code, 8, 9, 21, 208
  - source file, 293
  - source-level compatibility, 195
  - space, 8, 283
  - special character, 3
  - spread, 106
  - spread array, 106
  - spread expression, 59, 92, 105
  - spread operator, 59, 60, 230
  - square bracket, 11
  - standalone expression, 75, 145
  - standard annotation, 288
  - standard library, 29, 36, 250, 274, 278
  - state, 31
  - statement, 19, 94, 151–157, 159, 186, 217, 272
  - statement block, 50
  - statement execution, 151
  - static context, 184
  - static field, 107, 172, 173, 181
  - static field access, 107
  - static member, 6
  - static member, 48, 50
  - static method, 108, 176, 179, 186, 238, 239, 271, 294
  - static method call, 109
  - static modifier, 176
  - static type, 230, 231
  - statically typed language, 21
  - storage management, 2

string, 15, 28, 29, 77, 78, 87, 89, 127, 128, 133, 134, 144, 146, 155, 158, 196, 197, 208, 255, 256, 284

string comparison, 131

string concatenation, 33, 76, 127, 128, 144

string concatenation operator, 27, 28, 30, 146

string context, 78

string conversion, 78, 84, 128

string equality operator, 134

string interpolation, 17

string interpolation expression, 146

string literal, 16, 17, 33, 35, 99, 146, 208, 222

string name, 64

string object, 33

string operator, 78

string type, 30, 31, 33, 72, 146, 150, 218

string value, 131, 132

strong typing, 230

struct, 295

struct type, 258

structured coroutine, 272

structuring rule, 3

subclass, 80, 164, 166, 170, 177, 178, 200, 227, 239, 262, 301

subclass relationship, 166

subcomponent (*derived component, child component*), 6

subexpression, 94, 111, 142, 144

subinterface, 80, 190, 191, 227, 240

substitution, 167, 190

subtraction, 11, 121, 123, 128, 129

subtype, 31, 227, 234

subtyping, 67, 227, 229

superclass, 80, 92, 101, 109, 163, 165–167, 173, 175, 177, 181, 184, 189, 227, 228, 237, 239, 301

superclass constructor, 184, 185

superclass constructor call, 184, 185

superclass constructor call statement, 184, 185

superclass property, 108

supercomponent (*base component, parent component*), 6

superinterface, 80, 163, 167, 168, 173, 175, 186, 189–191, 194, 227, 228, 240

supertype, 35, 55, 227, 228

supertype conversion, 90

supertyping, 67, 228

surrounding context, 75, 107, 148

surrounding function, 156, 157

surrounding method, 157

surrounding scope, 161

surrounding type, 148

switch expression, 157

switch statement, 156–158

syntactic grammar, 3, 8, 9, 18

syntactic notation, 3

syntactic structure, 3

syntactical form, 115

syntax, 2, 96, 108

syntax production, 19

## T

target type, 75–80, 82, 85, 86, 116, 277

template, 2

terminal, 6

terminal, 3

terminal symbol, 6

terminal symbol, 3, 7, 9

termination, 157

ternary conditional operator, 150

third-party library API, 226

throw, 29, 31

throw statement, 159, 181, 200

thrown object, 53, 159

thrown value, 159

token, 6

token, 3, 7–9, 11, 48

tokenization, 6

tokenization, 9

top-level declaration, 202, 207, 210, 211, 214

top-level entity, 278

top-level function, 186, 187, 219

top-level statement, 56, 157, 202, 217, 219

top-level statements, 218

top-level type, 210

top-level variable, 60, 210

trailing comma, 98, 100

trailing lambda, 269, 270

trailing lambda call, 108, 110

transformation, 99, 242

transitive closure, 166, 191, 227

truncated number, 29

truncated operand, 129

truncation, 29, 34, 126, 129, 130, 139

truthiness, 6

truthiness, 244

try block, 159, 161

try statement, 159, 161, 181, 200

try-catch, 160

tuple, 98, 99, 106

tuple argument, 59

tuple type, 24, 37, 57, 59, 89, 99, 105, 234

two's-complement format, 124, 128

two's-complement integer, 129

two's-complement representation, 123

two's-complement value, 123

type, 45, 47, 54, 55, 57, 59, 70–73, 75, 76, 82, 85–87, 89, 93, 98–100, 106, 109, 113, 114, 116, 130, 136, 155, 166, 169, 210, 227–231, 233, 252, 277, 284, 287, 288, 294  
 type alias, 24–27, 37, 38, 51, 52, 63, 64, 284  
 type alias declaration, 25, 51, 52  
 type annotation, 32–34, 53–55, 76, 98, 110, 148, 222, 259  
 type argument, 25, 27, 32, 33, 52, 64, 66, 68–70, 109, 166, 190, 232, 259, 260, 273  
 type bigint, 23, 34  
 type boolean, 138  
 type call expression, 257, 258  
 type char, 250  
 type constraint, 136  
 type declaration, 37, 39, 43, 51, 152  
 type definition, 21  
 type enum, 195  
 type expression, 61  
 type identity, 36  
 type in parentheses, 24  
 type inference, 21, 42, 43, 55, 98–102, 148, 230  
 type int, 78, 218, 252, 271  
 type key, 103  
 type long, 34  
 type name, 25, 27  
 type never, 30, 31, 41  
 type null, 18, 30, 33  
 type Object, 37  
 type operand, 128  
 type parameter, 23, 25, 30, 44, 52, 56, 63–67, 69, 109, 115, 116, 119, 132, 133, 135–137, 147, 170, 228, 232–234, 280  
 type parameter constraint, 69  
 type parameter declaration, 25  
 type parameter default, 66  
 type parameter scope, 6  
 type parameterized entity, 2  
 type property, 103  
 type readonly, 72  
 type reference, 6  
 type reference, 24–27, 64  
 type safety, 41, 230, 299  
 type string, 23, 33, 72, 117, 271  
 type structure, 24  
 type undefined, 18, 30, 32, 33, 38  
 type value, 103  
 type void, 30, 32, 60, 218  
 type-parameterized declaration, 64, 67, 190  
 typed catch clause, 159  
 typeof expression, 117  
 types conversion, 139

## U

unary, 78  
 unary bitwise complement expression, 123, 124  
 unary expression, 120  
 unary logical complement expression, 124  
 unary minus, 123  
 unary minus operator, 28  
 unary negation operation, 123  
 unary numeric promotion, 123, 129  
 unary operator, 27, 28, 120, 122–124, 150  
 unary plus, 122  
 unary plus operator, 28, 122  
 unboxed type, 41  
 unboxing conversion, 84, 86, 124, 135, 139, 140, 145, 158  
 undefined, 45, 58, 110, 119, 137, 138  
 undefined literal, 18, 33, 35  
 undefined value, 43, 103  
 underflow, 28, 29, 125–127, 129  
 underlying object, 275–277  
 underscore character, 13, 15  
 Unicode character, 7, 9  
 Unicode code point, 7, 9  
 Unicode code unit, 33  
 Unicode escape sequence, 17  
 Unicode input character, 9  
 Unicode Standard, 9  
 union, 31, 39, 82  
 union conversion, 86  
 union equality, 136  
 union type, 23, 24, 30, 39–43, 51, 55, 58, 61, 64, 72, 82, 86, 99, 100, 103, 104, 116, 119, 120, 133, 135–137, 145, 190, 231, 273  
 union type normalization, 100  
 unqualified form, 202  
 unqualified identifier, 63, 65  
 unqualified import, 287  
 unqualified name, 47  
 unqualified superclass constructor call, 184  
 unqualified superclass constructor call statement, 184  
 unsigned right shift, 129  
 unsigned shift operator, 27, 28  
 user-defined getter, 71  
 user-defined type, 21–23, 26, 43, 195  
 user-defined type declaration, 43  
 user-defined type name, 11, 297  
 utility type, 63, 70–73, 300

## V

value, 11, 13, 31, 32, 37, 50, 53, 54, 64, 72, 76, 85, 86, 90, 93, 94, 96, 98, 103, 104, 106, 107, 113, 114,

- 116, 121–124, 126, 129–132, 134–136, 141, 142, 144, 196, 222, 241, 252, 271, 274, 286, 298
- value equality, 132–135
- value equality operator, 135
- value set, 129, 142
- value set conversion, 123
- value type, 21, 26, 33, 34, 41, 58, 68, 132
- variable, **6**
- variable, 29, 31, 35, 36, 44, 47–49, 53, 54, 57, 71, 76, 89, 92–94, 112, 116, 121–124, 136, 138, 141–145, 148, 149, 155, 189, 201, 210, 230, 255, 271, 272, 279, 280, 294, 299
- variable declaration, **6**
- variable declaration, 14, 53–55, 98, 202, 272
- variance, 63, 66, 67, 229, 234
- variance modifier, 67
- vertical tab, 17
- virtual machine, 94

## W

- while statement, 154, 156, 244
- white space, **6**
- white space, 3, 7–9
- widening, 28, 29, 77, 84–86, 133
- widening conversion, **6**
- widening conversion, 84, 85
- wrap, 22
- wrapper, 275–277

## Z

- zero-extension, 129, 130
- zero-width joiner, 9
- zero-width no-break space, 8
- zero-width non-joiner, 9